

从 C++ 到 x86-64、现代 CPU 与 AI 算子优化

一条面向高性能计算和 AI CPU 算子的系统路径

CPU Performance Study

序言

这本书的目标不是让读者“看懂一些汇编”，而是建立一套完整的性能工程能力：

- C/C++ 源码语义
 - > 编译器 IR
 - > x86-64 汇编
 - > 指令与微架构
 - > cache / TLB / branch / OS
 - > benchmark / PMU / profiling
 - > SIMD / GEMM / AI CPU 算子

本书面向刚学完 C 语言和 C++ 基础、希望进入现代计算机体系结构、操作系统、编译器和高性能计算的读者。它会从“程序如何运行”“CPU 如何执行指令”“内存中的字节如何解释”这些基础问题讲起，再逐步进入汇编、ABI、编译器优化、现代 CPU 微架构、SIMD、HPC kernel 和 AI 算子优化。

本书写作遵循一个原则：任何性能结论都必须能被解释、观察和验证。只看源码不够，只看汇编不够，只看运行时间也不够。真正可靠的性能工程需要把抽象、实现、工具和实验连接起来。

本书不是速查手册

本书不是指令表，不是编译参数列表，也不是算法模板合集。每个核心机制都会按以下问题展开：

- 它为什么存在？
- 如果没有它，会遇到什么问题？
- 它给程序员提供了什么抽象？
- 底层大致如何实现？
- C/C++ 代码如何映射到这个机制？
- 如何用工具观察它？
- 它对性能有什么影响？
- 初学者最容易误解什么？

学习要求

读者需要已经写过基本 C/C++ 程序，理解变量、函数、指针、数组、结构体、类、模板的基础语法。除此之外，本书会补齐学习汇编、CPU、操作系统和性能优化所需的底层知识。

每一章都包含实验和作业。只读正文不算完成。你需要运行命令、观察输出、写代码、看汇编、做 benchmark，并写报告。

如何使用本书

本书按经典教材方式组织，但它同时也是一本实验书。每章都应按下面顺序学习：

1. 阅读本章目标和起始问题。
2. 读正文，手工推导关键例子。
3. 运行章节中的命令。
4. 修改例子，观察输出如何变化。
5. 查看汇编、IR、二进制或性能数据。
6. 完成实验。
7. 完成作业并写报告。
8. 对照验收标准检查自己是否真正掌握。

三条主线

本书有三条主线。

第一条：抽象到实现

从 C/C++ 语言抽象出发，追到编译器、汇编、机器码、CPU 执行和操作系统机制。

第二条：模型到证据

先建立模型，再用工具观察。比如先理解 cache line 和局部性，再做工作集扫描；先理解 uops 和端口，再用 `llvm-mca` 分析。

第三条：基础到工程

先掌握最小函数、简单循环和数组访问，再进入 SIMD、GEMM、softmax、layernorm、attention 和生产库。

报告要求

每个性能实验报告至少包含：

- 实验目标。
- 机器和环境。
- 编译器和编译参数。
- correctness reference。
- 输入规模。
- benchmark 方法。
- 汇编、IR 或 profiling 证据。
- 结果表格。
- 失败实验。
- 结论和下一步验证。

全书架构设计

本书按百万字级教材规划，采用三卷、十二个大章、若干附录的结构。每个大章对应一个完整专题，原先按课程周次拆开的细小章节已经合并为大章内部的节和小节；后续写作会在这些节内继续加厚，而不是继续增加大量短章。

写作目标

本书最终目标是覆盖：

- C/C++ 程序如何变成机器执行。
- x86-64 汇编、ABI、ELF、链接和加载。
- 编译器 IR、优化、自动向量化和未定义行为。
- 现代 CPU 微架构、cache、TLB、分支、乱序执行。
- 可信性能测量、benchmark、PMU 和 profiling。
- AVX2/FMA、AVX-VNNI、int8 量化。
- GEMM、packing、microkernel、MiniBLAS。
- softmax、layernorm、RMSNorm、GELU、attention、operator fusion。
- 线程、NUMA、ISA dispatch、生产库阅读。

十二个大章

1. 程序、机器与 C++ 底层地基。
2. 二进制、链接、进程与运行时系统。
3. x86-64 汇编、ABI 与 C++ 二进制接口。
4. 可信性能测量、Benchmark 工程与 Lab 00。
5. 编译器、LLVM IR 与优化。
6. 现代 CPU 微架构性能模型。
7. 内存层级、数据布局、TLB 与 Roofline。
8. SIMD、AVX2/FMA 与低精度计算。
9. HPC Kernel：从循环优化到 MiniBLAS。
10. AI CPU 算子优化。
11. 性能工具、并行、NUMA 与生产库阅读。
12. Capstone：大师级性能工程项目。

章节深度标准

每个大章按三层写：

1. 必须掌握：刚学完 C/C++ 的读者能跟上。
2. 工程理解：能用于真实代码分析和优化。
3. 深入方向：连接体系结构、OS、编译器或生产库更深内容。

交付物标准

每个大章最终应包含：

- 不少于一个贯穿例子。
- 不少于三个实验。
- 不少于三组习题。
- 一个报告型大作业。
- 一组常见误区。
- 一组验收标准。

目录

序言	ii
如何使用本书	iii
全书架构设计	iv
第一部分 第一卷：程序、机器、汇编和测量地基	1
第 1 章 程序、机器与 C++ 底层地基	2
1.1 学习地图：把程序、机器、系统和性能连成一条主线	2
1.2 程序如何从源码变成正在运行的进程	8
1.3 CPU 如何执行指令	16
1.4 数据表示、内存、指针和对象布局	30
1.5 Linux、工具链、调试器和学习 workflow	57
第 2 章 二进制、链接、进程与运行时系统	59
2.1 构建流程、编译参数和工具链心智模型	59
2.2 ELF、符号、重定位、链接和加载	59
2.3 进程、虚拟内存和地址空间	60
2.4 栈、堆、运行时和对象生命周期	60
2.5 操作系统噪声、时间源和性能实验边界	61
第 3 章 x86-64 汇编、ABI 与 C++ 二进制接口	62
3.1 x86-64 汇编语法总览	62
3.2 整数指令、位运算和地址计算	73
3.3 控制流：跳转、循环、调用与 switch	83
3.4 System V AMD64 ABI	104
3.5 C++ 与汇编互操作	119
第 4 章 可信性能测量、Benchmark 工程与 Lab 00	139
4.1 可信性能测量基础	139
4.2 Benchmark 设计：输入、热路径和报告	154
4.3 性能统计、噪声分析和实验复现	171
4.4 Lab 00：建立可信性能实验室	182
第二部分 第二卷：编译器、CPU、内存和 SIMD 性能模型	197
第 5 章 编译器、LLVM IR 与优化	198
5.1 编译器流水线：从源码到 IR	198
5.2 LLVM IR、SSA 和控制流图	210
5.3 标量优化：DCE、常量传播、内联和 LICM	216
5.4 循环优化：展开、重排、归约和依赖	223
5.5 自动向量化	228
5.6 Alias、未定义行为和浮点语义	235
第 6 章 现代 CPU 微架构性能模型	241
6.1 Pipeline、前端和后端总览	241
6.2 Latency、Throughput、依赖链和 ILP	241
6.3 uops、执行端口和 llvm-mca	242
6.4 分支预测和推测执行	242
6.5 前端带宽、I-cache、uop cache 和代码尺寸	243
6.6 乱序执行、寄存器重命名和提交	243
第 7 章 内存层级、数据布局、TLB 与 Roofline	245
7.1 内存层级：寄存器、cache、DRAM	245
7.2 Cache line、局部性和 miss 类型	245
7.3 TLB、页和地址翻译性能	246
7.4 Load/Store、store buffer、prefetch 和写入策略	246
7.5 数据布局：AoS、SoA、AoSoA 和 tensor layout	247
7.6 Roofline：FLOP、Bytes 和 Arithmetic Intensity	247
第 8 章 SIMD、AVX2/FMA 与低精度计算	249
8.1 SIMD 思维模型	249
8.2 AVX2 Intrinsics 从入门到可审计代码	249
8.3 FMA、归约和浮点误差	250
8.4 Shuffle、Permute、Gather 和数据移动	250

8.5	Tail、Mask、Alignment 和边界策略	251
8.6	int8 量化、AVX-VNNI 和低精度推理	251
第三部分 第三卷：HPC、AI 算子、生产工具和综合项目		253
第 9 章	HPC Kernel：从循环优化到 MiniBLAS	254
9.1	Loop Nest 性能模型	254
9.2	Transpose、Stencil 和 Reduction	254
9.3	GEMM：从 naive 到 cache blocking	255
9.4	GEMM Packing 和 Panel Layout	255
9.5	AVX2 GEMM Microkernel	256
9.6	MiniBLAS：把 GEMM 做成可维护库	256
第 10 章	AI CPU 算子优化	258
10.1	AI CPU 算子性能模型	258
10.2	Softmax：数值稳定性、归约和向量化	258
10.3	LayerNorm 和 RMSNorm	259
10.4	GELU、激活函数和近似计算	259
10.5	Attention、KV Cache 和 Streaming Softmax	260
10.6	Operator Fusion：收益、代价和边界	260
10.7	MiniDNN：教学版 AI CPU 算子库	261
第 11 章	性能工具、并行、NUMA 与生产库阅读	262
11.1	perf、PMU 和 Top-Down 分析	262
11.2	VTune、AMD uProf 和图形化性能分析	262
11.3	线程、Atomics 和内存模型入门	263
11.4	False Sharing、Cache Coherence 和 NUMA	263
11.5	CPUID、ISA Dispatch 和多版本 Kernel	264
11.6	生产库阅读：oneDNN、BLIS、OpenBLAS	264
第 12 章	Capstone：大师级性能工程项目	266
12.1	Capstone 项目设计	266
12.2	性能事故复盘方法	266
12.3	最终评审：从学习者到性能工程师	267
附录 A	x86-64 速查表	268
A.1	通用寄存器	268
A.2	常见指令	268
附录 B	Linux 与二进制工具命令速查	269
B.1	编译与反汇编	269
B.2	ELF 和符号	269
B.3	调试	269
附录 C	LaTeX 写作规范	270
C.1	章节规则	270
C.2	代码规则	270
C.3	实验和习题	270
C.4	扩写节奏	270
附录 D	参考资料和延伸阅读	271
D.1	公开课程	271
D.2	官方和一手资料	271
D.3	性能数据和生产库	271
附录 E	实验和作业评分标准	272
E.1	正确性	272
E.2	复现性	272
E.3	证据链	272
E.4	失败实验	272
	术语表	273

Part I

第一卷：程序、机器、汇编和测量地基

第 1 章 程序、机器与 C++ 底层地基

1.1 学习地图：把程序、机器、系统和性能连成一条主线

1.1.1 本章不是普通导读

如果一本系统性能书只在开头说“我们会学习 C++、汇编、CPU、操作系统和性能工具”，这并不能真正帮助初学者。因为初学者最缺的不是名词清单，而是一张能不断回来的地图：当你在第 10 章看到 `mov`，第 31 章看到 `cache line`，第 56 章看到 `PMU event`，第 50 章优化 `softmax` 时，你必须知道这些东西为什么属于同一件事。

本章的目标就是建立这张地图。

刚学完 C/C++ 的读者通常已经知道变量、函数、数组、指针、结构体、类和简单的标准库调用。但从性能工程角度看，这些只是最上层的表达。程序真正运行时，下面还存在许多层：

```
C/C++ source
-> language semantics
-> compiler frontend
-> IR and optimization
-> target-specific backend
-> assembly
-> machine-code bytes
-> object file and executable
-> loader and process
-> virtual memory
-> ISA architectural state
-> microarchitectural execution
-> cache / TLB / branch predictor / memory system
-> measured time and performance counters
```

性能优化的难点在于：每一层都在隐藏下一层的复杂性，但每一层隐藏得都不完美。平时写业务代码时，隐藏是好事；做高性能计算和 AI CPU 算子时，隐藏经常变成障碍。你必须知道抽象在什么时候可靠，什么时候会泄漏，什么时候会被编译器、CPU 或操作系统用一种你没想到的方式重新解释。

核心理想

本书的主线不是“先学一堆概念，再做优化”。本书的主线是：从一个 C/C++ 表达式出发，追踪它如何变成机器码、如何改变寄存器和内存、如何穿过 CPU 流水线和缓存层级、如何受到操作系统影响，最后如何变成可信或不可信的性能数据。

1.1.2 一个贯穿全书的例子

先看一个非常普通的函数：

```
int sum(int const* a, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += a[i];
    }
    return s;
}
```

如果只从 C++ 层看，它很简单：遍历数组，把元素加起来。可是为了真正理解它的性能，你至少要能回答下面这些问题：

- `int const* a` 是什么？它只是一个地址，还是还携带了类型和别名语义？
- `a[i]` 如何变成地址计算？为什么是 `base+i*4`？
- `s+=a[i]` 在汇编中会使用哪些寄存器？会不会每次都访问内存？
- 编译器能否把循环展开？能否向量化？为什么有时候不能？
- 如果 `n` 很小，循环开销和函数调用开销是否比内存访问还重要？
- 如果 `n` 很大，性能瓶颈是整数加法，还是内存带宽？
- 如果 `a` 没有按 `cache line` 对齐，是否一定慢？
- 如果多个线程同时处理数组，是否会发生 `false sharing`？

- benchmark 的结果为什么每次都不一样？操作系统调度、CPU 频率、cache 状态分别可能贡献什么噪声？
- 这些问题没有一个可以只靠“懂 C++ 语法”解决。它们分别落在语言语义、编译器、汇编、ISA、微架构、操作系统和测量方法上。

心智模型

以后看到任何性能代码，都先把它拆成四个视角：

1. 源码视角：程序员写了什么语义。
2. 编译器视角：哪些语义限制了优化，哪些语义允许优化。
3. 机器视角：实际执行了哪些指令，访问了哪些地址。
4. 测量视角：数据是否真的证明了你的判断。

1.1.3 计算机系统为什么要分层

现代计算机太复杂，不可能让程序员直接控制每个晶体管、每条内部总线、每个缓存替换决策和每个乱序执行细节。因此系统必须分层。分层的作用是让上层只依赖下层的一部分承诺。

比如 C++ 不要求你知道整数加法电路如何实现，但它要求机器能表示某些整数值。汇编不要求你知道 CPU 内部有多少执行端口，但它要求 CPU 能按照 ISA 的语义执行 `add`。操作系统不要求应用知道物理内存条的真实地址，但它提供虚拟地址空间，让进程以为自己拥有连续内存。

分层带来的好处是可编程性，代价是性能信息被藏起来。性能工程正是在这些隐藏处工作。

1.1.3.1 物理层：信号、晶体管和时钟

最底层是物理世界。电路通过电压高低表示 0 和 1。晶体管可以被看作受控制的开关。大量晶体管组合成逻辑门、触发器、寄存器、加法器、选择器、缓存 SRAM 单元和控制逻辑。

本书不会把你训练成芯片设计工程师，但你必须知道两个事实。

第一，硬件不是“瞬间完成”的。信号传播需要时间，门电路切换需要时间，存储单元读写需要时间。因此 CPU 有时钟周期，有关键路径，有流水线。性能中的 latency 和 throughput，最终都与“工作需要经过多少硬件阶段”和“硬件能否并行处理多个工作”有关。

第二，硬件资源是有限的。一个核心不可能无限发射指令，不可能无限读取内存，不可能无限预测分支，不可能无限保存乱序执行中的临时状态。因此现代 CPU 性能常常不是由某一条高级语句决定，而是由执行端口、load/store 单元、ROB 容量、物理寄存器、L1/L2/L3 cache、TLB、内存带宽等资源共同决定。

1.1.3.2 数字逻辑层：组合逻辑和时序逻辑

组合逻辑的输出只由当前输入决定，比如加法器、比较器、多路选择器。时序逻辑会记住状态，比如寄存器、触发器、计数器和缓存 tag。CPU 可以被看作一个巨大的状态机：每个时钟周期，它根据当前状态和输入，计算下一状态。

这个模型非常重要，因为 ISA 也可以被理解为状态机规则。以 `add rax, rbx` 为例，抽象语义是：

```
RAX_next = RAX_old + RBX_old
RFLAGS_next = flags produced by the addition
```

真实 CPU 内部可能把这条指令拆成微操作，可能乱序执行，可能重命名寄存器，但最终对软件可见的结果必须像上面的状态转移一样。这就是后面“架构状态”和“微架构实现”的区别。

1.1.4 ISA：软件和硬件之间的合同

ISA (Instruction Set Architecture) 是指令集体系结构。它定义软件能看到的机器模型：有哪些寄存器、有哪些指令、指令如何编码、内存寻址如何表达、异常如何发生、哪些状态对程序可见。

x86-64 ISA 给程序员承诺了很多东西，例如：

- 有一组通用寄存器，如 `rax`、`rbx`、`rcx`、`rdx`、`rdi`、`rsi`、`rsp`、`rbp`、`r8` 到 `r15`。
- 指令可以读取寄存器、写寄存器、读取内存、写内存、改变控制流。
- 内存可以按字节寻址。
- 常见寻址模式可以写成 `base+index*scale+displacement`。
- `rip` 表示当前执行位置，`rsp` 表示栈顶。

但 ISA 不告诉你 CPU 内部有多少流水线阶段，也不告诉你某条指令占用哪个执行端口，不告诉你 L1 cache 多大，不告诉你分支预测器怎么猜。这些属于微架构。

常见误区

初学者经常把 ISA 和 CPU 实现混在一起。说“x86-64 有 `rax`”是 ISA 层；说“某 Intel 核心上整数加法吞吐约为每周期若干条”是微架构层。优化时必须区分这两层，否则会把一台机器上的性能经验误当成所有 x86-64 机器的规律。

1.1.5 微架构：同一份 ISA 可以有不同的执行机器

同样执行 x86-64 指令，不同 CPU 的内部结构可以非常不同。一个简单核心可以顺序执行；一个高性能核心会取指、解码、分派、重命名、乱序执行、访存、提交。CPU 还会预测分支、预取数据、维护多级缓存、处理 TLB、解决内存一致性问题。

微架构的存在解释了一个看似奇怪的事实：两段汇编指令数量相同，性能可能差很多；两段 C++ 源码看起来复杂度相同，真实性能也可能差很多。

例如：

```
add eax, dword ptr [rdi + rcx*4]
```

这条指令表面上是“从数组加载一个 int 并加到 `eax`”。微架构层至少可能发生：

- 地址生成单元计算 `rdi+rcx*4`。
- load 单元向 L1 data cache 发起读取。
- 如果虚拟地址还没有 TLB 命中，需要页表相关机制参与。
- 如果 L1 miss，访问 L2；L2 miss，访问 L3；再 miss，访问内存。
- 加法依赖 load 的结果，因此 load latency 会影响后续加法。
- 如果循环中有多个独立 load，乱序执行可以重叠等待。

同一条汇编，在 cache 命中和 cache miss 时的代价完全不同。这就是为什么本书必须把“指令”和“数据所在位置”一起讲。

1.1.6 操作系统：程序不是独占机器

C++ 程序不是直接裸奔在 CPU 上。普通 Linux 程序运行在操作系统提供的进程环境中。操作系统负责创建进程、建立虚拟地址空间、加载可执行文件、处理系统调用、调度线程、管理页面、处理中断和异常。

从性能角度看，操作系统至少带来五类重要影响：

1. 虚拟内存让程序看到的是虚拟地址，不是物理地址。
2. 页面和 TLB 会影响大数组、随机访问和多线程程序。
3. 调度会让线程在不同核心之间迁移，影响 cache 热度和测量稳定性。
4. 系统调用和 page fault 会造成远高于普通指令的开销。
5. CPU 频率、电源管理、后台任务和中断会污染 benchmark。

因此，真正的性能工程不是“写一段快代码”这么简单。你还要控制运行环境，知道什么时候该固定 CPU 频率，什么时候该 pin 线程，什么时候该丢弃第一次运行，什么时候要看 page fault、context switch 和 PMU 事件。

1.1.7 编译器：它不是翻译员，而是证明器和重写器

很多初学者把编译器理解成“把 C++ 翻译成汇编的工具”。这个理解只对了一半。现代优化编译器更像一个在语言规则约束下工作的证明器和重写器。

它会问：

- 这个变量的值是否一定不被使用？
- 这个循环是否可以展开？
- 两个指针是否可能指向同一块对象？
- 这个函数是否可以内联？
- 这个条件是否永远成立或永远不成立？
- 这个浮点表达式能不能重排？
- 这个内存访问是否连续，能不能向量化？

只要 C++ 标准允许，编译器就可能把源码改写成完全不同的机器执行形式。比如下面的代码：

```
int f(int x)
{
    return x + 1 > x;
}
```

在没有有符号整数溢出的前提下，编译器可以认为 `x+1>x` 恒为真，从而返回 1。可是如果你按机器补码回绕去想，`x` 等于最大正整数时似乎会溢出。关键在于：C++ 有符号整数溢出是未定义行为，编译器可以假设它不会发生。

深入理解

性能优化必须尊重语言规则。很多“我看机器码应该这样”的推理，如果违反 C++ 对对象生命周期、别名、对齐、未定义行为的规定，就不是可靠优化，而是在给编译器提供错误前提。后续讲 `alias`、`restrict`、`std::bit_cast`、`fast-math` 和自动向量化时，会反复回到这一点。

1.1.8 ABI：函数之间、模块之间必须说同一种话

ABI (Application Binary Interface) 是二进制接口。它规定函数调用时参数如何传递、返回值放在哪里、哪些寄存器由调用者保存、哪些由被调用者保存、栈如何对齐、结构体如何返回、异常和动态链接如何约定。源码层面两个函数看起来只是：

```
int g(int x, int y);
int z = g(1, 2);
```

机器层面却必须回答：

- 1 放进哪个寄存器？
- 2 放进哪个寄存器？
- 调用前 `rsp` 要满足什么对齐要求？
- `g` 能随便改哪些寄存器？
- 返回值从哪里取？

System V AMD64 ABI 是 Linux x86-64 上的核心规则之一。后面学习汇编时，如果不懂 ABI，就看不懂函数序言、函数尾声、栈帧、寄存器保存和 C++ 与汇编互操作。

1.1.9 内存：性能工程的第二个主角

第一个主角是指令，第二个主角是数据。现代 CPU 的算术单元非常快，但内存层级相对慢得多。一个 AI 算子是否快，常常不取决于“乘法会不会写”，而取决于数据如何布局、是否连续、是否复用、是否落在 cache、是否适合 SIMD load/store。

看下面两个结构：

```
struct ParticleAoS {
    float x;
    float y;
    float z;
    float mass;
};

struct ParticleSoA {
    float* x;
    float* y;
    float* z;
    float* mass;
};
```

AoS 和 SoA 的差异不是语法审美问题，而是内存访问模式问题。如果一个 kernel 只需要所有粒子的 `x`，AoS 会把不需要的 `y`、`z`、`mass` 一起拖进 cache line；SoA 则能连续加载 `x`。后续讲 SIMD、cache line、GEMM packing 和 AI 算子 fusion 时，这个思想会不断出现。

1.1.10 性能不是运行一次得到的数字

性能工程最危险的习惯之一，是把一次运行时间当成真相。真实机器上，一次运行结果可能受到很多因素影响：

- 编译器是否把代码优化掉。
- 输入数据是否还在 cache 中。
- CPU 当前频率是多少。
- 线程是否被调度到另一个核心。
- 后台进程和中断是否打断了运行。
- 分支预测器和 TLB 是否已经被预热。
- 第一次执行是否触发动态链接、page fault 或内存分配。

所以本书要求你形成“证据链”意识。一个高质量性能结论通常至少包含：

1. correctness reference：优化前后结果是否一致。
2. 编译参数：是否启用合理优化，是否说明目标 ISA。
3. 汇编或 IR：机器实际执行的形态是什么。
4. benchmark 方法：如何预热、重复、统计、避免被优化删除。

5. profiling 或 PMU：瓶颈证据支持哪个解释。
6. 反例检查：有没有输入规模或机器配置让结论失效。

常见误区

“我的版本快了 20%”不是结论，只是观察。真正的结论必须说明：在哪台机器、什么编译器、什么参数、什么输入规模、什么统计方法下快了 20%；为什么快；有什么证据排除了测量噪声和错误优化；在哪些条件下不保证快。

1.1.11 本书的四条长期能力线

为了在 3 到 4 个月内尽量逼近专家水平，本书不按“轻松每周一点点”的节奏写，而是按高强度训练设计。你要同时推进四条能力线。

1.1.11.1 能力线一：机器阅读能力

机器阅读能力是指你能从源码一路读到汇编、机器码、寄存器和内存变化。最低要求是能读懂简单函数的汇编；更高要求是能看出编译器为什么生成这种形式，性能瓶颈可能在哪里。

这条线的训练材料包括：

- objdump 和 llvm-objdump。
- gdb 单步执行。
- Compiler Explorer。
- clang-S、clang-emit-llvm。
- 手工追踪寄存器和栈。

1.1.11.2 能力线二：性能模型能力

性能模型能力是指你能先不用跑程序，就对瓶颈做出合理预测。比如一个循环每次加载 64 字节、做 16 次浮点运算，你应该能估算它更可能受内存带宽限制还是计算吞吐限制。

这条线会反复训练：

- latency 和 throughput 的区别。
- dependency chain 和 instruction-level parallelism。
- cache line、working set、reuse distance。
- memory bandwidth 和 arithmetic intensity。
- SIMD lane、向量宽度、tail 处理。
- Roofline 模型。

1.1.11.3 能力线三：测量和诊断能力

测量能力是指你知道怎样设计 benchmark，怎样避免错误测量，怎样用工具定位问题。性能工程中，错误测量比不会优化更危险，因为它会把你带到错误方向。

这条线的工具包括：

- perfstat 和 perfrecored。
- time、taskset、numactl。
- llvm-mca。
- Intel VTune 或 AMD uProf。
- 自己写 benchmark harness。

1.1.11.4 能力线四：kernel 设计能力

kernel 设计能力是指你能把机器模型用到真实计算核心里。最终目标不是会背 CPU 概念，而是能写出、解释和验证高性能 kernel，例如 reduction、transpose、stencil、GEMM microkernel、softmax、layernorm、GELU、attention 中的小核心。

这条线要求你能同时处理：

- 数据布局。
- 分块和复用。
- SIMD intrinsic。
- 数值稳定性。
- 多线程和 NUMA。
- ISA dispatch。

- benchmark 和 profiling。

1.1.12 一条完整的优化闭环

本书要求你以后按下面流程做优化：

1. 写出清晰正确的 baseline。
2. 准备 correctness reference 和误差标准。
3. 建立粗略性能模型：计算量、访存量、预期瓶颈。
4. 设计 benchmark，避免死代码消除和输入偏差。
5. 编译并查看汇编或 IR。
6. 运行测量，记录环境和统计结果。
7. 用 profiling 或 PMU 验证瓶颈。
8. 修改一个因素：布局、循环、分块、SIMD、预取或并行。
9. 再次验证正确性和性能。
10. 写报告，说明哪些假设被证实，哪些被推翻。

这就是专业性能工程和“凭感觉改代码”的区别。

1.1.13 本章实验：建立你的机器档案

实验

本实验不是为了追求速度，而是为了建立后续所有实验都要引用的机器档案。报告保存目录：

```
results/machine-profile/
```

任务 1：记录 CPU 和系统信息

运行：

```
lscpu
cat /proc/cpuinfo | sed -n '1,80p'
cat /proc/meminfo | sed -n '1,40p'
uname -a
gcc --version
clang --version
ldd --version
```

报告中至少记录：

- CPU 型号、核心数、线程数。
- L1d/L1i/L2/L3 cache 信息。
- 支持的 ISA 扩展，例如 SSE、AVX、AVX2、AVX-512、FMA、VNNI。
- 内存总量。
- Linux kernel 版本。
- GCC/Clang 版本。

任务 2：建立第一个源码到汇编证据链

创建 labs/ch00_machine_map/sum.cpp：

```
#include <cstdlib>
#include <stdint>

extern "C" std::int64_t sum_i32(std::int32_t const* a, std::size_t n)
{
    std::int64_t s = 0;
    for (std::size_t i = 0; i < n; ++i) {
        s += a[i];
    }
    return s;
}
```

分别生成未优化和优化后的汇编：

```
clang++ -std=c++20 -O0 -S -masm=intel sum.cpp -o sum_00.s
clang++ -std=c++20 -O3 -S -masm=intel sum.cpp -o sum_03.s
```

报告必须回答：

1. -O0 和 -O3 的循环形态有什么差异？
2. 参数 a 和 n 分别从哪些寄存器进入函数？
3. 汇编中哪里体现了 sizeof(std::int32_t)==4？

4. 编译器有没有展开或向量化？证据是什么？

任务 3：第一次测量但不急着下结论

写一个小 benchmark，测量 $n=1024$ 、 $n=1048576$ 、 $n=67108864$ 三种规模。每个规模运行多次，记录最小值、中位数和最大值。报告中必须写明为什么三种规模可能对应不同 cache 行为。

1.1.14 本章作业：写出你的第一份性能工程备忘录

习题与作业

提交一份不少于 1500 字的中文报告，题目为“从 C++ 到机器执行的第一张地图”。报告必须包含以下部分：

1. 画出从 C++ 源码到 CPU 执行的链路图，并用自己的话解释每一层的职责。
2. 用 `sum_i32` 例子解释源码、汇编、寄存器、内存访问之间的关系。
3. 说明 ISA 和微架构的区别，并举一个“同一条汇编在不同数据状态下性能不同”的例子。
4. 说明操作系统为什么会影响 benchmark。
5. 列出你当前最不熟悉的 10 个概念，并为每个概念写一句你准备如何验证它。

评分重点不是语言华丽，而是证据完整、层次清楚、没有把不同抽象层混在一起。

1.1.15 验收标准

完成本章后，你应该能做到：

- 画出源码到机器执行的完整粗粒度链路。
- 区分语言语义、编译器优化、ISA、微架构、操作系统和测量方法。
- 对一个简单循环提出至少三种可能性能瓶颈。
- 用汇编证据回答“编译器实际生成了什么”。
- 解释为什么一次运行时间不能直接当成性能结论。

后续章节会把这张地图逐层放大。第 1 章放大“源码到进程”；第 2 章放大“CPU 如何执行指令”；第 3 章放大“数据、内存、指针和对象布局”；第 10 章开始正式进入 x86-64 汇编；第 31 章以后进入内存层级；第 37 章以后进入 SIMD；第 45 章以后进入 GEMM 和 AI 算子。

1.2 程序如何从源码变成正在运行的进程

1.2.1 本章要解决的问题

刚学完 C/C++ 时，你通常知道三件事：

1. 写一个 `.cpp` 文件。
2. 用编译器生成可执行文件。
3. 在终端运行它。

但你可能还不知道：

- 头文件到底是不是被“编译进来”。
- `g++main.cpp-oapp` 背后调用了哪些工具。
- `main` 为什么通常不是程序真正执行的第一条指令。
- 可执行文件里除了机器码还有什么。
- 操作系统如何把文件变成一个进程。
- 指针地址为什么每次运行可能不同。
- 这些机制为什么会影响性能测量。

核心思想

程序运行不是“源码直接交给 CPU”。源码要经过预处理、编译、优化、汇编、链接、加载和运行时启动，最后 CPU 才开始执行机器指令。性能工程的很多错误判断，都来自看不见这条链路。

1.2.2 从一个最小程序开始

先看一个最小但真实的 C++ 程序：

```
#include <iostream>

int main()
{
    int x = 1;
    int y = 2;
    std::cout << x + y << '\n';
    return 0;
}
```

从人的角度看，它做了三件事：

创建两个整数
计算 $x + y$
打印结果

从系统角度看，它至少涉及：

```
source file
-> preprocessor
-> compiler frontend
-> optimizer
-> compiler backend
-> assembly
-> assembler
-> object file
-> linker
-> executable ELF
-> loader
-> process address space
-> runtime startup
-> main
-> CPU executes instructions
```

本章的任务就是把这条路径铺开。

1.2.3 为什么这条路径必须学

你最终的目标是高性能计算和 AI CPU 算子优化。为什么第一个基础问题不是 AVX2，而是“程序怎么运行”？

因为很多性能现象发生在源码之外：

- 编译器可能删除你以为正在测量的循环。
- 函数可能被 inline，二进制里不再有独立函数体。
- 动态链接和全局构造可能影响第一次运行时间。
- 不同编译参数可能生成完全不同的指令。
- 同一个地址表达式可能映射到栈、堆、全局区或 mmap 区域。
- 共享库调用、PLT/GOT、lazy binding 可能影响调用路径。
- 程序启动成本不是 kernel 成本。

如果你不知道这些层，就很容易把系统行为误认为算法行为。

1.2.4 第一层：源文件、头文件和翻译单元

C/C++ 源文件常见扩展名：

```
.c
.cc
.cpp
.cxx
```

头文件常见扩展名：

```
.h
.hpp
```

初学者容易以为头文件会被单独编译，然后和源文件“合并”。通常不是这样。C/C++ 的基本编译单位是 **翻译单元** (translation unit)：一个源文件经过预处理后的结果。

如果你写：

```
#include <vector>
#include "kernel.hpp"
```

预处理器会把这些头文件的内容展开到当前源文件中。最终编译器看到的是一个很大的文件。

1.2.4.1 为什么头文件模型影响性能

头文件不是小事。它影响：

- inline 函数是否对编译器可见。
- 模板代码在哪里实例化。
- 宏是否改变源码。
- 条件编译是否选择了不同 ISA 路径。
- 编译时间和代码膨胀。

例如，一个高性能库可能在头文件里写：

```
#if defined(__AVX2__)
// AVX2 implementation
#else
// scalar fallback
#endif
```

你以为自己读的是同一份代码，但不同编译参数会选择完全不同的实现。

1.2.5 第二层：预处理器

预处理器 (preprocessor) 处理 include directive、宏、条件编译和注释删除。

例子：

```
#define SCALE 4

int f(int x)
{
    return x * SCALE;
}
```

预处理后大致变成：

```
int f(int x)
{
    return x * 4;
}
```

命令：

```
mkdir -p scratch/ch01
cat > scratch/ch01/preprocess.cpp <<'EOF'
#include <iostream>
```

```
#define SCALE 4

int f(int x)
{
    return x * SCALE;
}
```

```
int main()
{
    std::cout << f(3) << '\n';
}
EOF
```

```
g++ -E scratch/ch01/preprocess.cpp -o scratch/ch01/preprocess.i
wc -l scratch/ch01/preprocess.cpp scratch/ch01/preprocess.i
```

你通常会看到 **preprocess.i** 远比原文件长，因为标准库头文件展开了大量声明和模板。

常见误区

不要以为编译器看到的就是你手写的 **.cpp**。宏、include 和条件编译会先改写输入。性能分析遇到不符合直觉的行为时，必要时要看预处理结果。

1.2.6 第三层：编译器前端

预处理结果仍然是 C++ 文本。编译器前端要把它转成内部结构。

简化流程：

```
characters
-> tokens
-> parser
-> AST
-> semantic analysis
-> IR generation
```

1.2.6.1 Token

代码：

```
int x = a + 3;
```

会被词法分析成类似：

```
int
x
=
a
+
3
;
```

1.2.6.2 AST

抽象语法树 (Abstract Syntax Tree) 描述语法结构。上面的声明可理解成：

```
declaration x
  type: int
  initializer:
    binary +
      left: a
      right: 3
```

1.2.6.3 语义分析

语义分析检查：

- 名字是否声明。
- 类型是否匹配。
- 函数调用参数是否正确。
- 重载解析选择哪个函数。
- 模板实例化是否合法。
- `const`、引用和生命周期规则是否满足。

这一步决定了程序的 C++ 语义。后续优化必须在这个语义边界内进行。

1.2.7 第四层：IR 和优化器

编译器通常不会直接从 C++ 生成最终机器码，而是先生成 **中间表示** (intermediate representation)，简称 IR。

IR 的作用是提供一个比 C++ 简单、比机器码抽象的优化平台。

例如：

```
extern "C" int add_i32(int a, int b)
{
    return a + b;
}
```

LLVM IR 可能类似：

```
define i32 @add_i32(i32 %a, i32 %b) {
entry:
    %sum = add i32 %a, %b
    ret i32 %sum
}
```

优化器会在 IR 上做：

- 删除无用代码。
- 常量折叠。
- 函数内联。
- 循环优化。
- 自动向量化。
- 别名分析。

1.2.7.1 一个重要例子：无用计算会消失

如果你写：

```
int sum(int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += i;
    }
    return s;
}

int main()
{
    sum(1000000);
}
```

如果返回值没有可观察用途，优化器可能删除整个调用。这就是 benchmark 必须防止 dead-code elimination 的原因。

1.2.8 第五层：后端和汇编

编译器后端把优化后的 IR 降低到目标机器。

后端要解决：

- 指令选择：使用哪些 x86-64 指令。
- 寄存器分配：值放在哪些寄存器。
- 指令调度：指令如何排序。
- 调用约定：参数和返回值放哪里。
- 汇编输出：生成 .s 文件。

生成汇编：

```
g++ -O3 -S -masm=intel scratch/ch01/add.cpp -o scratch/ch01/add.s
```

汇编是机器码的人类可读表示。CPU 最终执行的不是汇编文本，而是汇编器编码后的字节。

1.2.9 第六层：汇编器和目标文件

汇编器（assembler）把汇编文本变成目标文件。

命令：

```
g++ -c -O3 scratch/ch01/add.cpp -o scratch/ch01/add.o
```

目标文件中包含：

- 机器码。
- 符号表。
- section。
- relocation。
- 可选调试信息。

它通常还不能单独运行，因为外部函数和最终地址还没处理。

1.2.10 第七层：链接器

一个真实程序通常由多个目标文件和库组成。

例子：

```
// add.cpp
extern "C" int add_i32(int a, int b)
{
    return a + b;
}
```

```
// main.cpp
extern "C" int add_i32(int, int);

int main()
{
    return add_i32(1, 2);
}
```

构建：

```
g++ -c add.cpp -o add.o
g++ -c main.cpp -o main.o
g++ add.o main.o -o app
```

链接器 (linker) 负责把 `main.o` 里对 `add_i32` 的引用解析到 `add.o` 中的定义。它还会处理：

- 标准库。
- 启动代码。
- 静态库。
- 共享库引用。
- relocation。
- 符号可见性。

1.2.11 第八层：ELF 可执行文件

Linux x86-64 上的可执行文件通常是 **ELF** (Executable and Linkable Format)。

ELF 不是“一段纯机器码”。它是一个带元数据的二进制容器，包含：

- ELF header。
- program headers。
- sections。
- symbol table。
- relocation information。
- dynamic linking information。
- debug information。

查看：

```
file app
readelf -h app
readelf -l app
readelf -S app
nm -C app
objdump -drwC -Mintel app
```

现在不要求你完全读懂每一项。第 6 章会系统展开 ELF。这里你只需要知道：可执行文件是操作系统加载器能理解的一种结构化文件。

1.2.12 第九层：操作系统加载程序

当你运行：

```
./app
```

shell 会请求内核创建新进程。内核和加载器大致做：

```
read ELF header
create process address space
map executable segments
map shared libraries if needed
prepare stack
prepare argc / argv / envp
start dynamic linker if dynamically linked
transfer control to entry point
```

如果是动态链接程序，动态链接器还会解析共享库和符号，然后运行 C/C++ runtime 初始化。

核心思想

`main` 是 C/C++ 程序员约定的入口函数，但进程真正开始执行的位置通常是 ELF header 中的 entry point，例如 `_start`。

1.2.13 第十层：进程和虚拟地址空间

进程（process）可以理解为：

```
running program instance
+ virtual address space
+ OS-managed resources
```

进程拥有：

- 虚拟地址空间。
- 文件描述符。
- 当前工作目录。
- 信号处理状态。
- 一个或多个线程。
- 权限和资源限制。

简化的虚拟地址空间：

```
high address
stack
...
shared libraries / mmap
...
heap
bss
data
rodata
text
low address
```

这只是模型。真实地址受 ASLR、动态链接、内核配置和运行环境影响。

1.2.14 入口点不是 main

查看入口点：

```
readelf -h app | rg "Entry point"
objdump -d -Mintel app | sed -n '/<_start>:/,+40p'
nm -C app | rg "main$_start"
```

你会看到 `_start` 一类符号。它最终会调用运行时库，再调用你的 `main`。

这对性能测量很重要：

- 程序启动成本不是函数成本。
- 动态链接和首次解析可能影响第一次运行。
- 全局对象构造可能在 `main` 前发生。
- benchmark 应该把 `setup` 和 `timed region` 分开。

1.2.15 贯穿实验：拆开一个程序

实验

创建文件：

```
mkdir -p scratch/ch01
cat > scratch/ch01/add.cpp <<'EOF'
extern "C" int add_i32(int a, int b)
{
    return a + b;
}
EOF
```

逐层生成中间产物：

```
g++ -E scratch/ch01/add.cpp -o scratch/ch01/add.i
g++ -S -O0 -masm=intel scratch/ch01/add.cpp -o scratch/ch01/add_00.s
g++ -S -O3 -masm=intel scratch/ch01/add.cpp -o scratch/ch01/add_03.s
g++ -c -O3 scratch/ch01/add.cpp -o scratch/ch01/add.o
objdump -drwC -Mintel scratch/ch01/add.o
```

观察任务：

1. `.i`、`.s`、`.o` 分别是什么。

2. `-O0` 和 `-O3` 汇编差异。
3. `objdump` 输出中的机器码字节。
4. 函数符号名是否保留为 `add_i32`。

1.2.16 实验：观察地址空间

实验

```
cat > scratch/ch01/address.cpp <<'EOF'
#include <iostream>
#include <vector>

int global_initialized = 42;
int global_zero;
char const* message = "hello";

int main()
{
    int stack_value = 1;
    auto* heap_value = new int(2);
    std::vector<int> vec(16, 3);

    std::cout << "main" << reinterpret_cast<void*>(&main) << '\n';
    std::cout << "global_initialized" << &global_initialized << '\n';
    std::cout << "global_zero" << &global_zero << '\n';
    std::cout << "message variable" << &message << '\n';
    std::cout << "message literal" << static_cast<void const*>(message) << '\n';
    std::cout << "stack_value" << &stack_value << '\n';
    std::cout << "heap_value" << heap_value << '\n';
    std::cout << "vector data" << vec.data() << '\n';

    delete heap_value;
}
EOF

g++ -O0 -g scratch/ch01/address.cpp -o scratch/ch01/address
./scratch/ch01/address
./scratch/ch01/address
```

报告任务：

1. 比较两次运行地址是否一致。
2. 推测哪些属于 `text`、`data`、`BSS`、`rodata`、`stack`、`heap`。
3. 解释为什么地址可能每次变化。
4. 查看 `/proc/self/maps` 的方式，并解释它能提供什么信息。

1.2.17 反例：看似合理但错误的理解

常见误区

误区一：C++ 程序从 `main` 的第一行开始。

不准确。进程入口通常是 `runtime` 启动代码，`main` 是被运行时调用的用户入口。

误区二：头文件会被单独编译。

通常不对。头文件被包含进源文件，形成翻译单元。

误区三：目标文件就是可执行文件。

不对。目标文件还需要链接。

误区四：源码里有函数，机器码里一定有这个函数。

不对。优化后函数可能被 `inline`、删除、合并或改变可见性。

误区五：指针就是物理地址。

用户态程序看到的是虚拟地址。

1.2.18 作业

习题与作业

作业 1：画出完整流程。

画出从 `hello.cpp` 到正在运行进程的完整路径。每一层写明：

- 输入是什么。
- 输出是什么。
- 哪个工具或系统组件负责。
- 性能学习为什么要关心。

作业 2：函数消失实验。

写三个函数：

- 一个允许 `inline`。
- 一个使用 `noinline` 阻止 `inline`。
- 一个从不调用。

分别用 `-O0` 和 `-O3` 编译，用 `nm` 和 `objdump` 观察哪些符号存在，哪些函数消失。

作业 3：启动成本和 kernel 成本。

写一个程序，让 `main` 调用某个小函数一次，再调用同一函数一亿次。分别测整个程序时间和循环区域时间。解释为什么启动成本不能代表 kernel 成本。

1.2.19 验收标准

完成本章后，你应该能：

- 解释预处理、编译、汇编、链接、加载的区别。
- 生成并解释 `.i`、`.s`、`.o` 和可执行文件。
- 用 `readelf`、`nm`、`objdump` 做基础观察。
- 解释为什么 `main` 不是真正的进程入口。
- 画出简化虚拟地址空间。
- 说明为什么性能测量要排除启动、初始化、动态链接和全局构造等因素。

1.3 CPU 如何执行指令

1.3.1 本章要解决的问题

上一章已经把一份 C/C++ 源码拆成了预处理、编译、汇编、链接、加载和进程执行。那条路径告诉我们：源代码最后会变成可执行文件中的机器码字节，操作系统把这些字节映射到进程的虚拟地址空间里，然后 CPU 从某个入口地址开始执行。

本章继续往下一层走。我们要回答一个看似简单、实际非常关键的问题：

CPU 到底怎样把一条指令变成一次真实的机器行为？

对初学者来说，这个问题通常会被几种不准确的说法遮住：

- “CPU 执行 C++ 代码”。
- “汇编就是 CPU 直接看到的东西”。
- “一条汇编就是一个硬件动作”。
- “CPU 一条一条顺序执行，所以源代码顺序就是真实执行顺序”。
- “内存就是一个大数组，访问一个变量就是直接去内存取它”。

这些说法都有一点影子，但都不够精确。为了做高性能计算、AI 算子优化和 CPU kernel 开发，必须建立更细的模型。我们不要求你在本章就理解现代乱序执行核心的所有细节，但你必须形成一条稳定的链路：

C/C++ 表达式 → 汇编文本 → 机器码字节 → 指令语义 → 架构状态变化 → 微架构执行过程 → 性能现象

后面所有优化都会反复使用这条链路。你看一个循环为什么慢，最终还是要问：

- 它生成了哪些指令？
- 这些指令读写哪些寄存器和内存？
- 哪些指令依赖前面的结果？
- 哪些访问会打到 cache，哪些会去更远的内存层级？
- 哪些分支会打断前端供给？
- CPU 内部能否把多条指令重叠执行？

核心思想

本章不是为了把 CPU 讲成神秘黑盒，而是为了把 CPU 拆成几个可观察、可推理、可实验验证的层次。性能工程的第一原则是：先知道机器真正执行了什么，再讨论优化。

1.3.2 从一个最小函数开始

考虑这个 C++ 函数：

```
extern "C" int add_i32(int x, int y)
{
    return x + y;
}
```

在 x86-64 System V ABI 下，前两个整数参数通常通过 `edi` 和 `esi` 传入，32 位整数返回值通常放在 `eax`。一个容易理解的汇编版本可以写成：

```
mov eax, edi
add eax, esi
ret
```

这三行已经包含了本章的核心问题：

- `mov` 为什么能把一个参数复制到返回寄存器？
- `add` 读了什么，写了什么？
- `ret` 为什么能回到调用者？
- CPU 怎样知道下一条该执行哪条指令？
- 这些汇编文本进入可执行文件后到底是什么？

先强调一个事实：CPU 不知道变量名 `x` 和 `y`。变量名、函数名、类型名是源代码和调试信息层面的东西。CPU 执行时看到的是指令字节，操作的是寄存器、内存地址和状态位。

如果把这个函数放进目标文件，用 `objdump-d-Mintel` 观察，机器码字节可能类似：

```
89 f8      mov eax, edi
01 f0      add eax, esi
c3         ret
```

也可能在优化后看到：

```
8d 04 37   lea eax, [rdi+rsi]
c3         ret
```

编译器选择哪一种，取决于优化级别、目标 CPU、上下文和它是否需要保留 flags。这里先不急判断哪种更好。本章先建立一个原则：

心智模型

汇编是人类可读的指令表示，机器码是 CPU 取指单元读取的字节序列。C++ 源码不是 CPU 的输入，汇编文本通常也不是 CPU 的最终输入；真正被执行的是内存中的机器码字节。

1.3.3 CPU 是什么：从状态机开始理解

很多教材会直接说 CPU 由控制器、运算器和寄存器组成。这个说法没错，但对性能学习还不够。我们需要更底层一点的心智模型：CPU 是一个巨大而高速的状态机。

所谓状态机，就是它有一组状态，每个时钟周期根据当前状态和输入计算出下一个状态。对程序员来说，最重要的状态包括：

- 当前要执行的位置，也就是 instruction pointer，在 x86-64 中叫 `rip`。
- 通用寄存器，例如 `rax`、`rbx`、`rcx`、`rdx`、`rdi`、`rsi`。
- 栈指针 `rsp`。
- 条件标志位 RFLAGS，例如 zero flag、carry flag、sign flag、overflow flag。
- 内存中可见的数据。

从硬件实现角度看，CPU 内部有大量组合逻辑和时序逻辑。

组合逻辑 (combinational logic) 的输出只由当前输入决定。加法器、比较器、多路选择器、地址生成单元都可以从这个角度理解。给加法器两个输入位模式，它经过门电路传播后给出结果和进位。

时序逻辑 (sequential logic) 会保存状态。寄存器、流水线寄存器、重排序缓冲区中的条目、cache tag 阵列中的状态都属于这一类。时钟边沿到来时，一部分计算结果被锁存成新的状态。

你不需要在本书前几章就掌握电路设计，但必须理解一点：机器执行不是抽象概念，它最终落实为状态的读、组合逻辑的计算和状态的写。


```

old state + instruction bytes
      |
      v
decode instruction meaning
      |
      v
read operands -> compute -> write results
      |
      v
new state

```

这就是最朴素的 CPU 执行图景。现代 CPU 会把这个过程拆开、重叠、预测、乱序执行、再按程序顺序提交结果，但它必须最终制造出和 ISA 规定一致的可见状态。

1.3.4 ISA：软件和硬件之间的契约

ISA (instruction set architecture, 指令集体系结构) 是软件和处理器之间的契约。对 x86-64 来说, ISA 规定了：

- 有哪些寄存器。
- 每条指令的编码格式。
- 每条指令读写什么操作数。
- 指令如何改变寄存器、内存、flags 和 `rip`。
- 异常、中断、权限、页表等系统级行为如何表现。

ISA 不规定 CPU 内部必须如何实现。例如同样是执行 `add eax, esi`：

- 一个简单教学 CPU 可以顺序取指、解码、读寄存器、加法、写回。
- 一个现代桌面 CPU 可能把它解码为 micro-op，进入调度队列，等待操作数 ready，在某个整数执行端口上执行，最后在 retire 阶段提交。
- 一个仿真器可以用软件解释这条指令。

只要对程序可见的结果一致，它们都符合 ISA。

架构状态 (architectural state) 是 ISA 规定的软件可见状态，例如通用寄存器、`rip`、RFLAGS 和内存。**微架构** (microarchitecture) 是某个具体 CPU 实现 ISA 的内部方式，例如 pipeline、decode queue、uop cache、register renaming、reservation station、load/store queue、L1 cache、TLB 和 branch predictor。

核心思想

ISA 决定正确性语义，微架构决定性能表现。同一份 x86-64 程序在不同 CPU 上结果应该一致，但速度可能差很多，因为内部实现不同。

1.3.5 x86-64 程序员模型

学习汇编时，先不要把 CPU 想成无限复杂的乱序机器。先从程序员模型开始。所谓程序员模型，就是写汇编、读反汇编、理解 ABI 时必须看到的那部分 CPU 状态。

1.3.5.1 通用寄存器

x86-64 有 16 个常用通用整数寄存器：

64 位	32 位低半部分	常见用途
<code>rax</code>	<code>eax</code>	返回值、通用计算
<code>rbx</code>	<code>ebx</code>	callee-saved 通用寄存器
<code>rcx</code>	<code>ecx</code>	第 4 个整数参数、计数、通用计算
<code>rdx</code>	<code>edx</code>	第 3 个整数参数、乘除辅助、通用计算
<code>rsi</code>	<code>esi</code>	第 2 个整数/指针参数
<code>rdi</code>	<code>edi</code>	第 1 个整数/指针参数
<code>rbp</code>	<code>ebp</code>	栈帧指针或通用 callee-saved 寄存器
<code>rsp</code>	<code>esp</code>	栈指针
<code>r8</code>	<code>r8d</code>	第 5 个整数/指针参数
<code>r9</code>	<code>r9d</code>	第 6 个整数/指针参数
<code>r10</code>	<code>r10d</code>	caller-saved 临时寄存器
<code>r11</code>	<code>r11d</code>	caller-saved 临时寄存器
<code>r12</code>	<code>r12d</code>	callee-saved 通用寄存器
<code>r13</code>	<code>r13d</code>	callee-saved 通用寄存器
<code>r14</code>	<code>r14d</code>	callee-saved 通用寄存器
<code>r15</code>	<code>r15d</code>	callee-saved 通用寄存器

寄存器不是变量。寄存器是 CPU 内部的高速状态存储单元。编译器会把源代码里的变量、临时值、地址、中间计算结果分配到寄存器或栈位置。一个变量可能根本没有固定的寄存器；一个寄存器在不同指令之间也可能承载不同含义。

1.3.5.2 32 位写入会清零高 32 位

x86-64 有一个非常重要的规则：写入 32 位寄存器通常会把对应 64 位寄存器的高 32 位清零。例如：

```
mov eax, edi
```

这条指令写 `eax`，同时会让 `rax` 的高 32 位变成 0。这个规则经常帮助 CPU 避免旧高位造成的依赖，也让 32 位整数计算自然产生零扩展结果。

常见误区

不要把 `eax` 理解成完全独立于 `rax` 的寄存器。它是 `rax` 的低 32 位视图。类似地，`ax`、`al` 也是同一个物理架构寄存器的更小视图。部分寄存器写入在某些老架构上还会带来额外依赖问题，后面讲性能时会再次出现。

1.3.5.3 RIP: CPU 下一步从哪里取指

`rip` 是 instruction pointer。它保存下一条要执行的指令地址。普通顺序执行时，CPU 取出当前指令后，会把 `rip` 更新到下一条指令地址。

x86 指令长度不是固定的。刚才的例子中：

```
89 f8      mov eax, edi
01 f0      add eax, esi
c3         ret
```

`mov eax, edi` 长 2 字节，`add eax, esi` 长 2 字节，`ret` 长 1 字节。所以如果第一条指令地址是 `0x1000`，顺序取指时大致会这样：

执行前 <code>rip</code>	指令字节	顺序下一地址
<code>0x1000</code>	<code>89f8</code>	<code>0x1002</code>
<code>0x1002</code>	<code>01f0</code>	<code>0x1004</code>
<code>0x1004</code>	<code>c3</code>	由栈中的返回地址决定

跳转、调用、返回和异常都会改变 `rip`。控制流的本质就是改变 CPU 接下来从哪里取指。

1.3.5.4 RFLAGS: 条件判断的隐含状态

很多整数指令会设置 `RFLAGS`。例如 `add` 不只写目标寄存器，还会更新 `zero flag`、`sign flag`、`carry flag`、`overflow flag` 等状态。条件跳转指令读取这些 `flags` 来决定是否跳转。

例如：

```
cmp edi, esi
jg greater
```

`cmp edi, esi` 的语义类似计算 `edi-esi` 并更新 `flags`，但不保存结果。`jb` 读取 `flags`，判断有符号意义下 `edi` 是否大于 `esi`。

这解释了为什么 `lea eax, [rdi+rsi]` 有时会替代 `add: lea` 做地址形式的整数计算，但通常不改 `flags`。如果后续代码还需要旧 `flags`，使用 `lea` 可以避免破坏它们；如果后续不需要 `flags`，编译器也可能因为编码或调度原因选择其中一种。

1.3.6 机器码字节和汇编文本

汇编文本是给人看的。机器码字节才是 CPU 取指单元读取的内容。汇编器负责把：

```
mov eax, edi
```

变成：

```
89 f8
```

这两个字节不是随意来的。x86 指令编码中包含 opcode、寄存器编号、寻址模式、立即数、位移、前缀等部分。完整 x86 编码非常复杂，本章只建立直觉。

以 `mov eax, edi` 为例：

- 89 是一类 `mov` 指令的 opcode。
- f8 是 ModRM 字节，描述源寄存器和目标寄存器。

以 `ret` 为例：

- c3 就是一字节 `near return` 指令。

x86-64 是变长指令集。一条指令可以很短，也可以很长。变长编码带来代码密度优势，也让前端解码变复杂。现代 x86 CPU 为了高速执行，必须在前端投入大量硬件：取指、边界识别、解码、缓存已经解码过的 uops 等。后面讲前端瓶颈时，这一点会非常重要。

心智模型

一条汇编指令不是一个字符串，而是一段字节的解释方式。反汇编器把字节翻译回人类可读文本；汇编器把文本翻译成字节。性能分析时，二者都要看，但最终以实际二进制中的字节和指令为准。

1.3.7 最朴素的 Fetch-Decode-Execute 模型

先构造一个极简 CPU 模型。它每次只执行一条指令，每条指令完整执行完才开始下一条：

```
while running:
    bytes = fetch memory[RIP]
    inst = decode(bytes)
    next_RIP = RIP + length(inst)
    operands = read_registers_or_memory(inst)
    result = execute(inst, operands)
    write_registers_or_memory(result)
    RIP = choose_next_RIP(inst, next_RIP)
```

这个模型非常简化，但它的每个词都值得解释。

1.3.7.1 Fetch: 取指

取指就是根据 `rip` 到内存层级中取机器码字节。现代 CPU 通常不是每次直接去主内存取字节，而是从 L1 instruction cache、uop cache 或其他前端结构中拿。第一次学习时可以先想象：

`rip` → instruction memory → instruction bytes

如果指令字节不在前端 cache 里，取指就可能变慢。代码太大、分支太乱、热循环跨越不友好的边界，都可能影响前端供给。

1.3.7.2 Decode: 解码

解码器把机器码字节解释成指令语义。它要知道：

- 这是什么指令。
- 指令有多长。
- 操作数在哪里。
- 是否有立即数或内存位移。

- 这条指令应该读写哪些架构状态。

现代 x86 CPU 通常会把复杂 x86 指令解码成更接近内部执行格式的 micro-ops, 简称 uops。不要把 uops 当成另一套必须立刻精通的汇编语言; 现在只需要知道: 软件看到 x86 指令, 硬件内部可能执行一个或多个更小的操作。

1.3.7.3 Read operands: 读取操作数

指令需要输入。输入可能来自:

- 寄存器, 例如 `edi`、`esi`。
- 立即数, 例如 `add eax, 5` 里的 5。
- 内存, 例如 `moveax, dword ptr[rdi]`。
- 隐含状态, 例如 `ret` 隐含读取 `rsp` 指向的栈内存。

读寄存器通常很快。读内存则复杂得多, 因为地址可能还要计算, 虚拟地址可能要经过 TLB 翻译, 数据可能在 L1/L2/L3 cache 或主内存。

1.3.7.4 Execute: 执行

执行是根据指令语义完成计算或动作:

- 整数加法由整数执行单元完成。
- 地址计算由地址生成单元完成。
- `load` 从内存层级读取数据。
- `store` 把数据写向内存系统。
- `branch` 计算下一条指令地址。
- `floating-point` 和 `SIMD` 指令使用向量/浮点执行单元。

1.3.7.5 Write back / Commit: 写回和提交

计算结果最终要改变状态。简单 CPU 可以执行完就写架构寄存器。现代乱序 CPU 会先把结果写到内部物理寄存器或缓冲结构中, 再在 `retire` 阶段按程序顺序提交。这样可以保证异常和中断时仍然呈现精确的架构状态。

本章先记住一句话:

核心思想

CPU 可以在内部提前做很多事, 但最终必须让软件看到像是按照程序顺序执行每条指令一样的结果。这是理解乱序执行和精确异常的基础。

1.3.8 逐条执行 `add_i32`

现在手工执行这个版本:

```
mov eax, edi
add eax, esi
ret
```

假设函数入口处:

- `edi` = 10。
- `esi` = 32。
- `rsp` 指向栈顶。
- 栈顶 8 字节保存调用者的返回地址。

第一条:

```
mov eax, edi
```

语义是把 `edi` 的低 32 位复制到 `eax`。执行后:

- `eax` = 10。
- `rax` 高 32 位清零。
- 通常不改变 `flags`。
- `rip` 指向下一条指令。

第二条:

```
add eax, esi
```

语义是:

$$eax \leftarrow eax + esi$$

执行后：

- `eax = 42`。
- `RFLAGS` 根据结果更新。
- `rax` 高 32 位仍然是 0。
- `rip` 指向下一条指令。

第三条：

```
ret
```

近似语义是：

```
target = memory[rsp]
rsp = rsp + 8
rip = target
```

也就是说，`ret` 从栈顶弹出返回地址，更新 `rsp`，然后把 `rip` 改成返回地址。函数返回值已经在 `eax` 中，调用者按照 ABI 从那里读取返回值。

1.3.9 调用和返回：栈为什么参与控制流

函数调用不是魔法。假设调用者执行：

```
call add_i32
```

`call` 的关键语义可以近似理解为：

```
rsp = rsp - 8
memory[rsp] = address_of_next_instruction
rip = address_of_add_i32
```

所以 `call` 做了两件事：

- 保存返回地址。
- 跳转到被调用函数入口。

`ret` 则做反向操作：

- 从栈顶取回返回地址。
- 跳回调用者。

这解释了为什么破坏 `rsp` 会让程序崩溃，也解释了为什么栈溢出、返回地址覆盖和控制流劫持是安全问题。对性能学习来说，理解 `call/ret` 也很重要，因为函数调用涉及 ABI、寄存器保存、栈对齐、返回地址预测和 `inline` 决策。

常见误区

`call` 和 `ret` 改变的是控制流，不是普通数据流。你必须同时追踪 `rip`、`rsp` 和栈内存，才能正确理解函数调用。

1.3.10 内存访问：一条 `load` 背后的路径

再看一个稍微复杂的函数：

```
extern "C" int add_load(const int* p, int y)
{
    return p[0] + y;
}
```

可能出现这样的汇编：

```
mov eax, dword ptr [rdi]
add eax, esi
ret
```

这里 `rdi` 保存指针 `p`，`dwordptr[rdi]` 表示从地址 `rdi` 处读取 4 字节整数。从 ISA 语义看，这条 `load` 很简单：

$$eax \leftarrow \text{memory}[\text{rdi} \dots \text{rdi}+3]$$

从微架构看，它可能经过很多步骤：

1. 地址生成单元计算有效地址。
2. 如果使用虚拟内存，CPU 用 TLB 查询虚拟地址到物理地址的翻译。

3. L1 data cache 根据地址查找 cache line。
4. 如果 L1 命中，数据很快返回。
5. 如果 L1 未命中，继续查 L2、L3，甚至访问主内存。
6. 取回 4 字节并送到目标寄存器。

这就是性能工程中经常说的：寄存器计算和内存访问不是一个数量级的问题。一个整数加法可能吞吐很高，而一次真正去主内存的 load 会慢很多。

1.3.11 地址计算：数组访问并不神秘

考虑：

```
extern "C" int get_i(const int* a, long i)
{
    return a[i];
}
```

核心地址计算是：

$$\text{address} = a + i * \text{sizeof}(\text{int})$$

x86-64 寻址模式能直接表达常见的 $\text{base} + \text{index} * \text{scale} + \text{displacement}$ ：

```
mov eax, dword ptr [rdi + rsi*4]
ret
```

这里：

- `rdi` 是 base，也就是数组起始地址。
- `rsi` 是 index，也就是下标。
- 4 是 scale，因为 `int` 通常 4 字节。
- 没有额外 displacement。

这条指令同时包含地址计算和内存读取。现代 CPU 内部通常会把地址生成、TLB 查询、cache 访问等步骤分到 load pipeline 中。后面学习 cache、TLB 和 prefetch 时，这个例子会继续使用。

1.3.12 控制流：顺序、跳转和分支

没有跳转时，`rip` 顺序前进。有跳转时，下一条指令地址由跳转目标决定。

例如：

```
extern "C" int abs_i32(int x)
{
    if (x < 0) {
        return -x;
    }
    return x;
}
```

一个未必最优但容易理解的汇编形态是：

```
mov eax, edi
test edi, edi
jge done
neg eax
done:
ret
```

`test edi, edi` 会计算 `edi` 和自身的按位与，只更新 flags，不保存结果。它常用于判断是否为 0 或正负。`jge done` 根据 flags 决定是否跳到 `done`。

控制流引入了性能问题：CPU 前端必须知道接下来取哪里。如果等分支真正执行完再取下一条，流水线会经常停住。现代 CPU 因此引入 branch predictor，提前猜测分支方向和目标地址。

1.3.13 为什么需要流水线

假设一条指令要经历五步：

1. 取指。
2. 解码。
3. 读操作数。
4. 执行。
5. 写回。

如果一条指令完成所有步骤后才开始下一条，硬件资源大部分时间都在空闲。流水线的想法是把不同指令的不同阶段重叠起来：

周期	阶段 1	阶段 2	阶段 3	阶段 4	阶段 5
1	I1 取指				
2	I2 取指	I1 解码			
3	I3 取指	I2 解码	I1 读数		
4	I4 取指	I3 解码	I2 读数	I1 执行	
5	I5 取指	I4 解码	I3 读数	I2 执行	I1 写回

流水线提高的是吞吐，不一定降低单条指令延迟。这里要提前区分两个性能概念：

延迟（latency）是一个操作从开始到结果可用需要多久。**吞吐**（throughput）是单位时间内可以完成多少操作。

一条加法可能延迟 1 个周期，但如果 CPU 每周期能发射多条独立加法，那么吞吐可以高于每周期 1 条。后面讲 latency、throughput、ILP 时会严格量化这些概念。

1.3.14 依赖：为什么有些指令不能并行

考虑：

```
add eax, esi
add eax, edx
add eax, ecx
```

每条指令都读写 `eax`。第二条必须等待第一条产生新的 `eax`，第三条必须等待第二条。这叫数据依赖。

再考虑：

```
add eax, esi
add r8d, r9d
add r10d, r11d
```

这三条指令互相独立。现代 CPU 更容易把它们重叠执行。高性能代码经常要制造足够多的独立工作，让 CPU 的多个执行单元都有活干。

核心思想

性能优化不是简单减少源代码行数，而是控制机器指令的依赖图、内存访问模式和前端供给。短代码不一定快，能让硬件持续高效工作的代码才快。

1.3.15 为什么需要乱序执行

顺序流水线遇到长延迟操作会卡住。例如：

```
mov eax, dword ptr [rdi] ; 可能 cache miss
add r8d, r9d             ; 与 eax 无关
add r10d, r11d           ; 与 eax 无关
add eax, esi             ; 依赖 load
```

如果第一条 `load` 很慢，而后两条加法与它无关，理想情况下 CPU 不应该傻等。乱序执行的目标就是：在不改变架构可见结果的前提下，先执行已经 `ready` 的指令。

现代乱序 CPU 大致会：

- 按程序顺序取指和解码。
- 通过 `register renaming` 消除一部分假依赖。
- 把 `uops` 放入调度结构。
- 哪些操作数 `ready`，哪些执行单元可用，就先执行哪些。
- 最后按程序顺序 `retire`，保证异常和结果可见性正确。

这解释了一个重要现象：你在汇编里看到的是程序顺序，但 CPU 内部的实际执行顺序可能不同。性能分析不能只看文本顺序，还要看依赖和资源。

1.3.16 寄存器重命名：为什么架构寄存器不等于物理寄存器

ISA 只暴露 16 个通用寄存器，但现代 CPU 内部通常有更多物理寄存器。寄存器重命名会把架构寄存器名映射到物理寄存器。

看这个例子：

```
mov eax, edi
add eax, esi
mov eax, r8d
add eax, r9d
```

从文本看，四条指令都写 `eax`。但第二组计算并不需要第一组计算的结果。没有重命名时，对同一个架构寄存器的写会制造不必要的顺序限制。重命名可以让两个不同生命期的 `eax` 映射到不同物理寄存器，从而消除假依赖。

这对 C++ 优化有一个启发：不要用源代码变量名想当然地判断寄存器压力和依赖。编译器和 CPU 都会重命名、合并、消除和重排中间值。真正可靠的方式是看编译后的汇编，再结合微架构模型分析。

1.3.17 Cache：为什么内存访问不是一个速度

程序员经常说“访问内存”，但 CPU 看到的是层级结构：

层级	大致特点	学习重点
寄存器	最靠近执行单元，数量少	分配、依赖、寄存器压力
L1 data cache	很小但很快	cache line、局部性、load/store 吞吐
L2 cache	更大但更慢	工作集大小、miss 代价
L3 cache	多核心共享或分片	跨核心影响、容量
主内存	容量大但延迟高	bandwidth、NUMA、预取

CPU 不是按单个 `int` 从内存取数据，而是按 cache line 在 cache 和更低层级之间移动数据。常见 cache line 大小是 64 字节。顺序扫描数组通常比随机访问快，一个重要原因就是顺序访问能利用 cache line 和硬件预取。

本章只建立直觉：内存访问速度取决于数据在哪一级，以及访问模式能否被 cache 和 prefetcher 利用。后面内存层级部分会用实验系统展开。

1.3.18 分支预测：CPU 为什么要猜

流水线越深、前端越宽，分支的代价越高。如果 CPU 不猜，它遇到条件跳转就必须等条件计算完成，然后才知道下一条从哪里取。这会让前端经常空转。

branch predictor 试图预测：

- 条件分支会不会跳。
- 跳转目标在哪里。
- `ret` 会返回到哪里。

如果预测正确，流水线保持忙碌。如果预测错误，CPU 必须丢弃错误路径上的推测工作，回到正确路径。这会造成明显性能损失。

这解释了为什么数据相关、难预测的分支会慢，也解释了为什么有时编译器或手写优化会使用条件移动、mask、SIMD blend 等方式减少分支。不是所有分支都坏，坏的是高频、难预测、代价明显的分支。

1.3.19 从 C++ 到 CPU 行为的阅读方法

以后看到一段 C++ hot loop，不要只停留在源代码层。建议按下面步骤阅读：

1. 写出这段代码的输入、输出和允许的 C++ 语义。
2. 编译成汇编，分别看 `-O0` 和 `-O3`，但以优化版本为性能分析对象。
3. 标出每条关键指令读写的寄存器和内存。
4. 找出循环携带依赖，例如 reduction 中的累加变量。
5. 找出内存访问模式，例如连续、跨步、随机、gather。
6. 判断控制流是否有难预测分支。
7. 用工具验证，不靠纯想象。

本书会不断训练这套方法。它看起来繁琐，但熟练以后会变成直觉。真正高手读性能代码时，脑子里会自动浮现数据路径、指令路径和瓶颈假设。

1.3.20 贯穿例子：求和循环

考虑：

```
extern "C" int sum_i32(const int* a, long n)
{
    int s = 0;
    for (long i = 0; i < n; ++i) {
        s += a[i];
    }
    return s;
}
```

一个标量汇编形态可能包含：

```

xor eax, eax
xor ecx, ecx
loop:
add eax, dword ptr [rdi + rcx*4]
inc rcx
cmp rcx, rsi
jle loop
ret

```

逐层理解：

- `rdi` 保存数组起始地址。
- `rsi` 保存长度 `n`。
- `eax` 保存累加和。
- `rcx` 保存循环下标。
- `[rdi+rcx*4]` 是数组元素地址。
- `add eax, dword ptr [...]` 同时做 `load` 和加法。
- `cmp` 和 `jle` 控制循环。

性能上，这段循环至少有几个问题可以讨论：

- 累加变量 `eax` 形成循环携带依赖。
- 每次迭代有一次 `load`。
- 每次迭代有循环分支。
- 如果编译器能证明安全并且目标支持 SIMD，优化版本可能向量化。

这就是从源代码到性能模型的第一步。你不需要在本章就优化它，但必须能把它拆成寄存器、内存、控制流和依赖。

1.3.21 实验 2.1：观察机器码字节

实验

目标：确认 CPU 执行的是机器码字节，汇编只是人类可读表示。

步骤：

```

mkdir -p scratch/ch02
cat > scratch/ch02/add.cpp <<'EOF'
extern "C" int add_i32(int x, int y)
{
    return x + y;
}
EOF

g++ -O0 -c scratch/ch02/add.cpp -o scratch/ch02/add_00.o
g++ -O3 -c scratch/ch02/add.cpp -o scratch/ch02/add_03.o

objdump -drwC -Mintel scratch/ch02/add_00.o
objdump -drwC -Mintel scratch/ch02/add_03.o

```

交付物：

- 截取 `add_i32` 的反汇编。
- 标出每条指令前面的机器码字节。
- 解释 `-O0` 和 `-O3` 的差异。
- 说明返回值为什么在 `eax`。

1.3.22 实验 2.2：用 GDB 单步观察寄存器

实验

目标：用调试器观察 `rip`、`rsp`、`eax`、`edi`、`esi` 如何变化。

准备代码：

```

#include <stdio>

extern "C" __attribute__((noinline))
int add_i32(int x, int y)
{
    return x + y;
}

int main()
{
    int z = add_i32(10, 32);
}

```



```
std::printf("%d\n", z);
}
```

编译:

```
g++ -O0 -g scratch/ch02/debug_add.cpp -o scratch/ch02/debug_add
gdb scratch/ch02/debug_add
```

GDB 命令:

```
set disassembly-flavor intel
break add_i32
run
disassemble /r add_i32
info registers rip rsp rax eax rdi edi rsi esi
si
info registers rip rsp rax eax rdi edi rsi esi
si
info registers rip rsp rax eax rdi edi rsi esi
```

交付物:

- 每次 `si` 后记录关键寄存器。
- 解释 `rip` 为什么变化。
- 解释 `rsp` 在函数入口和返回附近怎样变化。
- 解释调试版本为什么比优化版本多很多栈帧相关指令。

1.3.23 实验 2.3: flags 和条件跳转

实验

目标: 理解 `cmp`、`test`、条件跳转如何配合。

任务: 写三个函数:

```
extern "C" int is_zero(int x)
{
    return x == 0;
}

extern "C" int max_i32(int a, int b)
{
    return a > b ? a : b;
}

extern "C" int abs_i32(int x)
{
    return x < 0 ? -x : x;
}
```

分别使用 `-O0`、`-O2`、`-O3` 编译, 观察是否出现:

- `cmp`
- `test`
- `jcc`
- `cmov`
- `setcc`

交付物:

- 为每个函数画出控制流或数据流。
- 解释哪条指令设置 flags, 哪条指令读取 flags。
- 判断优化版本是否减少了分支。

1.3.24 实验 2.4: call、ret 和栈

实验

目标: 理解函数调用如何使用栈保存返回地址。

任务: 写三个 `noinline` 函数:

```
extern "C" __attribute__((noinline)) int f3(int x)
{
    return x + 3;
}

extern "C" __attribute__((noinline)) int f2(int x)
{
```

```

    return f3(x + 2);
}

extern "C" __attribute__((noinline)) int f1(int x)
{
    return f2(x + 1);
}

```

用 GDB 在 `f3` 里停住，执行：

```

bt
info registers rip rsp rbp
x/8gx $rsp
disassemble /r f1
disassemble /r f2
disassemble /r f3

```

交付物：

- 画出调用链。
- 标出每一层返回地址。
- 解释 `call` 和 `ret` 怎样配合。
- 比较 `-O0` 和 `-O2` 下调用链是否变化，解释 `inline` 的影响。

1.3.25 实验 2.5：顺序访问和随机访问的第一印象

实验

目标：用简单实验感受 cache 和访问模式对性能的影响。本实验不要求建立完整 cache 模型，只要求形成问题意识。

任务：分配一个较大的 `std::vector<int>`，分别做：

- 顺序求和。
- 固定 stride 求和，例如 `stride = 16`。
- 按一个打乱后的 `index` 数组随机求和。

要求：

- 用相同数据规模。
- 防止编译器把循环优化掉。
- 每个 case 至少运行多次，报告中位数。
- 记录 CPU 型号和编译命令。

交付物：

- 三种访问模式的时间。
- 你对差异的解释。
- 你认为还需要哪些证据才能证明解释是正确的。

1.3.26 常见误区

常见误区

误区一：CPU 执行的是 C++。

不准确。CPU 执行机器码字节。C++ 是高层语言，编译器把它翻译到目标 ISA。调试信息可以帮助你吧机器状态映射回源代码，但那不是 CPU 的原生输入。

常见误区

误区二：汇编和机器码是一回事。

不准确。汇编是文本表示，机器码是字节。它们可以互相转换，但反汇编不一定恢复原始汇编源文件中的标签、注释和宏结构。

常见误区

误区三：一条汇编一定对应一个硬件动作。

不准确。复杂 x86 指令可能解码成多个 uops；简单指令也可能在流水线中经历多个阶段。性能上要区分 ISA 指令、uops、执行端口、延迟和吞吐。

常见误区

误区四：程序顺序就是内部执行顺序。

不准确。现代 CPU 可以乱序执行 ready 的操作，但 retire 时必须保持架构可见结果符合程序顺序。性能分析要看依赖图，而不只是文本顺序。

常见误区

误区五：内存访问速度固定。

不准确。同样是一条 load，数据在 L1、L2、L3、主内存时成本完全不同。访问模式、工作集大小、TLB、预取和 NUMA 都会影响性能。

1.3.27 本章作业

习题与作业

作业 1：手工执行指令。

给定函数入口状态：

- `edi = 7`。
- `esi = 5`。
- `edx = 3`。
- `rsp = 0x7fffffff0000`。
- `memory[0x7fffffff0000]` 保存返回地址 `0x401234`。

手工执行：

```
mov eax, edi
add eax, esi
sub eax, edx
ret
```

写出每条指令后的 `eax`、`rax`、`rsp`、`rip` 的变化，并说明哪条指令会更新 `flags`。

习题与作业

作业 2：解释一个数组访问。

对下面函数：

```
extern "C" int load2(const int* a, long i)
{
    return a[i + 2];
}
```

生成优化汇编。解释地址表达式中 `base`、`index`、`scale`、`displacement` 分别是什么。说明为什么 `int` 下标会乘以 4。

习题与作业

作业 3：比较 `add` 和 `lea`。

写两个或更多简单加法函数，尝试让编译器生成 `add` 或 `lea`。报告中必须包含：

- 源码。
- 编译命令。
- 反汇编。
- 你对编译器选择的解释。
- 这两类指令对 `flags` 的影响差异。

习题与作业

作业 4：调用链报告。

基于实验 2.4，写一份不少于 800 字的报告，回答：

- 函数调用前参数在哪里？
- 返回值在哪里？
- 返回地址在哪里？
- `rsp` 如何变化？
- 优化级别改变后，为什么调用链可能消失？

习题与作业

作业 5：第一次性能解释。

完成实验 2.5，写一份性能报告。报告不能只写“随机访问慢”，必须包含：

- 数据规模。
- 编译器版本和命令。
- CPU 型号。
- 每种访问模式的时间统计。
- 你基于 cache line 和局部性的解释。
- 你认为本实验还有哪些不严谨之处。

1.3.28 验收标准

学完本章，你应该能够做到：

- 解释 CPU 不直接执行 C++，而是执行机器码字节。
- 区分汇编文本、机器码、ISA 语义和微架构实现。
- 说清楚 `rip`、通用寄存器、`rsp`、`RFLAGS` 的基本作用。
- 手工执行简单整数指令，追踪寄存器和 `rip` 变化。
- 解释 `call` 和 `ret` 如何用栈保存和恢复控制流。
- 解释一条 `load` 指令为什么可能涉及地址生成、TLB 和 cache。
- 用 `objdump` 观察机器码字节和反汇编。
- 用 GDB 单步观察寄存器变化。
- 初步解释 pipeline、cache、branch prediction、out-of-order execution 为什么存在。

如果你能完成这些目标，后面正式进入 x86-64 汇编、ABI、性能测量和微架构时，就不会把术语当成孤立概念，而能把它们放回真实执行链路中理解。

1.4 数据表示、内存、指针和对象布局

1.4.1 本章要解决的问题：`a[i]` 到底访问了什么

刚学 C/C++ 时，数组访问通常被解释成“取第 `i` 个元素”。这个说法对写程序有用，但对理解机器执行和性能远远不够。高性能计算和 AI CPU 算子优化中，大量核心问题都可以追到一个更底层的问题：

某条指令到底访问了哪个地址上的哪些字节？

先看一个简单函数：

```
int get(int const* a, int i)
{
    return a[i];
}
```

在 x86-64 System V ABI 下，编译器可能生成类似汇编：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4]
ret
```

这三行非常值得细看。

- `rdi` 保存第一个参数 `a`，也就是数组首元素地址。
- `esi` 保存第二个参数 `i` 的低 32 位。
- `movsxd rsi, esi` 把 32 位有符号整数扩展成 64 位索引。
- `[rdi+rsi*4]` 是地址表达式，意思是 `a+i*4`。
- `dword ptr` 说明这次从内存读取 4 字节。
- `eax` 是返回值寄存器的低 32 位。

于是 `a[i]` 在机器层面不是一个神秘的“数组操作”，而是一次地址计算加一次内存读取：

```
address = base_address(a) + i * sizeof(int)
load 4 bytes from address
interpret those 4 bytes as int
```

核心思想

机器没有“数组第几个元素”的概念。机器看到的是地址、字节数、寄存器和指令。C/C++ 类型系统把“第几个元素”翻译成“从哪个基地址开始，按多大步长计算地址，读写多少字节，并按什么规则解释这些字节”。

本章要把这个过程讲清楚。后面所有性能优化几乎都会反复依赖本章内容：cache line 为什么一次拉 64 字节，SIMD load 为什么要求连续布局，结构体字段顺序为什么影响性能，未定义行为为什么会让编译器大胆优化，AoS 和 SoA 为什么差异巨大，GEMM packing 为什么本质上是重排内存访问。

1.4.2 本章学习路线：从一个字节走到一个 kernel

本章容易学散，因为它同时出现位、整数、指针、数组、结构体、padding、cache line 和汇编寻址。不要把它们当成孤立知识点背。它们之间有一条非常明确的依赖链：

```
bit -> byte -> object representation -> type rule
    -> address -> pointer expression -> layout
    -> x86-64 addressing mode -> cache line behavior
    -> SIMD / HPC / AI kernel data movement
```

这条链路的意思是：

1. **bit 和 byte** 让你知道机器保存信息的最小粒度。
2. **对象表示** 让你知道一个 C++ 对象最终落成哪些字节。
3. **类型规则** 让你知道这些字节如何被解释，编译器能假设什么。
4. **地址和指针** 让你把源码表达式变成具体内存位置。
5. **布局** 让你知道数组元素、结构体字段、padding 在地址空间里的相对位置。
6. **x86-64 寻址模式** 让你把 C++ 访问表达式对应到 $[base + index * scale + displacement]$ 。
7. **cache line** 让你解释为什么同样的运算量会因为访问模式不同而差很多。
8. **SIMD 和 kernel 优化** 则要求你主动设计数据布局，让硬件以更少指令、更少 cache miss、更高带宽效率完成同样计算。

如果你只是记住“int 通常 4 字节”“double 通常 8 字节”，本章价值很小。真正要训练的是一种固定能力：看到任意 C++ 表达式，能回答它需要访问哪些字节；看到任意内存操作汇编，能反推出它可能来自什么数据布局；看到一个慢循环，能判断它是不是被布局、步长、对齐或别名拖慢。

1.4.2.1 本章的四个观察层次

同一个程序可以从四个层次观察。初学者常犯的问题，是只停在第一层或把四层混在一起。

层次	你看到什么	本章要你学会的问题
C++ 源码层	<code>a[i]</code> 、 <code>p->x</code> 、 <code>sizeof(S)</code>	这个表达式的类型是什么？对象生命周期存在吗？访问是否越界？
对象布局层	字段 offset、padding、对齐、数组步长	目标字节从基地址偏移多少？一次访问读写几字节？
汇编层	<code>moveax, [rdi+rsi*4]</code>	基址、索引、缩放和常量偏移分别来自源码里的什么？
微架构预告层	cache line、步长、连续性、带宽	这些访问会顺序命中 cache，还是浪费 cache line、破坏预取和 SIMD？

学习时必须强迫自己每次都跨层解释。例如：

```
float z = particles[i].z;
```

不要只说“取第 i 个粒子的 z”。更完整的解释应该是：

1. `particles` 是 `Particle*`，所以 `particles+i` 以 `sizeof(Particle)` 为步长。
2. 如果 `Particle` 大小为 16，`z` 字段 offset 为 8，那么目标地址是“首元素地址 + $i * 16 + 8$ ”。
3. 目标字段是 `float`，因此 load 宽度是 4 字节。
4. 在 x86-64 汇编里，因为 scale 不能直接取 16，编译器可能先计算 $i * 16$ ，再访问 `dword ptr [base+offset+8]`。
5. 如果循环只读 `z`，AoS 布局中每 16 字节只用 4 字节，cache line 利用率可能只有四分之一。

对应的汇编形态可以放进单独代码块观察：

```
shl    rax, 4
movss  xmm0, dword ptr [rdi + rax + 8]
```

这种解释方式看起来繁琐，但它是性能优化的基本功。后面做矩阵乘、卷积、attention、quantization、packing、prefetch、SIMD load/store 时，所有复杂技术都只是这条链路的规模化版本。

1.4.2.2 符号约定和推理习惯

为了让后面的推导不混乱，本章使用下面几个约定：

- `addr(x)` 表示对象或子对象 `x` 的起始地址。
- `sizeof(T)` 表示类型 `T` 的对象大小，单位是字节。
- `offset(S, field)` 表示字段 `field` 相对结构体 `S` 起始地址的字节偏移。
- `stride` 表示相邻两个逻辑元素之间的地址距离，单位通常是字节。
- `loadwidth` 表示一次 `load` 指令读取多少字节。

你每次推导内存访问时，都应该把“元素下标”和“字节偏移”分开写。下标是源码层概念，偏移是机器地址层概念。例如：

```
a[3] index      = 3
a[3] byte_offset = 3 * sizeof(int)
               = 12
```

把这两个数混在一起，是阅读汇编时最常见的早期错误。

习题与作业

随堂检查 3.0：把一句话说完整

请不要查资料，直接回答下面三个问题。每个答案至少写出“类型、基地址、字节偏移、访问宽度”四个词。

1. `double*p;p[i]` 访问的字节地址如何计算？
2. `structV{\floatx,y,z,w;\};V*v;v[i].z` 访问的字节地址如何计算？
3. 下面两条汇编合起来可能对应什么 C++ 访问？

```
shl    rax, 4
movss  xmm0, dword ptr [rdi + rax + 8]
```

如果你的答案里只有“取第几个元素”，说明你还停留在源码层；本章后面要反复训练到机器层。

1.4.3 位、字节和机器字：数据表示的最小地基

1.4.3.1 位是信息，字节是寻址单位

bit (binary digit) 是二进制位，只能表示 0 或 1。一个 bit 可以表达一个真假状态，但真实程序中的整数、地址、浮点数和指令都需要多个 bit 组合。

byte (字节) 在现代通用机器上通常是 8 bit。C++ 标准中 `char` 是最小可寻址对象单位；在我们学习的 x86-64/Linux 环境中，一个 byte 就是 8 bit。

单位	大小	常见用途
bit	1 个二进制位	标志位、掩码的一位
byte	8 bit	最小寻址单位、 <code>char</code> 、对象表示
word	依上下文变化	CPU 自然处理宽度或 ISA 术语
dword	32 bit	x86 汇编中的双字
qword	64 bit	x86 汇编中的四字

注意 **word** (字) 是一个容易混淆的词。在 x86 历史语境里，word 常指 16 bit，dword 是 32 bit，qword 是 64 bit；在体系结构教材中，word 有时指机器自然处理宽度；在内存系统里，cache line 又是另一种粒度。读文档时必须看上下文。

1.4.3.2 二进制、十六进制和为什么程序员爱用十六进制

二进制直接对应 bit，但写起来太长。十六进制每一位刚好表示 4 bit，因此两个十六进制数字刚好表示 1 byte：

十六进制	二进制	十进制
0x0	0000	0
0x1	0001	1
0x7	0111	7
0x8	1000	8
0xF	1111	15

例如：

```
0x12 = 0001 0010
0x34 = 0011 0100
0x1234 = 0001 0010 0011 0100
```


十六进制非常适合观察内存、机器码和地址。后面你会经常看到：

- 对象字节：78563412。
- 机器码字节：8b04b7。
- 地址：0x7ffe...。
- 位掩码：0xff、0xffff0000。

1.4.4 内存的第一模型：按字节寻址的巨大数组

从最简单的机器模型看，内存可以被想象成一个按地址索引的字节数组：

```
address  byte
0x1000  -> 0x78
0x1001  -> 0x56
0x1002  -> 0x34
0x1003  -> 0x12
0x1004  -> ...
```

地址是“某个字节的位置编号”。如果一个 `int` 占 4 字节，地址 `0x1000` 处的 `int` 会覆盖：

```
0x1000
0x1001
0x1002
0x1003
```

但是这个模型还不完整，因为普通 Linux 进程看到的是虚拟地址，不是物理内存条上的真实位置。更准确的说法是：对用户态程序而言，指针值通常是虚拟地址；CPU 执行 `load/store` 时会通过地址翻译机制把虚拟地址转换为物理地址，期间会用到 TLB 和页表。

本章先使用“内存是字节数组”的模型来理解对象表示、指针和布局。第 7 章、第 33 章会把虚拟内存、页表和 TLB 展开。

心智模型

先把内存想成字节数组，但永远记住它只是第一模型。性能优化时，你还要逐步叠加 `cache line`、`cache set`、`TLB`、`page`、`NUMA`、预取器和内存带宽。

1.4.5 类型不是字节本身，而是解释字节规则

内存中的 `byte` 本身没有“这是 `int`”或“这是 `float`”的标签。类型存在于程序语言、编译器和指令语义中。

例如同样 4 个字节：

```
00 00 80 3f
```

如果按 32 位整数解释，它是某个整数位模式；如果按 IEEE 754 `float` 解释，它是 `1.0f`；如果按 4 个 `unsignedchar` 解释，它只是 4 个字节。

C++ 的类型系统至少做了四件事：

1. 决定对象需要多少字节，例如 `sizeof(int)`。
2. 决定对象要求什么对齐，例如 `alignof(double)`。
3. 决定表达式如何生成地址，例如 `p+1` 前进多少字节。
4. 决定编译器可以做哪些优化，例如别名规则和对象生命周期规则。

常见误区

“内存只是字节，所以我想怎么解释就怎么解释”是非常危险的半真半假。硬件层面确实是字节；C++ 语言层面有对象、类型、生命周期、对齐和别名规则。高性能代码经常需要看字节，但必须用语言允许的方式看。

1.4.6 对象表示：C++ 对象在内存中的字节

C++ 中的对象有类型、存储期、生命周期和值。对象在内存中的底层字节叫作 **object representation** (对象表示)。对于一个可平凡复制的对象，你可以通过 `unsignedchar`、`std::byte` 或 `char` 观察它的对象表示。

下面的程序打印一个 32 位整数的字节：

```
#include <cstdint>
#include <iomanip>
#include <iostream>

int main()
{
```



```
std::uint32_t x = 0x12345678U;
auto const* p = reinterpret_cast<unsigned char const*>(&x);

for (std::size_t i = 0; i < sizeof(x); ++i) {
    std::cout << std::hex << std::setw(2) << std::setfill('0')
        << static_cast<unsigned>(p[i]) << '\n';
}
}
```

在 x86-64 上，你通常会看到：

78
56
34
12

这就是小端字节序。

1.4.7 字节序：为什么 0x12345678 在内存里像反过来

endianness（字节序）描述多字节对象的字节如何放进连续地址。

1.4.7.1 小端和大端

如果最低有效字节放在最低地址，叫 little-endian（小端）。x86-64 使用小端。对 0x12345678：

地址偏移	字节
+0	0x78
+1	0x56
+2	0x34
+3	0x12

如果最高有效字节放在最低地址，叫 big-endian（大端）。同一个值会存成：

地址偏移	字节
+0	0x12
+1	0x34
+2	0x56
+3	0x78

1.4.7.2 字节序影响什么，不影响什么

字节序影响的是多字节对象在内存中的排列。它不影响整数的数学值，也不影响寄存器里 0x12345678 这个值本身。

字节序经常在这些场景中变重要：

- 打印对象字节。
- 解析二进制文件格式。
- 网络协议和磁盘格式。
- 手写 SIMD shuffle 或处理 packed data。
- 调试机器码和内存 dump。

多数纯 C++ 算法不需要直接关心字节序，但系统性能工程师必须能看懂字节序，否则看到内存 dump 时会误读数据。

1.4.8 整数表示：无符号模运算和有符号补码

1.4.8.1 无符号整数是按位宽取模的环

一个 N 位无符号整数可以表示 0 到 $2^N - 1$ 。运算按模 2^N 回绕。例如 8 位无符号整数：

$255 + 1 = 0$
 $0 - 1 = 255$

C++ 对无符号整数溢出有明确规定：按模回绕。这让无符号整数适合写位运算、哈希、掩码、循环计数的一些底层逻辑。

1.4.8.2 有符号整数通常用补码表示

现代 x86-64 使用 **two's complement**（二进制补码）表示有符号整数。以 8 位为例：

位模式	无符号解释	有符号补码解释
00000000	0	0
00000001	1	1
01111111	127	127
10000000	128	-128
11111111	255	-1

补码的好处是加法电路可以同时服务有符号和无符号加法。比如 -1 的 8 位补码是 11111111，加 1 得到 00000000，自然回到 0。

1.4.8.3 语言规则和机器行为不能混为一谈

机器补码加法会回绕，但 C++ 有符号整数溢出是未定义行为。也就是说，下面代码在 C++ 语言层面不能被解释成“最大值加一变最小值”：

```
int f(int x)
{
    return x + 1;
}
```

如果 `x==INT_MAX`，表达式 `x+1` 发生有符号溢出，程序有未定义行为。编译器可以假设这种情况不会发生，并据此优化。

这不是抠字眼。性能优化中，编译器对循环边界、条件判断、向量化和死代码删除的很多决定，都依赖“程序没有未定义行为”的假设。

深入理解

后续学习编译器优化时，你会看到 signed overflow、strict aliasing、out-of-bounds、未初始化读取、对象生命周期等规则如何影响优化。底层工程不是绕开语言规则，而是理解语言规则如何给编译器提供证明空间。

1.4.9 浮点数只先建立直觉

本章重点是整数、地址和对象布局。浮点数会在 AI 算子、数值稳定性、SIMD 和 FMA 章节中深入。本章先建立三个直觉：

1. `float` 和 `double` 也是固定字节数的对象表示。
2. 它们通常使用 IEEE 754 格式，包含符号、指数和尾数。
3. 浮点运算不是实数运算，存在舍入、特殊值、非结合性和 fast-math 语义问题。

例如 `float x=1.0f;` 的对象字节在小端 x86-64 上通常是：

00 00 80 3f

按整数看这只是一个位模式；按 IEEE 754 `float` 看它表示 1.0。这个例子再次提醒你：字节没有类型，解释规则来自类型和上下文。

1.4.10 指针：地址值加类型语义

1.4.10.1 指针值通常是地址

在普通 x86-64/Linux 用户态程序中，指针值通常可以理解为虚拟地址。比如：

```
int x = 42;
int* p = &x;
```

`p` 保存的是对象 `x` 的地址。打印 `p` 可能得到类似：

0x7ffdf2c3a6bc

但指针不仅是一个整数地址。C++ 指针还携带类型语义：`int*` 指向 `int` 对象，`double*` 指向 `double` 对象，`Particle*` 指向 `Particle` 对象。类型决定了解引用读取多少字节、指针加法前进多少字节、编译器如何判断别名。

1.4.10.2 解引用是一次按类型解释的内存访问

表达式 `*p` 不是“取地址里的东西”这么笼统。更准确地说：

1. 读取指针表达式 `p` 的值，得到一个地址。
2. 根据 `p` 的静态类型知道目标对象类型是 `int`。
3. 从该地址开始读取 `sizeof(int)` 个字节。
4. 按 `int` 的对象表示解释这些字节。

如果地址没有对齐、对象生命周期不存在、类型不匹配或越界，语言层面可能已经出错。硬件是否“碰巧能读”不是 C++ 程序正确性的充分条件。

1.4.10.3 空指针、野指针和悬垂指针

指针错误是底层学习必须严肃对待的内容。

空指针 不指向任何对象。解引用空指针是错误。

野指针 未初始化或保存无效地址的指针。它可能指向任意位置。

悬垂指针 原来指向有效对象，但对象生命周期已经结束。

例如：

```
int* bad()
{
    int x = 1;
    return &x;
}
```

`x` 是局部自动对象，函数返回后生命周期结束。返回的指针值可能仍然看起来像一个地址，但它不再指向活着的 `int` 对象。用它做性能实验，结果没有意义。

1.4.11 指针加法：类型决定缩放

指针加法不是简单整数加法。若 `p` 类型是 `T*`，则 `p+k` 的地址通常是：

$$\text{address}(p + k) = \text{address}(p) + k * \text{sizeof}(T)$$

例如：

```
int* p = reinterpret_cast<int*>(0x1000);
int* q = p + 3;
```

如果 `sizeof(int)==4`，则 `q` 的数值地址是 `0x100c`，不是 `0x1003`。这解释了 `a[i]`：

$$a[i] = *(a + i)$$

如果 `a` 是 `int*`，`a+i` 自动按 4 缩放；如果 `a` 是 `double*`，按 8 缩放；如果 `a` 是 `Particle*`，按 `sizeof(Particle)` 缩放。

常见误区

`reinterpret_cast<std::uintptr_t>(p)+1` 和 `p+1` 不是同一个概念。前者是整数地址加 1 字节；后者是指针按元素大小前进 1 个对象。写底层代码时必须分清“字节偏移”和“元素偏移”。

1.4.12 数组：连续对象序列

C/C++ 数组是连续存放的同类型对象序列：

```
int a[4] = {10, 20, 30, 40};
```

如果 `a[0]` 地址为 `0x1000`，且 `sizeof(int)==4`，则布局通常是：

元素	起始地址	覆盖字节范围
<code>a[0]</code>	<code>0x1000</code>	<code>0x1000..0x1003</code>
<code>a[1]</code>	<code>0x1004</code>	<code>0x1004..0x1007</code>
<code>a[2]</code>	<code>0x1008</code>	<code>0x1008..0x100b</code>
<code>a[3]</code>	<code>0x100c</code>	<code>0x100c..0x100f</code>

1.4.12.1 数组名退化为指针

在很多表达式中，数组名会退化为指向首元素的指针：

```
int a[4];
int* p = a;    // p = &a[0]
```

但数组本身不是指针。下面三个 `sizeof` 结果不同：

```
int a[4];

sizeof(a);    // 4 * sizeof(int)
sizeof(&a[0]); // pointer size
sizeof(int*); // pointer size
```

这一区别对性能代码很重要。模板、`span`、数组引用、指针和长度参数会影响编译器是否知道长度、是否能展开、是否能做边界相关优化。

1.4.12.2 越界不是“多走几个字节”这么简单

从机器地址角度看，`a[4]` 似乎只是访问 `a[0]` 后面第 16 个字节。但从 C++ 语言角度，访问数组范围外对象是未定义行为。

```
int a[4] = {1, 2, 3, 4};
int x = a[4];    // undefined behavior
```

它可能读到某个栈上的值，可能触发崩溃，也可能被编译器假设为永不发生并重写代码。性能工程中不要用越界行为做“技巧”。

1.4.13 四步推导法：从 C++ 表达到机器访问

前面已经讲了指针、数组和类型，但真正读汇编时，你需要一个固定动作。本节给出一个可以反复使用的四步推导法。以后看到任何内存访问表达式，都按这个顺序走：

1. **确定表达式的对象类型和访问宽度。**它最终读写的是 `int`、`double`、`float`，还是某个结构体字段？访问宽度是 1、2、4、8、16、32 还是 64 字节？
2. **确定基地址。**这个访问从哪个指针、数组首元素、结构体对象或栈帧位置出发？
3. **计算字节偏移。**把下标、字段 `offset`、数组步长、结构体大小都换成字节单位。
4. **映射到 x86-64 寻址。**尽量写成 `[base+index*scale+displacement]`，同时标出 `load/store` 宽度。

这四步看似简单，但它可以把 C++ 源码、对象布局和汇编连接成一条线。

1.4.13.1 例 1: `inta[5];a[3]`

假设 `a[0]` 的地址是 `0x1000`，并且 `sizeof(int)==4`。

步骤	推导
对象类型	<code>a[3]</code> 是 <code>int</code> 左值，访问宽度 4 字节。
基地址	数组首元素地址 <code>addr(a[0])=0x1000</code> 。
字节偏移	下标 3，元素大小 4，所以 <code>offset = 3*4=12=0x0c</code> 。
最终地址	<code>0x1000+0x0c=0x100c</code> ，覆盖字节 <code>0x100c..0x100f</code> 。

如果 `a` 作为第一个参数传入函数，`i` 作为第二个参数，汇编可能是：

```
mov eax, dword ptr [rdi + rsi*4]
```

这里 `rdi` 是基地址，`rsi` 是元素下标，`scale 4` 来自 `sizeof(int)`，`dword ptr` 表示访问 4 字节。

1.4.13.2 例 2: `doublea[5];a[3]`

同样是下标 3，但类型换成 `double` 后，字节偏移不同。先把条件单独写清楚：

```
addr(a[0])    = 0x1000
sizeof(double) = 8
```

因此：

```
offset = 3 * sizeof(double) = 3 * 8 = 24 = 0x18
addr(a[3]) = 0x1000 + 0x18 = 0x1018
```

对应汇编可能是：

```
movsd xmm0, qword ptr [rdi + rsi*8]
```

这次 `scale` 是 8，`load width` 是 `qword`，返回值进入 `xmm0`。同一个下标，因为类型不同，机器地址计算不同；同一个地址，因为解释规则不同，得到的值也可能不同。

1.4.13.3 例 3: $p[i].z$ 如何拆成两层偏移

看一个对 AI/HPC 很常见的结构：

```
struct Vec4 {
    float x;
    float y;
    float z;
    float w;
};

float get_z(Vec4 const* p, std::size_t i)
{
    return p[i].z;
}
```

在常见 ABI 下，四个 `float` 连续排布：

字段	offset	大小	说明
x	0	4	第一个 <code>float</code>
y	4	4	第二个 <code>float</code>
z	8	4	第三个 <code>float</code>
w	12	4	第四个 <code>float</code>

所以：

```
addr(p[i].z) = addr(p[0]) + i * sizeof(Vec4) + offset(Vec4, z)
              = base      + i * 16          + 8
```

这个地址公式不能直接写成单条 x86-64 内存操作数，因为 x86-64 的 `scale` 只有 1、2、4、8。真实编译器可能先把 $i \times 16$ 算到寄存器里，再做 `load`：

```
shl    rsi, 4
movss  xmm0, dword ptr [rdi + rsi + 8]
```

这里第一条指令把元素下标变成字节偏移。对于 $i \times 16$ 这种 2 的幂乘法，左移通常最直观。对于 12、24、40 这类非 2 的幂步长，后面会专门用 `lea` 解释。

不要死记某一种汇编形态。关键是看出地址公式里有三个来源：数组基址、元素步长、字段 `offset`。

1.4.13.4 例 4: 二维数组下标不是两个 `load`

考虑真正的 C++ 二维数组：

```
int a[3][5];
int x = a[i][j];
```

`a` 是 3 行 5 列的连续数组。按行主序存放，`a[i][j]` 的线性元素下标是：

```
i * 5 + j
```

字节地址是：

```
addr(a[i][j]) = addr(a[0][0]) + (i * 5 + j) * sizeof(int)
```

这不是“先找到第 i 行指针，再从那个指针找第 j 个元素”。真正的 `inta[3][5]` 是一整块连续内存，行长度 5 是类型的一部分。只有 `int**` 或“指针数组”才是另一种布局。

常见误区

`inta[3][5]`、`int(*p)[5]`、`int**p` 是三种不同模型。前两者知道每行 5 个 `int`，可以用线性地址公式；`int**` 是指向指针的指针，通常需要先读取行指针，再访问该行的数据。它们的性能、局部性和汇编形态都可能不同。

1.4.13.5 例 5: 把 $a[i] += b[i]$ 展开成 `load-compute-store`

一个赋值表达式往往不止一次内存访问：

```
void add_into(float* a, float const* b, std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        a[i] += b[i];
    }
}
```

循环体在标量层面可以拆成：

```
tmp_a = load 4 bytes from addr(a[0]) + i * 4
tmp_b = load 4 bytes from addr(b[0]) + i * 4
tmp    = tmp_a + tmp_b
store 4 bytes to  addr(a[0]) + i * 4
```

这对性能很关键。你不能只数“一次加法”。真实循环每个元素至少有两个 load、一个 store、一个浮点加法、一个循环控制，以及可能的别名检查、边界处理和向量化 remainder。后面做 roofline 模型时，算术强度就是从这种拆解开始的。

习题与作业

随堂检查 3.1：按四步推导法写答案

对下面表达式分别写出对象类型、基地址、字节偏移公式和可能的 x86-64 内存操作形式：

1. `std::uint64_t*p;p[i]`
2. `float*x;x[i+1]`
3. `struct Pair{inta;intb;};Pair*p;p[i].b`
4. `double a[4][8];a[i][j]`
5. `struct Node{intkey;Node*next;};Node*p;p->next`

要求：不要只写 C++ 解释，必须写成字节地址公式。

1.4.14 x86-64 寻址模式：数组访问如何进入指令

x86-64 内存操作数常见形式是：

```
[base + index * scale + displacement]
```

其中：

- **base** 是基址寄存器。
- **index** 是索引寄存器。
- **scale** 通常只能是 1、2、4、8。
- **displacement** 是编码在指令中的常量偏移。

这非常适合表达 C/C++ 的数组和结构体字段访问。

1.4.14.1 访问 `inta[i]`

```
mov eax, dword ptr [rdi + rsi*4]
```

含义：

```
load 4 bytes from address rdi + rsi * 4
```

如果执行到这条指令时寄存器是：

寄存器	值	含义
<code>rdi</code>	<code>0x1000</code>	<code>a[0]</code> 的地址
<code>rsi</code>	<code>3</code>	下标 <code>i</code>

那么有效地址是：

```
effective_address = 0x1000 + 3 * 4
                  = 0x100c
```

`mov eax, dword ptr [0x100c]` 的语义是从 `0x100c` 开始读取 4 字节，写入 `eax`。在 x86-64 中，对 `eax` 的写入还会清零 `rax` 的高 32 位。这一点读汇编时经常重要：`int` 返回值写 `eax`，但调用者看到的完整 `rax` 高位已经被清零。

1.4.14.2 访问结构体字段

如果有：

```
struct S {
    int a;
    double b;
    int c;
};

int get_c(S const* p)
{
    return p->c;
}
```

汇编可能是：

```
mov eax, dword ptr [rdi + 16]
ret
```

16 就是字段 `c` 相对结构体起始地址的 `offset`。编译器根据类型布局规则算出它，然后把常量写进指令。这个例子可以完整展开。常见 x86-64 ABI 下，`S` 的布局通常是：

字节范围	内容	大小	原因
0..3	<code>a</code>	4	<code>int</code> 字段
4..7	padding	4	让 <code>b</code> 满足 8 字节对齐
8..15	<code>b</code>	8	<code>double</code> 字段
16..19	<code>c</code>	4	<code>int</code> 字段
20..23	tail padding	4	让 <code>sizeof(S)</code> 是 8 的倍数

假设 `rdi` 保存 `p==0x2000`，那么：

```
addr(p->c) = addr(*p) + offset(S, c)
            = 0x2000 + 16
            = 0x2010
```

因此：

```
mov eax, dword ptr [rdi + 16]
```

就是从 `0x2010..0x2013` 读取 4 字节。这里没有 `index` 和 `scale`，因为没有数组下标；只有结构体基地址和字段偏移。

1.4.14.3 结构体数组字段访问：scale 和 displacement 同时出现

更接近真实 kernel 的情况是结构体数组：

```
struct Pixel {
    std::uint8_t r;
    std::uint8_t g;
    std::uint8_t b;
    std::uint8_t a;
};

std::uint8_t load_b(Pixel const* p, std::size_t i)
{
    return p[i].b;
}
```

`sizeof(Pixel)==4`，字段 `b` 的 `offset` 是 2，所以地址公式是：

```
addr(p[i].b) = addr(p[0]) + i * sizeof(Pixel) + offset(Pixel, b)
              = base      + i * 4                + 2
```

可能的汇编是：

```
movzx eax, byte ptr [rdi + rsi*4 + 2]
ret
```

这条指令有几个细节：

- `byteptr` 表示从内存读取 1 字节。
- `movzx` 表示 zero extend，读取 8 位后零扩展到 32 位 `eax`。
- `rsi*4` 来自 `sizeof(Pixel)`。
- `+2` 来自字段 `b` 的 `offset`。

假设：

寄存器	值	含义
<code>rdi</code>	<code>0x3000</code>	<code>p[0]</code> 的地址
<code>rsi</code>	<code>5</code>	下标 <code>i</code>

则：

```
effective_address = 0x3000 + 5 * 4 + 2
                  = 0x3016
```

CPU 会读取 `0x3016` 这个地址上的 1 字节，把它零扩展到 32 位返回值。这个案例比 `inta[i]` 更接近图像处理、量化张量、packed struct、RGBA buffer 这类真实工作负载。

1.4.14.4 当 scale 不够用时：sizeof(T) 大于 8 怎么办

x86-64 内存操作数的 `scale` 只有 1、2、4、8。如果结构体大小是 16、24、32，编译器必须用额外指令先算出字节偏移。

例如：


```

struct Vec4 {
    float x;
    float y;
    float z;
    float w;
};

float load_z(Vec4 const* p, std::size_t i)
{
    return p[i].z;
}

```

地址公式是：

```
addr(p[i].z) = base + i * 16 + 8
```

一种可能汇编是：

```

shl    rsi, 4
movss  xmm0, dword ptr [rdi + rsi + 8]
ret

```

如果寄存器初值是：

寄存器	值	含义
rdi	0x4000	p[0] 的地址
rsi	3	下标 i

执行 `shl rsi, 4` 后, `rsi` 变成 48, 也就是 3×16 。随后：

```

effective_address = 0x4000 + 48 + 8
                  = 0x4038

```

`movss` 从 `0x4038..0x403b` 读取 4 字节到 `xmm0` 的低 32 位。读汇编时要注意：有些寄存器一开始表示“元素下标”，经过 `shl` 或 `lea` 后变成“字节偏移”。如果不跟踪这种语义变化，就会把地址算错。

1.4.14.5 `lea`：地址计算指令不一定访问内存

`lea` 的名字是 load effective address，但它不从内存读取数据，只做地址形式的整数计算。例如：

```
lea rax, [rsi + rsi*2]
```

这条指令计算的是：

```

rax = rsi + rsi * 2
    = 3 * rsi

```

它没有访问 `[rsi+rsi*2]` 这个内存地址。编译器经常用 `lea` 做乘法常数、数组偏移和指针计算，因为它可以把“加法 + scale + 常量”合到一条指令里。

例如结构体大小为 24 时：

```

struct Item24 {
    double a;
    double b;
    int c;
    int d;
};

int load_d(Item24 const* p, std::size_t i)
{
    return p[i].d;
}

```

若 `sizeof(Item24)==24`, `offset(Item24,d)==20`, 地址公式是：

```
addr(p[i].d) = base + i * 24 + 20
```

编译器可能生成类似：

```

lea eax, [rsi + rsi*2]      ; eax = i * 3
mov eax, dword ptr [rdi + rax*8 + 20]
ret

```

这里第二条指令的 scale 是 8，所以总乘数是 $3 \times 8 = 24$ 。这种组合在真实汇编里很常见。读它时不要只看最后一条内存操作，要把前面的 `lea` 一起纳入地址公式。

核心思想

C++ 的数组下标、指针解引用、结构体字段访问，在机器层面都落到地址计算。理解地址计算，是读汇编、理解 cache 行为和优化数据布局的共同入口。

1.4.15 对象的存储期：栈、堆、静态区只是第一步分类

C++ 对象可以有不同存储期。存储期决定对象的存储何时获得、何时释放。常见分类如下：

存储期	示例	生命周期大致特点
自动存储期	局部变量	进入作用域创建，离开作用域销毁
动态存储期	<code>new</code> 、 <code>malloc</code>	程序显式申请和释放
静态存储期	全局变量、 <code>static</code>	程序启动到结束
线程存储期	<code>thread_local</code>	线程开始到线程结束

初学者常说“局部变量在栈上，`new` 出来的在堆上，全局变量在静态区”。这作为入门直觉可以，但要知道它不是完整语言规则。编译器优化可能把局部变量放在寄存器里，甚至完全消除；动态分配器背后可能使用 `mmap`；静态对象还涉及 ELF 段、重定位和运行时初始化。

1.4.15.1 栈对象

局部自动对象常见实现是在栈帧中分配：

```
void f()
{
    int x = 1;
    int y = 2;
}
```

未优化时，编译器可能给 `x` 和 `y` 分配栈槽。优化后，如果它们不需要真实地址，可能只存在于寄存器甚至常量传播结果中。

1.4.15.2 堆对象

动态对象常由分配器管理：

```
auto* p = new int[1024];
delete[] p;
```

堆分配有额外成本：分配器元数据、锁或线程缓存、对齐、碎片、`page fault`、`NUMA` 位置等。性能关键路径中频繁分配小对象通常很危险。

1.4.15.3 静态对象

全局变量和静态变量通常在可执行文件或共享库的段中描述，并在进程加载时映射：

```
int global_counter = 0;

int f()
{
    static int local_static = 1;
    return ++local_static;
}
```

静态对象会牵涉链接、加载、初始化顺序、线程安全局部静态初始化等问题。本书后面讲 ELF、链接和运行时时会展开。

1.4.16 对齐：对象地址不是随便放的

alignment（对齐）是对象地址必须满足的约束。若一个类型 `T` 的对齐要求是 `A`，则 `T` 对象地址通常必须是 `A` 的倍数。

例如在常见 `x86-64 ABI` 下：

类型	常见大小	常见对齐
<code>char</code>	1	1
<code>int</code>	4	4
<code>float</code>	4	4
<code>double</code>	8	8
<code>void*</code>	8	8

你可以用 `alignof` 查询：

```
#include <iostream>

int main()
{
    std::cout << alignof(char) << '\n';
    std::cout << alignof(int) << '\n';
    std::cout << alignof(double) << '\n';
    std::cout << alignof(void*) << '\n';
}
```

1.4.16.1 为什么需要对齐

对齐来自硬件和 ABI 的共同需要。硬件访问自然对齐的数据通常更简单、更快。有些架构上未对齐访问会触发异常；x86-64 通常允许许多未对齐访问，但可能带来性能代价，尤其是跨 cache line、跨页或用于某些 SIMD 指令时。

例如，一个 8 字节 load 如果地址是 8 字节对齐，且不跨 cache line，硬件处理更直接。如果它从 cache line 最后 4 字节开始，就会跨两个 cache line，可能需要两个 cache line 都可用。

深入理解

“x86 支持未对齐访问”不等于“对齐不重要”。标量普通 load/store 可能只是轻微变慢，但跨 cache line、跨 page、原子操作、SIMD 对齐指令、false sharing 场景下，对齐会变成明显性能因素。

1.4.17 结构体布局：字段、padding 和 ABI

结构体不是简单把字段大小相加。编译器需要为每个字段选择 offset，并在必要时插入 padding（填充字节），以满足字段和整个结构体的对齐要求。

1.4.17.1 先掌握一个机械算法

学习结构体布局时，不要一上来凭感觉。对普通标准布局结构体，可以先用下面这个机械算法训练。真实 ABI 有更多细节，例如继承、虚函数、空基类优化、位域、向量类型、显式 alignas、packing pragma，但本算法足够支撑本章和大量性能分析。

1. 令当前 offset 为 0。
2. 从第一个字段开始，按声明顺序处理。
3. 对当前字段，先查出字段大小 `sizeof(field)` 和对齐 `alignof(field)`。
4. 如果当前 offset 不是字段对齐的倍数，就插入 padding，把 offset 向上补齐到下一个满足对齐的位置。
5. 当前 offset 就是该字段的 offset。
6. offset 增加字段大小。
7. 所有字段处理完后，结构体整体对齐通常是所有字段对齐要求的最大值。
8. 最终 `sizeof(struct)` 必须补齐到结构体整体对齐的倍数，因此末尾可能有 tail padding。

其中“向上补齐”可以写成一个函数：

$$\text{align_up}(x, A) = \text{smallest } y \text{ such that } y \geq x \text{ and } y \% A = 0$$

如果 A 是 2 的幂，机器代码和系统代码里经常写成：

```
std::size_t align_up(std::size_t x, std::size_t a)
{
    return (x + a - 1) & ~(a - 1);
}
```

例如 `align_up(5,4)==8`, `align_up(16,8)==16`。第一个例子需要补 3 字节，第二个例子本来已经对齐，不需要补。

核心思想

padding 不是编译器“随便浪费空间”。padding 是为了让每个字段和数组中每个结构体元素都满足对齐约束。结构体大小必须包含尾部 padding，否则 `arr[1]` 的起始地址可能不满足结构体自身对齐。

看第一个例子：

```
struct A {
    char c;
    int x;
};
```

在常见 x86-64 ABI 下，char 大小 1、对齐 1；int 大小 4、对齐 4。若 c 放在 offset 0，那么 x 不能放在 offset 1，因为 offset 1 不是 4 的倍数。编译器会插入 3 字节 padding：

offset	内容	大小	说明
0	c	1	字段
1..3	padding	3	让下个字段 4 字节对齐
4..7	x	4	字段

所以 `sizeof(A)` 通常是 8，不是 5。

1.4.17.2 完整例题：逐字段模拟布局

现在用算法推一个稍复杂的例子：

```
struct Record {
    char tag;
    double score;
    int count;
    short flags;
};
```

假设常见 x86-64 ABI 下：

类型	大小	对齐
char	1	1
short	2	2
int	4	4
double	8	8

逐字段推导如下：

字段	进入前 offset	对齐动作	字段 offset	写入后 offset
tag	0	已满足 1 对齐	0	1
score	1	补到 8，插入 7 字节 padding	8	16
count	16	已满足 4 对齐	16	20
flags	20	已满足 2 对齐	20	22

字段处理完后，最大对齐是 8，所以结构体大小必须补到 8 的倍数。当前 offset 是 22，`align_up(22,8)` ==24，因此尾部 padding 是 2 字节：

字节范围	内容	大小	说明
0	tag	1	字段
1..7	padding	7	让 score 8 字节对齐
8..15	score	8	字段
16..19	count	4	字段
20..21	flags	2	字段
22..23	tail padding	2	让 sizeof(Record) 是 8 的倍数

如果有数组：

```
Record r[2];
```

那么：

$$\begin{aligned} \text{addr}(r[1]) &= \text{addr}(r[0]) + \text{sizeof}(\text{Record}) \\ &= \text{addr}(r[0]) + 24 \end{aligned}$$

24 是 8 的倍数，因此 `r[1].score` 仍然可以落在 8 字节对齐地址上。如果结构体大小只是字段结束时的 22，数组第二个元素就会破坏字段对齐，这就是尾部 padding 存在的根本原因。

1.4.17.3 把字段重排也算法化

如果只是追求更小的结构体大小，一个常见启发式是把对齐要求大的字段放前面：

```
struct RecordPackedOrder {
    double score;
    int count;
    short flags;
    char tag;
};
```

逐字段推导：

字段	进入前 offset	对齐动作	字段 offset	写入后 offset
score	0	已满足 8 对齐	0	8
count	8	已满足 4 对齐	8	12
flags	12	已满足 2 对齐	12	14
tag	14	已满足 1 对齐	14	15

最大对齐仍然是 8，最终大小补到 16。和原来的 24 相比，减少了 8 字节。如果这种对象有一千万个，节省的是约 80 MB，不是小数。

但这仍然不是“所有结构体都按大到小排序”的命令。性能结构体设计要同时考虑三类目标：

- **空间密度**：减少 padding，让更多对象进入 cache。
- **访问局部性**：把同一个 hot loop 经常一起使用的字段放近，把冷字段挪远。
- **并发隔离**：多线程频繁写的字段有时要故意分开，避免 false sharing。

字段顺序是数据布局设计的一部分，不只是“省几个字节”。对于 AI runtime 的 tensor metadata、operator descriptor、thread task、work queue node，字段布局会影响 cache 占用、预取效果、false sharing 和 ABI 稳定性。

1.4.17.4 尾部 padding

再看：

```
struct B {
    int x;
    char c;
};
```

x 放在 offset 0，占 4 字节；c 放在 offset 4，占 1 字节。看起来总共 5 字节。但结构体整体对齐是其成员最大对齐，通常是 4。为了让 `Barr[2]` 中每个元素都满足对齐，`sizeof(B)` 必须是 4 的倍数，于是尾部会有 3 字节 padding：

offset	内容	大小	说明
0..3	x	4	字段
4	c	1	字段
5..7	tail padding	3	让数组中下个元素对齐

1.4.17.5 字段顺序会影响大小

比较：

```
struct Bad {
    char a;
    double b;
    char c;
    int d;
};

struct Better {
    double b;
    int d;
    char a;
    char c;
};
```

Bad 可能因为多次对齐插入大量 padding；Better 把大对齐字段放前面，通常更紧凑。

但字段重排不是无脑优化。你还要考虑：

- ABI 兼容：公开结构体布局不能随便改。
- 序列化和文件格式：布局变化会破坏二进制兼容。
- 热字段和冷字段：有时为了 cache，把常用字段聚在一起比最小 `sizeof` 更重要。
- false sharing：多线程写的字段可能需要故意分离到不同 cache line。

1.4.18 sizeof、alignof 和 offsetof

研究布局时，三个工具非常重要：

```
#include <cstddef>
#include <iostream>

struct S {
    char a;
    double b;
    int c;
};

int main()
{
    std::cout << "sizeof(S) = " << sizeof(S) << '\n';
    std::cout << "alignof(S) = " << alignof(S) << '\n';
    std::cout << "offset a = " << offsetof(S, a) << '\n';
    std::cout << "offset b = " << offsetof(S, b) << '\n';
    std::cout << "offset c = " << offsetof(S, c) << '\n';
}
```

`offsetof` 对标准布局类型可靠。学习阶段你应该先手工预测，再用程序验证。不要只看输出，因为真正的训练是形成布局推导能力。

1.4.19 padding 字节不一定有稳定值

结构体 padding 是对象表示的一部分，但不一定参与对象的值。它可能未初始化，也可能包含旧栈数据。比较结构体对象时，不能简单用 `memcmp` 替代字段级比较，除非你非常清楚类型性质和初始化方式。

例如：

```
struct A {
    char c;
    int x;
};

bool equal_bad(A const& lhs, A const& rhs)
{
    return std::memcmp(&lhs, &rhs, sizeof(A)) == 0;
}
```

即使两个对象的 `c` 和 `x` 相等，padding 字节也可能不同，`memcmp` 可能返回不相等。这个问题在哈希、序列化、网络传输和 SIMD 批量比较中很常见。

常见误区

结构体的“值”和结构体的“对象字节”不是总能一一对应。padding、未初始化字节、浮点 NaN、不同但等价的表示，都可能让字节级比较和语义级比较不一致。

1.4.20 安全地观察和搬运字节

1.4.20.1 用 `unsignedchar` 或 `std::byte` 观察对象表示

观察对象字节时，可以使用 `unsignedcharconst*`：

```
template <class T>
void dump_bytes(T const& value)
{
    auto const* p = reinterpret_cast<unsigned char const*>(&value);
    for (std::size_t i = 0; i < sizeof(T); ++i) {
        std::cout << std::hex << static_cast<unsigned>(p[i]) << ' ';
    }
    std::cout << '\n';
}
```

1.4.20.2 用 `std::memcpy` 或 `std::bit_cast` 搬运位模式

如果你想在两种类型之间复制位模式，不要随便用错误类型指针解引用。C++20 提供 `std::bit_cast`：

```
#include <bit>
#include <cstdint>

float f = 1.0f;
std::uint32_t bits = std::bit_cast<std::uint32_t>(f);
```

也可以用 `std::memcpy`：

```
std::uint32_t bits;
std::memcpy(&bits, &f, sizeof(bits));
```

这两种方式都表达“复制对象表示”，比用不匹配类型的指针解引用更可靠。

1.4.21 别名规则：编译器如何知道两个指针会不会指向同一块内存

别名问题是从 C++ 到高性能优化的关键门槛。看下面代码：

```
void f(int* a, int* b)
{
    *a = 1;
    *b = 2;
    *a = *a + 3;
}
```

如果 `a` 和 `b` 指向不同对象，最后 `*a==4`。如果它们指向同一个 `int`，执行过程是：

1. 执行 `*a=1`，该对象变成 1。
2. 执行 `*b=2`。因为 `a` 和 `b` 指向同一对象，所以 `*a` 也变成 2。
3. 执行 `*a=*a+3`，最终该对象变成 5。

编译器如果不能证明 `a` 和 `b` 不别名，就必须保守处理。保守处理会影响寄存器缓存、load/store 重排、循环向量化和 common subexpression elimination。

1.4.21.1 不同类型通常提供别名信息

看另一个例子：

```
void g(int* a, double* b)
{
    *a = 1;
    *b = 2.0;
    *a = *a + 3;
}
```

在正常 C++ 别名规则下，`int*` 和 `double*` 通常不能指向同一个活着的对象。编译器可以利用这个事实，把 `*a` 保存在寄存器中，而不必担心 `*b=2.0` 改了同一个 `int`。

这就是为什么错误的类型双关不仅可能不正确，还可能导致编译器生成你无法预期的代码。

深入理解

C 的 `restrict`、C++ 编译器扩展中的 `__restrict__`、LLVM IR 的 `alias metadata`、Fortran 的数组别名语义，都会显著影响高性能循环能否向量化。后面讲 `auto-vectorization` 和 AI kernel 时会深入。

1.4.22 常见混淆：底层学习最容易卡住的地方

本章内容多，但很多错误可以归结为几组概念混淆。这里集中整理一次。你后面看汇编、写 SIMD、调 benchmark 时，只要结果不符合预期，就应该回到这些问题检查。

1.4.22.1 混淆 1：指针变量和被指对象不是同一个东西

看下面代码：

```
int x = 42;
int* p = &x;
```

`x` 是一个 `int` 对象，通常占 4 字节。`p` 也是一个对象，它的类型是 `int*`，在 x86-64 上通常占 8 字节。`p` 的值是 `x` 的地址，但 `p` 自己也需要存储空间。

如果假设：

```
addr(x) = 0x1000
addr(p) = 0x2000
value(p) = 0x1000
```

那么：

- 访问 `x`，读取的是 `0x1000..0x1003`。
- 访问 `p` 这个指针变量，读取的是 `0x2000..0x2007`。
- 访问 `*p`，先从 `p` 的值拿到 `0x1000`，再读取 `0x1000..0x1003`。

汇编里也会体现这个差异。如果 `rdi` 直接保存 `p` 的值，也就是 `x` 的地址，那么：

```
mov eax, dword ptr [rdi]
```

读取的是 `*p`。但如果 `rdi` 保存的是指针变量 `p` 自己的地址，也就是 `&p`，那么通常需要两次 load：

```
mov rax, qword ptr [rdi]    ; load p itself
mov eax, dword ptr [rax]    ; load *p
```

第一条读取指针值，第二条读取被指对象。链表、树、稀疏矩阵和 `int**` 的性能问题，很多都来自这种“先读地址，再按地址读数据”的间接访问。

1.4.22.2 混淆 2: 元素偏移和字节偏移不是同一个单位

下面两行很像，但单位不同：

```
int* q1 = p + 3;
auto q2 = reinterpret_cast<char*>(p) + 3;
```

如果 `p==0x1000`，则：

```
q1 = 0x1000 + 3 * sizeof(int) = 0x100c
q2 = 0x1000 + 3              = 0x1003
```

`p+3` 的 3 是元素个数；`reinterpret_cast<char*>(p)+3` 的 3 是字节数。读汇编时，所有地址最终都要落到字节偏移；读 C++ 时，指针加法先按类型缩放。

1.4.22.3 混淆 3: 数组对象、数组首元素指针、指向整个数组的指针

看代码：

```
int a[4];
int* p0 = a;
int (*p1)[4] = &a;
```

它们的数值地址可能相同，但类型不同：

表达式	类型	数值上常见地址	+1 的含义
<code>a</code>	表达式中常退化为 <code>int*</code>	<code>addr(a[0])</code>	下一个 <code>int</code>
<code>&a[0]</code>	<code>int*</code>	<code>addr(a[0])</code>	下一个 <code>int</code>
<code>&a</code>	<code>int(*)[4]</code>	<code>addr(a)</code>	下一个 4 元素数组

因此：

```
addr((&a[0]) + 1) = base + 4
addr((&a) + 1) = base + 16
```

这类区别在多维数组、固定 shape 张量、小矩阵 tile、编译期已知长度的 kernel 中非常重要。编译器知道更多类型信息时，往往能生成更直接的地址计算和更强的优化。

1.4.22.4 混淆 4: 硬件能执行不代表 C++ 定义良好

很多初学者会用“我运行没崩”来判断底层代码对不对。这在系统编程和性能优化里非常危险。看下面例子：

```
int a[4] = {1, 2, 3, 4};
int x = a[4];
```

硬件层面，这可能只是从 `a[0]` 后面 16 字节的位置发起一次 4 字节 load。但 C++ 层面，`a[4]` 越界，行为未定义。编译器可以假设这种情况不会发生，并据此重排、删除或改写代码。

再看类型双关：

```
int x = 0;
double* p = reinterpret_cast<double*>(&x);
double y = *p;
```

硬件也许能从 `&x` 开始读 8 字节，但 C++ 对象模型、对齐和别名规则都可能已经被破坏。正确观察位模式时，应优先使用：

```
std::bit_cast<T>(value)
std::memcpy(&dst, &src, sizeof(dst))
unsigned char const* bytes = reinterpret_cast<unsigned char const*>(&obj)
```

底层工程的目标不是“骗过编译器和硬件”，而是在语言规则允许的范围内，把编译器、ISA 和微架构都用到极限。

1.4.22.5 混淆 5: 汇编里的地址公式不等于源码里的唯一写法

同一个源码可能生成多种汇编，同一个汇编片段也可能来自多种源码。比如：

```
mov eax, dword ptr [rdi + rsi*4 + 8]
```

它可能来自：

```
return p[i + 2];
```

也可能来自：

```
struct S {
    int a;
    int b;
    int c;
};
```

```
return p[i].c;
```

区别要靠上下文判断：`rdi` 是不是数组首地址？`rsi` 是不是原始下标？结构体大小是否为 12？前面是否有 `lea` 改变了 `index`？函数参数 ABI、调试信息、周围 `load/store` 和循环结构都要一起看。

心智模型

读汇编不是把每条指令机械翻译回一行 C++。更可靠的方法是先恢复数据流和地址公式，再结合类型、ABI、循环结构和访问宽度推断源码意图。

1.4.23 对象生命周期：有地址不等于有对象

底层程序员容易犯的另一个错误是把“有一块字节存储”当成“有一个对象”。C++ 区分存储和对象生命周期。

例如，分配一块原始存储：

```
alignas(int) unsigned char buffer[sizeof(int)];
```

这块字节存在，但其中还没有一个 `int` 对象。要在其中创建对象，需要 `placement new` 或符合标准的对象创建方式：

```
int* p = new (buffer) int(42);
```

在高性能代码中，自定义 `allocator`、`memory pool`、`arena`、`SIMD buffer` 和序列化反序列化都会碰到对象生命周期问题。初学阶段不要求你立刻掌握所有细节，但必须建立警觉：地址、字节、对象不是同一个概念。

1.4.24 数据布局如何进入性能：cache line 预告

CPU 不是每次只从内存拿你请求的那几个字节。缓存层级通常以 `cache line` 为单位搬运数据。现代 x86-64 常见 `cache line` 大小是 64 字节。

如果你顺序遍历 `int` 数组：

```
for (std::size_t i = 0; i < n; ++i) {
    s += a[i];
}
```

每个 `int` 4 字节，一个 64 字节 `cache line` 可以容纳 16 个 `int`。当 CPU 加载 `a[i]` 所在 `cache line` 后，后面的 `a[i+1]` 到 `a[i+15]` 很可能已经在 `cache` 中。这叫空间局部性。

如果你随机访问：

```
for (std::size_t i = 0; i < n; ++i) {
    s += a[index[i]];
}
```

每次访问可能落在不同 `cache line`。即使只需要 4 字节，也可能为它搬来 64 字节，造成带宽浪费和高延迟。

1.4.24.1 步长访问

步长访问是理解 `cache` 的第一个实验：

```
for (std::size_t i = 0; i < n; i += stride) {
    s += a[i];
}
```

当 `stride==1` 时，空间局部性最好。当 `stride` 很大时，每次访问都可能使用一个新 `cache line`。访问元素数看似减少了，但每个访问的 `cache line` 利用率也下降了。

1.4.25 AoS 与 SoA：布局决定 SIMD 和 cache 效率

看一个粒子数组：

```
struct Particle {
    float x;
    float y;
    float z;
    float mass;
};

Particle* particles;
```

这是 **AoS** (Array of Structs)：数组中的每个元素是一个结构体。如果只计算所有 `x` 的和：

```
float sx = 0.0f;
for (std::size_t i = 0; i < n; ++i) {
    sx += particles[i].x;
}
```

内存中 `x` 被 `y`、`z`、`mass` 间隔开。每次 cache line 搬入很多不需要的字段。
另一种布局是 **SoA** (Struct of Arrays):

```
struct Particles {
    float* x;
    float* y;
    float* z;
    float* mass;
};
```

此时所有 `x` 连续存放。对只读 `x` 的 kernel, SoA 更利于 cache 和 SIMD。

但如果每次都同时使用 `x,y,z,mass`, AoS 可能更直观, 也可能有不错局部性。真正的布局选择必须围绕访问模式, 而不是口号。

核心想

数据布局不是“代码风格”, 而是性能模型的一部分。一个 kernel 的访问模式决定它需要什么布局; 布局决定 cache line 利用率、SIMD load/store 形态、预取效果和多线程 false sharing 风险。

1.4.26 从布局回到汇编: `offset` 和 `scale` 是布局的影子

读反汇编时, 不要只盯着某一条 `mov`。你要把它前面的地址计算指令也一起看。下面几组模式非常常见。

1.4.26.1 模式 1: 纯数组, `scale` 直接等于元素大小

当你看到:

```
movss xmm0, dword ptr [rdi + rax*4]
```

它很可能表示读取一个连续 `float` 数组:

```
float x = arr[i];
```

解释如下:

汇编部分	含义	可能来源
<code>rdi</code>	基地址	<code>arr</code>
<code>rax</code>	下标	<code>i</code>
<code>*4</code>	元素大小	<code>sizeof(float)</code>
<code>dword ptr</code>	访问 4 字节	<code>float load</code>

1.4.26.2 模式 2: 结构体数组字段, 先算大步长

如果结构体大小是 16, x86 内存操作数不能直接写 `rax*16`。真实汇编更可能是:

```
shl    rax, 4
movss  xmm0, dword ptr [rdi + rax + 4]
```

它可能表示:

```
struct Particle {
    float x;
    float y;
    float z;
    float mass;
};

float y = particles[i].y;
```

推导过程是:

```
sizeof(Particle)    = 16
offset(Particle, y)  = 4
byte_offset_for_i    = i * 16
addr(particles[i].y) = base + i * 16 + 4
```

所以 `shl rax, 4` 把元素下标变成字节偏移, `+4` 是字段 `y` 的 `offset`。

1.4.26.3 模式 3: 用 `lea` 拼出非 2 的幂步长

当结构体大小是 24 时, 常见组合是:

```
lea    rcx, [rax + rax*2]
mov     edx, dword ptr [rdi + rcx*8 + 20]
```

这可能表示:

```
struct Item24 {
    double a;
    double b;
    int c;
    int d;
};

int d = items[i].d;
```

因为：

```
rcx = rax + rax * 2 = i * 3
effective_address = base + (i * 3) * 8 + 20
                  = base + i * 24 + 20
```

20 是字段 d 的 offset。这个例子训练的是：不要看到最后一条里只有 `rcx*8` 就误判结构体大小是 8；`rcx` 本身已经被前一条 `lea` 变成了 `i*3`。

1.4.26.4 模式 4：二维连续数组，行长进入地址公式

如果源码是：

```
int get_2d(int const a[3][5], std::size_t i, std::size_t j)
{
    return a[i][j];
}
```

一种可能汇编形态是：

```
lea eax, [rsi + rsi*4]
add rax, rdx
mov eax, dword ptr [rdi + rax*4]
ret
```

其中：

```
rax = i + i * 4 = i * 5
rax = i * 5 + j
address = base + (i * 5 + j) * 4
```

这说明每行 5 个 `int` 这个类型信息已经进入机器地址计算。如果是 `int**`，则通常会先 load 行指针，再访问行内元素，汇编形态完全不同。

1.4.26.5 反汇编阅读清单

看到一条内存操作时，按下面清单写笔记：

1. 这条指令是 load 还是 store?
2. 访问宽度是多少：byte、word、dword、qword、xmmword 还是 ymmword?
3. base 寄存器可能是哪一个指针或对象起点?
4. index 寄存器现在是元素下标，还是已经被 `shl/lea` 变成字节偏移?
5. scale 来自元素大小、字段数组大小，还是地址计算的中间步骤?
6. displacement 是结构体字段 offset、固定下标、栈槽偏移，还是全局对象偏移?
7. 如果有前置 `lea`，它把下标乘成了多少?
8. 这次访问落到一个 cache line 内，还是可能跨 cache line?

核心思想

offset 和 scale 是数据布局在汇编里的影子；`shl` 和 `lea` 则经常是“影子被折叠之前”的地址计算过程。读 AI 算子汇编时，先恢复地址公式，再讨论 SIMD、cache、带宽和流水线。

1.4.27 实验 3.1：打印地址并区分存储期

实验

创建 `labs/ch03_data_layout/address_map.cpp`：

```
#include <cstdlib>
#include <iostream>
#include <memory>

int global_x = 1;

int main()
{
    static int static_x = 2;
    int stack_x = 3;
    auto heap_x = std::make_unique<int>(4);
```

```
std::cout << "&global_x = " << &global_x << '\n';
std::cout << "&static_x = " << &static_x << '\n';
std::cout << "&stack_x = " << &stack_x << '\n';
std::cout << "heap_x = " << heap_x.get() << '\n';
```

```
int stack_arr[4] = {1, 2, 3, 4};
for (int i = 0; i < 4; ++i) {
    std::cout << "&stack_arr[" << i << "] = "
              << &stack_arr[i] << '\n';
}
```

编译运行：

```
clang++ -std=c++20 -O0 -g address_map.cpp -o address_map
./address_map
./address_map
```

交付物：

1. 记录两次运行的地址，说明哪些地址变化明显。
2. 解释栈数组相邻元素地址差值为什么等于 `sizeof(int)`。
3. 说明这些地址为什么是虚拟地址，不应直接理解为物理内存地址。

1.4.28 实验 3.2：对象字节、字节序和 `std::bit_cast`

实验

创建 `labs/ch03_data_layout/dump_bytes.cpp`，实现一个通用 `dump_bytes`：

```
#include <bit>
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <iostream>
#include <type_traits>

template <class T>
void dump_bytes(char const* name, T const& value)
{
    static_assert(std::is_trivially_copyable_v<T>);

    auto const* p = reinterpret_cast<unsigned char const*>(&value);
    std::cout << name << " (" << sizeof(T) << " bytes): ";
    for (std::size_t i = 0; i < sizeof(T); ++i) {
        std::cout << std::hex << std::setw(2) << std::setfill('0')
                  << static_cast<unsigned>(p[i]) << ' ';
    }
    std::cout << std::dec << '\n';
}

struct A {
    char c;
    int x;
};

int main()
{
    std::uint32_t u = 0x12345678U;
    float f = 1.0f;
    A a{'Z', 0x12345678};

    dump_bytes("u", u);
    dump_bytes("f", f);
    dump_bytes("a", a);

    auto f_bits = std::bit_cast<std::uint32_t>(f);
    dump_bytes("f_bits", f_bits);
}
```

交付物：

1. 标出 `u` 的小端字节序。
2. 解释 `A` 中哪些字节是字段，哪些可能是 padding。
3. 比较 `f` 和 `f_bits` 的输出，说明 `bit_cast` 的含义。
4. 用 `-O0` 和 `-O3` 都运行一次，观察输出是否变化。

1.4.29 实验 3.3: 手工预测结构体布局

实验

对下面结构体，先手工预测 `sizeof`、`alignof` 和每个字段 `offset`，再写程序验证：

```
struct S1 {
    char a;
    int b;
    double c;
    char d;
};

struct S2 {
    double c;
    int b;
    char a;
    char d;
};

struct S3 {
    char tag;
    float x;
    float y;
    float z;
};
```

验证程序必须使用：

```
sizeof(T)
alignof(T)
offsetof(T, field)
```

交付物：

1. 每个结构体的布局表。
2. 手工预测和实际输出的对比。
3. 解释字段重排为什么改变或没有改变 `sizeof`。
4. 说明如果这些结构体已经写入二进制文件，为什么不能随便重排字段。

1.4.30 实验 3.4: 指针加法和汇编寻址模式

实验

创建 `labs/ch03_data_layout/pointer_scale.cpp`。本实验不是只看一条 `load` 指令，而是训练你把源码、结构体布局、汇编寻址和具体地址模拟连起来。

```
#include <stddef>
#include <stdint>

extern "C" int load_int(int const* p, int i)
{
    return p[i];
}

extern "C" double load_double(double const* p, int i)
{
    return p[i];
}

struct Pair {
    int a;
    int b;
};

extern "C" int load_pair_b(Pair const* p, int i)
{
    return p[i].b;
}

struct Pixel {
    std::uint8_t r;
    std::uint8_t g;
    std::uint8_t b;
    std::uint8_t a;
};

extern "C" std::uint8_t load_pixel_b(Pixel const* p, std::size_t i)
{
    return p[i].b;
}
```

```
        return p[i].b;
    }

    struct Vec4 {
        float x;
        float y;
        float z;
        float w;
    };

    extern "C" float load_vec4_z(Vec4 const* p, std::size_t i)
    {
        return p[i].z;
    }

    extern "C" int load_2d(int const (*a)[5], std::size_t i, std::size_t j)
    {
        return a[i][j];
    }
}
```

生成汇编：

```
clang++ -std=c++20 -O3 -S -masm=intel pointer_scale.cpp -o pointer_scale.s
clang++ -std=c++20 -O3 -c pointer_scale.cpp -o pointer_scale.o
objdump -d -Mintel pointer_scale.o > pointer_scale.objdump.txt
```

如果系统上没有 GNU objdump，可以尝试：

```
llvm-objdump -d --x86-asm-syntax=intel pointer_scale.o > pointer_scale.objdump.txt
```

第一部分交付物：为每个函数写一张汇编阅读表。

函数	load 指令	base	index/scale	displacement
load_int	你填写	你填写	你填写	你填写
load_double	你填写	你填写	你填写	你填写
load_pair_b	你填写	你填写	你填写	你填写
load_pixel_b	你填写	你填写	你填写	你填写
load_vec4_z	你填写	你填写	你填写	你填写
load_2d	你填写	你填写	你填写	你填写

第二部分交付物：为每个函数写地址公式。格式必须像下面这样明确：

```
load_pair_b:
    sizeof(Pair) = 8
    offset(Pair, b) = 4
    addr(p[i].b) = base + i * 8 + 4

load_vec4_z:
    sizeof(Vec4) = 16
    offset(Vec4, z) = 8
    addr(p[i].z) = base + i * 16 + 8
```

第三部分交付物：选择三个函数做具体地址模拟。至少包含一个 scale 为 4 或 8 的例子、一个结构体字段例子、一个需要 shl 或 lea 的例子。

示例格式：

```
Function: load_pixel_b
Assume:
    rdi = 0x3000
    rsi = 5
Formula:
    addr(p[i].b) = 0x3000 + 5 * 4 + 2
                = 0x3016
Access:
    load 1 byte from 0x3016
```

第四部分交付物：记录目标文件中的机器码字节。报告里必须贴出每个函数的反汇编片段，例如：

```
0000000000000000 <load_int>:
0: 48 63 f6          movsxd rsi, esi
3: 8b 04 b7          mov  eax, DWORD PTR [rdi+rsi*4]
6: c3              ret
```

如果你的机器生成的指令不同，不要照抄本书示例。你要解释自己的编译器为什么可能选择不同指令，例如用 lea、movsxd、shl、不同寄存器或不同指令顺序。

最终报告必须回答：

- 1. 哪些函数的元素大小可以直接放进 scale？
- 2. 哪些函数需要额外地址计算？额外地址计算由哪条指令完成？

3. 哪些 displacement 来自字段 offset?
4. `movzx` 和普通 `mov` 在 `std::uint8_t` 返回值中有什么区别?
5. `movss` 和 `movsd` 分别对应什么标量浮点类型?
6. `load_2d` 的行长 5 如何进入汇编?
7. 修改 `Pair` 字段顺序或添加字段后, `sizeof(Pair)`、字段 offset 和汇编如何变化?

1.4.31 实验 3.5: AoS 与 SoA 的第一次 benchmark

实验

实现两个版本, 计算所有粒子 `x` 坐标之和。

```
struct Particle {
    float x;
    float y;
    float z;
    float mass;
};

float sum_x_aos(Particle const* p, std::size_t n)
{
    float s = 0.0f;
    for (std::size_t i = 0; i < n; ++i) {
        s += p[i].x;
    }
    return s;
}

float sum_x_soa(float const* x, std::size_t n)
{
    float s = 0.0f;
    for (std::size_t i = 0; i < n; ++i) {
        s += x[i];
    }
    return s;
}
```

要求:

1. 使用相同的 `x` 数据初始化 AoS 和 SoA。
2. 至少测试 `n=1024`、`n=1048576`、`n=67108864`。
3. 每个规模重复运行至少 20 次。
4. 防止结果被编译器优化掉。
5. 输出最小值、中位数、最大值。
6. 查看 `-O3` 汇编, 记录是否自动向量化。

报告必须回答:

1. 哪个版本更快? 是否所有规模都一样?
2. 从 cache line 利用率解释结果。
3. 从汇编解释编译器是否生成了不同访问形态。
4. 如果同时使用 `x, y, z, mass`, 你的结论是否可能改变?

1.4.32 本章作业

习题与作业

作业 1: 字节级对象报告

选择 5 个对象:

- 一个 `std::uint32_t`。
- 一个负的 `std::int32_t`。
- 一个 `float`。
- 一个含 padding 的结构体。
- 一个你自己设计的嵌套结构体。

对每个对象提交:

1. 源码定义。
2. `sizeof` 和 `alignof`。

3. 字节 dump。
4. 手工解释每个字段或位模式。
5. 哪些字节不应该被当作稳定语义值。

作业 2: 结构体布局优化设计

设计一个结构体表示简化版神经网络张量元数据，至少包含：

- 数据指针。
- 维度数量。
- 4 个维度。
- 4 个 stride。
- dtype。
- flags。
- 引用计数或状态字段。

提交两个版本：

1. 直觉字段顺序版本。
2. 经过布局推导优化后的版本。

要求给出每个版本的 `sizeof`、字段 offset、padding 分析，并解释你是为了减小大小、改善 cache 局部性，还是为了 ABI 可读性做取舍。

作业 3: 指针和地址推导题

假设：

```
struct P {
    float x;
    float y;
    float z;
    float w;
};

P* p = reinterpret_cast<P*>(0x1000);
```

在 `sizeof(P)==16` 的前提下，手工计算：

1. `&p[0].x`
2. `&p[0].z`
3. `&p[3]`
4. `&p[3].w`
5. `reinterpret_cast<char*>(p)+3`

然后写程序在真实数组上验证相对偏移。报告中必须明确区分元素偏移和字节偏移。

作业 4: 行为分类

判断下列行为属于 well-defined、implementation-defined、unspecified 还是 undefined，并解释原因：

1. 读取 `std::uint32_t` 的对象字节。
2. 有符号 `int` 溢出。
3. 无符号 `uint32_t` 溢出。
4. 访问 `a[n]`，其中数组合法下标为 `0..n-1`。
5. 用 `std::bit_cast<std::uint32_t>(float_value)` 读取位模式。
6. 用 `reinterpret_cast<double*>(&int_obj)` 后解引用。
7. 比较两个含 padding 的结构体对象的 `memcmp` 结果。
8. 打印指针值并观察 ASLR 导致每次运行不同。

作业 5: 小型性能报告

完成 AoS vs SoA benchmark。报告不少于 2000 字，必须包括：

- 机器信息。
- 编译器和参数。
- 数据规模和初始化方式。
- correctness 验证方式。

- benchmark 方法。
- 汇编证据。
- 时间结果表。
- 用 cache line 利用率解释的性能模型。
- 至少一个失败或反直觉观察。

1.4.33 验收标准

完成本章后，你应该能做到：

- 解释位、字节、对象表示、类型和值之间的关系。
- 读懂小端内存 dump。
- 区分无符号回绕、有符号补码表示和 C++ 有符号溢出 UB。
- 从 `a[i]` 推导出地址计算。
- 从 x86-64 内存操作数识别 base、index、scale、displacement。
- 手工推导简单结构体的 offset、padding、`sizeof` 和 `alignof`。
- 知道如何安全观察对象字节，什么时候应使用 `std::bit_cast` 或 `std::memcpy`。
- 解释别名和对象生命周期为什么影响编译器优化。
- 用 cache line 利用率初步解释连续访问、步长访问、随机访问、AoS 和 SoA 的性能差异。

如果你能做到这些，第 31 章的 cache、第 35 章的 AoS/SoA、第 37 章的 SIMD load/store、第 45 章的 GEMM blocking、第 50 章以后的 AI 算子布局优化，才会真正建立在可推导的基础上，而不是建立在经验口号上。

1.5 Linux、工具链、调试器和学习 workflow

1.5.1 为什么需要工具基础

底层学习不是只读概念。你必须能编译、反汇编、调试、测量、保存结果和复现实验。本章建立最低工具能力。

1.5.2 基本命令

必须熟悉：

- `g++`、`clang++`：编译。
- `objdump`：反汇编。
- `readelf`：查看 ELF。
- `nm`：查看符号。
- `gdb`：调试和单步。
- `lscpu`：查看 CPU。
- `time`、`perf`：基础测量。

1.5.3 构建类型

Debug、Release、sanitizer、coverage build 目的不同。Debug build 不用于性能结论；Release build 不替代 correctness 测试。

1.5.4 GDB 最小 workflow

```
g++ -O0 -g main.cpp -o app
gdb -q ./app
```

常用命令：

```
break main
run
next
step
stepi
info registers
x/10i $rip
disassemble /m main
```

1.5.5 实验记录

每次实验至少记录：

- 日期。
- CPU 和 OS。
- 编译器版本。
- 编译 flags。
- 输入规模。
- 命令。
- 原始输出。
- 结论。

1.5.6 习题

习题与作业

选择一个三行 C++ 函数，分别用 `g++-00-g` 和 `g++-03` 编译。用 GDB 单步 Debug 版本，用 `objdump` 阅读 Release 版本。写一页报告说明两者差异。

第 2 章 二进制、链接、进程与运行时系统

2.1 构建流程、编译参数和工具链心智模型

2.1.1 本节定位

本章把 C/C++ 的实际构建流程讲清楚：源文件如何被组织，编译命令如何影响语义和性能，CMake、编译数据库和构建类型在工程中承担什么角色。

2.1.2 核心问题

- 为什么同一份源码在不同 flags 下会生成完全不同的机器码？
- Debug、Release、RelWithDebInfo、sanitizer、coverage build 分别解决什么问题？
- 什么是 translation unit、include path、library path、compile command database？

2.1.3 必须讲清楚的底层机制

- 编译驱动程序如何调用预处理器、编译器、汇编器和链接器。
- 优化级别、目标 ISA、调试信息、异常/RTTI、LTO 对二进制的影响。
- 构建系统如何决定哪些文件重新编译，为什么大型工程需要明确依赖。

2.1.4 实验和交付物

实验

建立一个多文件 C++ 小工程，用 GCC 和 Clang 分别构建；生成 `compile_commands.json`；比较 `-O0`、`-O3`、`-march=native`、`-g` 的汇编和符号差异。

2.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

2.2 ELF、符号、重定位、链接和加载

2.2.1 本节定位

本章系统讲解 Linux x86-64 二进制文件格式和链接加载流程，为后续 ABI、动态库、profiling 和生产库阅读打基础。

2.2.2 核心问题

- ELF 文件为什么需要 header、section、segment？
- symbol、relocation、PLT、GOT 分别解决什么问题？
- 静态链接和动态链接的性能与工程取舍是什么？

2.2.3 必须讲清楚的底层机制

- 目标文件中未解析引用如何通过 relocation 延迟到链接期。
- 动态链接器如何加载共享库并解析符号。
- 函数调用经过 PLT/GOT 的路径以及 lazy binding 的基本模型。

2.2.4 实验和交付物

实验

用 `readelf-h/-S/-l/-r`、`nm-C`、`objdump-drwC` 分析一个含外部函数调用的程序；写静态库和共享库，对比符号和加载行为。

2.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

2.3 进程、虚拟内存和地址空间

2.3.1 本节定位

本章讲清楚用户态程序看到的地址空间如何由操作系统和硬件共同实现，解释为什么指针是虚拟地址，为什么 `page`、`TLB` 和 `mmap` 会影响性能。

2.3.2 核心问题

- 进程和线程的资源边界是什么？
- 虚拟地址如何转换成物理地址？
- `ASLR`、`page fault`、`mmap`、`copy-on-write` 对程序有什么影响？

2.3.3 必须讲清楚的底层机制

- 页表、`TLB`、`page fault` 的简化实现模型。
- 用户态/内核态切换和系统调用的边界。
- 地址空间布局与 `text/data/bss/heap/stack/mmap regions` 的关系。

2.3.4 实验和交付物

实验

打印不同对象地址；查看 `/proc/self/maps`；用大数组触发 `page fault`；比较首次访问和重复访问的时间差异。

2.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

2.4 栈、堆、运行时和对象生命周期

2.4.1 本节定位

本章把函数调用栈、局部变量、堆分配、全局构造、`RAII` 和运行时支持连接起来，为 `ABI` 和性能测量打基础。

2.4.2 核心问题

- 函数调用为什么需要栈？
- 栈帧里可能有哪些内容？
- `new/malloc` 为什么不能随便放在 `hot loop`？

2.4.3 必须讲清楚的底层机制

- call/ret、返回地址、保存寄存器、局部变量和栈对齐。
- allocator、free list、系统调用和线程缓存的粗略模型。
- 全局构造、析构、异常展开和运行时库。

2.4.4 实验和交付物

实验

用 GDB 观察 `rsp/rbp`；写递归和非递归函数比较栈增长；benchmark 循环内分配与预分配。

2.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

2.5 操作系统噪声、时间源和性能实验边界

2.5.1 本节定位

本章解释为什么测量不是简单计时：OS 调度、CPU 频率、虚拟化、page fault、interrupt 和后台任务都会改变结果。

2.5.2 核心问题

- wall time、CPU time、cycle counter 有什么差别？
- 为什么第一次运行常常更慢？
- WSL2、虚拟机和裸机 Linux 的测量边界在哪里？

2.5.3 必须讲清楚的底层机制

- scheduler、context switch、timer interrupt 的基本模型。
- TSC、clock source、`std::chrono` 的抽象边界。
- 频率 scaling、turbo、温度和功耗限制。

2.5.4 实验和交付物

实验

写一个重复测量程序，记录 min/median/max；比较首次访问和 warmup 后访问；记录环境信息并解释噪声来源。

2.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 3 章 x86-64 汇编、ABI 与 C++ 二进制接口

3.1 x86-64 汇编语法总览

3.1.1 本章要解决的问题

前面几章已经建立了三条基础线：

```
C++ source -> compiler -> assembly text -> machine code bytes
object/type/layout -> address formula -> memory operand
instruction bytes -> CPU state transition -> performance behavior
```

现在正式进入 x86-64 汇编。很多初学者第一次看到汇编会有两个相反误区：一种是觉得汇编只是“更丑的 C”；另一种是觉得汇编完全不可读。两者都不对。汇编不是 C，它更接近 ISA 暴露给软件的机器动作；但汇编也不是随机符号，只要掌握语法、寄存器、操作数、指令宽度和 ABI 上下文，就可以逐条推导。

本章目标不是把所有 x86 指令一次性背完，而是建立读汇编的基本方法。你完成本章后，应该能做到：

- 看懂一条 Intel 语法汇编指令的基本结构。
- 区分立即数、寄存器操作数、内存操作数。
- 识别通用寄存器及其 64/32/16/8 位别名。
- 理解 `byteptr`、`dwordptr`、`qwordptr` 等宽度标记。
- 从 `[base+index*scale+displacement]` 还原 C++ 数组和结构体访问。
- 用“读哪些状态、写哪些状态”的方式注释汇编。
- 比较 `-O0` 和 `-O3` 汇编为什么差异巨大。
- 知道 Intel 语法和 AT&T 语法的关键差异，避免读资料时混淆。

核心理想

读汇编的核心不是把每条指令机械翻译成一行 C++，而是恢复数据流、控制流和地址公式。C++ 源码只是汇编的一种可能来源；真正可靠的是逐条说明指令读写了哪些寄存器、内存和 flags。

3.1.2 先生成一份你能控制的汇编

学习汇编不能只看网上片段。你要从自己写的 C++ 生成汇编，知道编译命令、优化级别、目标语法和反汇编工具。先创建 `labs/ch10_assembly_syntax/minimal.cpp`：

```
extern "C" int add3(int x, int y, int z)
{
    return x + y + z;
}

extern "C" int load_i32(int const* p, int i)
{
    return p[i];
}
```

生成 Intel 语法汇编：

```
clang++ -std=c++20 -O3 -S -masm=intel minimal.cpp -o minimal.s
```

生成目标文件并反汇编：

```
clang++ -std=c++20 -O3 -c minimal.cpp -o minimal.o
objdump -d -Mintel minimal.o > minimal.objdump.txt
```

如果你的系统主要使用 LLVM 工具链，也可以用：

```
llvm-objdump -d --x86-asm-syntax=intel minimal.o > minimal.objdump.txt
```

`minimal.s` 是汇编器输入，通常包含标签、伪指令、调试/展开信息等。`minimal.objdump.txt` 是从目标文件机器码反汇编出来的文本，会显示机器码字节和指令地址。学习时两者都要看：

- `.s` 更接近编译器输出，适合看函数边界和汇编器伪指令。
- `objdump` 更接近最终二进制，适合看机器码字节、真实指令地址和链接前符号。

常见误区

不同编译器、版本、优化级别、目标 CPU、命令行参数都会生成不同汇编。本书的汇编是“合理示例”，不是要求你的机器逐字一致。学习重点是解释自己的输出。

3.1.3 一条汇编指令的基本结构

Intel 语法里，一条普通指令大致长这样：

mnemonic destination, source

例如：

```
mov eax, edi
add eax, esi
ret
```

这段代码可以来自：

```
extern "C" int add2(int x, int y)
{
    return x + y;
}
```

在 System V AMD64 ABI 下，前两个 `int` 参数在 `edi` 和 `esi`，返回值在 `eax`。逐条解释：

指令	读什么	写什么
<code>moveax,edi</code>	<code>edi</code>	<code>eax</code> ，同时清零 <code>rax</code> 高 32 位
<code>addeax,esi</code>	<code>eax</code> 、 <code>esi</code>	<code>eax</code> 和部分 flags
<code>ret</code>	栈顶返回地址、 <code>rsp</code>	<code>rip</code> 、 <code>rsp</code>

这里有一个初学阶段必须接受的事实：`add eax, esi` 的目的操作数既是输入也是输出。它不是“产生一个新变量”，而是把结果写回 `eax`：

```
eax = eax + esi
```

因此读汇编时，你应该维护一个“当前每个寄存器代表什么”的笔记。比如：

```
before:
    edi = x
    esi = y

mov eax, edi
    eax = x

add eax, esi
    eax = x + y

ret
    return eax
```

3.1.4 操作数：立即数、寄存器和内存

x86-64 指令的操作数常见三类：立即数、寄存器、内存。

3.1.4.1 立即数

immediate（立即数）是编码在指令里的常量：

```
mov eax, 42
add rdi, 16
and eax, 255
```

立即数不是内存地址，也不需要额外 `load`。它是机器码字节的一部分。比如 `addrdi,16` 中的 `16` 会被编码进指令。

3.1.4.2 寄存器操作数

寄存器操作数直接读写 CPU 的架构寄存器：

```
mov rax, rdi
add eax, esi
xor ecx, ecx
```

寄存器访问通常比内存访问快得多，但寄存器数量有限。编译器优化很大一部分工作，就是尽量把热数据保存在寄存器中，减少 `load/store`。

3.1.4.3 内存操作数

方括号表示内存操作数：

```
mov eax, dword ptr [rdi]
mov eax, dword ptr [rdi + rsi*4]
mov qword ptr [rsp + 8], rax
```

方括号里的表达式计算地址；`byteptr`、`dwordptr`、`qwordptr` 表示访问宽度。注意：

```
mov eax, edi        ; register -> register
mov eax, [rdi]      ; memory at address rdi -> register
```

少一个方括号，语义完全不同。第一条把 `edi` 的值复制到 `eax`；第二条把 `rdi` 当作地址，从该地址读取内存。

3.1.4.4 一般不能内存到内存

多数普通 x86 指令不能同时有两个内存操作数。例如下面这种通常不合法：

```
mov dword ptr [rdi], dword ptr [rsi] ; invalid for ordinary mov
```

需要用寄存器中转：

```
mov eax, dword ptr [rsi]
mov dword ptr [rdi], eax
```

这对性能分析很重要。源码里看起来像一次赋值，机器层面可能是一次 load 加一次 store。

3.1.5 通用寄存器和别名

x86-64 有 16 个常用通用整数寄存器。每个寄存器可以用不同宽度的名字访问低位部分：

64 位	32 位	16 位	8 位低位	常见 ABI 含义
<code>rax</code>	<code>eax</code>	<code>ax</code>	<code>al</code>	返回值、临时
<code>rbx</code>	<code>ebx</code>	<code>bx</code>	<code>bl</code>	callee-saved
<code>rcx</code>	<code>ecx</code>	<code>cx</code>	<code>cl</code>	第 4 参数、移位计数
<code>rdx</code>	<code>edx</code>	<code>dx</code>	<code>dl</code>	第 3 参数、乘除辅助
<code>rsi</code>	<code>esi</code>	<code>si</code>	<code>sil</code>	第 2 参数
<code>rdi</code>	<code>edi</code>	<code>di</code>	<code>dil</code>	第 1 参数
<code>rbp</code>	<code>ebp</code>	<code>bp</code>	<code>bpl</code>	帧指针或 callee-saved
<code>rsp</code>	<code>esp</code>	<code>sp</code>	<code>spl</code>	栈指针

`r8` 到 `r15` 也有类似名字：

64 位	32 位	16 位	8 位
<code>r8</code>	<code>r8d</code>	<code>r8w</code>	<code>r8b</code>
<code>r9</code>	<code>r9d</code>	<code>r9w</code>	<code>r9b</code>
<code>r10</code>	<code>r10d</code>	<code>r10w</code>	<code>r10b</code>
<code>r11</code>	<code>r11d</code>	<code>r11w</code>	<code>r11b</code>
<code>r12</code>	<code>r12d</code>	<code>r12w</code>	<code>r12b</code>
<code>r13</code>	<code>r13d</code>	<code>r13w</code>	<code>r13b</code>
<code>r14</code>	<code>r14d</code>	<code>r14w</code>	<code>r14b</code>
<code>r15</code>	<code>r15d</code>	<code>r15w</code>	<code>r15b</code>

3.1.5.1 32 位写入会清零高 32 位

x86-64 一个非常重要的规则是：写 32 位寄存器会把对应 64 位寄存器高 32 位清零。

```
mov eax, edi
```

这条指令写 `eax`，也会让 `rax` 的高 32 位变成 0。可以这样理解：

```
rax = zero_extend_64(edi)
```

但写 16 位或 8 位寄存器不会自动清零高位：

```
mov ax, di ; only low 16 bits are written
mov al, dil ; only low 8 bits are written
```

因此部分寄存器写入可能保留旧值的高位。现代编译器通常更喜欢用 32 位写入来打破不必要的旧值依赖。

常见误区

不要把 `eax`、`ax`、`al` 当成完全独立寄存器。它们是 `rax` 的不同宽度视图。读汇编时必须知道一条指令到底写了多少位。

3.1.6 操作数宽度：机器到底读写几个字节

Intel 语法中，访问内存时常用宽度标记：

标记	字节数	常见 C++ 类型或用途
byteptr	1	char、std::uint8_t
wordptr	2	std::uint16_t、x86 word
dwordptr	4	int、float、std::uint32_t
qwordptr	8	long、指针、double、std::uint64_t
xmmwordptr	16	128-bit SIMD memory operand
ymmwordptr	32	256-bit AVX memory operand

例如：

```
movzx eax, byte ptr [rdi]
mov    eax, dword ptr [rdi]
mov    rax, qword ptr [rdi]
movss xmm0, dword ptr [rdi]
movsd xmm0, qword ptr [rdi]
```

这些指令即使地址相同，访问宽度和解释规则也不同：

- movzx eax, byte ptr [rdi] 读取 1 字节并零扩展。
- mov eax, dword ptr [rdi] 读取 4 字节整数到 eax。
- mov rax, qword ptr [rdi] 读取 8 字节整数或指针。
- movss xmm0, dword ptr [rdi] 读取单精度浮点标量。
- movsd xmm0, qword ptr [rdi] 读取双精度浮点标量。

核心思想

内存地址只说明从哪里开始，操作数宽度说明读写多少字节，指令语义和寄存器类型说明如何解释这些字节。三者缺一不可。

3.1.7 内存寻址模式：地址公式如何写进指令

x86-64 最常见的内存操作数形式是：

```
[base + index * scale + displacement]
```

其中 **scale** 只能是 1、2、4、8。不是所有部分都必须出现：

形式	含义
[rdi]	地址是 rdi
[rdi+8]	地址是 rdi 加 8 字节
[rdi+rsi*4]	地址是 rdi 加 rsi 乘 4
[rdi+rsi*4+12]	基址、索引缩放、常量偏移同时出现
[rip+symbol]	RIP-relative，常用于全局对象或常量池

3.1.7.1 案例：数组 load

C++：

```
extern "C" int load_i32(int const* p, int i)
{
    return p[i];
}
```

可能汇编：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4]
ret
```

假设执行前：

寄存器	值	含义
rdi	0x1000	p[0] 地址
esi	3	下标 i

执行：

```
movsxd rsi, esi
rsi = sign_extend_64(3)
    = 3
```

```
mov eax, dword ptr [rdi + rsi*4]
effective_address = 0x1000 + 3 * 4
                  = 0x100c
load 4 bytes from 0x100c..0x100f into eax
```

3.1.7.2 案例：结构体字段

C++:

```
struct S {
    int a;
    double b;
    int c;
};

extern "C" int get_c(S const* p)
{
    return p->c;
}
```

常见布局:

字节范围	内容	大小	说明
0..3	a	4	字段
4..7	padding	4	让 b 8 字节对齐
8..15	b	8	字段
16..19	c	4	字段
20..23	tail padding	4	让 sizeof(S) 为 8 的倍数

可能汇编:

```
mov eax, dword ptr [rdi + 16]
ret
```

这里 +16 不是魔法数字, 而是 offset(S,c):

```
addr(p->c) = addr(*p) + offset(S, c)
           = rdi      + 16
```

3.1.7.3 案例：结构体数组字段

C++:

```
struct Pair {
    int a;
    int b;
};

extern "C" int get_b(Pair const* p, int i)
{
    return p[i].b;
}
```

公式:

```
sizeof(Pair)    = 8
offset(Pair, b) = 4
addr(p[i].b)    = base + i * 8 + 4
```

可能汇编:

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*8 + 4]
ret
```

如果 rdi 是 0x2000, rsi 是 5:

```
effective_address = 0x2000 + 5 * 8 + 4
                  = 0x202c
load 4 bytes from 0x202c..0x202f
```

3.1.8 lea: 看起来像内存, 实际不访问内存

lea 是学习 x86 汇编必须尽早掌握的指令。它的名字是 load effective address, 但它只计算地址表达式, 不读取内存。

```
lea rax, [rdi + rsi*4 + 8]
```

语义是:

```
rax = rdi + rsi * 4 + 8
```

不是：

```
rax = memory[rdi + rsi * 4 + 8]
```

对比：

```
lea rax, [rdi + rsi*4 + 8] ; compute address-like integer
mov eax, dword ptr [rdi + rsi*4 + 8] ; load 4 bytes from memory
```

编译器经常用 `lea` 做整数乘法常数：

```
lea eax, [rdi + rdi*2] ; eax = x * 3
lea eax, [rdi + rdi*4] ; eax = x * 5
lea eax, [rdi*8 + 16] ; eax = x * 8 + 16
```

这解释了为什么你在非指针代码里也会看到 `lea`：

```
extern "C" int mul5_add1(int x)
{
    return x * 5 + 1;
}
```

可能生成：

```
lea eax, [rdi + rdi*4 + 1]
ret
```

心智模型

看到 `lea` 时先问：它的结果后面被当作地址使用，还是只是整数计算结果？`lea` 本身不产生 `cache miss`，因为它不访问内存。

3.1.9 flags：有些指令会留下条件信息

x86 有 `RFLAGS/EFLAGS`。很多整数指令会更新 `flags`，例如 `add`、`sub`、`cmp`、`test`。后续条件跳转或条件移动会读取这些 `flags`。

本章先只建立直觉：

flag	名称	常见用途
ZF	zero flag	结果是否为 0，相等比较
SF	sign flag	结果最高位是否为 1
CF	carry flag	无符号进位/借位
OF	overflow flag	有符号溢出

例如：

```
cmp edi, esi
setg al
```

`cmp edi, esi` 本质上计算 `edi-esi` 来设置 `flags`，但不保存减法结果。`setg al` 根据 `flags` 把 `al` 设置为 0 或 1。第 12 章会系统讲控制流和条件码；这里只要记住：读汇编时，有些数据依赖不在通用寄存器里，而在 `flags` 里。

3.1.10 标签、函数和汇编器伪指令

看一段 `clang++-S` 生成的汇编，除了真正的机器指令，还会看到伪指令：

```
.text
.globl add2
.type add2,@function
add2:
mov    eax, edi
add    eax, esi
ret
.size  add2, .-add2
```

常见项：

写法	含义
<code>.text</code>	后面进入代码段
<code>.globladd2</code>	符号 <code>add2</code> 对链接器可见
<code>.typeadd2,@function</code>	标记符号类型是函数
<code>add2:</code>	标签，代表当前位置地址
<code>.sizeadd2, .-add2</code>	记录函数大小

这些伪指令不是 CPU 执行的指令。它们给汇编器、链接器、调试器和异常展开机制使用。后续讲 ELF、链接、ABI 和 unwinding 时会展开。

3.1.11 Intel 语法和 AT&T 语法

本书默认使用 Intel 语法，因为它和 Intel/AMD 手册、Compiler Explorer 的 Intel 输出、很多性能资料更接近。你仍然必须认识 AT&T 语法，因为 GNU 工具链和很多历史资料会用它。

同一条指令：

```
mov eax, dword ptr [rdi + rsi*4]    ; Intel
movl (%rdi,%rsi,4), %eax          ; AT&T
```

关键差异：

差异	Intel	AT&T
操作数顺序	destination, source	source, destination
寄存器写法	rax	%rax
立即数写法	42	\protect\TU\textdollar42
内存写法	[rdi+rsi*4]	(%rdi,%rsi,4)
宽度表示	dwordptr	指令后缀 movl

再看几个对应关系：

```
Intel: mov rax, rdi
AT&T : movq %rdi, %rax

Intel: add eax, esi
AT&T : addl %esi, %eax

Intel: mov qword ptr [rsp + 8], rax
AT&T : movq %rax, 8(%rsp)
```

常见误区

最容易错的是操作数顺序。Intel 是 dst,src，AT&T 是 src,dst。读资料时先确认语法，否则会把数据流方向看反。

3.1.12 -O0 和 -O3 为什么差异巨大

同一个 C++ 函数，在不同优化级别下可以完全不像同一个程序。看：

```
extern "C" int f(int x)
{
    int y = x + 1;
    int z = y * 3;
    return z - 2;
}
```

-O0 的目标是容易调试，不是快。它可能生成栈帧，把局部变量放在栈上：

```
push rbp
mov rbp, rsp
mov dword ptr [rbp - 4], edi
mov eax, dword ptr [rbp - 4]
add eax, 1
mov dword ptr [rbp - 8], eax
mov eax, dword ptr [rbp - 8]
imul eax, eax, 3
mov dword ptr [rbp - 12], eax
mov eax, dword ptr [rbp - 12]
sub eax, 2
pop rbp
ret
```

-O3 会做代数化简和寄存器分配，可能变成：

```
lea eax, [rdi + rdi*2 + 1]
ret
```

因为：

```
y = x + 1
z = y * 3 = 3*x + 3
return z - 2 = 3*x + 1
```

这个案例说明：

- O0 适合观察源代码结构、栈帧和调试信息。

- -03 适合观察真实性能路径。
- 性能优化不能根据 -00 汇编下结论。
- -03 汇编可能已经和源码结构差很多，需要恢复语义而不是逐句对应。

3.1.13 读汇编的固定流程

以后每次读一个函数，按下面流程做笔记：

1. 写出函数签名和 ABI 参数位置。
2. 标出每个入口寄存器的含义，例如 `rdi` 是第一个指针，`esi` 是第二个 `int`。
3. 从上到下维护寄存器含义表。
4. 对每条内存操作写出有效地址公式。
5. 标出访问宽度和读写字节范围。
6. 标出哪些指令更新 flags，后面哪里读取 flags。
7. 区分真正 load/store 和纯地址计算 `lea`。
8. 最后再尝试恢复 C-like 伪代码。

例如：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4 + 8]
add    eax, 1
ret
```

笔记应该像这样：

```
entry:
    rdi = base pointer p
    esi = int index i

movsxd rsi, esi
    rsi = sign_extend_64(i)

mov    eax, dword ptr [rdi + rsi*4 + 8]
    address = p + i * 4 + 8
    load width = 4 bytes
    possible source = p[i + 2] if p is int*

add    eax, 1
    eax = loaded_value + 1

ret
    return eax
```

这种笔记比“这大概是数组访问”更有价值。高性能工程里，证据链必须精确到地址公式和访问宽度。

3.1.14 案例：从 C++ 到汇编逐条解释

创建函数：

```
struct Item {
    int id;
    float score;
    int count;
};

extern "C" int update(Item* items, int i, int delta)
{
    items[i].count += delta;
    return items[i].count;
}
```

常见布局：

字段	offset	大小	说明
<code>id</code>	0	4	<code>int</code>
<code>score</code>	4	4	<code>float</code>
<code>count</code>	8	4	<code>int</code>

`sizeof(Item)` 通常是 12。因为 x86 scale 没有 12，编译器可能使用 `lea` 生成 `i*3`，再乘 4：

```
movsxd rax, esi
lea    rax, [rax + rax*2]
add    dword ptr [rdi + rax*4 + 8], edx
mov    eax, dword ptr [rdi + rax*4 + 8]
ret
```

逐条解释：

```

entry:
    rdi = items
    esi = i
    edx = delta

    movsxd rax, esi
    rax = sign_extend_64(i)

    lea rax, [rax + rax*2]
    rax = i * 3

    add dword ptr [rdi + rax*4 + 8], edx
    address = items + (i * 3) * 4 + 8
    = items + i * 12 + 8
    = addr(items[i].count)
    load old 4-byte count
    add delta
    store new 4-byte count
    update flags

    mov eax, dword ptr [rdi + rax*4 + 8]
    reload 4-byte count into eax as return value

ret

```

为什么第二条 `mov` 要 reload? 因为 `add dword ptr [mem], edx` 是 read-modify-write 内存指令，它把结果写回内存，但不把结果写入 `eax`。返回值需要在 `eax`，所以还要再 load。编译器也可能生成不同形式，比如先 load 到寄存器、加、store、再 return 寄存器。你要解释自己的输出。

深入理解

read-modify-write 指令表面是一条汇编，但微架构内部通常涉及 load、ALU、store，并会占用 load/store 相关资源。后面讲 uops、ports、store buffer 和原子指令时，这一点会变得非常重要。

3.1.15 实验 10.1：建立第一份汇编阅读报告

实验

创建 `labs/ch10_assembly_syntax/basic_reading.cpp`：

```

#include <cstdint>
#include <cstdio>

extern "C" int add2(int x, int y)
{
    return x + y;
}

extern "C" int mix(int x)
{
    return x * 5 + 17;
}

extern "C" std::uint8_t load_u8(std::uint8_t const* p, std::size_t i)
{
    return p[i];
}

extern "C" int load_i32(int const* p, int i)
{
    return p[i + 2];
}

```

生成汇编和反汇编：

```

clang++ -std=c++20 -O0 -S -masm=intel basic_reading.cpp -o basic_reading_00.s
clang++ -std=c++20 -O3 -S -masm=intel basic_reading.cpp -o basic_reading_03.s
clang++ -std=c++20 -O3 -c basic_reading.cpp -o basic_reading.o
objdump -d -Mintel basic_reading.o > basic_reading.objdump.txt

```

交付物：

1. 对每个函数列出 ABI 参数寄存器和返回值寄存器。
2. 从 `basic_reading_03.s` 中摘出每个函数的核心指令。
3. 对每条核心指令写“读什么、写什么、是否访问内存、是否更新 flags”。
4. 对 `load_u8` 解释为什么可能出现 `movzx`。
5. 对 `mix` 解释编译器是否使用 `lea`。

6. 对 `load_i32` 写出地址公式，并说明 `+8` 从哪里来。
7. 从 `objdump` 中记录机器码字节，说明汇编文本和机器码字节的关系。

3.1.16 实验 10.2: Intel 和 AT&T 语法互译

实验

用同一个源码生成两种语法：

```
clang++ -std=c++20 -O3 -S -masm=intel basic_reading.cpp -o intel.s
clang++ -std=c++20 -O3 -S basic_reading.cpp -o att.s
```

交付物：

1. 从两个文件中各选 8 条对应指令。
2. 写成三列表：Intel、AT&T、语义。
3. 标出操作数顺序差异。
4. 标出内存寻址写法差异。
5. 标出宽度写法差异，例如 `dwordptr` 和 `movl`。

报告中必须包含下面这种格式：

Intel	AT&T	语义
<code>moveax,edi</code>	<code>movl%edi,%eax</code>	<code>eax=edi</code>
<code>addeax,esi</code>	<code>addl%esi,%eax</code>	<code>eax+=esi</code>

3.1.17 实验 10.3: -O0 和 -O3 对比

实验

创建 `labs/ch10_assembly_syntax/o0_o3_compare.cpp`：

```
extern "C" int formula(int x)
{
    int a = x + 1;
    int b = a * 3;
    int c = b - x;
    return c + 7;
}
```

生成：

```
clang++ -std=c++20 -O0 -S -masm=intel o0_o3_compare.cpp -o formula_00.s
clang++ -std=c++20 -O3 -S -masm=intel o0_o3_compare.cpp -o formula_03.s
```

交付物：

1. 注释 `-O0` 中的栈帧建立、局部变量栈槽和返回路径。
2. 注释 `-O3` 中的代数化简结果。
3. 用数学式证明 `formula(x)` 可以化简成什么。
4. 解释为什么不能用 `-O0` 汇编评估性能。
5. 说明 `-O0` 对调试有什么价值。

3.1.18 本章作业

习题与作业

作业 1: 逐条注释 20 条汇编

从本章实验生成的 `-O3` 汇编中选择至少 20 条真实指令。每条指令必须写：

- 指令文本。
- 操作数类型：立即数、寄存器、内存。
- 读取的寄存器或内存。
- 写入的寄存器或内存。
- 访问宽度。
- 是否更新 `flags`。
- 你认为它对应的 C++ 语义。

作业 2：地址公式恢复

对下面汇编片段恢复可能的地址公式和 C-like 伪代码：

```

movsxd rax, esi
mov     ecx, dword ptr [rdi + rax*4 + 12]
add     ecx, edx
mov     dword ptr [rdi + rax*4 + 12], ecx
mov     eax, ecx
ret

```

要求至少给出两种可能来源：

1. `int*` 数组访问版本。
2. 结构体数组字段访问版本。

每种版本都必须解释 +12 的来源。

作业 3：语法互译

把下面 Intel 语法翻译成 AT&T 语法，并写出语义：

```

mov rax, rdi
add eax, esi
movzx eax, byte ptr [rdi + rsi]
mov qword ptr [rsp + 8], rax
lea rax, [rdi + rsi*8 + 16]

```

再把下面 AT&T 语法翻译成 Intel 语法：

```

movl %edi, %eax
addl %esi, %eax
movzbl (%rdi,%rsi), %eax
movq %rax, 8(%rsp)
leaq 16(%rdi,%rsi,8), %rax

```

作业 4：机器码和指令长度

用 `objdump` 记录至少 10 条指令的机器码字节。回答：

1. 哪些指令是 1 字节？
2. 哪些指令长度超过 4 字节？
3. 立即数和 displacement 如何让指令变长？
4. 为什么 x86 前端需要处理变长指令？

作业 5：小型汇编阅读报告

选择一个你自己写的 10 到 20 行 C++ 函数，要求包含：

- 至少一个数组访问。
- 至少一个结构体字段访问。
- 至少一个整数表达式。
- 至少一个条件判断或比较。

生成 -O0 和 -O3 汇编。提交不少于 2000 字报告，必须包含：

- 源码。
- 编译命令。
- 函数签名和 ABI 参数寄存器。
- -O0 和 -O3 汇编对比。
- 至少 15 条指令逐条注释。
- 所有内存访问的地址公式。
- 至少 5 条机器码字节记录。
- 你遇到的一个误判，以及如何纠正。

3.1.19 验收标准

完成本章后，你应该能做到：

- 熟练区分 Intel 和 AT&T 语法。
- 识别立即数、寄存器和内存操作数。
- 解释通用寄存器别名和 32 位写入清零高位。

- 从内存操作数写出有效地址公式。
- 解释 `byteptr`、`dwordptr`、`qwordptr`、`xmmwordptr` 的访问宽度。
- 解释 `lea` 为什么不访问内存。
- 对简单函数生成 `-00` 和 `-03` 汇编，并逐条注释核心指令。
- 从反汇编中记录机器码字节，知道汇编文本最终对应指令编码。
- 不再把 `-00` 汇编当作性能路径。

如果这些能力稳定，第 12 章的整数指令、第 13 章的控制流、第 14 章的 ABI、第 24 章的自动向量化和第 48 章的 AVX2 microkernel 才能真正读得进去。

3.2 整数指令、位运算和地址计算

3.2.1 本章要解决的问题

上一章建立了读 x86-64 汇编的基本语法：指令、寄存器、立即数、内存操作数、操作数宽度和 `lea`。本章继续深入最常见的一组指令：整数数据移动、加减、乘法、符号扩展、零扩展、移位、位运算、比较和地址计算。

这些指令看起来简单，但它们是后面所有内容的底层材料：

- 数组下标和结构体字段访问需要整数地址计算。
- 循环计数、边界判断、指针递增都依赖加减和比较。
- 类型转换会生成符号扩展或零扩展。
- 对齐、掩码、量化、哈希、SIMD lane 处理会大量使用位运算。
- 性能优化里经常要判断一条指令是否更新 `flags`、是否破坏依赖、是否可被 `lea` 替代。

本章不要求你背完 Intel 手册。目标是让你看到一个整数汇编片段时，能回答：

- 1. 每条指令读哪些寄存器、内存和 `flags`？
- 2. 每条指令写哪些寄存器、内存和 `flags`？
- 3. 操作数宽度是多少？
- 4. 这是有符号语义、无符号语义，还是纯位模式语义？
- 5. 如果它来自 C++，语言规则是否允许这种变换？
- 6. 它对性能有什么直接影响？

核心思想

整数指令处理的是位模式。signed/unsigned 很多时候不是硬件存了不同东西，而是某条指令如何解释这些位、编译器如何根据 C++ 类型选择指令、语言规则允许编译器做哪些假设。

3.2.2 整数寄存器里的值：位模式先于解释

先看一个 32 位寄存器：

`eax = 0xffffffff`

这个位模式可以解释成：

解释方式	值	说明
<code>std::uint32_t</code>	4294967295	无符号解释
<code>std::int32_t</code>	-1	二进制补码解释
位掩码	所有 32 位都是 1	mask 解释

寄存器本身没有贴着“这是 signed int”的标签。不同指令会以不同方式解释这些位：

- `add` 对低 N 位加法而言，有符号和无符号使用同一个加法器；区别主要体现在 `flags` 如何解释。
- `cmp` 后接有符号条件跳转和无符号条件跳转时，读取的 `flags` 组合不同。
- `movsx` 和 `movzx` 明确区分符号扩展和零扩展。
- `sar` 是算术右移，保留符号位；`shr` 是逻辑右移，高位补 0。

常见误区

不要说“这个寄存器是 signed”。更准确的说法是：“这段代码把这个寄存器的位模式按 signed 语义使用”。寄存器存的是位，解释来自指令、类型和上下文。

3.2.3 `mov`：复制位模式，不是高级赋值

`mov` 是最常见的指令之一，但它的名字容易误导。它不是“移动后源消失”，而是复制位模式：

`mov eax, edi`

语义：

```
eax = edi
rax high 32 bits = 0
```

因为写 32 位寄存器会清零高 32 位。

3.2.3.1 寄存器到寄存器

C++:

```
extern "C" int id(int x)
{
    return x;
}
```

可能汇编：

```
mov eax, edi
ret
```

edi 是参数，eax 是返回值。这里没有内存访问。

3.2.3.2 内存到寄存器：load

C++:

```
extern "C" int load(int const* p)
{
    return *p;
}
```

可能汇编：

```
mov eax, dword ptr [rdi]
ret
```

解释：

```
address = rdi
load 4 bytes from address
write eax
```

3.2.3.3 寄存器到内存：store

C++:

```
extern "C" void store(int* p, int x)
{
    *p = x;
}
```

可能汇编：

```
mov dword ptr [rdi], esi
ret
```

解释：

```
address = rdi
store low 32 bits of esi to address
```

3.2.3.4 立即数到寄存器或内存

```
mov eax, 123
mov dword ptr [rdi], 123
```

第一条把立即数写入寄存器；第二条把立即数写入内存。第二条是 store，会修改内存。

3.2.4 清零寄存器：xor reg, reg 和 mov reg, 0

你经常看到：

```
xor eax, eax
```

它把 `eax` 清零。因为任何位和自己异或都是 0：

```
x ^ x = 0
```

也可以写：

```
mov eax, 0
```

但编译器常用 `xor eax, eax`，原因包括编码短、CPU 识别 zero idiom、可打破旧值依赖等。注意 `xor` 会更新 flags，而 `mov` 通常不更新 flags。上下文不同，编译器选择也可能不同。

深入理解

现代 x86 核心通常能识别 `xor reg, reg`、`sub reg, reg` 这类 zero idiom，把它们当成不依赖旧寄存器值的清零操作。依赖打断对乱序执行和寄存器重命名很重要，后面讲微架构时会展开。

3.2.5 加法和减法：结果、flags 和语言规则

3.2.5.1 `add` 和 `sub`

C++:

```
extern "C" int add_i32(int x, int y)
{
    return x + y;
}

extern "C" int sub_i32(int x, int y)
{
    return x - y;
}
```

可能汇编:

```
add_i32:
    lea eax, [rdi + rsi]
    ret

sub_i32:
    mov eax, edi
    sub eax, esi
    ret
```

`add` 和 `sub` 都会更新 flags。编译器有时用 `lea` 做加法，因为 `lea` 不更新 flags:

```
lea eax, [rdi + rsi]    ; eax = x + y, flags unchanged
add eax, esi            ; eax = eax + y, flags updated
```

如果后续不需要 flags，`lea` 可能更合适。如果后续需要根据结果跳转，`add` 或 `sub` 设置 flags 就有价值。

3.2.5.2 有符号溢出和无符号回绕

机器的 32 位加法天然按低 32 位结果回绕。但 C++ 语言规则不同:

C++ 表达式	溢出规则
<code>std::uint32_t+a</code>	定义良好，按模 2^{32} 回绕
<code>int+a</code>	有符号溢出是 undefined behavior

所以同样可能生成一条 `add`，编译器对 signed 和 unsigned 源码能做的优化假设并不一样。不要把“机器会回绕”直接等同为“C++ 程序定义良好”。

3.2.5.3 `inc` 和 `dec`

`inc` 加 1，`dec` 减 1:

```
inc eax
dec ecx
```

它们会更新大多数 flags，但不更新 CF。现代编译器在一些场景下更偏向 `add reg, 1` 或 `sub reg, 1`，因为 flags 行为更规则，也可能更利于后续优化。你看到哪种要按上下文解释，不要死记“循环一定用 `inc`”。

3.2.6 乘法：`imul`、常数乘法和 `lea`

3.2.6.1 常数乘法经常不用乘法指令

C++:

```
extern "C" int mul5(int x)
{
    return x * 5;
}
```

可能汇编:

```
lea eax, [rdi + rdi*4]
ret
```

因为:


```
x * 5 = x + x * 4
```

lea 可表达 base+index*scale+displacement，其中 scale 为 1、2、4、8。因此乘以 3、5、9 等常数经常可用一条 lea。

C++ 表达式	可能汇编	语义
x*3	leaeax,[rdi+rdi*2]	x+2*x
x*5	leaeax,[rdi+rdi*4]	x+4*x
x*8+7	leaeax,[rdi*8+7]	8*x+7

3.2.6.2 imul 的常见形式

当乘法不能简单用 lea 或 shift 表达时，会看到 imul：

```
imul eax, edi, 37      ; eax = edi * 37
imul eax, esi          ; eax = eax * esi, low 32 bits
```

对低 N 位乘法结果来说，补码 signed 和 unsigned 的低 N 位相同。因此很多普通乘法可以使用同一类指令。但当需要完整宽度结果、高半部分、溢出判断时，signed/unsigned 就会影响指令选择。

3.2.6.3 性能直觉

整数乘法通常比简单加法、位运算、lea 延迟更高、吞吐更低。现代 CPU 乘法已经很快，但在 tight loop 里仍然要关注。如果乘以常数，编译器经常自动替换成 lea、shift 或加法组合。不要手写“聪明替换”之前先看编译器输出。

3.2.7 符号扩展和零扩展：movsx、movzx、movsxd

从小整数类型转换到大整数类型时，编译器必须决定高位如何填充。

3.2.7.1 零扩展

C++：

```
#include <cstdint>

extern "C" int from_u8(std::uint8_t x)
{
    return x;
}
```

std::uint8_t 到 int 要保留 0..255 的值，因此高位补 0。可能汇编：

```
movzx eax, dil
ret
```

语义：

```
eax = zero_extend_32(dil)
```

如果是从内存读取：

```
movzx eax, byte ptr [rdi]
```

表示读取 1 字节，再零扩展到 32 位。

3.2.7.2 符号扩展

C++：

```
#include <cstdint>

extern "C" int from_i8(std::int8_t x)
{
    return x;
}
```

std::int8_t 到 int 要保留 -128..127 的值，因此复制符号位。可能汇编：

```
movsx eax, dil
ret
```

语义：

```
eax = sign_extend_32(dil)
```

举例：

8 位输入	movzx 到 32 位	movsx 到 32 位
0x7f	0x0000007f	0x0000007f
0x80	0x00000080	0xffffffff80
0xff	0x000000ff	0xffffffffff

3.2.7.3 movsxd: 32 位到 64 位符号扩展

数组下标是 `int`，地址计算需要 64 位时，经常看到：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4]
```

`movsxd rsi, esi` 把 32 位有符号下标扩展成 64 位。如果 `i == -1`，扩展后 `rsi` 是 `0xffffffffffffffff`，地址计算就会向前移动 4 字节。C++ 中真正访问负下标通常越界，是 UB；但从指令语义看，它就是补码地址计算。

常见误区

`char` 是否 `signed` 在 C++ 中与实现有关。写底层实验时需要明确使用 `std::int8_t` 或 `std::uint8_t`，不要靠普通 `char` 猜测编译器会生成 `movsx` 还是 `movzx`。

3.2.8 移位：乘除 2 的幂、位域和符号

3.2.8.1 shl 和 sal

`shl` 是逻辑左移，`sal` 是算术左移；在 x86 上它们对整数位模式的效果相同：

```
shl eax, 3 ; eax = eax << 3
```

如果没有溢出语义问题，左移 3 位相当于乘以 8。但 C++ 对 `signed` 左移有规则限制，尤其是溢出和负数左移。机器指令只是移位，语言层面未必定义良好。

3.2.8.2 shr 和 sar

右移分两种：

```
shr eax, 1 ; logical right shift, high bits filled with 0
sar eax, 1 ; arithmetic right shift, high bits filled with sign bit
```

例子：

输入 8 位	shr 1	sar 1
01111110	00111111	00111111
10000000	01000000	11000000
11111110	01111111	11111111

`shr` 适合 `unsigned` 语义；`sar` 适合补码 `signed` 语义。C++ 中 `signed` 负数右移的具体行为在现代主流实现通常是算术右移，但学习标准规则时要注意实现定义历史和标准版本细节。本书在 x86-64/Linux 实验中按补码和算术右移观察。

3.2.8.3 移位计数

x86 中变量移位计数通常使用 `cl`：

```
shl eax, cl
shr rdx, cl
sar edi, cl
```

C++ 中如果移位计数小于 0 或大于等于位宽，行为未定义：

```
std::uint32_t x = 1;
auto y = x << 32; // undefined behavior in C++
```

硬件可能会对计数取模或屏蔽低位，但这不是你依赖的 C++ 语义。性能代码要么证明计数合法，要么显式处理。

3.2.9 位运算：mask、alignment 和 flags

3.2.9.1 and、or、xor、not

常见位运算指令：

指令	简化语义	常见用途
and	dst=dst&src	清位、取低位、对齐 mask
or	dst=dst src	置位、合并 flags
xor	dst=dst^src	翻转位、清零寄存器、差异检测
not	dst=~dst	位取反，不更新 flags

例如判断偶数：

```
extern "C" bool is_even(unsigned x)
{
    return (x & 1U) == 0;
}
```

可能汇编：

```
test dil, 1
sete al
ret
```

test a, b 计算 a&b 并设置 flags，但不保存结果。这里用最低位判断奇偶。

3.2.9.2 对齐向下：清掉低位

如果 align 是 2 的幂，对地址向下对齐可写成：

```
std::uintptr_t align_down(std::uintptr_t x)
{
    return x & ~std::uintptr_t(63);
}
```

对应位运算：

```
and rax, -64
```

因为 64 字节对齐要求低 6 位为 0。清掉低 6 位就是向下对齐到 cache line 边界。
例子：

```
x = 0x1234
mask = ~0x3f = ...ffc0
x & mask = 0x1200
```

3.2.9.3 对齐向上：加再清低位

向上对齐：

```
std::uintptr_t align_up_64(std::uintptr_t x)
{
    return (x + 63) & ~std::uintptr_t(63);
}
```

地址例子：

```
x = 0x1234
x + 63 = 0x1273
(x + 63) & ~0x3f = 0x1240
```

这个模式在 allocator、SIMD buffer、cache line padding、tensor storage 对齐中非常常见。

3.2.9.4 位标志

位运算经常用来维护 flags：

```
constexpr unsigned kReadable = 1u << 0;
constexpr unsigned kWritable = 1u << 1;
constexpr unsigned kPinned   = 1u << 2;

bool writable(unsigned flags)
{
    return (flags & kWritable) != 0;
}
```

可能汇编：

```
test dil, 2
setne al
ret
```

test 不保存 flags&2，只设置 ZF。后续 setne 根据 ZF 生成布尔值。

3.2.10 cmp 和 test：只设置 flags 的计算

cmp a, b 类似计算：

`a - b`

但不保存结果，只更新 flags。

`test a, b` 类似计算：

`a & b`

也不保存结果，只更新 flags。

对比：

```
sub eax, esi      ; eax = eax - esi, flags updated
cmp eax, esi      ; compute eax - esi only for flags

and eax, 1        ; eax = eax & 1, flags updated
test eax, 1       ; compute eax & 1 only for flags
```

第 13 章会系统讲条件跳转。本章先掌握一个读法：看到 `cmp` 或 `test`，不要找“结果存哪里”；结果不存，flags 才是输出。

3.2.11 read-modify-write：一条指令不等于一次微操作

看：

```
add dword ptr [rdi], esi
```

ISA 语义可以写成：

```
tmp = load 4 bytes from [rdi]
tmp = tmp + esi
store low 4 bytes of tmp to [rdi]
update flags
```

它是一条汇编，但语义包含 load、ALU、store。对于普通内存操作，这已经比寄存器加法重；对于原子 read-modify-write，例如 `lockadd`，还会牵涉 cache coherence 和内存序。

性能工程里要区分：

```
add eax, esi      ; register ALU
add dword ptr [rdi], esi ; memory RMW
lock add dword ptr [rdi], esi ; atomic RMW, much heavier
```

本章只要求你看到内存目的操作数时提高警觉：这不是普通寄存器加法。

3.2.12 案例：从 C++ 类型转换到扩展指令

创建：

```
#include <cstdint>

extern "C" int sum_bytes(std::uint8_t const* a, std::int8_t const* b, int i)
{
    return a[i] + b[i];
}
```

可能汇编：

```
movsxd rax, edx
movzx ecx, byte ptr [rdi + rax]
movsx eax, byte ptr [rsi + rax]
add eax, ecx
ret
```

逐条解释：

```
entry:
    rdi = a
    rsi = b
    edx = i

    movsxd rax, edx
    rax = sign_extend_64(i)

    movzx ecx, byte ptr [rdi + rax]
    load a[i] as uint8_t
    ecx = zero_extend_32(a[i])

    movsx eax, byte ptr [rsi + rax]
    load b[i] as int8_t
    eax = sign_extend_32(b[i])

    add eax, ecx
    eax = int(b[i]) + int(a[i])
```

这里 `movzx` 和 `movsx` 的差异来自 C++ 类型。换成两个 `std::uint8_t`，第二个 load 很可能也变成 `MOVZX`。

3.2.13 案例：地址计算、常数乘法和结构体大小

C++:

```
struct Node {
    int key;
    int value;
    int next;
};

extern "C" int get_value(Node const* nodes, int i)
{
    return nodes[i].value;
}
```

布局:

```
sizeof(Node)      = 12
offset(Node,value) = 4
addr(nodes[i].value) = base + i * 12 + 4
```

可能汇编:

```
movsxd rax, esi
lea    rax, [rax + rax*2]
mov    eax, dword ptr [rdi + rax*4 + 4]
ret
```

解释:

```
rax = sign_extend_64(i)
rax = rax + rax * 2 = i * 3
address = base + (i * 3) * 4 + 4
        = base + i * 12 + 4
```

这个案例很典型：结构体大小是 12，不能直接作为 x86 scale，于是编译器拆成 $i*3$ 再 $*4$ 。

3.2.14 案例：branchless mask 的第一眼

后面会系统讲 branchless 技术，这里先看一个基础模式：

```
extern "C" unsigned mask_if_nonzero(unsigned x)
{
    return x != 0 ? 0xffffffffu : 0u;
}
```

一种可能汇编:

```
neg edi
sbb eax, eax
ret
```

简化解释:

- `neg edi` 计算 $0-x$ ，并根据是否借位设置 CF。
- 如果 $x!=0$ ，CF 通常为 1；如果 $x==0$ ，CF 为 0。
- `sbb eax, eax` 计算 $eax=eax-eax-CF$ 。
- 当 CF 为 1，结果是 $0xffffffff$ ；当 CF 为 0，结果是 0。

这个例子不要求你现在熟练使用 `sbb`，但要建立意识：整数指令和 flags 可以组合出没有分支的 mask。SIMD tail mask、选择、量化 clamp、条件累加等场景会反复出现这种思想。

3.2.15 性能注意点：延迟、吞吐、依赖和 flags

本章不做完整微架构表，但必须先建立几个判断：

1. 寄存器 ALU 指令通常很便宜。
2. 乘法通常比加法和位运算更重。
3. 内存操作比寄存器操作更容易受 cache、TLB、store buffer 影响。
4. 更新 flags 会产生 flags 依赖；如果后续不需要 flags，`lea` 可能避免不必要的 flags 写。
5. 32 位写入清零高位，经常用于打断旧高位依赖。
6. 小宽度部分寄存器写入可能引入额外依赖或需要合并旧值。

例如:

```
add eax, esi ; writes eax and flags
lea eax, [rdi+rsi]; writes eax, does not write flags
```

如果后面紧跟条件跳转读取上一条 `cmp` 的 flags，那么中间插入会写 flags 的指令可能破坏条件。编译器会避开这种错误，但手写汇编必须自己负责。

3.2.16 实验 11.1: 类型扩展指令观察

实验

创建 labs/ch11_integer_instructions/extend.cpp:

```
#include <stdint>

extern "C" int from_u8(std::uint8_t x) { return x; }
extern "C" int from_i8(std::int8_t x) { return x; }
extern "C" int load_u8(std::uint8_t const* p, int i) { return p[i]; }
extern "C" int load_i8(std::int8_t const* p, int i) { return p[i]; }
```

生成汇编:

```
clang++ -std=c++20 -O3 -S -masm=intel extend.cpp -o extend.s
clang++ -std=c++20 -O3 -c extend.cpp -o extend.o
objdump -d -Mintel extend.o > extend.objdump.txt
```

交付物:

1. 找出所有 `movsx`、`movzx`、`movsxd`。
2. 解释每条扩展指令对应的 C++ 类型。
3. 对 `i==1` 的情况, 写出地址公式会如何变化, 并说明 C++ 是否允许访问。
4. 记录机器码字节。

3.2.17 实验 11.2: lea、乘法和结构体步长

实验

创建 labs/ch11_integer_instructions/lea_patterns.cpp:

```
extern "C" int mul3(int x) { return x * 3; }
extern "C" int mul5_add7(int x) { return x * 5 + 7; }

struct S12 {
    int a;
    int b;
    int c;
};

extern "C" int load_s12_c(S12 const* p, int i)
{
    return p[i].c;
}

struct S24 {
    double a;
    double b;
    int c;
    int d;
};

extern "C" int load_s24_d(S24 const* p, int i)
{
    return p[i].d;
}
```

交付物:

1. 找出所有 `lea`。
2. 解释每个 `lea` 是否访问内存。
3. 写出 `S12` 和 `S24` 的布局、`sizeof` 和字段 `offset`。
4. 从汇编恢复 `i*12+offset` 和 `i*24+offset`。
5. 对比编译器何时用 `imul`, 何时用 `lea`。

3.2.18 实验 11.3: 位运算和对齐

实验

创建 labs/ch11_integer_instructions/bit_alignment.cpp:

```
#include <stdint>

extern "C" std::uintptr_t align_down_64(std::uintptr_t x)
{
    return x & ~std::uintptr_t(63);
}
```

```
extern "C" std::uintptr_t align_up_64(std::uintptr_t x)
{
    return (x + 63) & ~std::uintptr_t(63);
}

extern "C" bool has_flag(unsigned flags, unsigned bit)
{
    return (flags & bit) != 0;
}

extern "C" unsigned set_flag(unsigned flags, unsigned bit)
{
    return flags | bit;
}
```

交付物：

1. 找出 `and`、`or`、`test`。
2. 用二进制解释 `align_down_64(0x1234)` 和 `align_up_64(0x1234)`。
3. 解释 `test` 为什么不保存结果。
4. 说明 64 字节对齐为什么和低 6 位有关。

3.2.19 本章作业

习题与作业

作业 1：整数指令注释表

从本章实验中选择至少 30 条整数指令，建立表格：

- 指令文本。
- 分类：move、ALU、shift、bitwise、extension、compare、address calculation。
- 读寄存器/内存。
- 写寄存器/内存。
- 是否更新 flags。
- 对应 C++ 语义。

作业 2：扩展语义推导

给定输入字节：

```
0x00
0x7f
0x80
0xff
```

分别写出：

1. `movzx eax, byte ptr [p]` 的结果。
2. `movsx eax, byte ptr [p]` 的结果。
3. 对应 `std::uint8_t` 和 `std::int8_t` 到 `int` 的 C++ 转换结果。

作业 3：恢复地址公式

对下面汇编恢复结构体布局的可能信息：

```
movsxd rax, esi
lea    rax, [rax + rax*2]
mov    ecx, dword ptr [rdi + rax*8 + 20]
ret
```

要求：

1. 写出元素步长。
2. 写出字段 offset。
3. 设计一个可能的 C++ 结构体。
4. 解释为什么不能只看 `rax*8` 就说元素大小是 8。

作业 4：对齐函数证明

证明下面函数对 2 的幂 A 有效：


```
std::uintptr_t align_up(std::uintptr_t x, std::uintptr_t A)
{
    return (x + A - 1) & ~(A - 1);
}
```

要求用二进制低位解释，不少于 800 字，并举三个例子：

- 已经对齐的地址。
- 只差 1 字节对齐的地址。
- 随机未对齐地址。

作业 5：小型整数汇编报告

写一个 20 行以内 C++ 文件，包含：

- `std::uint8_t` 和 `std::int8_t` load。
- 一个结构体数组字段访问，结构体大小不是 1、2、4、8。
- 一个对齐函数。
- 一个位标志测试函数。

生成 -O3 汇编并提交不少于 2500 字报告，必须包括：

- 源码和编译命令。
- 类型和结构体布局表。
- 每个函数的核心汇编。
- 所有地址公式。
- 扩展指令解释。
- flags 相关指令解释。
- 至少 10 条机器码字节记录。

3.2.20 验收标准

完成本章后，你应该能做到：

- 区分位模式、signed 解释和 unsigned 解释。
- 解释 mov、load、store 的差异。
- 解释 xor reg, reg 清零和 32 位写清零高位。
- 读懂 add、sub、imul、lea 的基本语义。
- 解释 movsx、movzx、movsxd 对应的 C++ 类型转换。
- 区分 shr 和 sar。
- 用 and、or、test 解释 mask、flag 和 alignment。
- 看到 cmp 和 test 时知道真正输出是 flags。
- 从 lea 组合恢复非 2 的幂结构体步长。
- 初步判断整数指令对性能的影响：寄存器 ALU、内存 RMW、flags 依赖、乘法和地址计算。

如果这些能力稳定，第 13 章的控制流、第 23 章的 loop optimization、第 24 章的 alias/UB、第 42 章的低精度量化和第 48 章的 AVX2 microkernel 都会更容易进入。

3.3 控制流：跳转、循环、调用与 switch

3.3.1 本章要解决的问题

前两章已经能读懂单条整数指令、寄存器、内存操作数、地址公式和 flags。现在要把这些材料组合成真正的程序执行路径。

一个 C++ 程序不是从第一行一直直线执行到最后一行。它会判断、跳转、循环、调用函数、返回调用点，也会通过函数指针或虚函数跳到运行时才知道的目标。机器层面没有高级的 if、while、switch、return 语法，只有一件更底层的事情：

```
normal instruction:
    rip = address_of_next_instruction

branch/call/ret:
    rip = some_other_address
```

rip 是下一条要取指的地址。只要你能解释一段代码什么时候写 rip、写成哪个地址、为什么写成那个地址，就能读懂控制流。

本章目标是让你完成下面几件事：

- 把 C++ 的 `if/else`、条件表达式、循环、`switch` 和函数调用还原成基本块和边。
- 理解 `cmp`、`test` 如何生产 flags, `jcc`、`setcc`、`cmovcc` 如何消费 flags。
- 区分 signed 比较和 unsigned 比较使用的条件码。
- 能画出一段汇编的控制流图。
- 能解释循环中的入口、回边、退出条件、归纳变量和地址递推。
- 能看懂 `switch` 的比较链和跳转表。
- 能用具体地址解释 `call` 写入返回地址、`ret` 取回返回地址。
- 初步理解分支预测、间接跳转和返回预测为什么影响现代 CPU 性能。

核心思想

控制流的本质是对 `rip` 的选择。顺序执行是默认选择；条件跳转、无条件跳转、调用、返回、间接跳转都是显式改变下一条指令地址。

3.3.2 从源代码到基本块

先看一个很普通的 C++ 函数：

```
extern "C" int classify(int x)
{
    if (x < 0) {
        return -1;
    }
    if (x == 0) {
        return 0;
    }
    return 1;
}
```

源代码有三条返回路径。编译器降低它时，会先把代码切成若干 **basic block**（基本块）。基本块是一段只有一个入口、一个出口的连续指令序列。进入基本块之后，中间不会跳走；要跳走只能发生在块尾。

控制流图可以写成：

```
entry:
    test x
    if x < 0 goto neg
    goto check_zero

neg:
    return -1

check_zero:
    test x
    if x == 0 goto zero
    goto pos

zero:
    return 0

pos:
    return 1
```

真实编译器可能用分支，也可能用 `setcc` 或 `cmovcc` 生成无分支代码。先不要急着要求“这段 C++ 必须长成某种汇编”。正确训练方式是：

1. 找出基本块。
2. 找出每个基本块最后如何选择下一块。
3. 找出每条边的条件。
4. 找出每个块里产生或使用的数据。
5. 再分析编译器是否把控制流改写成数据流。

深入理解

编译器中间表示通常会显式表达基本块、边和控制依赖。后面讲 LLVM IR 时，你会看到 `br`、`switch`、`phi`。汇编阶段虽然没有 `phi`，但基本块和边仍然存在，只是隐藏在标签、跳转和寄存器赋值里。

3.3.3 RIP：CPU 正在执行哪里

从 ISA 视角看，CPU 有一个架构状态：通用寄存器、flags、内存可见状态和 `rip`。普通指令完成后，`rip` 指向下一条指令：

address	bytes	instruction
0x1000	89 f8	mov eax, edi
0x1002	01 f0	add eax, esi

```
0x1004 c3      ret
```

执行过程：

当前 rip	执行指令	下一步 rip
0x1000	moveax,edi	0x1002
0x1002	addeax,esi	0x1004
0x1004	ret	从栈顶取出的返回地址

为什么 0x1000 后面是 0x1002？因为 `mov eax, edi` 的机器码长度是 2 字节。x86 是变长指令集，一条指令可以是 1 字节，也可以更长。取指和解码前端的复杂性，一部分就来自“下一条指令地址不是固定加 4”。

分支指令则不同。比如：

```
0x2000: 85 ff      test edi, edi
0x2002: 7e 08      jle 0x200c
0x2004: 8d 04 36    lea eax, [rsi + rsi]
0x2007: 83 c0 01    add eax, 1
0x200a: c3        ret
0x200c: 8d 46 03    lea eax, [rsi + 3]
0x200f: c3        ret
```

`jle` 的条件为真时，下一条 `rip` 是 0x200c；条件为假时，下一条 `rip` 是顺序地址 0x2004。这就是控制流分叉。

心智模型

读控制流时，把每条跳转都看成一次对 `rip` 的赋值：

```
if condition:
    rip = target
else:
    rip = next_instruction
```

3.3.4 Flags 回顾

上一章已经见过 flags。本章要把它们放进控制流里使用。x86-64 的很多条件跳转不是直接读取 C++ 的布尔变量，而是读取前面指令设置的 flags。

3.3.4.1 `cmp` 的本质

Intel 语法：

```
cmp destination, source
```

语义近似是：

```
temporary = destination - source
set flags according to temporary
discard temporary
```

例如：

```
cmp edi, esi
jg .Lgreater
```

这表示先按 `edi-esi` 设置 flags，然后 `jg` 判断 signed 意义下 `edi>esi` 是否成立。

最常用 flags：

flag	名称	典型含义
ZF	zero flag	结果为 0；常用于相等判断
SF	sign flag	结果最高位为 1；常用于 signed 判断的一部分
CF	carry flag	unsigned 加法进位或减法借位
OF	overflow flag	signed 加减溢出
PF	parity flag	低 8 位奇偶；现代 C++ 性能代码较少直接关心

3.3.4.2 `test` 的本质

`test` 做按位与，只设置 flags，不保存结果：

```
test edi, edi
je .Lzero
```

语义近似：

```
temporary = edi & edi
set flags according to temporary
discard temporary
```

因为 $x \& x == x$ ，所以 `test edi, edi` 常用于判断 $x == 0$ 、 $x < 0$ 或 $x > 0$ 。它不会像 `cmp edi, 0` 那样编码一个立即数 0，机器码通常更短。

```
test edi, edi
je zero      ; ZF=1, x == 0
js negative  ; SF=1, x < 0 in signed interpretation
jne nonzero  ; ZF=0, x != 0
```

常见误区

flags 是隐式状态。很多指令会写 flags，很多条件指令会读 flags。手写汇编时，如果在 `cmp` 和 `jcc` 中间插入会改 flags 的指令，条件判断就会改变。编译器会维护这种依赖；人写汇编必须自己负责。

3.3.5 条件跳转

`jcc` 是一族条件跳转。这里的 `cc` 是 condition code，例如 `je`、`jne`、`jl`、`jg`、`ja`、`jb`。基本形态：

```
cmp a, b
jcc target
```

含义：

```
compute flags for a - b
if condition(flags):
    rip = target
else:
    rip = next_instruction
```

3.3.5.1 相等和不相等

相等判断只需要看 ZF：

指令	条件	常见 C++ 关系
<code>je</code> 或 <code>jz</code>	$ZF == 1$	$a == b$
<code>jne</code> 或 <code>jnz</code>	$ZF == 0$	$a != b$

3.3.5.2 signed 比较

signed 比较需要考虑 SF 和 OF：

指令	读 flags 的条件	<code>cmp a, b</code> 后的含义
<code>jl</code> 或 <code>jnge</code>	$SF != 0F$	signed $a < b$
<code>jle</code> 或 <code>jng</code>	$ZF == 1 SF != 0F$	signed $a <= b$
<code>jg</code> 或 <code>jnl</code>	$ZF == 0 \&\& SF == 0F$	signed $a > b$
<code>jge</code> 或 <code>jnl</code>	$SF == 0F$	signed $a >= b$

3.3.5.3 unsigned 比较

unsigned 比较主要看 CF 和 ZF：

指令	读 flags 的条件	<code>cmp a, b</code> 后的含义
<code>jb</code> 或 <code>jnae</code>	$CF == 1$	unsigned $a < b$
<code>jbe</code> 或 <code>jna</code>	$CF == 1 ZF == 1$	unsigned $a <= b$
<code>ja</code> 或 <code>jnb</code>	$CF == 0 \&\& ZF == 0$	unsigned $a > b$
<code>jae</code> 或 <code>jnb</code>	$CF == 0$	unsigned $a >= b$

核心理想

同一个 `cmp` 之后，接 signed 条件跳转还是 unsigned 条件跳转，会得到不同语义。硬件没有问变量类型；类型信息已经体现在编译器选择了哪条条件指令。

3.3.6 signed 和 unsigned 的分叉

下面两个函数源代码几乎一样，但类型不同：

```
extern "C" int less_i32(int a, int b)
{
    return a < b;
}

extern "C" int less_u32(unsigned a, unsigned b)
{
    return a < b;
}
```

可能汇编：

```
less_i32:
    xor eax, eax
    cmp edi, esi
    setl al
    ret

less_u32:
    xor eax, eax
    cmp edi, esi
    setb al
    ret
```

两段都用 `cmp edi, esi`，但是 signed 版本使用 `setl`，unsigned 版本使用 `setb`。这不是小细节，而是语义核心。
设：

```
edi = 0xffffffff
esi = 0x00000001
```

解释：

解释方式	edi 的值	edi<esi
std::int32_t	-1	true
std::uint32_t	4294967295	false

所以：

```
cmp edi, esi
setl al    ; signed:  -1 < 1, true
setb al    ; unsigned: 4294967295 < 1, false
```

常见误区

很多性能 bug 和安全 bug 都来自 signed/unsigned 混用。看到 `jl`、`jg` 时要想到 signed；看到 `jb`、`ja` 时要想到 unsigned。尤其是数组长度、下标、`size_t`、负数检查混在一起时，要非常警惕。

3.3.7 if/else 的降低

考虑函数：

```
extern "C" int branchy(int x, int y)
{
    if (x > 0) {
        return y * 2 + 1;
    } else {
        return y + 3;
    }
}
```

为了教学，我们先看一种有显式分支的写法：

```
branchy:
    test edi, edi
    jle .lElse
    lea eax, [rsi + rsi]
    add eax, 1
    ret
.lElse:
    lea eax, [rsi + 3]
    ret
```

控制流图：

```
entry:
    test x
    if x <= 0 goto else
```

```

goto then

then:
    eax = y * 2 + 1
    return

else:
    eax = y + 3
    return

```

注意编译器经常会把源代码条件取反。源代码写的是 $x > 0$ ，汇编却可能写 `jle .Lelse`。这不是语义反了，而是把不满足 `then` 的情况跳到 `else`，让 `then` 作为 `fallthrough`。

逐条解释一次：

```

entry:
    edi = x
    esi = y

test edi, edi
    flags = flags_of(x & x)

jle .Lelse
    if signed x <= 0:
        rip = .Lelse
    else:
        rip = next instruction

lea eax, [rsi + rsi]
    eax = y * 2
    flags unchanged

add eax, 1
    eax = y * 2 + 1
    flags updated, but no later use here

ret
    return eax

.Lelse:
lea eax, [rsi + 3]
    eax = y + 3

ret

```

3.3.7.1 带机器码地址的观察

反汇编里可能看到类似：

```

0000000000000000 <branchy>:
0: 85 ff          test    edi, edi
2: 7e 08          jle     c <branchy+0xc>
4: 8d 04 36       lea     eax, [rsi+rsi]
7: 83 c0 01       add     eax, 0x1
a: c3            ret
c: 8d 46 03       lea     eax, [rsi+0x3]
f: c3            ret

```

`jle` 的机器码 `7e08` 使用 8 位相对位移。位移不是从当前指令开头算，而是从下一条指令地址算：

```

jle address      = 0x0002
jle length       = 2
next instruction = 0x0004
encoded offset   = 0x08
target           = 0x0004 + 0x08 = 0x000c

```

这就是 `c<branchy+0xc>` 的来源。

深入理解

x86 条件跳转既可以使用短位移，也可以使用更长的相对位移。汇编器会根据目标距离选择编码。链接和重定位还可能调整最终地址。理解“相对下一条指令”的规则，对读机器码、二进制补丁和反汇编非常重要。

3.3.8 setcc：把条件变成 0 或 1

C++ 经常需要返回布尔值：

```

extern "C" int is_positive(int x)
{
    return x > 0;
}

```

```
}

```

可能汇编：

```
is_positive:
    xor    eax, eax
    test   edi, edi
    setg   al
    ret

```

`setg al` 表示如果 `signed x > 0`，就把 `al` 写成 1，否则写成 0。由于 `setcc` 只写 8 位目标，前面常见 `xor eax, eax`，先把整个 `eax` 清零，避免高位残留：

```
xor eax, eax
    eax = 0

test edi, edi
    flags = flags_of(x)

setg al
    if x > 0:
        al = 1
    else:
        al = 0
    eax is now 0 or 1

```

常见 `setcc`：

指令	条件	常见含义
<code>sete</code>	<code>ZF==1</code>	等于
<code>setne</code>	<code>ZF==0</code>	不等于
<code>setl</code>	signed 小于	<code>a < b</code> for signed
<code>setg</code>	signed 大于	<code>a > b</code> for signed
<code>setb</code>	unsigned 小于	<code>a < b</code> for unsigned
<code>seta</code>	unsigned 大于	<code>a > b</code> for unsigned

3.3.9 `cmovcc`：把控制选择变成数据选择

分支会改变 `rip`。有时候编译器不想改变控制流，而是先算出两个候选值，然后用 `cmovcc` 选择一个。C++：

```
extern "C" int min_i32(int a, int b)
{
    return a < b ? a : b;
}

```

可能汇编：

```
min_i32:
    mov    eax, esi
    cmp    edi, esi
    cmovl  eax, edi
    ret

```

逐条解释：

```
entry:
    edi = a
    esi = b

mov    eax, esi
    result candidate = b

cmp    edi, esi
    flags = flags_of(a - b)

cmovl  eax, edi
    if signed a < b:
        eax = a
    else:
        eax remains b

ret

```

`cmovcc` 不改变 `rip`，但它读取 `flags`，并且目标寄存器会依赖条件和两个候选值。它适合分支难预测、两边计算都便宜的情况。

3.3.9.1 什么时候不要盲目追求 *branchless*

无分支不等于一定更快。考虑：

```
extern "C" int maybe_load(int cond, int const* p, int fallback)
{
    if (cond) {
        return *p;
    }
    return fallback;
}
```

这类代码不能简单改成“先 load 再 cmov”，因为 `cond==0` 时 `p` 可能为空或无效。C++ 语义只允许在条件为真时访问 `*p`。机器硬件可以执行 load，但语言层面和异常行为都不允许你随便提前访问。

性能判断也要看代价：

- 分支高度可预测时，普通分支可能非常快。
- 分支接近随机时，错误预测代价可能很高，`cmovcc` 可能更好。
- `cmovcc` 会保留数据依赖，可能拉长关键路径。
- 如果两边计算很重，*branchless* 可能让本来不需要执行的一边也产生代价。
- 如果一边包含潜在非法内存访问，不能用简单数据选择替代控制选择。

核心思想

分支优化不是“消灭所有分支”。正确问题是：这条分支是否可预测？错误预测代价多大？两边计算是否便宜？数据依赖是否变长？语言语义是否允许提前计算？

3.3.10 循环的机器结构

循环不是一个神秘结构。它由入口判断、循环体、回边和退出边组成。

C++：

```
extern "C" int sum_i32(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += p[i];
    }
    return s;
}
```

一种容易教学的汇编：

```
sum_i32:
    xor eax, eax
    test esi, esi
    jle .ldone
    xor ecx, ecx
.Lloop:
    add eax, dword ptr [rdi + rcx*4]
    inc rcx
    cmp ecx, esi
    jl .Lloop
.Ldone:
    ret
```

这里假设 `rdi` 是 `p`，`esi` 是 `n`，`eax` 是累加器，`ecx/rcx` 是循环下标。

控制流图：

```
entry:
    s = 0
    if n <= 0 goto done
    i = 0
    goto loop

loop:
    s += p[i]
    i += 1
    if i < n goto loop
    goto done

done:
    return s
```

回边是：

```
jl .Lloop
```

它让 `rip` 回到更小地址处的循环体开头。后面讲分支预测时会看到，循环回边往往非常可预测：大多数迭代跳回，最后一次不跳。

3.3.10.1 地址和循环变量一起推导

对循环体核心指令：

```
add eax, dword ptr [rdi + rcx*4]
```

推导：

```
rdi = p
rcx = i
element width = 4 bytes

address = p + i * 4
load    = *(int*)(p + i * 4)
eax     = s + load
```

每轮迭代：

迭代	rcx	访问地址	语义
0	0	p+0	s+=p[0]
1	1	p+4	s+=p[1]
2	2	p+8	s+=p[2]
k	k	p+4*k	s+=p[k]

3.3.10.2 指针递增形式

编译器也可能不用显式下标，而是让指针前进：

```
sum_i32_ptr:
    xor eax, eax
    test esi, esi
    jle .ldone
    movsxd rsi, esi
    lea rdx, [rdi + rsi*4]
.Lloop:
    add eax, dword ptr [rdi]
    add rdi, 4
    cmp rdi, rdx
    jne .Lloop
.Ldone:
    ret
```

这里：

```
rdi = current pointer
rdx = end pointer = p + n * 4

loop:
    s += *current
    current += 4
    continue while current != end
```

这和源代码的 `i < n` 语义等价，但汇编里看不到 `i`。你要恢复的是数据流和地址递推，而不是强行找源代码变量名。

常见误区

不要看到 `cmp rdi, rdx` 就以为源代码一定写了两个指针比较。它也可能来自数组下标循环，编译器把下标形式优化成了指针递增形式。

3.3.11 循环形态：while、do-while 和 for

高级语言有不同循环语法，机器层面差异主要在第一次检查条件的位置。

3.3.11.1 while 循环

C++：

```
extern "C" int count_down(int n)
{
    int s = 0;
    while (n > 0) {
        s += n;
        --n;
    }
    return s;
}
```

结构：

```
entry:
    s = 0
    if n <= 0 goto done

loop:
    s += n
    n -= 1
    if n > 0 goto loop

done:
    return s
```

while 是先判断，再执行。若初始条件不成立，循环体一次都不执行。

3.3.11.2 do-while 循环

C++：

```
extern "C" int count_down_do(int n)
{
    int s = 0;
    do {
        s += n;
        --n;
    } while (n > 0);
    return s;
}
```

结构：

```
entry:
    s = 0

loop:
    s += n
    n -= 1
    if n > 0 goto loop

done:
    return s
```

do-while 至少执行一次。汇编里常见特征是循环体前面没有入口条件跳转，条件跳转在尾部。

3.3.11.3 for 循环

for(init;cond;step) 常见结构：

```
init
if not cond goto done

loop:
    body
    step
    if cond goto loop

done:
```

编译器会根据优化目标重排。比如把第一次条件判断放到循环尾，或者把不变量挪到循环外。读汇编时不要纠结源代码写法，先识别基本块和边。

3.3.12 归纳变量和循环优化预告

induction variable（归纳变量）是每次迭代按固定规律变化的变量。最常见：

```
i = i + 1
ptr = ptr + 4
offset = offset + stride
```

在 HPC 和 AI 算子里，归纳变量非常重要，因为它决定内存访问模式、向量化条件、预取距离和 cache line 利用率。

考虑：

```
extern "C" void axpy(float* y, float const* x, float a, int n)
{
    for (int i = 0; i < n; ++i) {
        y[i] += a * x[i];
    }
}
```

标量汇编的核心形态可能是：

```
.Lloop:
    movss xmm1, dword ptr [rsi + rcx*4]
    mulss xmm1, xmm0
    addss xmm1, dword ptr [rdi + rcx*4]
    movss dword ptr [rdi + rcx*4], xmm1
    inc rcx
    cmp ecx, edx
    jl .Lloop
```

控制流只是循环；性能核心则在数据流：

```
x address = x + i * 4
y address = y + i * 4
two loads + one store per element
one multiply + one add per element
branch once per scalar iteration
```

后面讲 SIMD 时，这个循环会变成每轮处理多个元素，回边次数减少，地址递增步长变大，tail 处理成为新问题。本章先把标量控制流看清楚。

3.3.13 switch：比较链和跳转表

switch 的降低取决于 case 值是否密集、数量是否多、每个分支体大小和编译器策略。

3.3.13.1 比较链

C++:

```
extern "C" int small_switch(int x)
{
    switch (x) {
        case 1: return 10;
        case 7: return 70;
        default: return -1;
    }
}
```

可能汇编：

```
small_switch:
    cmp edi, 1
    je .Lcase1
    cmp edi, 7
    je .Lcase7
    mov eax, -1
    ret
.Lcase1:
    mov eax, 10
    ret
.Lcase7:
    mov eax, 70
    ret
```

控制流就是多次比较加条件跳转。case 稀疏时，这种形式很常见。

3.3.13.2 跳转表

case 密集时，编译器可能生成跳转表：

```
extern "C" int dense_switch(int x)
{
    switch (x) {
        case 0: return 3;
        case 1: return 5;
        case 2: return 8;
        case 3: return 13;
        case 4: return 21;
        default: return -1;
    }
}
```

如果每个 case 只是返回常量，编译器可能生成数据表而不是代码跳转表。为了看清跳转表，可以让每个分支调用不同函数。教学形式如下：

```
dense_switch:
    cmp edi, 4
    ja .Ldefault
    mov eax, edi
    jmp qword ptr [rip + .LJTI0_0 + rax*8]

.Lcase0:
    mov eax, 3
```

```

    ret
.Lcase1:
    mov eax, 5
    ret
.Lcase2:
    mov eax, 8
    ret
.Lcase3:
    mov eax, 13
    ret
.Lcase4:
    mov eax, 21
    ret
.Ldefault:
    mov eax, -1
    ret

.LJTI0_0:
    .quad .Lcase0
    .quad .Lcase1
    .quad .Lcase2
    .quad .Lcase3
    .quad .Lcase4

```

分解：

```

cmp edi, 4
ja .Ldefault
    unsigned if x > 4 goto default

mov eax, edi
    rax = zero-extended x

jmp qword ptr [rip + table + rax*8]
    target_address = *(table + x * 8)
    rip = target_address

```

为什么使用 `ja` 而不是 `jg`? 因为这里常见技巧是用 `unsigned` 范围检查同时处理负数：

```

x = -1 as int32 bits = 0xffffffff
as unsigned          = 4294967295
4294967295 > 4       = true

```

所以 `x<0` 和 `x>4` 都会跳到 `default`。

核心思想

跳转表把多路选择变成一次范围检查、一次表访问、一次间接跳转。它减少比较次数，但引入间接分支和额外取表访问。它是性能优化、二进制分析和安全缓解都会关心的结构。

3.3.13.3 跳转表的地址推导

设：

```

rip at jmp next instruction base = 0x401050
.LJTI0_0 runtime address          = 0x402000
rax                               = 3
entry width                       = 8 bytes

```

则：

```

entry_address = 0x402000 + 3 * 8
               = 0x402018

target = *(uint64_t*)0x402018
        = address of .Lcase3

rip = target

```

反汇编时，`rip+displacement` 里的 `rip` 通常指下一条指令地址，不是当前指令开头地址。这一点和前面的相对跳转一致。

3.3.14 无条件跳转和尾调用

`jmp` 直接改变 `rip`：

```
jmp .Ltarget
```

语义：

```
rip = .Ltarget
```

它不写返回地址。和 `call` 的区别非常关键。
一个函数末尾直接返回另一个函数结果：

```
extern "C" int callee(int);

extern "C" int wrapper(int x)
{
    return callee(x);
}
```

优化后可能是：

```
wrapper:
    jmp callee
```

这叫 **tail call**（尾调用）或 tail-call optimization。因为 `wrapper` 调用 `callee` 后没有额外工作，编译器可以复用调用者给 `wrapper` 的返回地址。这样 `callee` 返回时直接回到 `wrapper` 的调用者。

控制流：

```
caller -> wrapper -> callee -> caller
```

尾调用优化后：

```
caller -> wrapper
        jmp callee
callee -> caller
```

3.3.15 call 和 ret

`call` 做两件事：

1. 把返回地址压入栈。
2. 把 `rip` 改成被调用函数入口地址。

`ret` 做相反动作：

1. 从栈顶取出返回地址。
2. `rsp` 增加 8。
3. 把 `rip` 改成取出的返回地址。

以具体地址说明。假设：

```
main:
0x401120: e8 5b 00 00 00    call 0x401180 <f>
0x401125: 83 c0 01          add eax, 1

f:
0x401180: 8d 04 3f          lea eax, [rdi+rdi]
0x401183: c3               ret
```

执行 `call` 前：

```
rip = 0x401120
rsp = 0x7fffffffcd90
```

执行 `call 0x401180` 后：

```
rsp = 0x7fffffffcd88
[rsp] = 0x401125
rip = 0x401180
```

返回地址是 `0x401125`，也就是 `call` 的下一条指令地址。函数 `f` 执行 `ret` 时：

```
target = *(uint64_t*)rsp = 0x401125
rsp = rsp + 8
rip = target
```

于是执行回到 `main` 里的 `add eax, 1`。

指令	栈动作	rip 动作
<code>call target</code>	<code>pushnext_rip</code>	<code>rip=target</code>
<code>ret</code>	<code>poptarget</code>	<code>rip=target</code>
<code>jmp target</code>	无返回地址动作	<code>rip=target</code>

常见误区

`ret` 信任栈顶的返回地址。如果栈上的返回地址被破坏，控制流就会跳到错误位置。这是理解栈溢出、ROP、控制流完整性和影子栈等安全机制的基础。

3.3.16 直接调用和间接调用

直接调用目标在指令里：

```
call foo
```

间接调用目标来自寄存器或内存：

```
call rax
call qword ptr [rdi + 16]
```

C++ 函数指针：

```
using Fn = int (*)(int);

extern "C" int call_fn(Fn f, int x)
{
    return f(x) + 1;
}
```

可能汇编：

```
call_fn:
    push rbx
    mov  rbx, rdi
    mov  edi, esi
    call rbx
    add  eax, 1
    pop  rbx
    ret
```

这里 `rdi` 初始是函数指针，`esi` 是参数 `x`。调用前要把 `x` 放进第一个参数寄存器 `edi`，再 `call rbx`。

间接调用在 C++ 中还来自：

- 函数指针。
- 虚函数调用。
- `std::function` 或回调封装。
- 动态链接过程中的 PLT/GOT。
- JIT 生成代码或插件系统。

性能上，间接调用目标不固定，分支预测更难。安全上，间接调用目标需要控制流完整性保护。后面讲分支预测、动态链接和生产库 dispatch 时会继续深入。

3.3.17 现代 CPU 如何处理分支

ISA 语义很简单：条件满足就跳，不满足就不跳。但现代 CPU 为了性能不会慢慢等条件算完才取下一条指令。它会预测。

一个简化执行过程：

```
front-end:
    fetch bytes at predicted rip
    decode instructions

branch predictor:
    predict taken/not taken
    predict target address

back-end:
    execute compare/test
    resolve actual branch direction

retire:
    if prediction was correct:
        commit work
    else:
        flush wrong-path work
        restart from correct rip
```

常见预测结构包括：

- 方向预测：这次分支会不会跳。
- BTB：branch target buffer，预测跳转目标地址。
- RSB：return stack buffer，预测 `ret` 返回到哪里。
- 间接分支预测：预测 `jmp rax` 或 `call rax` 的目标。

这解释了为什么控制流会影响性能：

- 可预测分支通常成本低。
- 不可预测分支会造成错误路径清空，浪费前端和后端工作。
- 间接分支目标多时，预测更困难。
- 代码布局会影响取指、I-cache、BTB 和 fallthrough。
- `ret` 通常依赖 RSB 预测；异常调用模式可能让返回预测变差。

深入理解

这不是说每个分支都要手工消掉。高性能工程里要用数据证明：`perfstat` 的 `branch misses`、采样热点、输入分布、汇编路径长度、内存行为都要一起看。单独盯一条 `jcc`，很容易做出错误优化。

3.3.18 控制流阅读流程

读一段陌生控制流汇编，按下面顺序：

1. 标出函数入口和所有标签。
2. 按跳转、调用、返回切分基本块。
3. 给每个基本块编号，例如 `B0`、`B1`、`B2`。
4. 对每个块写入口寄存器含义。
5. 找出块尾的 `jcc`、`jmp`、`call`、`ret`。
6. 如果是 `jcc`，向上找最近一次设置 `flags` 的指令。
7. 判断条件是 `signed`、`unsigned`、相等、非零还是位测试。
8. 画边：`taken edge` 和 `fallthrough edge`。
9. 对循环找回边和退出边。
10. 对每条内存访问写地址公式。

报告里建议使用这种格式：

```
B0:
instructions:
    test edi, edi
    jle B2
state:
    edi = x
    esi = y
outgoing edges:
    if signed x <= 0 -> B2
    otherwise       -> B1

B1:
instructions:
    lea eax, [rsi+rsi]
    add eax, 1
    ret
meaning:
    return y * 2 + 1

B2:
instructions:
    lea eax, [rsi+3]
    ret
meaning:
    return y + 3
```

3.3.19 案例：从 C++ 到 CFG 到汇编

综合前面内容，分析一个带循环和条件的函数：

```
extern "C" int sum_positive(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        if (v > 0) {
            s += v;
        }
    }
    return s;
}
```

一种分支式汇编：

```
sum_positive:
    xor eax, eax
    test esi, esi
    jle .Ldone
    xor ecx, ecx
.Lloop:
    mov edx, dword ptr [rdi + rcx*4]
    test edx, edx
    jle .Lskip
    add eax, edx
.Lskip:
```

```
inc rcx
cmp ecx, esi
jl .Lloop
.Ldone:
ret
```

基本块:

```
B0 entry:
s = 0
if n <= 0 goto Bdone
i = 0
goto Bloop

Bloop:
v = p[i]
if v <= 0 goto Bskip
goto Badd

Badd:
s += v
goto Bskip

Bskip:
i += 1
if i < n goto Bloop
goto Bdone

Bdone:
return s
```

地址公式:

```
p base      = rdi
i           = rcx
element width = 4
load address = rdi + rcx * 4
```

flags 生产和消费:

生产 flags	消费 flags	语义
testesi,esi	jle.Ldone	signed n<=0
testedx,edx	jle.Lskip	signed v<=0
cmpecx,esi	jl.Lloop	signed i<n

编译器也可能使用无分支形式:

```
.Lloop:
mov edx, dword ptr [rdi + rcx*4]
test edx, edx
cmovle edx, r8d
add eax, edx
inc rcx
cmp ecx, esi
jl .Lloop
```

这里假设 r8d 已经是 0。含义是如果 v<=0，把 edx 变成 0，再无条件加到 s。这消除了循环体内部的数据相关分支，但引入 cmovle 和数据依赖。哪种更快取决于正数分布、分支可预测性、CPU 和内存瓶颈。

3.3.20 实验 12.1: signed 和 unsigned 条件码

实验

创建 labs/ch12_control_flow/signed_unsigned.cpp:

```
#include <stdint>

extern "C" int less_i32(std::int32_t a, std::int32_t b)
{
    return a < b;
}

extern "C" int less_u32(std::uint32_t a, std::uint32_t b)
{
    return a < b;
}

extern "C" int in_range_i32(std::int32_t x)
{
    return x >= 0 && x <= 4;
```

```

}

extern "C" int in_range_u32(std::uint32_t x)
{
    return x <= 4;
}

```

生成：

```

mkdir -p labs/ch12_control_flow/build
clang++ -std=c++20 -O3 -S -masm=intel \
    labs/ch12_control_flow/signed_unsigned.cpp \
    -o labs/ch12_control_flow/build/signed_unsigned.s

clang++ -std=c++20 -O3 -c \
    labs/ch12_control_flow/signed_unsigned.cpp \
    -o labs/ch12_control_flow/build/signed_unsigned.o

objdump -d -Mintel \
    labs/ch12_control_flow/build/signed_unsigned.o \
    > labs/ch12_control_flow/build/signed_unsigned.objdump.txt

```

交付物：

1. 找出 signed 版本使用的 `setcc` 或 `jcc`。
2. 找出 unsigned 版本使用的 `setcc` 或 `jcc`。
3. 用 `a=-1, b=1` 的位模式解释 signed 和 unsigned 结果差异。
4. 对 `in_range_i32(-1)` 和 `in_range_u32(0xffffffff)` 解释 flags 和返回值。
5. 从 `objdump` 中记录至少 6 条机器码字节，并说明每条指令的地址。

3.3.21 实验 12.2：画控制流程图

实验

创建 `labs/ch12_control_flow/cfg.cpp`：

```

#include <cstdint>

extern "C" int score(int const* p, int n, int threshold)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        if (v > threshold) {
            s += v - threshold;
        } else {
            s -= threshold - v;
        }
    }
    return s;
}

```

分别生成 `-O0` 和 `-O3`：

```

clang++ -std=c++20 -O0 -S -masm=intel cfg.cpp -o cfg_00.s
clang++ -std=c++20 -O3 -S -masm=intel cfg.cpp -o cfg_03.s
clang++ -std=c++20 -O3 -c cfg.cpp -o cfg.o
objdump -d -Mintel cfg.o > cfg.objdump.txt

```

交付物：

1. 对 `cfg_00.s` 切分基本块，至少标出入口块、循环条件块、then 块、else 块、回边块、退出块。
2. 对 `cfg_03.s` 切分基本块。如果编译器使用 `cmovcc` 或代数化简，要说明控制流如何变成数据流。
3. 画出 `-O0` 和 `-O3` 的控制流程图。可以用文本图、Graphviz 或手绘截图。
4. 对每个 `jcc` 写出：生产 flags 的指令、条件码、taken edge、fallthrough edge。
5. 对循环体里的 `p[i]` 写地址公式。

3.3.22 实验 12.3：分支和 branchless 对比

实验

创建 `labs/ch12_control_flow/branch_vs_cmov.cpp`：

```

#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdio>

```

```

#include <random>
#include <vector>

extern "C" int sum_branch(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        if (p[i] > 0) {
            s += p[i];
        }
    }
    return s;
}

extern "C" int sum_mask(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        int mask = -(v > 0);
        s += v & mask;
    }
    return s;
}

static std::uint64_t now_ns()
{
    auto t = std::chrono::steady_clock::now().time_since_epoch();
    return std::chrono::duration_cast<std::chrono::nanoseconds>(t).count();
}

int main()
{
    constexpr int n = 1 << 24;
    std::vector<int> data(n);

    // pattern 1: predictable, all positive
    std::fill(data.begin(), data.end(), 1);

    volatile int sink = 0;
    auto t0 = now_ns();
    for (int r = 0; r < 20; ++r) sink += sum_branch(data.data(), n);
    auto t1 = now_ns();
    for (int r = 0; r < 20; ++r) sink += sum_mask(data.data(), n);
    auto t2 = now_ns();

    std::printf("predictable branch=%llu ns mask=%llu ns sink=%d\n",
        (unsigned long long)(t1 - t0),
        (unsigned long long)(t2 - t1),
        sink);

    // pattern 2: random signs
    std::mt19937 rng(123);
    for (int& x : data) {
        x = (rng() & 1) ? 1 : -1;
    }

    t0 = now_ns();
    for (int r = 0; r < 20; ++r) sink += sum_branch(data.data(), n);
    t1 = now_ns();
    for (int r = 0; r < 20; ++r) sink += sum_mask(data.data(), n);
    t2 = now_ns();

    std::printf("random branch=%llu ns mask=%llu ns sink=%d\n",
        (unsigned long long)(t1 - t0),
        (unsigned long long)(t2 - t1),
        sink);
}

```

构建并观察汇编：

```

clang++ -std=c++20 -O3 -march=native \
    branch_vs_cmov.cpp -o branch_vs_cmov

clang++ -std=c++20 -O3 -march=native -S -masm=intel \
    branch_vs_cmov.cpp -o branch_vs_cmov.s

./branch_vs_cmov

perf stat -e branches,branch-misses ./branch_vs_cmov

```

交付物：

1. 找出 `sum_branch` 和 `sum_mask` 的汇编核心循环。
2. 说明编译器是否自动向量化。如果向量化了，额外用下面参数生成标量版本比较：
`-fno-vectorize -fno-slp-vectorize`
3. 比较 `predictable` 和 `random` 两种输入的时间。
4. 记录 `branches` 和 `branch-misses`。如果系统不允许访问 `perf` 事件，说明权限问题并至少报告运行时间。
5. 写出结论：什么时候分支更好，什么时候 `mask/branchless` 更好，本实验不能证明什么。

3.3.23 实验 12.4：switch 和跳转表

实验

创建 `labs/ch12_control_flow/switch_table.cpp`：

```
#include <cstdint>

extern "C" int f0(int x) { return x + 3; }
extern "C" int f1(int x) { return x + 5; }
extern "C" int f2(int x) { return x + 8; }
extern "C" int f3(int x) { return x + 13; }
extern "C" int f4(int x) { return x + 21; }

extern "C" int dense_dispatch(int op, int x)
{
    switch (op) {
        case 0: return f0(x);
        case 1: return f1(x);
        case 2: return f2(x);
        case 3: return f3(x);
        case 4: return f4(x);
        default: return -1;
    }
}

extern "C" int sparse_dispatch(int op, int x)
{
    switch (op) {
        case 1: return f0(x);
        case 17: return f1(x);
        case 101: return f2(x);
        default: return -1;
    }
}
```

生成：

```
clang++ -std=c++20 -O3 -fno-inline -S -masm=intel \
    switch_table.cpp -o switch_table.s

clang++ -std=c++20 -O3 -fno-inline -c \
    switch_table.cpp -o switch_table.o

objdump -dr -Mintel switch_table.o > switch_table.objdump.txt
readelf -r switch_table.o > switch_table.reloc.txt
```

交付物：

1. 判断 `dense_dispatch` 是否使用跳转表。
2. 判断 `sparse_dispatch` 是否使用比较链或其他策略。
3. 找出范围检查指令，并解释为什么可能使用 `unsigned` 条件。
4. 如果出现 `jmpqwordptr[rip+...+rax*8]`，写出表项地址公式。
5. 在 `readelf-r` 中找出和跳转表相关的重定位项，说明目标文件阶段为什么还需要重定位。

3.3.24 实验 12.5：call/ret 和返回地址

实验

创建 `labs/ch12_control_flow/call_ret.cpp`：

```
#include <cstdio>

extern "C" int twice(int x)
{
```

```

    return x * 2;
}

extern "C" int caller(int x)
{
    int y = twice(x);
    return y + 1;
}

int main()
{
    std::printf("%d\n", caller(21));
}

```

构建带调试信息的版本：

```

clang++ -std=c++20 -O0 -g call_ret.cpp -o call_ret_00
objdump -d -Mintel call_ret_00 > call_ret_00.objdump.txt

```

用 GDB 观察：

```

gdb ./call_ret_00
(gdb) break caller
(gdb) run
(gdb) disassemble /r caller
(gdb) info registers rip rsp rbp
(gdb) x/4gx $rsp
(gdb) stepi
(gdb) info registers rip rsp rbp
(gdb) x/4gx $rsp

```

交付物：

1. 找到 `call` 指令地址和下一条指令地址。
2. 在执行 `call` 前记录 `rsp`。
3. 在进入 `twice` 后记录 `rsp` 和栈顶 8 字节。
4. 证明栈顶保存的是返回地址。
5. 执行到 `ret` 前后，记录 `rip` 和 `rsp` 的变化。
6. 对 -03 再构建一次，观察是否发生内联或尾调用，并解释差异。

3.3.25 本章作业

习题与作业

作业 1：条件码推导表

给定下面位模式：

```

a = 0xffffffff
b = 0x00000001

```

完成表格：

- 把 `a`、`b` 分别解释成 `std::int32_t` 和 `std::uint32_t`。
- 对 `cmpa,b` 写出 signed 小于、signed 大于、unsigned 小于、unsigned 大于的真假。
- 说明 `setl`、`setg`、`setb`、`seta` 各会产生 0 还是 1。
- 用文字解释为什么同一个位模式会得到不同结果。

作业 2：控制流图恢复

对下面汇编恢复控制流图和可能 C-like 伪代码：

```

foo:
    xor eax, eax
    test esi, esi
    jle .ldone
    xor ecx, ecx
.Lloop:
    mov edx, dword ptr [rdi + rcx*4]
    cmp edx, 10
    jl .lskip
    add eax, edx
.Lskip:
    inc rcx
    cmp ecx, esi
    jl .lloop
.Ldone:
    ret

```

要求：

1. 切分基本块。
2. 给每条边标注条件。
3. 说明每个 `jcc` 使用 `signed` 还是 `unsigned` 语义。
4. 写出数组访问地址公式。
5. 给出至少一种可能的 C++ 源码。

作业 3: `switch` 反推

给定：

```
cmp edi, 5
ja .Ldefault
mov eax, edi
jmp qword ptr [rip + .Ltable + rax*8]
```

要求：

1. 解释为什么 `ja` 可以同时过滤负数和大于 5 的值。
2. 写出跳转表索引公式。
3. 假设 `.Ltable=0x5000`, `edi=4`, 写出表项地址。
4. 说明这是直接跳转还是间接跳转。
5. 说明这种结构对分支预测有什么潜在影响。

作业 4: `call/ret` 地址审计

从任意一个你自己编译的 `-O0` 程序中选择一个非内联函数调用，提交：

- `objdump-d-Mintel` 中调用者和被调用者的反汇编。
- `call` 的机器码字节。
- `call` 指令地址、下一条指令地址、目标函数地址。
- GDB 中 `call` 前后的 `rsp`、`rip` 和栈顶值。
- 用具体数值证明返回地址被压栈。

作业 5: 分支性能小报告

基于实验 12.3 写一份 800 字以上报告，必须包含：

- CPU 型号、编译器版本、编译参数。
- 输入数据分布。
- 汇编核心循环。
- 运行时间表。
- `branch` 和 `branch-miss` 数据；若无法获得，解释原因。
- 结论和限制：不要只写“`branchless` 更快”或“`branch` 更快”，必须说明条件。

评分标准

本章作业按下面标准评价：

1. 是否能从 `flags` 生产指令追踪到 `flags` 消费指令。
2. 是否能正确区分 `signed` 和 `unsigned` 条件码。
3. 是否能切分基本块并画出完整边。
4. 是否能把地址公式、循环变量和内存访问联系起来。
5. 是否能用真实工具输出支持自己的结论。
6. 是否明确区分 C++ 语言语义、ISA 语义和微架构性能现象。

3.3.26 本章小结

本章把单条指令扩展成了程序路径。你应该把控制流理解成三层：

```
source level:
if / while / for / switch / function call

ISA level:
cmp/test -> flags -> jcc/setcc/cmov
call -> push return address + jump
ret -> pop return address + jump
```

```
jmp -> direct or indirect rip assignment
```

```
microarchitecture level:
prediction, speculation, BTB, RSB, misprediction recovery
```

从性能工程角度，控制流不是孤立问题。它和输入分布、数据布局、内存访问、向量化、代码布局、前端带宽和预测器状态交织在一起。下一章进入 System V AMD64 ABI，把函数调用的参数、返回值、寄存器保存、栈对齐和结构体传递规则讲清楚。到那时，`call` 和 `ret` 就不只是跳转，而会成为二进制接口的一部分。

3.4 System V AMD64 ABI

3.4.1 本章要解决的问题

前面几章已经能看懂一段函数内部的汇编：整数指令、地址计算、条件跳转、循环、`call` 和 `ret`。但只知道这些还不够。真实程序不是一个函数孤立运行，而是由大量函数、目标文件、静态库、动态库、运行时和操作系统共同组成。

问题来了：一个 C++ 文件里编译出来的函数，为什么能被另一个 C 文件调用？一个手写汇编函数，为什么能返回给 C++ 调用者？动态库里的 `printf`，为什么知道你的参数放在哪里？异常展开、调试器栈回溯、profiler 采样，为什么能沿着调用链走？

答案是 **ABI** (Application Binary Interface)。ABI 是二进制层面的契约。它规定：

- 函数参数放在哪些寄存器或栈位置。
- 返回值放在哪里。
- 哪些寄存器由调用者保存，哪些由被调用者保存。
- 栈在调用点如何对齐。
- 结构体、浮点、向量、变参函数如何传递。
- 函数入口、函数尾声、调试信息和异常展开如何约定。

本章讲 Linux、macOS 等类 Unix x86-64 平台常用的 System V AMD64 ABI。Windows x64 使用另一套 ABI，参数寄存器、shadow space、保存规则都不同。本书后续默认讨论 System V AMD64，除非明确说明。

核心理想

ISA 告诉 CPU 一条指令怎么执行；ABI 告诉不同二进制代码如何互相调用。读函数级汇编时，如果不懂 ABI，就无法可靠判断参数、返回值、栈槽、寄存器保存和跨语言边界。

3.4.2 ABI 为什么必须存在

考虑两个源文件：

```
// add.cpp
extern "C" int add2(int a, int b)
{
    return a + b;
}
```

```
// main.cpp
#include <cstdio>

extern "C" int add2(int, int);

int main()
{
    std::printf("%d\n", add2(10, 32));
}
```

它们可以分开编译：

```
clang++ -std=c++20 -O3 -c add.cpp -o add.o
clang++ -std=c++20 -O3 -c main.cpp -o main.o
clang++ add.o main.o -o app
```

`main.o` 编译时并不知道 `add2` 的机器码是什么，但它知道怎样调用 `add2`：

```
edi = 10
esi = 32
call add2
read return value from eax
```

`add.o` 编译时也不知道谁会调用它，但它知道入口时参数在哪里：

```
on entry:
    edi = first int argument
    esi = second int argument

on return:
    eax = int return value
```


这就是 ABI 的意义。调用者和被调用者不需要共享源代码，不需要由同一个编译器同时优化，只要遵守同一套二进制约定，就可以链接和运行。

心智模型

函数调用不是“跳到另一个 C++ 函数”。机器层面更像一次协议交接：

```
caller:
    arrange arguments
    preserve caller-owned live values if needed
    align stack
    call callee
    read return value

callee:
    assume arguments are in ABI-defined locations
    preserve callee-saved registers it modifies
    restore stack
    ret
```

3.4.3 先掌握最常见的整数参数

System V AMD64 ABI 中，前 6 个整数或指针类参数通常放在这些寄存器里：

参数序号	64 位寄存器	常见 32 位整数视图
第 1 个	<code>rdi</code>	<code>edi</code>
第 2 个	<code>rsi</code>	<code>esi</code>
第 3 个	<code>rdx</code>	<code>edx</code>
第 4 个	<code>rcx</code>	<code>ecx</code>
第 5 个	<code>r8</code>	<code>r8d</code>
第 6 个	<code>r9</code>	<code>r9d</code>

整数返回值通常放在 `rax`；如果返回 `int`，有效结果在 `eax`。

C++：

```
extern "C" int sum6(int a, int b, int c, int d, int e, int f)
{
    return a + b + c + d + e + f;
}
```

可能汇编：

```
sum6:
    lea eax, [rdi + rsi]
    add eax, edx
    add eax, ecx
    add eax, r8d
    add eax, r9d
    ret
```

入口状态：

```
edi = a
esi = b
edx = c
ecx = d
r8d = e
r9d = f
```

返回约定：

```
eax = a + b + c + d + e + f
ret
```

常见误区

`rcx` 在 System V ABI 中是第 4 个整数参数；在 Windows x64 ABI 中，`rcx` 是第 1 个整数参数。看资料或反汇编时必须先确认目标 ABI。

3.4.4 超过 6 个整数参数：栈上传参

第 7 个及以后的整数或指针类参数通常通过栈传递。看例子：

```
extern "C" long sum8(long a, long b, long c, long d,
                    long e, long f, long g, long h)
{
    return a + b + c + d + e + f + g + h;
}
```

一种可能汇编：

```
sum8:
    lea rax, [rdi + rsi]
    add rax, rdx
    add rax, rcx
    add rax, r8
    add rax, r9
    add rax, qword ptr [rsp + 8]
    add rax, qword ptr [rsp + 16]
    ret
```

函数刚进入时，`rsp` 指向返回地址：

```
on entry to sum8:

    [rsp + 0]   return address
    [rsp + 8]   7th argument: g
    [rsp + 16]  8th argument: h

    rdi = a
    rsi = b
    rdx = c
    rcx = d
    r8  = e
    r9  = f
```

为什么第 7 个参数是 `[rsp+8]` 而不是 `[rsp]`？因为 `call` 已经把返回地址压到了栈顶。被调用函数入口处，栈顶首先是返回地址，栈上传递的第一个参数在返回地址上方 8 字节。

深入理解

如果被调用函数一进来就 `push rbp` 或 `sub rsp, ...`，参数相对 `rsp` 的偏移会改变。因此读汇编时必须知道当前 `rsp` 已经移动了多少。使用帧指针时，编译器常用 `rbp` 作为稳定基准。

3.4.5 浮点和向量参数

浮点参数走 SSE/AVX 寄存器通道。常见规则：

- `float`、`double` 参数通常放在 `xmm0` 到 `xmm7`。
- 浮点返回值通常放在 `xmm0`。
- 整数参数寄存器序列和浮点参数寄存器序列分别计数。

C++：

```
extern "C" double mix_args(int a, double b, int c, float d)
{
    return double(a + c) + b + double(d);
}
```

入口位置通常是：

```
edi = a
xmm0 = b
esi = c
xmm1 = d
```

注意 `c` 是第 2 个整数类参数，所以在 `esi`，不是 `edx`。因为 `doubleb` 消耗的是浮点寄存器 `xmm0`，不消耗整数参数寄存器。

可能汇编片段：

```
mix_args:
    add edi, esi
    cvtsi2sd xmm2, edi
    addsd xmm2, xmm0
    cvtss2sd xmm1, xmm1
    addsd xmm1, xmm2
    movapd xmm0, xmm1
    ret
```

返回值在 `xmm0`。这里的 `cvtsi2sd` 把整数转换成 `double`，`cvtss2sd` 把 `float` 扩展成 `double`。

常见误区

`xmm0` 既可能是第 1 个浮点参数，也可能是浮点返回值。函数入口和函数返回是两个不同时间点。读汇编时要区分“入口时 `xmm0` 是参数”和“返回时 `xmm0` 是结果”。

3.4.6 返回值位置

常见返回值约定：

返回类型	返回位置	说明
int、unsigned	eax	写 32 位会清零 rax 高 32 位
long、指针	rax	64 位整数或地址
double、float	xmm0	标量浮点返回
两个整数机器字	rax、rdx	常见小结构体返回形式
大结构体	隐藏 sret 指针	调用者提供返回对象地址

小结构体例子：

```
struct Pair {
    long x;
    long y;
};

extern "C" Pair make_pair(long a, long b)
{
    return Pair{a + 1, b + 2};
}
```

可能汇编：

```
make_pair:
    lea rax, [rdi + 1]
    lea rdx, [rsi + 2]
    ret
```

返回时：

```
rax = result.x
rdx = result.y
```

调用者按 ABI 知道这两个寄存器共同组成返回对象。

3.4.7 大结构体返回：隐藏 sret 指针

大结构体通常不能只靠 rax 和 rdx 返回。调用者会先分配好返回对象空间，然后把地址作为隐藏参数传给被调用者。

C++：

```
struct Big {
    long a;
    long b;
    long c;
};

extern "C" Big make_big(long x)
{
    return Big{x, x + 1, x + 2};
}
```

逻辑上像这样：

```
extern "C" Big make_big(long x);
```

机器层面更像：

```
extern "C" void make_big_hidden(Big* out, long x);
```

可能汇编：

```
make_big:
    mov rax, rdi
    mov qword ptr [rdi], rsi
    lea rcx, [rsi + 1]
    mov qword ptr [rdi + 8], rcx
    add rsi, 2
    mov qword ptr [rdi + 16], rsi
    ret
```

入口状态：

```
rdi = hidden pointer to caller-allocated result object
rsi = x
```

返回状态：

```
memory at rdi:
    [rdi + 0] = x
    [rdi + 8] = x + 1
    [rdi + 16] = x + 2

rax = rdi
```

这解释了一个重要现象：源代码里看起来只有一个参数 `x`，但汇编入口时 `rdi` 可能不是 `x`，而是隐藏返回对象指针。真正的 `x` 变成了 `rsi`。

核心理想

看到函数第一个参数“不对劲”时，不要马上怀疑编译器。先检查返回类型是否是大结构体，是否存在隐藏 `sret` 指针。

3.4.8 结构体参数的简化分类规则

System V AMD64 ABI 对聚合类型有详细分类算法。完整规则涉及 `INTEGER`、`SSE`、`SSEUP`、`X87`、`COMPLEX_X87`、`MEMORY` 等类别，还会把对象按 8 字节分块。本书先给出足以读大多数性能代码的简化流程：

1. 把结构体按 8 字节为单位切成一个或多个 `eightbyte`。
2. 如果对象太大、字段未对齐、或 C++ 类型有非平凡拷贝/析构语义，通常走内存或隐藏引用。
3. 如果不超过两个 `eightbyte`，且每块能分类成 `INTEGER` 或 `SSE`，就可能通过寄存器传递。
4. `INTEGER` 类通常走通用寄存器。
5. `SSE` 类通常走 `xmm` 寄存器。
6. 一旦需要的寄存器不够，相关参数可能改走栈。

例子 1：

```
struct TwoLong {
    long a;
    long b;
};

extern "C" long use_two_long(TwoLong x)
{
    return x.a + x.b;
}
```

两个 8 字节整数块，可能入口为：

```
rdi = x.a
rsi = x.b
```

例子 2：

```
struct TwoDouble {
    double a;
    double b;
};

extern "C" double use_two_double(TwoDouble x)
{
    return x.a + x.b;
}
```

两个 8 字节 SSE 块，可能入口为：

```
xmm0 = x.a
xmm1 = x.b
```

例子 3：

```
struct Mixed {
    int a;
    double b;
};

extern "C" double use_mixed(Mixed x)
{
    return double(x.a) + x.b;
}
```

常见入口可能是：

```
edi = x.a
xmm0 = x.b
```

这说明结构体并不总是“整体放到内存”。小结构体可能被拆分到多个寄存器里。

常见误区

结构体 ABI 分类是容易出错的地方。字段顺序、padding、对齐、浮点/整数混合、C++ 非平凡类型都会改变传参方式。遇到公开二进制接口时，不要凭感觉改结构体布局。

3.4.9 caller-saved 和 callee-saved

寄存器数量有限。函数调用前后，哪些寄存器可以被破坏？哪些必须保持？ABI 用 caller-saved 和 callee-saved 分工。

类别	寄存器	含义
caller-saved	<code>rax</code> 、 <code>rcx</code> 、 <code>rdx</code> 、 <code>rsi</code> 、 <code>rdi</code> 、 <code>r8–r11</code>	调用者若还需要这些值，调用前自己保存
callee-saved	<code>rbx</code> 、 <code>rbp</code> 、 <code>r12–r15</code>	被调用者如果修改，返回前必须恢复
特殊	<code>rsp</code>	返回前必须恢复到入口约定状态
浮点/向量	<code>xmm0–xmm15</code>	System V 下通常 caller-saved

为什么这样分？如果所有寄存器都由调用者保存，每次调用都很重；如果所有寄存器都由被调用者保存，每个函数入口都很重。ABI 把一部分寄存器定义为 caller-saved，适合短生命周期临时值；把另一部分定义为 callee-saved，适合跨调用仍要保留的值。

3.4.9.1 callee-saved 示例

C++:

```
extern "C" long g(long);

extern "C" long keep_across_call(long x)
{
    long y = x * 3;
    long z = g(x);
    return y + z;
}
```

`y` 必须跨过对 `g` 的调用继续存在。编译器可能把 `y` 放在 callee-saved 寄存器 `rbx`：

```
keep_across_call:
    push rbx
    lea rbx, [rdi + rdi*2]
    call g
    add rax, rbx
    pop rbx
    ret
```

解释：

```
push rbx
    save caller's original rbx

lea rbx, [rdi + rdi*2]
    rbx = x * 3

call g
    g may clobber caller-saved registers
    g must preserve rbx if it uses rbx

add rax, rbx
    rax = g(x) + x * 3

pop rbx
    restore caller's original rbx

ret
```

如果 `keep_across_call` 修改了 `rbx` 却没有恢复，调用它的上层函数可能崩溃或产生错误结果。

深入理解

callee-saved 不是说这个寄存器“不能用”。它的意思是：可以用，但你要把调用者看到的旧值还回去。常见保存方式是 `push` 和 `pop`，也可以保存到栈帧中的固定偏移。

3.4.10 栈对齐规则

System V AMD64 ABI 要求调用点满足 16 字节栈对齐。更具体地说：

```
before call:
    rsp is 16-byte aligned

call pushes 8-byte return address

on callee entry:
    rsp + 8 is 16-byte aligned
```

```
rsp itself is 8 mod 16
```

一个函数入口常见状态：

```
rsp points to return address
rsp mod 16 = 8
```

如果函数要再调用别的函数，它必须在自己的 `call` 前把 `rsp` 调整到 16 字节对齐。

例子：

```
caller:
  sub rsp, 8
  call callee
  add rsp, 8
  ret
```

假设 `caller` 入口时 `rsp mod 16 = 8`：

```
entry:
  rsp mod 16 = 8

sub rsp, 8:
  rsp mod 16 = 0

call callee:
  pushes return address
  callee entry rsp mod 16 = 8

add rsp, 8:
  restore caller's stack pointer
```

为什么要对齐？一方面 ABI 需要统一约定；另一方面 SIMD load/store、局部对象对齐、调用约定和编译器生成代码都依赖这个规则。一些指令如 `movaps` 对内存地址有对齐要求；即使现代 CPU 对很多未对齐访问能处理，错误的 ABI 栈对齐仍然可能导致崩溃或性能下降。

常见误区

手写汇编调用 C/C++ 函数时，最常见错误之一就是忘记在 `call` 前对齐 `rsp`。这种错误可能在简单测试里不暴露，换一个编译器版本、库函数或 CPU 状态就崩。

3.4.11 栈帧、帧指针和局部变量

带帧指针的函数常见形态：

```
push rbp
mov rbp, rsp
sub rsp, 32
...
add rsp, 32
pop rbp
ret
```

或等价尾声：

```
leave
ret
```

进入函数后栈可能长这样：

```
higher addresses

[rbp + 16]  stack argument 2 if present
[rbp + 8]   return address
[rbp + 0]   saved old rbp
[rbp - 8]   local/spill slot
[rbp - 16]  local/spill slot
[rbp - 24]  local/spill slot

lower addresses
```

`rbp` 的值是稳定。函数里 `rsp` 可能因为 `push`、`sub rsp, ...`、调用准备而变化，但 `rbp` 通常保持不变，方便调试器、低优化级代码和人工阅读。

优化级较高时，编译器常省略帧指针：

```
clang++ -std=c++20 -O3 -fomit-frame-pointer ...
```

性能分析时，很多工程会保留帧指针以改善采样栈回溯：

```
clang++ -std=c++20 -O3 -fno-omit-frame-pointer ...
```

是否保留帧指针是性能、调试、profiling 之间的工程权衡。现代生产服务常为了 profiler 可用性保留帧指针。

3.4.12 red zone

System V AMD64 ABI 定义了 **red zone** (红区): 用户态函数入口时, **rsp** 下方 128 字节区域可以被叶子函数临时使用, 而不必移动 **rsp**。

位置:

```
[rsp - 128] ... [rsp - 1]  red zone
[rsp + 0]                return address
```

叶子函数是不调用其他函数的函数。例子:

```
extern "C" long leaf(long x)
{
    long tmp[4];
    tmp[0] = x + 1;
    tmp[1] = x + 2;
    tmp[2] = x + 3;
    tmp[3] = x + 4;
    return tmp[0] + tmp[1] + tmp[2] + tmp[3];
}
```

如果优化器不能完全消除数组, 叶子函数可能使用 **rsp** 下方空间, 而不执行 **sub rsp, ...**。

red zone 的限制:

- 只适合不调用其他函数的叶子函数。
- 一旦执行 **call**, 新的返回地址会写到更低地址, 可能覆盖调用者 red zone 中的内容。
- 内核、裸机、中断上下文通常不能依赖 red zone, 常用 **-mno-red-zone**。
- 手写汇编使用 red zone 时必须确认目标环境允许。

```
clang++ -std=c++20 -O3 -mno-red-zone ...
```

3.4.13 变参函数和 va_list

变参函数比普通函数复杂, 因为被调用者不知道静态参数列表之后还有多少参数、类型是什么。

典型调用:

```
#include <cstdio>

extern "C" void print_values(int x, double y)
{
    std::printf("x=%d y=%f\n", x, y);
}
```

调用 **printf** 时:

```
rdi = pointer to format string
esi = x
xmm0 = y
al = number of vector registers used for floating arguments
call printf
```

System V ABI 要求调用变参函数时, **al** 携带用于传递浮点/向量参数的向量寄存器数量上界。你经常在调用 **printf** 前看到:

```
mov eax, 1
call printf
```

如果没有浮点参数, 可能看到:

```
xor eax, eax
call printf
```

变参函数内部使用 **va_start** 时, 编译器会维护一个 **va_list**, 里面通常包含:

```
gp_offset
fp_offset
overflow_arg_area
reg_save_area
```

直觉上:

- 前几个普通参数可能来自寄存器保存区。
- 超出寄存器容量的参数来自栈上的 **overflow** 区域。
- 浮点参数和整数参数有不同 **offset**。

深入理解

手写汇编调用变参函数时, 不能只把参数放到寄存器就结束。尤其是调用带浮点参数的 **printf**, 还要正确设置 **al**。这个细节非常适合实验, 因为错误代码有时能运行, 有时输出错, 有时在不同 **libc** 或优化级别下表现不同。

3.4.14 C++ 名字修饰和 extern"C"

ABI 不只包含寄存器和栈，也包含符号名约定。C++ 支持函数重载、命名空间、类成员函数，因此编译器会做 **name mangling**（名字修饰）。

C++:

```
int add(int, int);
double add(double, double);
```

两个函数源代码名都叫 **add**，目标文件里的符号名必须不同。编译器可能生成类似：

```
_Z3addii
_Z3adddd
```

如果希望 C++ 函数使用 C ABI 符号名，可以写：

```
extern "C" int add(int, int);
```

这样符号名通常就是 **add**。本书实验里大量使用 **extern"C**”，不是因为 C++ 不好，而是为了让汇编符号名稳定、易读、易链接。

3.4.15 手写汇编函数的最低正确性清单

如果你要写一个被 C++ 调用的 **.S** 函数，至少检查：

1. 符号名和 C++ 声明是否一致。
2. 参数寄存器是否按 System V ABI 读取。
3. 返回值是否写到正确寄存器。
4. 修改 callee-saved 寄存器前是否保存，返回前是否恢复。
5. 返回前 **rsp** 是否恢复。
6. 调用其他函数前 **rsp** 是否 16 字节对齐。
7. 调用变参函数前 **al** 是否正确。
8. 是否误用 red zone。
9. 是否需要 **.type**、**.size** 和 **.note.GNU-stack**。

一个最小汇编函数：

```
.intel_syntax noprefix
.text
.globl asm_add2
.type asm_add2, @function
asm_add2:
    lea eax, [rdi + rsi]
    ret
.size asm_add2, .-asm_add2
.section .note.GNU-stack,"",@progbits
```

对应 C++ 声明：

```
extern "C" int asm_add2(int, int);
```

3.4.16 案例：C++ 调汇编

创建两个文件。

labs/ch13_system_v_abi/asm_add.S:

```
.intel_syntax noprefix
.text
.globl asm_add6
.type asm_add6, @function
asm_add6:
    lea eax, [rdi + rsi]
    add eax, edx
    add eax, ecx
    add eax, r8d
    add eax, r9d
    ret
.size asm_add6, .-asm_add6
.section .note.GNU-stack,"",@progbits
```

labs/ch13_system_v_abi/main.cpp:

```
#include <cstdio>

extern "C" int asm_add6(int, int, int, int, int, int);

int main()
{
    int r = asm_add6(1, 2, 3, 4, 5, 6);
    std::printf("%d\n", r);
}
```


}

构建:

```
clang++ -std=c++20 -O2 -c main.cpp -o main.o
clang++ -c asm_add.S -o asm_add.o
clang++ main.o asm_add.o -o abi_demo
./abi_demo
```

预期输出:

21

反汇编:

```
objdump -d -Mintel abi_demo > abi_demo.objdump.txt
```

你应该在 `main` 的调用点看到参数被放入 ABI 寄存器, 在 `asm_add6` 中看到这些寄存器被读取。

3.4.17 案例: 汇编调 C++

C++:

```
#include <cstdio>

extern "C" int cpp_square(int x)
{
    return x * x;
}

extern "C" int asm_call_square(int);

int main()
{
    std::printf("%d\n", asm_call_square(9));
}
```

汇编:

```
.intel_syntax noprefix
.text
.globl asm_call_square
.type asm_call_square, @function
asm_call_square:
    sub rsp, 8
    call cpp_square
    add eax, 1
    add rsp, 8
    ret
.size asm_call_square, .-asm_call_square
.section .note.GNU-stack,"",@progbits
```

为什么要 `subrsp, 8`? 因为 `asm_call_square` 入口时 `rspmod16=8`。在它执行 `call cpp_square` 之前, 必须让 `rspmod16=0`。

执行路径:

```
main:
    edi = 9
    call asm_call_square

asm_call_square:
    align stack
    call cpp_square
    eax = cpp_square(9) + 1
    restore stack
    ret
```

返回值:

```
cpp_square(9) = 81
asm_call_square(9) = 82
```

3.4.18 实验 13.1: 参数寄存器和栈参数地图

实验

创建 `labs/ch13_system_v_abi/args.cpp`:

```
#include <stdint>

extern "C" long sum8(long a, long b, long c, long d,
                    long e, long f, long g, long h)
{
    return a + b + c + d + e + f + g + h;
}
```

```

}

extern "C" double mix(int a, double b, int c, float d)
{
    return double(a + c) + b + double(d);
}

struct TwoLong {
    long a;
    long b;
};

extern "C" long use_two_long(TwoLong x)
{
    return x.a * 10 + x.b;
}

```

生成:

```

mkdir -p labs/ch13_system_v_abi/build
clang++ -std=c++20 -O0 -S -masm=intel args.cpp \
-o labs/ch13_system_v_abi/build/args_00.s
clang++ -std=c++20 -O3 -S -masm=intel args.cpp \
-o labs/ch13_system_v_abi/build/args_03.s
clang++ -std=c++20 -O3 -c args.cpp \
-o labs/ch13_system_v_abi/build/args.o
objdump -d -Mintel labs/ch13_system_v_abi/build/args.o \
> labs/ch13_system_v_abi/build/args.objdump.txt

```

交付物:

1. 对 `sum8` 列出 8 个参数分别在哪里。
2. 解释为什么第 7、第 8 个参数从栈读取。
3. 对 `mix` 列出整数参数和浮点参数的寄存器序列。
4. 对 `use_two_long` 判断结构体是否拆到寄存器。
5. 对比 `-O0` 和 `-O3` 栈帧差异。
6. 从 `objdump` 记录至少 8 条机器码字节。

3.4.19 实验 13.2: 破坏 callee-saved 寄存器

实验

创建 `labs/ch13_system_v_abi/callee_saved.S`:

```

.intel_syntax noprefix
.text

.globl bad_clobber_rbx
.type bad_clobber_rbx, @function
bad_clobber_rbx:
    mov rbx, 0x1234567812345678
    ret
.size bad_clobber_rbx, .-bad_clobber_rbx

.globl good_clobber_rbx
.type good_clobber_rbx, @function
good_clobber_rbx:
    push rbx
    mov rbx, 0x1234567812345678
    pop rbx
    ret
.size good_clobber_rbx, .-good_clobber_rbx

.globl check_bad
.type check_bad, @function
check_bad:
    push rbx
    mov rbx, 0x1122334455667788
    call bad_clobber_rbx
    mov rax, rbx
    pop rbx
    ret
.size check_bad, .-check_bad

.globl check_good
.type check_good, @function
check_good:
    push rbx

```

```

    mov rbx, 0x1122334455667788
    call good_clobber_rbx
    mov rax, rbx
    pop rbx
    ret
.size check_good, .-check_good

.section .note.GNU-stack,"",@progbits

```

创建 labs/ch13_system_v_abi/callee_saved.cpp:

```

#include <stdint>
#include <stdio>

extern "C" std::uint64_t check_bad();
extern "C" std::uint64_t check_good();

int main()
{
    std::printf("bad = 0x%llx\n",
        (unsigned long long)check_bad());
    std::printf("good = 0x%llx\n",
        (unsigned long long)check_good());
}

```

构建:

```

clang++ -std=c++20 -O2 -c callee_saved.cpp -o callee_saved.o
clang++ -c callee_saved.S -o callee_saved_asm.o
clang++ callee_saved.o callee_saved_asm.o -o callee_saved_demo
./callee_saved_demo
objdump -d -Mintel callee_saved_demo > callee_saved_demo.objdump.txt

```

交付物:

1. 解释 `bad_clobber_rbx` 违反了哪条 ABI 规则。
2. 解释 `good_clobber_rbx` 如何保存和恢复 `rbx`。
3. 说明 `check_bad` 为什么返回被破坏后的值。
4. 在 GDB 中单步观察 `rbx` 变化。
5. 把这个实验推广到 `r12`，写一个新的坏例子和修复例子。

3.4.20 实验 13.3: 栈对齐和 `movaps`

实验

创建 labs/ch13_system_v_abi/stack_align.S:

```

.intel_syntax noprefix
.text

.globl bad_movaps_store
.type bad_movaps_store, @function
bad_movaps_store:
    sub rsp, 16
    xorps xmm0, xmm0
    movaps xmmword ptr [rsp], xmm0
    add rsp, 16
    ret
.size bad_movaps_store, .-bad_movaps_store

.globl good_movaps_store
.type good_movaps_store, @function
good_movaps_store:
    sub rsp, 24
    xorps xmm0, xmm0
    movaps xmmword ptr [rsp], xmm0
    add rsp, 24
    ret
.size good_movaps_store, .-good_movaps_store

.section .note.GNU-stack,"",@progbits

```

创建 labs/ch13_system_v_abi/stack_align.cpp:

```

#include <stdio>

extern "C" void bad_movaps_store();
extern "C" void good_movaps_store();

int main()

```

```
{
    std::puts("calling good");
    good_movaps_store();
    std::puts("calling bad");
    bad_movaps_store();
    std::puts("done");
}
```

构建并运行：

```
clang++ -std=c++20 -O2 -c stack_align.cpp -o stack_align.o
clang++ -c stack_align.S -o stack_align_asm.o
clang++ stack_align.o stack_align_asm.o -o stack_align_demo
./stack_align_demo
```

交付物：

1. 计算 `bad_movaps_store` 中 `sub rsp, 16` 后的栈对齐状态。
2. 计算 `good_movaps_store` 中 `sub rsp, 24` 后的栈对齐状态。
3. 解释 `movaps` 为什么要求目标地址对齐。
4. 如果程序崩溃，用 GDB 或 shell 记录信号和崩溃位置。
5. 把 `movaps` 改成 `movups`，观察行为是否变化，并解释为什么不能把这当成“不需要 ABI 对齐”的证据。

3.4.21 实验 13.4：大结构体返回和隐藏指针

实验

创建 `labs/ch13_system_v_abi/sret.cpp`：

```
#include <stdint>

struct Small {
    std::uint64_t a;
    std::uint64_t b;
};

struct Big {
    std::uint64_t a;
    std::uint64_t b;
    std::uint64_t c;
};

extern "C" Small make_small(std::uint64_t x)
{
    return Small{x, x + 1};
}

extern "C" Big make_big(std::uint64_t x)
{
    return Big{x, x + 1, x + 2};
}
```

生成：

```
clang++ -std=c++20 -O0 -S -masm=intel sret.cpp -o sret_00.s
clang++ -std=c++20 -O3 -S -masm=intel sret.cpp -o sret_03.s
clang++ -std=c++20 -O3 -c sret.cpp -o sret.o
objdump -d -Mintel sret.o > sret.objdump.txt
```

交付物：

1. 说明 `Small` 如何返回。
2. 说明 `Big` 如何返回。
3. 找出 `make_big` 中隐藏 `sret` 指针的位置。
4. 解释为什么源代码参数 `x` 在汇编中可能不是第一个参数寄存器。
5. 画出调用者为 `Big` 分配返回对象并传入地址的过程。

3.4.22 实验 13.5：变参函数和 `al`

实验

创建 `labs/ch13_system_v_abi/varargs.cpp`：

```
#include <stdio>

extern "C" void print_int(int x)
```

```

{
    std::printf("x=%d\n", x);
}

extern "C" void print_double(double x)
{
    std::printf("x=%f\n", x);
}

extern "C" void print_mix(int x, double y)
{
    std::printf("x=%d y=%f\n", x, y);
}

```

生成：

```

clang++ -std=c++20 -O2 -S -masm=intel varargs.cpp -o varargs.s
clang++ -std=c++20 -O2 -c varargs.cpp -o varargs.o
objdump -d -Mintel varargs.o > varargs.objdump.txt

```

交付物：

1. 找出每次调用 `printf` 前 `eax` 或 `al` 如何设置。
2. 解释 `print_int` 为什么通常设置为 0。
3. 解释 `print_double` 和 `print_mix` 为什么通常设置为 1。
4. 写一个手写汇编函数调用 `printf` 打印 `double`，先故意错误设置 `al`，再修复。
5. 记录错误版本和修复版本在你的系统上的表现。

3.4.23 本章作业

习题与作业

作业 1: ABI 参数表

对下面函数签名，给出 System V AMD64 下每个参数的常见传递位置：

```

extern "C" long f1(long a, int b, void* p, long d,
                  long e, long f, long g);

extern "C" double f2(int a, double b, int c,
                    float d, double e);

struct P { long a; long b; };
extern "C" long f3(P p, long x);

```

要求：

1. 区分通用寄存器和 `xmm` 寄存器。
2. 标出哪些参数会走栈。
3. 对结构体参数说明你的分类依据。
4. 说明返回值位置。

作业 2: 栈布局推导

给定函数入口汇编：

```

push rbp
mov rbp, rsp
push rbx
sub rsp, 24

```

要求画出执行完这些指令后的栈布局，至少包含：

- 返回地址。
- saved `rbp`。
- saved `rbx`。
- 24 字节局部区域。
- `rbp` 和 `rsp` 分别指向哪里。

作业 3: 修复错误汇编

下面函数被 C++ 调用：

```

.intel_syntax noprefix
.globl broken
broken:

```

```

mov rbx, rdi
call helper
add rax, rbx
ret

```

要求：

1. 找出所有 ABI 风险。
2. 修复 callee-saved 寄存器问题。
3. 修复调用前栈对齐问题。
4. 给出修复后的完整汇编。
5. 解释每条新增指令为什么必要。

作业 4: *sret* 逆向

给定汇编：

```

make_obj:
    mov rax, rdi
    mov qword ptr [rdi], rsi
    mov qword ptr [rdi + 8], rdx
    mov qword ptr [rdi + 16], rcx
    ret

```

要求：

1. 判断是否存在隐藏返回对象指针。
2. 推测源代码返回类型大小。
3. 推测显式参数可能有哪些。
4. 解释为什么返回值还会写 *rax*。

作业 5: 跨语言 ABI 小项目

写一个小项目，包含：

- 一个 C++ 文件声明并调用 3 个手写汇编函数。
- 汇编函数 1：整数参数求和，至少 8 个参数。
- 汇编函数 2：调用一个 C++ helper，并正确保存 callee-saved 寄存器。
- 汇编函数 3：返回一个两字段结构体。
- CMake 或 Makefile 构建脚本。
- 单元测试或简单断言。
- *objdump* 报告，解释每个函数的 ABI 行为。

评分标准

本章作业按下面标准评价：

1. 参数位置是否和 ABI 一致。
2. 是否能区分寄存器参数和栈参数。
3. 是否能正确处理 caller-saved 与 callee-saved。
4. 手写汇编是否保持栈对齐。
5. 是否能识别隐藏 *sret* 指针。
6. 是否能用 GDB、*objdump* 和实际运行结果证明结论。
7. 是否明确区分 System V AMD64 和其他 ABI。

3.4.24 本章小结

ABI 是从“单个函数内部汇编”走向“真实二进制程序”的关键桥梁。本章你应该掌握：

```

integer/pointer args:
    rdi, rsi, rdx, rcx, r8, r9, then stack

floating args:
    xmm0 - xmm7

return:
    rax / rdx for many integer-like results
    xmm0 for scalar floating results
    hidden sret pointer for large aggregate results

```

```

register ownership:
  caller-saved: caller protects live values
  callee-saved: callee restores what it modifies

stack:
  on entry, rsp points to return address
  before call, rsp must be 16-byte aligned
  leaf functions may use red zone in allowed environments

```

从高性能计算和 AI 算子开发角度，ABI 决定了 kernel 函数边界的成本和正确性：参数如何进入寄存器，哪些寄存器可以自由使用，调用 helper 前要保存什么，返回 tensor metadata 或小结构体会不会产生隐藏内存写。下一章会把这些规则用于 C++ 和汇编互操作，进一步进入可测试、可链接、可维护的手写汇编工程。

3.5 C++ 与汇编互操作

3.5.1 本章要解决的问题

上一章讲清楚了 System V AMD64 ABI：参数放在哪里、返回值放在哪里、哪些寄存器要保存、栈如何对齐。那些规则看起来像表格，但真正的价值在工程边界上才会显现出来：一个 C++ 文件如何调用手写汇编？手写汇编如何再调用 C++ helper？链接器如何把两个目标文件接到一起？内联汇编为什么经常比你想象中危险？为什么同样一条 `call`，在目标文件里还没有最终地址，链接之后才变成真实跳转？

本章把“能读汇编”推进到“能写可维护、可测试、可审计的汇编边界”。你需要掌握的不只是语法，而是一套工程流程：

```

source declaration:
  C++ header says which symbol exists and what ABI contract it has

assembly implementation:
  .S file exports that symbol and obeys ABI

object file:
  assembler emits symbols, sections, relocations, debug/unwind hints

linking:
  linker resolves references and patches call/data addresses

runtime:
  CPU only executes bytes; correctness depends on the binary contract

```

从高性能计算和 AI 算子开发角度，本章非常关键。真实算子库经常出现这样的边界：

- C++ 调度层选择某个 kernel，再调用汇编或 intrinsic 实现。
- 汇编 kernel 接收指针、维度、步长、常量、尾部长度等参数。
- kernel 内部尽量不调用其他函数；一旦调用，就必须恢复 ABI 约束。
- profiler、debugger、sanitizer、异常展开和符号工具都要能看懂这段代码。

因此，本章的核心不是“写几条汇编指令”，而是建立下面这条链路：

```

C++ type/signature
-> ABI-level parameter and return-value locations
-> assembly symbol and prologue/epilogue
-> object-file symbols and relocations
-> linked executable
-> debugger/profiler observable behavior
-> tests and performance evidence

```

核心思想

C++ 与汇编互操作的本质是二进制契约工程。C++ 编译器不会理解你手写汇编的意图；汇编器也不会检查你的 C++ 类型语义。两边能正确合作，完全依赖符号名、ABI、目标文件格式、链接器和测试共同维持边界。

3.5.2 先建立三层边界

很多初学者写手写汇编失败，不是因为不会写 `add`，而是混淆了三层不同的规则。

3.5.2.1 第一层：C++ 语言层

C++ 源代码里看到的是函数声明、类型、重载、命名空间、引用、对象生命周期、异常、访问控制和模板实例化。例如：

```

extern "C" long asm_sum8(long a, long b, long c, long d,
                        long e, long f, long g, long h);

```

这个声明告诉 C++ 编译器：

- 存在一个可调用函数。
- 函数名在链接层叫 `asm_sum8`，不要使用 C++ name mangling。
- 参数和返回值都是 `long`。
- 调用时按当前平台默认调用约定生成代码。

但这个声明没有告诉编译器函数体在哪里，也没有证明汇编实现真的返回 `long`。如果你的汇编函数把结果放进了 `rcx` 而不是 `rax`，C++ 编译器不会替你发现。

3.5.2.2 第二层：ABI 层

上一章的 ABI 表格在这里变成实际契约。对于上面的 `asm_sum8`，System V AMD64 下入口状态应当是：

```
on entry to asm_sum8:
    rdi    = a
    rsi    = b
    rdx    = c
    rcx    = d
    r8     = e
    r9     = f
    [rsp + 0] = return address
    [rsp + 8] = g
    [rsp + 16] = h

on return:
    rax = result
```

如果函数修改了 `rbx`、`rbp`、`r12` 到 `r15`，它必须在返回前恢复。若它调用其他函数，调用点前还要满足栈对齐约定。

3.5.2.3 第三层：目标文件和链接层

汇编文件最终不是直接和 C++ 源码连接，而是先变成目标文件。目标文件里包含：

- section，例如 `.text`、`.rodata`、`.data`。
- symbol，例如 `asm_sum8`、`cpp_helper`。
- relocation，例如“这里有一个 `call`，目标符号稍后由链接器填”。
- debug/unwind metadata，例如调试器和异常展开需要的信息。

可以用下面工具观察：

```
nm -C file.o
readelf -Ws file.o
readelf -r file.o
objdump -drwC -Mintel file.o
```

心智模型

源代码层问“这个函数叫什么、类型是什么”；ABI 层问“参数和返回值在哪些寄存器或栈槽”；目标文件层问“哪个符号定义在哪里，哪个地址需要链接器以后修补”。这三层都正确，跨语言调用才正确。

3.5.3 一个最小可运行工程

先做最小例子。目录结构如下：

```
labs/ch14_interop/minimal/
  CMakeLists.txt
  main.cpp
  asm_add.S
```

3.5.3.1 C++ 调用者

```
// main.cpp
#include <cassert>
#include <cstdint>
#include <cstdio>

extern "C" std::int64_t asm_add2(std::int64_t a, std::int64_t b);

int main()
{
    const std::int64_t x = 40;
    const std::int64_t y = 2;
    const std::int64_t got = asm_add2(x, y);

    assert(got == 42);
```



```
std::printf("asm_add2(%ld, %ld) = %ld\n", x, y, got);
}
```

这里的 `extern"C` 不是说这个函数由 C 写成，而是说这个声明使用 C 链接名。没有它，C++ 可能把符号名改写成带类型信息的 mangled name。

3.5.3.2 汇编实现

```
// asm_add.S
.intel_syntax noprefix
.text

.globl asm_add2
.type asm_add2, @function
asm_add2:
    lea rax, [rdi + rsi]
    ret
.size asm_add2, .-asm_add2

.section .note.GNU-stack,"",@progbits
```

逐行解释：

代码	含义
<code>.intel_syntaxnoprefix</code>	使用 Intel 语法，寄存器不写 % 前缀。
<code>.text</code>	后续内容进入代码段。
<code>.globlasm_add2</code>	导出符号，让其他目标文件可以引用。
<code>.typeasm_add2,@function</code>	标记符号类型为函数，便于工具识别。
<code>asm_add2:</code>	函数入口标签，也是符号地址。
<code>leairax,[rdi+rsi]</code>	计算第 1、2 个参数之和，放入返回寄存器。
<code>ret</code>	返回到调用者。
<code>.sizeasm_add2,.-asm_add2</code>	记录函数大小，从入口到当前位置。
<code>.note.GNU-stack</code>	告诉链接器不需要可执行栈。

常见误区

`.section.note.GNU-stack,"",@progbits` 不会生成运行时代码，但在 Linux ELF 工程中应当保留。省略它可能导致链接器给出 executable stack 警告，或者在某些环境里产生不必要的安全风险。

3.5.3.3 构建脚本

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
project(ch14_minimal LANGUAGES CXX ASM)

add_executable(ch14_minimal
    main.cpp
    asm_add.S
)

target_compile_features(ch14_minimal PRIVATE cxx_std_20)
target_compile_options(ch14_minimal PRIVATE
    $<$<COMPILE_LANGUAGE:CXX>:-Wall -Wextra -Wpedantic -O2 -g>
)
```

构建：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build -j
./build/ch14_minimal
```

审计：

```
nm -C build/ch14_minimal | rg 'asm_add2|main'
objdump -drwC -Mintel build/ch14_minimal > ch14_minimal.objdump.txt
rg -n '<asm_add2>|call.*asm_add2' ch14_minimal.objdump.txt
```

3.5.3.4 为什么文件后缀用 .S

GNU/Clang 工具链里常见约定：

后缀	含义
.s	汇编源文件，通常不经过 C 预处理器。
.S	汇编源文件，先经过 C 预处理器，再交给汇编器。

使用 .S 后，可以写：

```
#if defined(__linux__) && defined(__x86_64__)
.section .note.GNU-stack,"",@progbits
#endif
```

也可以在以后用宏生成不同 ISA 版本。但宏会降低可读性。本书训练阶段优先写直白的汇编，只有重复结构明显时才引入宏。

3.5.4 把函数签名当成二进制接口

互操作时，最危险的习惯是“先写汇编，能跑就行”。正确顺序应该是：

- 1. 写 C++ 声明，明确类型和链接名。
- 2. 按 ABI 推导参数位置、返回位置、保存责任。
- 3. 写汇编函数框架。
- 4. 写测试，覆盖普通值、边界值、随机值。
- 5. 用 objdump、nm、readelf 审计二进制。
- 6. 再考虑性能。

例如：

```
extern "C" std::int64_t asm_sum8(std::int64_t a0, std::int64_t a1,
                                std::int64_t a2, std::int64_t a3,
                                std::int64_t a4, std::int64_t a5,
                                std::int64_t a6, std::int64_t a7);
```

ABI 推导表：

源代码参数	入口位置	说明
a0	rdi	第 1 个整数类参数。
a1	rsi	第 2 个整数类参数。
a2	rdx	第 3 个整数类参数。
a3	rcx	第 4 个整数类参数。
a4	r8	第 5 个整数类参数。
a5	r9	第 6 个整数类参数。
a6	[rsp+8]	第 7 个参数，返回地址之后。
a7	[rsp+16]	第 8 个参数。
return	rax	64 位整数返回。

汇编：

```
.intel_syntax noprefix
.text

.globl asm_sum8
.type asm_sum8, @function
asm_sum8:
    lea rax, [rdi + rsi]
    add rax, rdx
    add rax, rcx
    add rax, r8
    add rax, r9
    add rax, qword ptr [rsp + 8]
    add rax, qword ptr [rsp + 16]
    ret
.size asm_sum8, .-asm_sum8

.section .note.GNU-stack,"",@progbits
```

注意这里没有函数序言。它是一个 leaf function，不调用其他函数，不需要局部栈空间，不修改 callee-saved 寄存器。这样的函数可以非常短。但这个短小建立在 ABI 推导正确之上，不是随便省略。

深入理解

如果函数入口后执行了 push rbp，那么第 7 个参数就不再位于当前 rsp 的 [rsp+8]。入口原始布局被改变了：

```
| on entry:
```

```

[rsp + 0] return address
[rsp + 8]  a6
[rsp +16]  a7

after push rbp:
[rsp + 0]  saved rbp
[rsp + 8]  return address
[rsp +16]  a6
[rsp +24]  a7

```

所以读写栈参数前必须先确定当前 `rsp` 相对入口移动了多少。很多手写汇编 bug 就来自这里。

3.5.5 测试不是附属品

手写汇编很容易出现“几个样例能跑，但边界条件错”的情况。测试要直接围绕二进制契约设计。

3.5.5.1 确定性测试

```

#include <array>
#include <cassert>
#include <cstdint>
#include <limits>

extern "C" std::int64_t asm_sum8(std::int64_t, std::int64_t,
                                std::int64_t, std::int64_t,
                                std::int64_t, std::int64_t,
                                std::int64_t, std::int64_t);

static std::int64_t ref_sum8(const std::array<std::int64_t, 8>& values)
{
    std::int64_t sum = 0;
    for (const std::int64_t value : values) {
        sum += value;
    }
    return sum;
}

int main()
{
    const std::array<std::array<std::int64_t, 8>, 4> cases {{
        {{0, 0, 0, 0, 0, 0, 0, 0}},
        {{1, 2, 3, 4, 5, 6, 7, 8}},
        {{-1, -2, -3, -4, -5, -6, -7, -8}},
        {{100, -50, 7, -8, 9, -10, 11, -12}},
    }};

    for (const auto& c : cases) {
        const std::int64_t got = asm_sum8(c[0], c[1], c[2], c[3],
                                           c[4], c[5], c[6], c[7]);
        assert(got == ref_sum8(c));
    }
}

```

这个测试至少覆盖了：

- 寄存器参数。
- 栈参数 `a6` 和 `a7`。
- 正数、负数、零。
- 多次调用。

3.5.5.2 随机测试

```

#include <array>
#include <cassert>
#include <cstdint>
#include <random>

extern "C" std::int64_t asm_sum8(std::int64_t, std::int64_t,
                                std::int64_t, std::int64_t,
                                std::int64_t, std::int64_t,
                                std::int64_t, std::int64_t);

static std::int64_t ref_sum8(const std::array<std::int64_t, 8>& values)
{
    std::int64_t sum = 0;
    for (const std::int64_t value : values) {

```

```

        sum += value;
    }
    return sum;
}

int main()
{
    std::mt19937_64 rng(0xC001D00DULL);
    std::uniform_int_distribution<std::int64_t> dist(-1000000, 1000000);

    for (int iter = 0; iter < 100000; ++iter) {
        std::array<std::int64_t, 8> values {};
        for (std::int64_t& value : values) {
            value = dist(rng);
        }

        const std::int64_t got = asm_sum8(values[0], values[1],
                                           values[2], values[3],
                                           values[4], values[5],
                                           values[6], values[7]);

        assert(got == ref_sum8(values));
    }
}

```

常见误区

如果 C++ reference 函数使用 signed overflow，而测试数据可能溢出，那么 reference 本身可能有未定义行为。训练早期可以限制输入范围；后续如果要测试完整 64 位 wraparound，应使用 `std::uint64_t` 或明确使用编译器支持的溢出内建函数。

3.5.6 C++ name mangling 与 extern"C"

C++ 支持重载、命名空间、类成员函数、模板实例化。为了让链接器区分这些函数，编译器会把源代码函数名编码成更复杂的链接符号，这叫 **name mangling**（名称修饰）。

```

int add(int a, int b)
{
    return a + b;
}

double add(double a, double b)
{
    return a + b;
}

```

编译并查看符号：

```

clang++ -std=c++20 -O2 -c overload.cpp -o overload.o
nm overload.o
nm -C overload.o

```

可能看到类似：

```

0000000000000000 T _Z3addii
0000000000000010 T _Z3adddd

```

nm-C 会 demangle：

```

0000000000000000 T add(int, int)
0000000000000010 T add(double, double)

```

如果写：

```
extern "C" int asm_add_int(int a, int b);
```

链接符号就是：

```
asm_add_int
```

这让手写汇编更简单。汇编文件只需要导出同名字符：

```

.global asm_add_int
.type asm_add_int, @function
asm_add_int:
    lea eax, [edi + esi]
    ret
.size asm_add_int, .-asm_add_int

```

深入理解

`extern"C` 只改变语言链接规则和符号名，不会把 C++ 类型系统变成 C，也不会禁用 System V ABI。比如 `extern"C"double f(double)` 仍然用 `xmm0` 传参与返回。它也不能用于重载同名函数，因为 C 链接名无法表达重载集合。

3.5.7 目标文件里的符号和重定位

很多人第一次看目标文件反汇编，会困惑为什么 `call` 后面是 0。
看 C++：

```
extern "C" long asm_add2(long, long);

extern "C" long call_add2()
{
    return asm_add2(10, 32);
}
```

编译目标文件：

```
clang++ -std=c++20 -O2 -c caller.cpp -o caller.o
objdump -drwC -Mintel caller.o
readelf -r caller.o
```

你可能看到：

```
0000000000000000 <call_add2>:
0: bf 0a 00 00 00      mov     edi,0xa
5: be 20 00 00 00      mov     esi,0x20
a: e8 00 00 00 00      call   f <call_add2+0xf>
    b: R_X86_64_PLT32    asm_add2-0x4
f: c3                 ret
```

这里的 `e800000000` 不是最终机器码。x86 的 near `call` 编码通常是：

```
e8 rel32
```

`rel32` 是相对下一条指令地址的有符号 32 位位移。目标文件阶段还不知道 `asm_add2` 最终地址，所以汇编器放一个临时值，并生成 relocation 记录：

```
at offset 0xb:
patch 4 bytes
relocation type = R_X86_64_PLT32
symbol = asm_add2
addend = -4
```

链接后，假设：

```
call instruction address = 0x40112a
next rip after call      = 0x40112f
asm_add2 address         = 0x401150
```

那么：

```
rel32 = target - next_rip
       = 0x401150 - 0x40112f
       = 0x21
```

最终编码会接近：

```
e8 21 00 00 00
```

核心思想

目标文件里的反汇编不是最终地址事实，而是“机器码字节 + 符号 + 重定位请求”的组合。读未链接目标文件时，必须同时看 `objdump-dr` 和 `readelf-r`。

3.5.8 汇编函数调用 C++ 函数

现在反过来：手写汇编调用 C++ helper。

3.5.8.1 C++ helper

```
#include <cstdint>

extern "C" std::int64_t cpp_square(std::int64_t x)
{
    return x * x;
}

extern "C" std::int64_t asm_square_plus(std::int64_t x);
```

目标：`asm_square_plus(x)` 调用 `cpp_square(x)`，再返回 `x*x+x`。

错误版本很容易写成：

```
.intel_syntax noprefix
.text

.globl asm_square_plus_broken
.type asm_square_plus_broken, @function
asm_square_plus_broken:
```

```

mov rbx, rdi
call cpp_square
add rax, rbx
ret
.size asm_square_plus_broken, .-asm_square_plus_broken

```

这段有两个风险：

- 修改了 callee-saved `rbx`，但没有恢复。
- 进入函数时 `rsp` 通常是 $16n+8$ ；直接 `call` 时调用点前栈没有 16 字节对齐。

3.5.8.2 修复版本

```

.intel_syntax noprefix
.text

.globl asm_square_plus
.type asm_square_plus, @function
asm_square_plus:
    push rbx
    mov  rbx, rdi

    call cpp_square

    add  rax, rbx
    pop  rbx
    ret
.size asm_square_plus, .-asm_square_plus

.section .note.GNU-stack,"",@progbits

```

为什么一个 `push rbx` 同时解决了两个问题？

入口时：

```
rsp = 16n + 8
```

执行 `push rbx` 后：

```
rsp = 16n
```

这时执行 `call cpp_square`，调用点前 `rsp` 满足 16 字节对齐。同时 `rbx` 原值被保存到栈上，返回前 `pop rbx` 恢复。

栈变化：

```

on entry to asm_square_plus:
    [rsp + 0] return address to C++ caller

after push rbx:
    [rsp + 0] saved rbx
    [rsp + 8] return address to C++ caller

inside cpp_square after call:
    [rsp + 0] return address to asm_square_plus
    [rsp + 8] saved rbx
    [rsp +16] return address to C++ caller

```

常见误区

不要把“push 一个寄存器刚好对齐”当成万能模板。如果你还要分配局部栈空间，必须重新计算总共改变了多少 `rsp`。原则是：每个 `call` 执行前，调用者的 `rsp` 应为 16 字节对齐。

3.5.9 leaf 和 non-leaf 汇编模板

手写汇编可以先从两个模板开始。

3.5.9.1 leaf function 模板

leaf function 不调用其他函数。它可以非常简洁：

```

.intel_syntax noprefix
.text

.globl function_name
.type function_name, @function
function_name:
    # use caller-saved registers freely:
    # rax, rcx, rdx, rsi, rdi, r8-r11, xmm0-xmm15
    # preserve callee-saved registers if you touch them

```

```
ret
.size function_name, .-function_name

.section .note.GNU-stack,"",@progbits
```

对于短小整数函数，常见策略是只用 caller-saved 寄存器，避免保存恢复开销。

3.5.9.2 non-leaf function 模板

non-leaf function 会调用其他函数，必须更谨慎：

```
.intel_syntax noprefix
.text

.globl function_name
.type function_name, @function
function_name:
    push rbp
    mov rbp, rsp
    push rbx
    sub rsp, 8

    # now rsp is 16-byte aligned before calls
    # save live caller-saved values yourself before each call if needed

    call some_helper

    add rsp, 8
    pop rbx
    pop rbp
    ret
.size function_name, .-function_name

.section .note.GNU-stack,"",@progbits
```

为什么有 `subrsp, 8`？计算一下：

```
entry:
    rsp = 16n + 8

push rbp:
    rsp = 16n

mov rbp, rsp:
    no change

push rbx:
    rsp = 16n - 8

sub rsp, 8:
    rsp = 16n - 16
```

所以在 `call some_helper` 前，`rsp` 是 16 字节对齐。

深入理解

真实编译器会根据局部变量大小、保存寄存器数量、是否省略帧指针、是否需要栈保护、是否有异常展开信息等因素生成不同序言。模板不是背诵答案，而是帮助你建立栈对齐和保存责任的算术模型。

3.5.10 C++ 对象、引用和指针在边界上的形状

汇编只看见寄存器、内存和地址。C++ 的引用、指针、小结构体、大结构体在 ABI 层有不同形状。

3.5.10.1 引用通常就是地址

C++：

```
extern "C" void asm_inc_ref(std::int64_t& x);
```

虽然源代码写的是引用，但汇编入口看到的通常是指针：

```
on entry:
    rdi = address of x
```

汇编实现：

```
.globl asm_inc_ref
.type asm_inc_ref, @function
asm_inc_ref:
    inc qword ptr [rdi]
    ret
.size asm_inc_ref, .-asm_inc_ref
```

这不意味着 C++ 引用“语言上等于指针”。语言层引用有别名、生命周期和绑定规则；ABI 层只是经常用地址传递它。

3.5.10.2 小结构体可能走寄存器

```
struct Pair64 {
    std::int64_t lo;
    std::int64_t hi;
};

extern "C" Pair64 asm_make_pair(std::int64_t x, std::int64_t y);
```

两个 64 位整数大小的结构体，在常见 System V AMD64 规则下可用 `rax` 和 `rdx` 返回：

```
.globl asm_make_pair
.type asm_make_pair, @function
asm_make_pair:
    mov rax, rdi
    mov rdx, rsi
    ret
.size asm_make_pair, .-asm_make_pair
```

3.5.10.3 大结构体通常通过隐藏指针返回

```
struct Triple64 {
    std::int64_t a;
    std::int64_t b;
    std::int64_t c;
};

extern "C" Triple64 asm_make_triple(std::int64_t x,
                                   std::int64_t y,
                                   std::int64_t z);
```

常见入口状态：

```
rdi = hidden pointer to return object
rsi = x
rdx = y
rcx = z
```

实现：

```
.globl asm_make_triple
.type asm_make_triple, @function
asm_make_triple:
    mov rax, rdi
    mov qword ptr [rdi + 0], rsi
    mov qword ptr [rdi + 8], rdx
    mov qword ptr [rdi + 16], rcx
    ret
.size asm_make_triple, .-asm_make_triple
```

注意显式参数 `x` 不在 `rdi`，因为 `rdi` 被隐藏返回对象指针占用了。这是读跨语言汇编时很常见的坑。

3.5.11 全局数据和 RIP-relative addressing

在 x86-64 位置无关代码中，访问静态数据常用 RIP-relative addressing。

```
.intel_syntax noprefix
.text

.globl asm_get_message
.type asm_get_message, @function
asm_get_message:
    lea rax, [rip + message]
    ret
.size asm_get_message, .-asm_get_message

.section .rodata
message:
    .asciz "hello from assembly"

.section .note.GNU-stack,"",@progbits
```

C++ 声明：

```
extern "C" const char* asm_get_message();
```

`lea` 不是读取字符串内容，而是计算字符串地址：

```
rax = address of message
```


反汇编目标文件时，仍然可能看到 relocation，因为 `message` 的最终地址要到链接阶段才知道：

```
lea rax, [rip + 0x0]
relocation: R_X86_64_PC32 .rodata-0x4
```

深入理解

RIP-relative addressing 的基准是下一条指令的地址，不是当前指令开头地址。这个规则和 `call rel32` 一样，都是 x86-64 位置无关代码的重要基础。后面讲动态库、PLT/GOT 和 PIC 时会继续展开。

3.5.12 内联汇编为什么危险

很多人一学汇编就想在 C++ 里写 inline asm。工程上要反过来：除非你非常清楚编译器优化器和寄存器分配器会怎样看它，否则优先使用普通 C++、compiler builtin、intrinsics 或单独的 .S 文件。

GNU extended asm 的基本形状：

```
asm volatile (
    "assembly template"
    : output operands
    : input operands
    : clobbers
);
```

它不是“把几行汇编插入 C++”这么简单。它是在告诉优化器：

- 哪些值被这个 asm 读。
- 哪些值被这个 asm 写。
- 哪些寄存器、flags、内存状态被破坏。
- 这个 asm 是否可以删除、移动或合并。

如果这些信息写错，编译器可能生成看似合理但实际错误的代码。

3.5.12.1 错误示例：没有声明输出

```
std::int64_t broken_add(std::int64_t a, std::int64_t b)
{
    std::int64_t result = a;
    asm("add %1, %0" : : "r"(result), "r"(b));
    return result;
}
```

这段的问题是：模板里修改了 `%0` 对应的寄存器，但约束里没有告诉编译器 `result` 是输出。优化器可以认为 `result` 没有改变。

修复：

```
std::int64_t fixed_add(std::int64_t a, std::int64_t b)
{
    std::int64_t result = a;
    asm("add %1, %0"
        : "+r"(result)
        : "r"(b)
        : "cc");
    return result;
}
```

`"+r"(result)` 表示这个操作数既输入又输出，并且放在某个通用寄存器中。`"cc"` 告诉编译器 condition codes 被修改。

3.5.12.2 错误示例：忘记 "memory" clobber

考虑一个人为的 store：

```
void broken_store(std::int64_t* p, std::int64_t value)
{
    asm volatile("mov %1, (%0)" : : "r"(p), "r"(value));
}
```

汇编确实写了 `*p`，但 C++ 优化器不知道这个 asm 会改变任意内存。若周围代码读取或写入同一内存，优化器可能重排。

更清晰的写法是把内存作为输出操作数：

```
void better_store(std::int64_t* p, std::int64_t value)
{
    asm volatile("mov %1, %0"
        : "=m"(*p)
        : "r"(value)
        : "memory");
}
```

这里 `"=m"(*p)` 明确描述被写的内存对象，`"memory"` 进一步作为编译器级内存屏障，防止优化器把其他内存访问跨过这个 `asm` 移动。

常见误区

`"memory"` clobber 是编译器屏障，不是 CPU 内存栅栏。它限制编译器重排，但不会自动生成 `mfence`、`lfence` 或 locked instruction。多线程内存序问题必须用 C++ atomics 或明确的硬件同步指令解决。

3.5.12.3 volatile 不是万能药

`asmvolatile` 的含义接近“这个 `asm` 有副作用，不要因为输出未使用就删除它，也不要随便合并”。但它不自动说明：

- `asm` 读写了哪些 C++ 变量。
- `asm` 修改了哪些寄存器。
- `asm` 修改了 `flags`。
- `asm` 访问了内存。
- `asm` 可以作为 CPU fence。

所以 `inline asm` 的正确性来自 `operands` 和 `clobbers`，而不是只靠 `volatile`。

3.5.12.4 常见约束

约束	含义
"r"	任意通用寄存器。
"m"	内存操作数。
"i"	编译期立即数。
"a"	使用 <code>rax/eax/al</code> 类寄存器。
"=r"	只写输出寄存器。
"+r"	读写同一个寄存器操作数。
"=&r"	early-clobber 输出，输出在所有输入读完前就可能被写。
"0"	与第 0 个输出操作数使用同一位置。
"cc"	<code>asm</code> 修改了 <code>flags</code> 。
"memory"	<code>asm</code> 可能读写未显式列出的内存，限制编译器重排。

深入理解

`inline asm` 的约束是你和编译器寄存器分配器之间的合同。你不应该在模板里随便写固定寄存器名，然后忘记告诉编译器。固定寄存器越多，寄存器分配越难，优化器越受限制，错误空间也越大。

3.5.13 intrinsics、inline asm 与 .S 的选择

现代高性能代码通常优先选择下面顺序：

方式	优点	风险
普通 C++ compiler builtin	可移植、优化器可理解、测试容易。 表达特殊语义，如 <code>prefetch</code> 、 <code>expect</code> 、 <code>overflow</code> 。	不一定生成你想要的指令。 依赖编译器扩展。
intrinsics	直接表达 SIMD 指令，寄存器分配仍由编译器做。	ISA 绑定强，代码可读性下降。
.S 文件	可完全控制函数级汇编和 ABI。	维护成本高，调试和 <code>unwind</code> 要额外注意。
inline asm	可在表达式附近插入特殊指令。	最容易和优化器合同写错。

对于 AVX2/AVX-512/FMA/VNNI 这类 AI 算子优化，本书后续会大量使用 `intrinsics`。原因是 `intrinsics` 让编译器仍然负责寄存器分配、调用约定、栈框架和调度的一部分，同时你可以明确控制核心 SIMD 操作。

单独 `.S` 文件适合：

- 你需要完全固定函数入口、寄存器分配和指令布局。
- 你正在写非常小的热路径 `kernel`。
- 你需要验证某个 ABI 或机器码现象。
- 你愿意维护多 ISA 版本和测试矩阵。

inline asm 适合：

- 少数没有合适 intrinsic 的特殊指令。
- 需要非常局部的硬件交互。
- 你已经能准确写出 operands、constraints 和 clobbers。

3.5.14 调试和审计工作流

写互操作代码时，永远保留可审计证据。

3.5.14.1 编译参数

```
clang++ -std=c++20 -O2 -g -fno-omit-frame-pointer \
main.cpp kernels.S -o app
```

训练阶段建议：

- 用 `-g` 保留调试信息。
- 初期加 `-fno-omit-frame-pointer`，让栈回溯更直观。
- 分别看 `-O0` 和 `-O2/-O3`，理解优化差异。
- 对汇编文件保留注释和清晰符号名。

3.5.14.2 符号审计

```
nm -C app | rg 'asm_|cpp_'
readelf -Ws app | rg 'asm_|cpp_'
```

要检查：

- 汇编函数是否真的导出。
- C++ 是否引用了预期符号名。
- 是否出现未预期的 mangled name。
- 符号类型是否是函数。

3.5.14.3 反汇编审计

```
objdump -drwC -Mintel app > app.objdump.txt
rg -n '<asm_sum8>|<asm_square_plus>|call.*cpp_square' app.objdump.txt
```

要检查：

- 参数寄存器使用是否和 ABI 一致。
- 栈参数偏移是否正确。
- callee-saved 寄存器是否成对保存恢复。
- 调用前 `rsp` 是否对齐。
- 是否出现你没预料到的 PLT 调用或间接跳转。

3.5.14.4 GDB 单步

```
gdb ./app
(gdb) break asm_square_plus
(gdb) run
(gdb) disassemble /r asm_square_plus
(gdb) info registers rdi rax rbx rsp rip
(gdb) x/6gx $rsp
(gdb) stepi
(gdb) info registers rdi rax rbx rsp rip
```

单步时不要只看源代码行。要看：

- `rip` 是否进入预期符号。
- `rsp` 每次 `push`、`pop`、`call`、`ret` 如何变化。
- 返回地址是否位于预期栈槽。
- 返回值是否写入 `rax` 或 `xmm0`。

3.5.15 sanitizer 和手写汇编的边界

AddressSanitizer、UndefinedBehaviorSanitizer、ThreadSanitizer 对 C++ 很有价值，但它们不能自动理解你所有手写汇编内存访问。比如汇编里：

```
mov rax, qword ptr [rdi + rcx*8]
```

如果 `rdi` 指向越界地址，ASan 未必能像检查 C++ 数组访问那样插桩发现。原因是插桩发生在编译器能理解的 IR 层；手写汇编已经绕过了这部分语义。

因此汇编边界测试要更保守：

- C++ wrapper 先检查尺寸、空指针和对齐前置条件。
- 汇编 kernel 文档明确要求输入合法。
- 随机测试使用保护页或额外 padding 检查越界。
- 对 debug 版本提供纯 C++ reference path。
- 必要时在汇编函数外层做 shadow check，而不是假设 sanitizer 能看见内部。

核心思想

优化 kernel 的工程边界通常是“外层 C++ 负责合法性和调度，内层汇编负责极少分支的热路径”。不要把复杂错误处理塞进手写汇编热循环。

3.5.16 一个接近算子 kernel 的例子：整数点积

先写标量整数点积，为后续 SIMD 章节做准备。

C++ 声明：

```
extern "C" std::int64_t asm_dot_i32(const std::int32_t* a,
                                   const std::int32_t* b,
                                   std::int64_t n);
```

语义：

```
sum = 0
for i in [0, n):
    sum += int64(a[i]) * int64(b[i])
return sum
```

ABI 入口：

```
rdi = a
rsi = b
rdx = n
rax = return value
```

汇编：

```
.intel_syntax noprefix
.text

.globl asm_dot_i32
.type asm_dot_i32, @function
asm_dot_i32:
    xor rax, rax
    test rdx, rdx
    jle .Ldone

    xor rcx, rcx
.Lloop:
    movsxd r8, dword ptr [rdi + rcx*4]
    movsxd r9, dword ptr [rsi + rcx*4]
    imul r8, r9
    add rax, r8
    inc rcx
    cmp rcx, rdx
    jl .Lloop

.Ldone:
    ret
.size asm_dot_i32, .-asm_dot_i32

.section .note.GNU-stack,"",@progbits
```

地址公式：

```
address of a[i] = rdi + rcx * 4
address of b[i] = rsi + rcx * 4
```

为什么 scale 是 4？因为 `std::int32_t` 元素大小是 4 字节。这个 scale 是 x86 地址模式支持的合法 scale。若元素结构大小是 12 字节，就不能写 `[base+index*12]`，必须用 `leaq` 或其他指令拆解。

这段代码还不是高性能点积。它每次循环只做一个元素，乘法和加载串行依赖较多，没有展开，没有 SIMD，也没有处理内存带宽。但它是非常好的互操作训练材料，因为它包含：

- 指针参数。
- signed load extension。

- 循环控制流。
- 数组地址公式。
- 64 位累加返回值。

3.5.17 汇编边界的文档模板

每个手写汇编函数都应有一段接口注释。示例：

```
# std::int64_t asm_dot_i32(const std::int32_t* a,
#                          const std::int32_t* b,
#                          std::int64_t n)
#
# ABI:
#   rdi = a
#   rsi = b
#   rdx = n
#   rax = return sum
#
# Preconditions:
#   a and b point to at least n int32 elements
#   n >= 0
#   pointers may be unaligned
#
# Clobbers:
#   rax, rcx, r8, r9, flags
#
# Callee-saved:
#   none modified
```

这不是形式主义。几个月后你优化这个 kernel 时，注释会告诉你哪些寄存器本来可以自由使用，哪些语义由外层保证，哪些行为必须保持。

3.5.18 实验 14.1：从零搭建 C++ 调汇编工程

实验

目标：建立一个最小但规范的 .S+C++CMake 工程，并用工具证明符号、调用和返回值正确。

创建目录：

```
mkdir -p labs/ch14_interop/lab14_1_minimal
cd labs/ch14_interop/lab14_1_minimal
```

文件：

```
lab14_1_minimal/
  CMakeLists.txt
  main.cpp
  asm_arith.S
```

任务：

1. 在 C++ 中声明 3 个函数：

```
extern "C" long asm_add2(long a, long b);
extern "C" long asm_sub2(long a, long b);
extern "C" long asm_mix4(long a, long b, long c, long d);
```

2. 在 asm_arith.S 中实现它们。
3. asm_mix4 的语义为 $(a+b)*3-(c-d)$ ，可以使用 lea、add、sub。
4. 写至少 20 个确定性测试，覆盖正数、负数、零和较大值。
5. 构建并运行。
6. 用 nm-C 证明符号存在。
7. 用 objdump-drwC-Mintel 找到 3 个函数的机器码。

推荐命令：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build -j
./build/lab14_1_minimal
nm -C build/lab14_1_minimal | rg 'asm_'
objdump -drwC -Mintel build/lab14_1_minimal > lab14_1.objdump.txt
```

交付物：

1. 源码。
2. 运行输出。
3. nm 输出中 3 个汇编符号的行。
4. 每个函数的反汇编。

5. 一张表：源代码参数、ABI 寄存器、汇编使用位置。

3.5.19 实验 14.2：符号、mangling 和 relocation 审计

实验

目标：理解 C++ 符号名、`extern "C"` 和链接器重定位。
创建两个版本：

```
with_c_linkage.cpp
without_c_linkage.cpp
caller.cpp
```

`with_c_linkage.cpp`:

```
extern "C" int add_c(int a, int b)
{
    return a + b;
}
```

`without_c_linkage.cpp`:

```
int add_cpp(int a, int b)
{
    return a + b;
}
```

`caller.cpp`:

```
extern "C" int add_c(int, int);
int add_cpp(int, int);

extern "C" int call_both(int x)
{
    return add_c(x, 1) + add_cpp(x, 2);
}
```

任务：

1. 分别编译成 `.o`。
2. 用 `nm` 和 `nm-C` 比较符号名。
3. 用 `objdump-drwC-Mintelcaller.o` 查看未链接目标文件。
4. 用 `readelf-rcaller.o` 查看 relocation。
5. 链接成可执行文件后，再看最终 `call` 的相对位移。

命令：

```
clang++ -std=c++20 -O2 -c with_c_linkage.cpp -o with_c_linkage.o
clang++ -std=c++20 -O2 -c without_c_linkage.cpp -o without_c_linkage.o
clang++ -std=c++20 -O2 -c caller.cpp -o caller.o
nm caller.o
nm -C caller.o
objdump -drwC -Mintel caller.o > caller.objdump.txt
readelf -r caller.o > caller.reloc.txt
```

交付物：

1. `add_c` 和 `add_cpp` 的原始符号名。
2. `call_both` 中两个 `call` 的 relocation 记录。
3. 解释 `R_X86_64_PLT32` 或你机器上出现的 relocation 类型。
4. 链接后任选一个 `call`，用 `target-next_rip` 计算其 `rel32`。
5. 一段 500 字总结：为什么目标文件反汇编不能只看机器码字节。

3.5.20 实验 14.3：ABI 安全的 8 参数汇编函数

实验

目标：实现并严格测试一个包含栈参数的汇编函数。
函数签名：

```
extern "C" long asm_weighted_sum8(long a0, long a1, long a2, long a3,
                                long a4, long a5, long a6, long a7);
```

语义：

```
return a0*1 + a1*2 + a2*3 + a3*4
       + a4*5 + a5*6 + a6*7 + a7*8
```

任务：

1. 写 ABI 参数位置表。
2. 实现汇编函数。第 7、8 个参数必须从栈读取。
3. 写 C++ reference 函数。
4. 写 100000 轮随机测试，限制输入范围避免 signed overflow。
5. 用 GDB 在函数入口断点，记录 `rsp`、`[rsp]`、`[rsp+8]`、`[rsp+16]`。
6. 用 `objdump` 标注每个参数在哪里被使用。

提示：可以用 `lea` 组合小常数乘法，但要保证结果语义清楚。例如 `x*3` 可以写成：

```
lea rax, [rdi + rdi*2]
```

交付物：

1. 参数映射表。
2. 完整汇编。
3. 测试源码和运行结果。
4. GDB 栈截图或文字记录。
5. 解释为什么 `a6` 是 `[rsp+8]` 而不是 `[rsp]`。

3.5.21 实验 14.4：汇编调用 C++ helper

实验

目标：写一个 non-leaf 汇编函数，正确处理 callee-saved 寄存器和栈对齐。

C++ helper：

```
extern "C" long cpp_mul_add(long x, long y, long z)
{
    return x * y + z;
}
```

汇编函数签名：

```
extern "C" long asm_call_helper(long x, long y, long z, long bias);
```

语义：

```
return cpp_mul_add(x, y, z) + bias
```

任务：

1. 先故意写一个错误版本：把 `bias` 放进 `rbx`，调用 `helper` 后不恢复。
2. 构造 C++ 调用者，让调用者在调用前后有一个长期存活变量，观察错误是否暴露。
3. 修复 `rbx` 保存恢复。
4. 检查调用 `cpp_mul_add` 前 `rsp` 是否 16 字节对齐。
5. 用 GDB 单步过 `call`，记录 `rsp` 和返回地址。

交付物：

1. 错误版本和修复版本的汇编。
2. 为什么错误版本违反 ABI。
3. 栈对齐计算过程。
4. GDB 记录。
5. 测试结果。

3.5.22 实验 14.5：inline asm 约束修复

实验

目标：理解 inline asm 不是文本替换，而是优化器合同。

给定错误代码：

```
#include <stdint>

std::int64_t broken_add(std::int64_t a, std::int64_t b)
{
    std::int64_t result = a;
    asm("add %1, %0" : : "r"(result), "r"(b));
    return result;
}

void broken_store(std::int64_t* p, std::int64_t value)
{
    asm volatile("mov %1, (%0)" : : "r"(p), "r"(value));
}
```



```
| }
```

任务：

1. 在 `-O0` 和 `-O3` 下分别编译运行，记录现象。
2. 查看反汇编，解释为什么错误版本可能“偶尔看起来能跑”。
3. 修复 `broken_add`，正确使用读写输出和 `"cc"` clobber。
4. 修复 `broken_store`，正确描述内存写和 `"memory"` clobber。
5. 写测试证明修复版本在 `-O3` 下稳定正确。

交付物：

1. 错误版本和修复版本源码。
2. `-O0`、`-O3` 反汇编对比。
3. 每个 `constraint` 的解释。
4. 说明 `volatile`、`"memory"`、`"cc"` 各自解决什么问题，不能解决什么问题。

3.5.23 实验 14.6：标量点积 kernel 的互操作审计

实验

目标：实现一个接近算子 `kernel` 形态的标量点积，并建立 `correctness report`。

函数：

```
extern "C" std::int64_t asm_dot_i32(const std::int32_t* a,
                                   const std::int32_t* b,
                                   std::int64_t n);
```

任务：

1. 写 C++ reference 实现。
2. 写汇编实现。
3. 测试 `n=0,1,2,15,16,17,1024`。
4. 随机生成数组，做 10000 轮测试。
5. 用 `objdump` 标注循环基本块、回边、数组地址公式。
6. 用 `perfstat` 初步测量运行时间、instructions、cycles；如果权限不足，记录错误信息并使用 `/usr/bin/time`。

交付物：

1. 源码和构建脚本。
2. 正确性测试输出。
3. 循环反汇编标注。
4. 地址公式说明。
5. 初步性能表。
6. 说明为什么这个版本还不是高性能实现，列出至少 5 个后续优化方向。

3.5.24 本章作业

习题与作业

作业 1：接口契约报告

选择你在实验中写的任意 3 个汇编函数，为每个函数写一份接口契约报告，必须包含：

- C++ 声明。
- 链接符号名。
- 参数位置表。
- 返回值位置。
- 修改的 `caller-saved` 寄存器。
- 是否修改 `callee-saved` 寄存器；如果修改，如何保存恢复。
- 是否调用其他函数；如果调用，如何保证栈对齐。
- 前置条件和未定义行为边界。

作业 2：修复破坏 ABI 的汇编

下面函数被 C++ 调用：


```
.intel_syntax noprefix
.text
.globl broken_kernel
broken_kernel:
    mov r12, rdi
    mov r13, rsi
    call helper
    add rax, r12
    add rax, r13
    ret
```

要求：

1. 找出所有 ABI 问题。
2. 说明 `r12`、`r13` 属于 caller-saved 还是 callee-saved。
3. 修复保存恢复。
4. 修复栈对齐。
5. 给出完整汇编。
6. 用栈布局图证明你的修复正确。

作业 3：重定位计算

给定未链接目标文件片段：

```
0000000000000000 <foo>:
  0: e8 00 00 00 00    call 5 <foo+0x5>
      1: R_X86_64_PLT32 bar-0x4
  5: c3                ret
```

链接后：

```
foo address = 0x401100
bar address = 0x401180
```

要求：

1. 写出 `call` 指令地址。
2. 写出 next `rip`。
3. 计算 `rel32`。
4. 写出小端序 4 字节。
5. 解释为什么 relocation addend 常见为 `-4`。

作业 4：inline asm 审计

阅读下面代码：

```
std::uint64_t rdtsc_bad()
{
    std::uint32_t lo = 0;
    std::uint32_t hi = 0;
    asm volatile("rdtsc");
    return (static_cast<std::uint64_t>(hi) << 32) | lo;
}
```

要求：

1. 解释为什么它错误。
2. 写出正确输出约束，让 `eax` 和 `edx` 分别写入 `lo`、`hi`。
3. 说明 `rdtsc` 的乱序风险。
4. 查阅本机反汇编，确认约束是否生成预期寄存器。
5. 说明为什么 benchmark 不能只靠裸 `rdtsc`。

作业 5：互操作小项目

完成一个小项目：

```
interop_project/
  CMakeLists.txt
  include/kernels.hpp
  src/main.cpp
  src/reference.cpp
  src/kernels.S
  tests/test_kernels.cpp
  report.md
```

功能：

- `asm_sum8`: 8 参数加权求和。
- `asm_make_pair`: 返回两个 64 位字段的小结构体。
- `asm_dot_i32`: 标量点积。
- `asm_call_helper`: 汇编调用 C++ helper。

质量要求:

1. 每个函数都有 reference 实现。
2. 每个函数都有确定性测试和随机测试。
3. 所有汇编函数都有接口注释。
4. 报告中包含 `nm`、`readelf-r`、`objdump` 证据。
5. 报告中至少分析 2 个栈布局。
6. 报告中至少解释 1 个 relocation。
7. 报告中明确列出 sanitizer 能覆盖什么、不能覆盖什么。

评分标准

本章作业按下面标准评价:

1. 能否从 C++ 签名正确推导 ABI 参数和返回值位置。
2. 汇编符号、`.type`、`.size`、`section` 是否规范。
3. 是否正确处理 caller-saved、callee-saved 和栈对齐。
4. 是否能用 `nm`、`readelf`、`objdump`、GDB 证明结论。
5. `inline asm` 是否准确描述输入、输出和 clobbers。
6. 测试是否覆盖边界值、随机值和跨优化级别行为。
7. 报告是否区分语言层语义、ABI 层契约、目标文件层机制和 CPU 执行行为。

3.5.25 本章小结

本章把 System V AMD64 ABI 放进了真实工程:

```
C++ declaration:
    type, language linkage, call site generation

assembly source:
    symbol definition, sections, prologue/epilogue, instructions

ABI:
    parameter locations, return locations, register ownership, stack alignment

object file:
    symbols, sections, relocations

tools:
    nm, readelf, objdump, gdb, perf

quality:
    deterministic tests, randomized tests, binary-interface report
```

你现在应该能够从一个 C++ 函数声明出发, 写出 ABI 映射表, 再写出手写汇编实现, 并用工具证明它真的按预期链接和运行。你也应该开始对 `inline asm` 保持谨慎: 它不是简单的“在 C++ 里写汇编”, 而是和优化器签订一份非常精确的合同。

后面的章节会进入测量、编译器优化、微架构和 SIMD。本章建立的互操作能力会反复使用: 你会写小 kernel、反汇编审计、测量性能、解释瓶颈, 再逐步把标量代码推进到 AVX、FMA、VNNI 和 AI 算子优化。

第 4 章 可信性能测量、Benchmark 工程与 Lab 00

4.1 可信性能测量基础

4.1.1 本章要解决的问题

从这一章开始，本书进入性能工程部分。很多人以为性能优化的第一步是“改代码”，但真正可靠的第一步是“知道自己测到的是什么”。如果测量方法不可信，后面所有结论都可能是错的：你以为某个优化快了 30%，实际可能是编译器删掉了循环；你以为某个版本慢了，实际可能是 CPU 降频、cache 状态、OS 调度、页错误、随机输入或 I/O 噪声在影响。

本章要建立一套性能测量的底层心智模型。你需要从一开始就把 benchmark 看成一次受控实验，而不是一次随手计时。

一个可信 benchmark 至少要回答：

- 被测对象是什么？是一个函数、一个循环、一个完整程序，还是一次系统调用？
- 输入是什么？规模、分布、对齐、cache 状态、随机种子是否明确？
- 计时区域在哪里？是否混入了初始化、分配、I/O、随机数生成、校验、日志输出？
- 编译器是否可能把被测代码删除、合并、常量折叠或移动？
- 结果是否先证明正确，再讨论性能？
- 测量重复了多少次？报告的是最小值、均值、中位数还是分位数？
- 环境是否记录？CPU 型号、频率策略、编译器版本、编译参数、操作系统状态是否可复现？

本章会先讲最核心的机制，后续第 16 章再系统讲 benchmark 设计，第 17 章再深入统计与噪声分析。你现在要掌握的是：看到一个性能数字时，能判断它是否有资格作为工程证据。

核心思想

性能测量不是“得到一个时间数字”，而是建立一条可复核的证据链：正确性、被测工作量、时间源、控制变量、重复测量、统计口径、二进制审计和环境记录。

4.1.2 为什么计时不等于 benchmark

先看一个常见错误：

```
#include <chrono>
#include <cstdint>
#include <cstdio>

int main()
{
    auto begin = std::chrono::high_resolution_clock::now();

    std::uint64_t sum = 0;
    for (std::uint64_t i = 0; i < 1000000000ULL; ++i) {
        sum += i;
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto ns = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin);
    std::printf("%lld\n", static_cast<long long>(ns.count()));
}
```

这个程序看起来在测 10 亿次加法。实际上它可能什么都没测到。优化器可以推导：

```
sum = 0 + 1 + 2 + ... + 999999999
```

如果 `sum` 没有被观察，甚至整个循环都可能被删除。就算打印了 `sum`，编译器也可能用公式替代循环。因为从 C++ 抽象机视角看，只要可观察行为相同，编译器可以自由改写。

另一个错误：

```
auto begin = clock_now();
std::vector<float> x(n);
fill_random(x);
auto result = kernel(x);
std::printf("%f\n", result);
auto end = clock_now();
```

这里计时区域包含了：

```
allocation + initialization + random generation + kernel + printf
```

如果目标是测 `kernel`，这个数字没有表达清楚。它可能主要测到了内存分配器、随机数生成器、首次触页、I/O flush 或动态链接的 `lazy binding`。

常见误区

“程序运行了多久”和“`kernel` 花了多久”不是同一个问题。高性能计算和 AI 算子优化通常关心热路径 `kernel` 的稳定吞吐；计时区域必须把 `setup`、`timed region`、`validation` 分开。

4.1.3 性能实验的基本结构

一个最小可信实验可以拆成五段：

```
1. describe environment:
    CPU, compiler, flags, OS, governor, command

2. setup:
    allocate data, initialize input, pin thread if needed

3. warmup:
    run kernel without recording final timing

4. timed region:
    measure only the intended workload

5. validation and report:
    compare with reference, compute statistics, save raw data
```

注意顺序：正确性在性能之前。如果一个优化版本结果错了，它再快也没有意义。

心智模型

benchmark 是一个实验系统。`kernel` 只是其中一个部件；输入、环境、计时器、统计、报告和二进制审计都是实验系统的一部分。训练自己时，不要只问“快不快”，还要问“我凭什么相信这个数字”。

4.1.4 被测对象：workload、kernel、measurement

先定义几个词。

术语	本书中的含义
workload （工作负载）	数据规模、数据分布、访问模式和运行条件。
kernel （核心计算）	希望重点优化和计时的函数或循环。
timed region （计时区域）	实际包在计时器之间的代码范围。
harness （测试框架）	负责 <code>setup</code> 、 <code>warmup</code> 、重复测量、统计和报告。
reference （参考实现）	语义清晰、容易验证正确性的版本，用来检查优化版本。

例子：

```
float dot_product_ref(std::span<const float> a, std::span<const float> b)
{
    float sum = 0.0f;
    for (std::size_t i = 0; i < a.size(); ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}
```

这里 `dot_product_ref` 是 `reference` 或 `baseline`，不一定是最终 `kernel`。`workload` 包括：

- `a` 和 `b` 的长度。
- 数据是否随机、全零、全一、递增、含 NaN、含 denormal。
- 指针是否对齐。
- 数据是否已经在 `cache` 中。
- 是否只测一个长度，还是扫描多个长度。

如果报告只写“`dot product` 需要 3 ms”，信息不够。至少应写：

```
kernel:
    dot_product_ref(float*, float*, n)

workload:
    n = 1048576
    data = uniform random in [-1, 1]
    arrays allocated once, reused
    warm-cache repeated measurement
```

```

compiler:
  clang++ -O3 -march=native -DNDEBUG

machine:
  CPU model, core count, governor, OS kernel

metric:
  median nanoseconds over 31 measured repetitions

```

4.1.5 正确性先于性能

性能优化常见灾难是“优化版本快了，因为少算了”。因此每个 benchmark 都要先有 correctness reference。以整数求和为例：

```

#include <stdint>
#include <span>

std::uint64_t sum_u64_ref(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}

```

为什么用 `std::uint64_t`？因为 unsigned overflow 在 C++ 中定义为模 2^{64} 回绕。若使用 `std::int64_t`，输入过大时 signed overflow 是未定义行为，reference 也可能被优化器按“不会溢出”的假设改写。

测试思路：

```

void check_sum_kernel(std::span<const std::uint64_t> values)
{
    const std::uint64_t expected = sum_u64_ref(values);
    const std::uint64_t actual = sum_u64_optimized(values);
    if (actual != expected) {
        std::abort();
    }
}

```

如果是浮点，正确性不能总用完全相等。应根据算法和数值误差选择容忍度：

```

absolute error:
abs(actual - expected) <= epsilon

relative error:
abs(actual - expected) <= epsilon * max(abs(expected), 1)

```

深入理解

浮点优化不仅是性能问题，也是语义问题。改变求和顺序会改变舍入误差。

启用 `-ffast-math` 还会改变 NaN、Inf、signed zero 和结合律相关的语义边界。后续讲 fast-math 和 AI 算子时会专门展开。现在先记住：浮点 benchmark 必须同时报告数值误差策略。

4.1.6 时间源：你到底在读什么钟

性能测量常见时间源有几类。

时间源	适合用途和主要风险
<code>steady_clock</code>	C++ 通用单调墙钟计时；分辨率和开销依平台而异。
<code>clock_gettime</code>	Linux 下可选择 <code>CLOCK_MONOTONIC</code> 等时间源；系统调用或 vDSO 路径需要了解。
TSC	低开销 cycle-like 计数；风险是乱序、迁核、频率解释和序列化问题。
PMU counters	观察硬件事件，例如 cycles、instructions、cache miss；风险是权限、事件语义、复用和平台差异。
<code>/usr/bin/time</code>	粗粒度进程级时间和资源统计；不适合细粒度 kernel。

4.1.6.1 `steady_clock`

C++ 中应优先使用 `std::chrono::steady_clock` 作为入门计时器，因为它承诺单调不倒退。

```
using Clock = std::chrono::steady_clock;

const auto begin = Clock::now();
run_kernel();
const auto end = Clock::now();

const auto elapsed_ns =
    std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();
```

不要默认 `high_resolution_clock` 一定更好。标准没有保证它一定是单调钟；在某些实现中它只是其他 clock 的别名。入门阶段用 `steady_clock` 更稳。

4.1.6.2 TSC 与 cycle 的关系

x86 上可以用 `rdtsc` 或 `rdtscp` 读取 time-stamp counter。它返回一个 64 位计数，常见写法是 `edx:eax` 组合：

```
#include <cstdint>
#include <x86intrin.h>

std::uint64_t read_tsc()
{
    return __rdtsc();
}
```

但是 TSC 不是新手测量的万能工具。风险包括：

- `rdtsc` 不是完全序列化指令，周围指令可能乱序跨过它。
- 如果线程迁移到另一个 core，早期或特殊系统上的 TSC 同步可能成为问题。
- 现代 CPU 的 TSC 常以 invariant rate 增长，不一定等于当前核心实际执行频率。
- Turbo、降频、C-state 会让“每秒多少 TSC tick”和“每条指令需要多少 core cycle”的解释变复杂。

更严谨的 TSC 计时需要 fence：

```
#include <cstdint>
#include <x86intrin.h>

std::uint64_t read_tsc_begin()
{
    _mm_lfence();
    const std::uint64_t value = __rdtsc();
    _mm_lfence();
    return value;
}

std::uint64_t read_tsc_end()
{
    _mm_lfence();
    const std::uint64_t value = __rdtsc();
    _mm_lfence();
    return value;
}
```

不同平台和论文中还会看到 `cpuid`、`rdtscp`、`lfence` 的不同组合。本书训练阶段的原则是：

- 初期先用 `steady_clock` 建立正确实验结构。
- 需要 cycle 级分析时，再引入 TSC，并明确 fence 策略。
- 真正分析微架构时，优先结合 PMU counter，而不是只看 TSC。

常见误区

不要把 TSC delta 直接称为“CPU cycles”而不解释。很多现代机器上 TSC 是稳定参考计数，核心实际频率会变化。报告时可以写 `TSCticks` 或说明换算假设。

4.1.7 编译器会删除你的 benchmark

优化器只需要保留 C++ 抽象机的可观察行为。benchmark 里最常见的错误是被测计算没有可观察结果。

4.1.7.1 dead-code elimination

错误：

```
void benchmark_bad(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
}
```

```
}
```

`sum` 没有被使用。优化器可以删除整个循环。
最简单的防护是把结果返回给调用者：

```
std::uint64_t benchmark_better(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}
```

但如果调用者也没有使用返回值，仍可能被删除。`benchmark harness` 需要显式制造可观察使用。

4.1.7.2 *do_not_optimize*

GCC/Clang 常用一个空 `inline asm` 阻止优化器认为值无用：

```
template <class T>
inline void do_not_optimize(const T& value)
{
    asm volatile("" : : "g"(value) : "memory");
}
```

用法：

```
const std::uint64_t result = benchmark_better(values);
do_not_optimize(result);
```

这段 `inline asm` 没有机器指令，但它告诉编译器：`value` 被某个不可见的 `asm` 使用，不能随便删除产生它的计算。“`memory`” 还限制周围内存访问重排。

常见误区

`do_not_optimize` 是 benchmark 工具，不是业务代码工具。它的目的是约束编译器优化，让实验可测；不要把它放进生产热路径。

4.1.7.3 *clobber_memory*

另一个常用工具：

```
inline void clobber_memory()
{
    asm volatile("" : : : "memory");
}
```

它告诉编译器：内存状态可能被未知 `asm` 改变，因此不要把内存读写随便跨过这个点移动。它不会清空 CPU cache，也不是硬件内存栅栏。

4.1.7.4 *constant folding*

错误：

```
std::uint64_t x = expensive(123);
```

如果 `expensive(123)` 在编译期可推导，优化器可能直接替换成常量。benchmark 输入应来自运行时数据，或者至少来自编译器无法完全证明的内存。

例子：

```
std::vector<std::uint64_t> make_input(std::size_t size)
{
    constexpr std::uint64_t INPUT_MULTIPLIER = 6364136223846793005ULL;
    constexpr std::uint64_t INPUT_INCREMENT = 1442695040888963407ULL;

    std::vector<std::uint64_t> values(size);
    std::uint64_t state = 1;
    for (std::uint64_t& value : values) {
        state = state * INPUT_MULTIPLIER + INPUT_INCREMENT;
        value = state;
    }
    return values;
}
```

这里用简单线性同余生成输入。它不是高质量随机数生成器，但适合构造可复现、运行时生成的数据。真正 benchmark 时，输入生成仍应放在 `setup`，不放入 `timed region`。

4.1.8 计时区域必须隔离

错误结构：

```
auto begin = Clock::now();
std::vector<std::uint64_t> values = make_input(size);
const std::uint64_t result = sum_u64(values);
check(result);
auto end = Clock::now();
```

这测的是：

allocation + input generation + sum + check

正确结构：

```
std::vector<std::uint64_t> values = make_input(size);
const std::uint64_t expected = sum_u64_ref(values);

auto begin = Clock::now();
const std::uint64_t result = sum_u64(values);
auto end = Clock::now();

if (result != expected) {
    std::abort();
}
```

如果要重复测量：

```
setup:
    allocate and initialize input once
    compute expected reference once

warmup:
    run kernel several times

measurement:
    for each repetition:
        begin timer
        run kernel
        end timer
        record elapsed
        consume result

validation:
    compare result with expected
```

深入理解

有些 workload 本身包含分配或 I/O，比如数据库查询、HTTP 服务、编译器编译一个文件。那不是不能测，而是要诚实命名 benchmark：你测的是 end-to-end workload，不是纯计算 kernel。不同问题需要不同边界。

4.1.9 warmup：第一次运行不代表稳态

第一次运行常常慢，因为它可能包含：

- 代码页和数据页首次触页。
- instruction cache 和 data cache 冷启动。
- branch predictor 尚未学习输入模式。
- 动态库 lazy binding。
- CPU 从低功耗状态升频。
- 内存分配器初始化。

所以 benchmark 通常先 warmup：

```
constexpr int BENCHMARK_WARMUP_REPETITIONS = 5;

for (int iter = 0; iter < BENCHMARK_WARMUP_REPETITIONS; ++iter) {
    const std::uint64_t result = sum_u64(values);
    do_not_optimize(result);
}
```

warmup 不是为了“作弊让 cache 热起来”，而是为了明确你测的是某个状态。你可以测 cold-cache，也可以测 warm-cache，但必须说清楚。

状态	含义
cold-cache	每次计时前尽量让数据不在 cache 中，模拟首次访问或大工作集。
warm-cache	同一数据重复运行，测稳定热路径吞吐。
steady-state	程序已经过初始化、预测器和 cache 已接近稳定状态。

4.1.10 重复测量和统计口径

单次测量几乎没有说服力。需要多次运行并保存原始数据。
常见统计量：

统计量	含义	风险
min	最小观测值，接近“噪声最少时”的能力。	可能过于乐观。
mean	平均值。	对 outlier 敏感。
median	中位数，代表典型运行。	不展示尾部风险。
p95	95 分位，展示较慢尾部。	需要足够样本。
max	最慢观测。	可能只是一次系统干扰。

HPC kernel 优化常看 min 或 median，因为目标是估计稳定热路径上限。服务端延迟常看 p95、p99，因为用户体验受尾延迟影响。AI 算子库通常会同时报告 median、min 和标准差或变异系数。

常见误区

不要只报告一个数字且不说明它是什么。写“用时 1.2 ms”是不完整的；应写“31 次重复，median = 1.2 ms，min = 1.17 ms，p95 = 1.35 ms”。

4.1.11 一个最小 C++20 benchmark harness

下面给一个教材用最小 harness。它不是要替代 Google Benchmark，而是让你看清楚每个部件为什么存在。

```
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <span>
#include <vector>

namespace {

constexpr std::size_t BENCHMARK_INPUT_SIZE = 1u << 20;
constexpr int BENCHMARK_WARMUP_REPETITIONS = 5;
constexpr int BENCHMARK_MEASURED_REPETITIONS = 31;
constexpr std::uint64_t INPUT_MULTIPLIER = 6364136223846793005ULL;
constexpr std::uint64_t INPUT_INCREMENT = 1442695040888963407ULL;

using Clock = std::chrono::steady_clock;

template <class T>
inline void do_not_optimize(const T& value)
{
    asm volatile("" : : "g"(value) : "memory");
}

std::vector<std::uint64_t> make_input(std::size_t size)
{
    std::vector<std::uint64_t> values(size);
    std::uint64_t state = 1;
    for (std::uint64_t& value : values) {
        state = state * INPUT_MULTIPLIER + INPUT_INCREMENT;
        value = state;
    }
    return values;
}

std::uint64_t sum_u64(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
}
```

```

    return sum;
}

std::uint64_t elapsed_ns_for_one_run(std::span<const std::uint64_t> values,
                                     std::uint64_t* result_out)
{
    const auto begin = Clock::now();
    const std::uint64_t result = sum_u64(values);
    const auto end = Clock::now();

    *result_out = result;
    do_not_optimize(result);

    return static_cast<std::uint64_t>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count());
}

double median_ns(std::vector<std::uint64_t> samples)
{
    std::sort(samples.begin(), samples.end());
    const std::size_t middle = samples.size() / 2;
    return static_cast<double>(samples[middle]);
}

} // namespace

int main()
{
    const std::vector<std::uint64_t> values = make_input(BENCHMARK_INPUT_SIZE);
    const std::uint64_t expected = sum_u64(values);

    for (int iter = 0; iter < BENCHMARK_WARMUP_REPETITIONS; ++iter) {
        const std::uint64_t result = sum_u64(values);
        do_not_optimize(result);
    }

    std::vector<std::uint64_t> samples;
    samples.reserve(BENCHMARK_MEASURED_REPETITIONS);

    for (int iter = 0; iter < BENCHMARK_MEASURED_REPETITIONS; ++iter) {
        std::uint64_t result = 0;
        const std::uint64_t elapsed_ns = elapsed_ns_for_one_run(values, &result);
        if (result != expected) {
            std::abort();
        }
        samples.push_back(elapsed_ns);
    }

    const auto [min_it, max_it] = std::minmax_element(samples.begin(), samples.end());
    std::printf("n=%zu min_ns=%llu median_ns=%.0f max_ns=%llu\n",
               values.size(),
               static_cast<unsigned long long>(*min_it),
               median_ns(samples),
               static_cast<unsigned long long>(*max_it));
}

```

编译:

```

clang++ -std=c++20 -O3 -march=native -DNDEBUG \
    bench_sum.cpp -o bench_sum
./bench_sum

```

这个 harness 做对了几件事:

- 输入在计时前生成。
- reference 结果在计时前计算。
- warmup 和 measurement 分离。
- 每次计时只包住 `sum_u64`。
- 结果被校验并被 `do_not_optimize` 消费。
- 重复测量并报告 min、median、max。

它还不够完整:

- 没有输出原始 CSV。
- 没有记录环境。
- 没有 CPU affinity。
- 没有 PMU counter。
- 没有多输入规模扫描。
- 没有统计 p95 和变异系数。

这些会在后续章节逐步补齐。

4.1.12 环境控制：CPU 不只是在跑你的代码

一次 benchmark 运行时，机器上还有很多因素：

- 操作系统可能调度其他进程。
- 中断可能打断当前 core。
- 线程可能迁移到另一个 core。
- CPU 可能升频或降频。
- SMT sibling 可能共享执行资源。
- 后台服务可能污染 cache 或占用内存带宽。
- 虚拟机或容器可能隐藏真实硬件行为。

入门阶段可以先做低成本控制：

```
# 查看 CPU 型号
lscpu

# 查看当前频率策略，具体路径依发行版而异
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor

# 绑定到某个 CPU 运行
taskset -c 2 ./bench_sum

# 粗略查看进程级时间
/usr/bin/time -v ./bench_sum
```

报告中至少记录：

```
CPU:
  model name, cores, SMT on/off if known

OS:
  kernel version, distribution

compiler:
  compiler name and version
  flags

run command:
  exact command line

environment:
  taskset core if used
  governor if known
  whether running on laptop battery, VM, container, remote server
```

深入理解

严格控制频率和隔离 CPU core 可能需要 root 权限和系统配置，例如 performance governor、isolcpus、关闭 turbo、关闭 SMT、固定 NUMA 节点。训练早期不一定要全部做到，但必须知道没有控制意味着结果会有更大噪声。

4.1.13 cache 状态会改变结论

同一个内存 kernel，在不同工作集大小下可能完全不同。

例子：

```
sum array of uint64:
  n = 1024          likely L1/L2 resident
  n = 1048576       likely LLC or memory bandwidth
  n = 134217728     definitely memory streaming and paging pressure
```

如果只测一个 n，你可能误以为某个优化“总是快”。实际上：

- 小数组可能受函数调用、循环开销、前端和分支影响。
- 中等数组可能受 cache bandwidth 和 latency 影响。
- 大数组可能受内存带宽、TLB、预取器和 NUMA 影响。

所以性能报告应把输入规模当作一维变量，而不是固定一个点：

```
n, median_ns, bytes, bandwidth_GBps
1024, ...
4096, ...
16384, ...
65536, ...
1048576, ...
```

```
16777216, ...

带宽计算:

bytes_read = n * sizeof(uint64_t)
bandwidth_GBps = bytes_read / seconds / 1e9
```

注意这个公式只计算源码层显式读取字节数。真实硬件可能按 cache line 传输，可能有 write allocate，可能有预取，可能有额外 page walk。后面内存层次章节会更精确。

4.1.14 PMU：从时间走向硬件证据

时间告诉你“慢了多少”，PMU counter 帮你猜“为什么慢”。
Linux 下可以先用：

```
perf stat ./bench_sum

更具体:

perf stat -r 10 \
-e cycles,instructions,branches,branch-misses,cache-references,cache-misses \
./bench_sum
```

常见指标：

指标	直觉	注意
cycles	花了多少周期。	受频率、停顿和事件定义影响。
instructions	retired instructions 数。	指令少不一定快。
IPC	instructions / cycles。	高 IPC 不一定代表高性能。
branch-misses	分支预测失败。	和输入分布强相关。
cache-misses	cache miss 事件。	不同 CPU 事件语义不同。

如果权限不足，你可能看到：

```
No permission to enable cycles event.

这不是 benchmark 失败，而是环境限制。报告里应写清楚，并改用可用工具：

cat /proc/sys/kernel/perf_event_paranoid
/usr/bin/time -v ./bench_sum
```

常见误区

PMU counter 不是绝对真理。事件可能被复用，事件名称和语义依 CPU 而异，虚拟机里可能不准确。PMU 的正确用法是和源码、汇编、输入规模、时间数据一起形成一致解释。

4.1.15 从源码到汇编再到测量

性能测量不能只看源码。至少要审计核心循环是否存在。
命令：

```
clang++ -std=c++20 -O3 -march=native -S -masm=intel bench_sum.cpp -o bench_sum.s
clang++ -std=c++20 -O3 -march=native bench_sum.cpp -o bench_sum
objdump -drwC -Mintel bench_sum > bench_sum.objdump.txt
rg -n "sum_u64|add|vpadd|ret" bench_sum.objdump.txt
```

你要看：

- **sum_u64** 是否被内联。
 - 循环是否还存在。
 - 是否被向量化。
 - 是否出现预期加载。
 - **timed region** 是否包含你以为的函数。
- 如果函数被内联，不一定是坏事。真实 -O3 性能经常依赖内联。但报告必须说明你测的是内联后的热路径，而不是一个独立函数调用。

深入理解

之后讲编译器优化时，你会看到 scalar replacement、loop unrolling、vectorization、LICM、DCE、constant propagation 等优化。benchmark 的可怕之处在于：这些优化可能正是你想研究的对象，也可能是把实验悄悄变成另一个实验的干扰因素。

4.1.16 性能数字的单位

不同 kernel 应选择不同单位。

单位	适用场景	例子
ns/op	小操作、函数调用、短 kernel。	每次 hash、每次分配。
cycles/op	微架构分析。	每元素周期数。
GB/s	内存带宽型 kernel。	copy、fill、sum、transpose。
GFLOP/s	浮点计算型 kernel。	dot、GEMM、卷积。
items/s	解析、压缩、推理 token。	rows/s、tokens/s。

例子: `sum uint64_t`:

```
elements = n
bytes_read = n * 8
seconds = median_ns / 1e9
GB/s = bytes_read / seconds / 1e9
ns/element = median_ns / n
```

例子: dot product:

```
for each element:
    one multiply
    one add

FLOPs = 2 * n
GFLOP/s = FLOPs / seconds / 1e9
```

要小心 FLOP 计数。FMA 指令 `vfmadd` 通常按 2 FLOPs 计，因为它完成乘加两个浮点操作。整数算子、量化算子、softmax、layernorm 的指标要根据业务语义定义。

4.1.17 常见 benchmark 反模式

4.1.17.1 反模式 1: 把打印放进 *timed region*

错误:

```
auto begin = Clock::now();
const auto result = kernel(input);
std::printf("%llu\n", static_cast<unsigned long long>(result));
auto end = Clock::now();
```

打印可能比 kernel 慢得多，也可能触发锁、缓冲、系统调用。

4.1.17.2 反模式 2: 每轮重新分配

错误:

```
for (int iter = 0; iter < repetitions; ++iter) {
    auto begin = Clock::now();
    std::vector<float> values(size);
    kernel(values);
    auto end = Clock::now();
}
```

这测了 allocator 和触页。除非目标就是测分配，否则分配应放在 setup。

4.1.17.3 反模式 3: 输入太小

如果 kernel 只运行几十纳秒，计时器开销、函数调用、分支、循环控制都会淹没结果。解决方法:

- 增大输入。
- 每次计时运行多个 batch。
- 单独测计时器 overhead。
- 使用硬件 counter 或专门 benchmark 框架。

4.1.17.4 反模式 4: 只测一次

一次运行可能被中断、调度、缺页、降频影响。至少重复几十次，并保存原始样本。

4.1.17.5 反模式 5: 比较不同编译参数

如果 A 用 `-O2`, B 用 `-O3-march=native`, 你比较的不是代码差异, 而是代码加编译策略的组合。除非这正是实验目标, 否则编译参数必须一致。

4.1.17.6 反模式 6: 没有审计汇编

如果你没有看反汇编，就不知道测的是手写循环、自动向量化循环、库函数调用，还是被优化器删掉的空代码。

4.1.18 报告模板

每次实验至少提交：

```
title:
  benchmark name and purpose

machine:
  CPU model
  memory if relevant
  OS and kernel
  governor / taskset / VM status if known

compiler:
  compiler version
  full flags

source:
  commit hash or file version
  kernel signature
  reference implementation

workload:
  input size
  data distribution
  cache state
  repetitions
  warmup count

measurement:
  timer or perf command
  raw data file
  summary statistics

binary audit:
  relevant objdump excerpt
  whether function was inlined/vectorized

correctness:
  validation method
  tolerance for floating point

conclusion:
  what the data supports
  what remains uncertain
```

核心理想

高质量性能报告不是“我的版本更快”。它应该让别人能复现实验、审计二进制、理解误差，并判断结论的适用范围。

4.1.19 实验 15.1: 修复一个会被优化器删除的 benchmark

实验

目标：亲手观察 dead-code elimination 和防优化工具的作用。

创建 `labs/ch15_measurement/lab15_1_dce/bench_bad.cpp`：

```
#include <stdint>
#include <vector>

int main()
{
    constexpr std::uint64_t LOOP_LIMIT = 1000000000ULL;
    std::uint64_t sum = 0;
    for (std::uint64_t i = 0; i < LOOP_LIMIT; ++i) {
        sum += i;
    }
}
```

任务：

1. 用 `-O0`、`-O2`、`-O3` 分别编译。

2. 用 `objdump-drwC-Mintel` 检查循环是否存在。
3. 加入返回值打印，再看汇编是否变成公式或仍保留循环。
4. 把输入上限改成运行时参数，例如从 `argv` 读取，再观察。
5. 加入 `do_not_optimize(sum)`，解释汇编变化。

命令：

```
clang++ -std=c++20 -O0 bench_bad.cpp -o bench_00
clang++ -std=c++20 -O3 bench_bad.cpp -o bench_03
objdump -drwC -Mintel bench_03 > bench_03.objdump.txt
rg -n "add|loop|main" bench_03.objdump.txt
```

交付物：

1. 三个优化级别的反汇编关键片段。
2. 哪个版本保留循环，哪个版本删除或改写循环。
3. 用 C++ 抽象机可观察行为解释为什么优化器有权这么做。
4. 修复后的 benchmark。

4.1.20 实验 15.2：写一个最小可信 sum benchmark

实验

目标：实现本章最小 harness，并形成第一份性能报告。

要求：

1. 使用 `std::chrono::steady_clock`。
2. `setup`、`warmup`、`timed region`、`validation` 分开。
3. 输入规模至少包含：

```
1024
16384
262144
1048576
16777216
```

4. 每个规模 warmup 5 次，正式测量 31 次。
5. 输出 CSV：

```
n,min_ns,median_ns,max_ns,bytes,bandwidth_Gbps
```

6. 用 `objdump` 审计核心循环。

交付物：

1. 源码和编译命令。
2. CSV 原始输出。
3. 一张结果表。
4. 汇编核心循环标注。
5. 500 字报告：随着 `n` 增大，瓶颈可能如何变化。

4.1.21 实验 15.3：比较 `steady_clock` 与 TSC

实验

目标：理解时间源差异，而不是盲目相信某一个计时器。

任务：

1. 对同一个 `sum_u64` kernel，同时记录 `steady_clock` 纳秒和 TSC delta。
2. 用 `rdtsc` 裸读一次。
3. 用 `lfence` 包围 `rdtsc` 再读一次。
4. 每种方式重复 100 次，保存原始数据。
5. 检查线程是否迁核；如果会迁核，用 `taskset` 绑定。

交付物：

1. 计时代码。
2. 原始数据。
3. `min`、`median`、`max` 表。
4. 解释 TSC delta 是否稳定。
5. 说明为什么 TSC delta 不应无条件等同于真实 core cycles。

4.1.22 实验 15.4：用 perfstat 建立硬件证据

实验

目标：从单纯时间测量进入硬件 counter 观察。

对实验 15.2 的 benchmark 运行：

```
perf stat -r 10 \
-e cycles,instructions,branches,branch-misses,cache-references,cache-misses \
./bench_sum
```

任务：

1. 记录 `cycles`、`instructions`、IPC。
2. 记录 cache miss 相关事件。
3. 如果权限不足，记录完整错误信息和 `perf_event_paranoid` 值。
4. 改变输入规模，观察事件是否随 `n` 变化。
5. 把 PMU 结果和时间结果放在同一份报告中解释。

交付物：

1. `perfstat` 输出。
2. 事件含义解释。
3. 输入规模与 counter 变化表。
4. 至少一个你不能确定的点，并说明还需要什么实验。

4.1.23 实验 15.5：cache 状态实验

实验

目标：观察 warm-cache 与 cold-ish-cache 对结果的影响。

设计两个版本：

1. warm-cache：同一数组连续重复求和。
2. cold-ish-cache：每次计时前扫描一个远大于 LLC 的 buffer，尽量污染 cache。

示例 cache 污染函数：

```
void touch_buffer(std::span<std::uint64_t> buffer)
{
    constexpr std::uint64_t TOUCH_INCREMENT = 1;
    for (std::uint64_t& value : buffer) {
        value += TOUCH_INCREMENT;
    }
}
```

任务：

1. 选择至少 3 个输入规模。
2. 每个规模分别测 warm-cache 和 cold-ish-cache。
3. 报告 median ns 和 GB/s。
4. 解释为什么 cold-ish-cache 只是近似，不是严格清空所有 cache。
5. 用 `perfstat` 观察 cache miss 是否变化。

交付物：

1. 源码。
2. 结果表。
3. cache 状态解释。
4. 对后续内存层次章节提出 3 个问题。

4.1.24 本章作业

习题与作业

作业 1：benchmark 批判性阅读

找一个你过去写过的性能测试，或者网上任意一段短 benchmark。写一份审计报告，回答：

1. 被测对象是否明确？
2. setup 和 timed region 是否分离？
3. 是否有 correctness reference？
4. 是否可能被 DCE、constant folding 或 loop invariant hoisting 影响？

5. 是否重复测量?
6. 是否报告统计口径?
7. 是否记录环境?
8. 是否审计汇编?

要求至少指出 5 个问题, 并给出修复方案。

作业 2: 最小 harness 代码评审

阅读本章最小 C++20 harness, 写一份代码评审:

- 哪些设计是为了防止编译器优化掉被测代码?
- 哪些设计是为了避免把 setup 混进计时区域?
- 哪些常量应该暴露成命令行参数?
- median_ns 当前实现有什么限制?
- 如果要支持多个 kernel, 应该如何拆分函数和数据结构?

要求给出不少于 800 字的工程评审, 不要只复述代码。

作业 3: 时间源误差分析

对一个空 timed region 测量计时器开销:

```
begin = now()
end = now()
```

再对一个很短 kernel 和一个较长 kernel 分别计时。要求:

1. 报告空计时开销分布。
2. 解释为什么短 kernel 更容易被计时器开销污染。
3. 设计 batch 策略降低相对误差。
4. 说明 batch 策略可能引入什么新问题。

作业 4: 汇编审计报告

选择实验 15.2 中一个输入规模, 对可执行文件生成反汇编报告:

```
objdump -drwC -Mintel ./bench_sum > bench_sum.objdump.txt
```

报告必须包含:

1. sum_u64 是否被内联。
2. 核心循环反汇编。
3. 是否出现 SIMD 指令。
4. 每次循环处理多少元素。
5. 循环退出条件。
6. 源码中的 do_not_optimize 在汇编层是否生成真实指令。

作业 5: 性能实验报告

提交一份完整性能实验报告, 主题是 sum_u64、copy_u64 或 dot_f32 三选一。报告必须包含:

- 环境记录。
- 编译命令。
- workload 定义。
- correctness reference。
- 原始测量数据。
- min、median、p95、max。
- 至少一段反汇编。
- 至少一次 perfstat 尝试。
- 结论和限制。

评分标准

本章作业按下面标准评价:

1. 是否能清楚划分 setup、warmup、timed region 和 validation。
2. 是否能解释编译器删除、移动或常量折叠 benchmark 的原因。

3. 是否能用 reference 证明结果正确。
4. 是否能保存和解释重复测量数据。
5. 是否能选择合适指标，例如 ns/op、GB/s、GFLOP/s。
6. 是否能用 `objdump` 和 `perfstat` 支撑结论。
7. 是否能诚实说明实验限制，而不是过度推断。

4.1.25 本章小结

本章建立了性能测量的第一层地基：

```
correctness:
  reference first, performance second

benchmark structure:
  setup -> warmup -> timed region -> validation -> report

compiler safety:
  avoid DCE, constant folding, unintended motion

time source:
  steady_clock for basic timing
  TSC and PMU require stronger interpretation

noise:
  OS scheduling, CPU frequency, cache state, migration, background load

evidence:
  raw samples, statistics, objdump, perf stat, environment record
```

从现在开始，你不应该再接受“这段代码跑了几毫秒”这种孤立数字。性能工程的基本素养是：每个数字都必须带着上下文、方法、证据和限制。下一章会继续把这些原则工程化，系统设计 benchmark 输入、热路径、CSV 输出、参数扫描和报告格式。

4.2 Benchmark 设计：输入、热路径和报告

4.2.1 本章要解决的问题

上一章讲的是“为什么性能测量容易错”。本章讲“怎样把测量原则做成可复现的 benchmark 工程”。

这一步很关键。你以后做 HPC kernel、AI 算子、编译器优化、内存布局优化时，不可能每次都手写一段临时代码计时。你需要的是一套可以扩展、可以审计、可以保存原始数据、可以扫描参数、可以比较版本的实验系统。

本章目标是让你能设计一个小型 benchmark suite，而不是只会写一个 `main()` 计时器。这个 suite 至少要处理：

- 多个 kernel：例如 `sum_u64`、`copy_u64`、`fill_u64`、`dot_f32`。
- 多个输入规模：从 L1 cache 级别到内存带宽级别。
- 多种数据分布：随机、全零、递增、分支友好、分支不友好。
- 多个重复次数：warmup、measured repetitions、batch。
- 正确性校验：每个 kernel 都有 reference 或可验证不变量。
- 原始数据和汇总数据：CSV 输出、环境记录、命令行记录。
- 二进制审计：可以把源码、反汇编和数字联系起来。

从工程角度看，benchmark suite 和普通测试框架很像：它要有清晰的数据流、明确的边界、稳定的输出格式和可维护的扩展点。差别在于，benchmark 还必须控制优化器、cache、频率、输入分布和统计口径。

核心思想

一个好的 benchmark 不是一段计时代码，而是一套实验协议。它规定被测对象、输入空间、计时边界、重复策略、输出格式、正确性检查、环境记录和审计方法。

4.2.2 Benchmark 设计的主线

把 benchmark 设计成下面的流水线：

```
configuration:
  parse command line
  choose sizes, repetitions, batch, seed, output path

environment record:
  compiler flags, command line, CPU info, OS info, git commit if available
```

```
workload construction:
  allocate buffers
  initialize inputs
  compute reference or invariant

warmup:
  run kernel without recording samples

measurement:
  repeat:
    run timed region
    consume result
    validate if cheap enough
    append raw sample

summary:
  compute min, median, p95, max, derived metrics
  write CSV

audit:
  keep binary, objdump excerpt, perf command, raw data
```

这条流水线背后的思想是：每个阶段只做一类事。输入生成不放进计时区域；统计不混进 kernel；报告不改变运行行为；校验不依赖性能版本本身。

心智模型

设计 benchmark 时，把它看成“实验编译器”：配置是源程序，workload 是输入实例，kernel 是被执行的目标代码，CSV 是输出产物，报告是解释器。任何隐式状态都会降低可复现性。

4.2.3 先写清楚问题陈述

动手写代码前，先写一段 problem statement。它不需要长，但必须具体。
差的描述：

```
measure sum performance
```

好的描述：

```
Goal:
  Measure warm-cache and streaming performance of unsigned 64-bit sum.

Kernel:
  std::uint64_t sum_u64(std::span<const std::uint64_t> values)

Input sizes:
  1 KiB, 16 KiB, 256 KiB, 4 MiB, 64 MiB worth of uint64_t elements

Data:
  deterministic pseudo-random uint64_t, generated once per size

Measurement:
  5 warmup runs, 31 measured repetitions
  timed region contains only the kernel call

Output:
  raw CSV and summary CSV
  fields include n, bytes, sample_ns, median_ns, bandwidth_GBps
```

为什么要这么写？因为性能结论总是有适用范围。一个只在 $n=1024$ 时更快的版本，不一定在 $n=16777216$ 时更快；一个 warm-cache 下更快的版本，不一定 cold-cache 下更快；一个随机数据下快的分支版本，不一定排序数据下快。

4.2.4 输入规模：不要只测一个点

输入规模决定瓶颈。对数组 kernel 来说，常见阶段如下：

工作集范围	常见瓶颈	观察重点
很小	调用开销、循环开销、前端开销。	ns/op、cycles/op。
L1/L2 内	指令吞吐、load/store 吞吐、向量化。	每周期元素数。
LLC 内	cache 带宽、预取、并行度。	GB/s、cache miss。
超过 LLC	内存带宽、地址转换和 NUMA。	GB/s、LLC miss、TLB。

推荐选择不是随便的 1000, 10000, 100000，而是能跨过 cache 层级的规模。例如对 `std::uint64_t`：

```
bytes per element = 8

sizes by bytes:
1 KiB      -> 128 elements
16 KiB     -> 2048 elements
256 KiB    -> 32768 elements
4 MiB      -> 524288 elements
64 MiB     -> 8388608 elements
```

在命令行或配置中可以写成：

```
--sizes-bytes 1024,16384,262144,4194304,67108864

还要测边界规模。比如 SIMD kernel 常见尾部处理问题：

vector width = 8 float elements

important n:
0, 1, 7, 8, 9, 15, 16, 17
1023, 1024, 1025
```

这些规模不一定用来报告最终吞吐，但必须用于 correctness 和尾部路径性能检查。

常见误区

只测一个大输入很容易掩盖小规模延迟；只测一个小输入又会掩盖内存带宽瓶颈。算子优化通常需要同时报告小规模、中规模、大规模和尾部边界。

4.2.5 数据分布：输入不是无关变量

数据分布会改变分支、向量化、数值路径、cache 压缩效果和异常值。
以 count_positive 为例：

```
std::uint64_t count_positive(std::span<const std::int32_t> values)
{
    std::uint64_t count = 0;
    for (const std::int32_t value : values) {
        if (value > 0) {
            ++count;
        }
    }
    return count;
}
```

这些输入会导致不同表现：

分布	特征	预测难度
全正数	分支总是 taken。	低
全负数	分支总是 not taken。	低
正负交替	模式简单但频繁变化。	中
随机正负	近似不可预测。	高
长 run	一段正、一段负。	低到中

如果报告只写 count_positive 的速度，不写数据分布，结论不完整。

4.2.5.1 确定性输入生成

benchmark 不应该依赖每次运行都不同的随机输入。使用固定 seed：

```
#include <cstdint>
#include <vector>

namespace {

constexpr std::uint64_t INPUT_MULTIPLIER = 6364136223846793005ULL;
constexpr std::uint64_t INPUT_INCREMENT = 1442695040888963407ULL;
constexpr std::uint64_t INPUT_DEFAULT_SEED = 1ULL;

std::vector<std::uint64_t> make_u64_input(std::size_t size, std::uint64_t seed)
{
    std::vector<std::uint64_t> values(size);
    std::uint64_t state = seed;
    for (std::uint64_t& value : values) {
        state = state * INPUT_MULTIPLIER + INPUT_INCREMENT;
        value = state;
    }
    return values;
}
```

```
} // namespace
```

这不是密码学随机数，也不是统计质量最好的生成器。它的作用是：

- 生成运行时数据，避免常量折叠。
- 固定 seed，保证可复现。
- 成本低，但仍然放在 setup，不放在 timed region。

4.2.5.2 浮点数据要避开隐藏语义

浮点 benchmark 要明确是否包含：

- NaN。
- Inf。
- signed zero。
- denormal/subnormal。
- 极大或极小值。

例如 dot product 入门实验可以先使用 `[-1,1]` 的普通有限值，避免把异常数值路径混进第一个实验。后续研究 denormal、fast-math、flush-to-zero 时，再单独设计对应 workload。

4.2.6 对齐、步长和别名

数组 kernel 的输入不只是长度，还有地址形状。

```
contiguous:
    a[i]

strided:
    a[i * stride]

offset-aligned:
    pointer address is 64-byte aligned

misaligned:
    pointer address = aligned_base + 4
```

这些差异会影响：

- load/store 是否跨 cache line。
- SIMD load 是否需要 unaligned 路径。
- 预取器能否识别访问模式。
- TLB 覆盖范围。
- 编译器能否证明无别名。

如果 kernel 接收两个指针：

```
void copy_u64(std::span<std::uint64_t> dst,
              std::span<const std::uint64_t> src);
```

要明确：

- dst 和 src 是否允许重叠。
- 若不允许重叠，C++ 实现是否能用 restrict-like 信息表达。
- 输入是否按 64 字节对齐。
- 是否测试非对齐偏移。

深入理解

C++ 标准没有标准化的 restrict 关键字，但 GCC/Clang 有 `__restrict__` 扩展。后续讲 alias analysis 和 vectorization 时会看到：编译器是否知道两个指针不别名，可能直接决定是否能向量化。

4.2.7 热路径边界：把 setup 赶出去

一个可维护 benchmark 应有显式阶段：

```
make_workload()
compute_reference()
warmup()
measure()
validate()
write_result()
```

不要在 timed region 里做这些事：

- 分配内存。

- 生成随机数。
- 打印日志。
- 写 CSV。
- 构造 `std::string`。
- 查询 CPU 信息。
- 解析命令行。
- 做复杂校验。

timed region 应尽量像这样：

```
const auto begin = Clock::now();
const std::uint64_t result = sum_u64(workload.values);
const auto end = Clock::now();
do_not_optimize(result);
```

如果 kernel 本身就是“分配一个对象”或“写一个文件”，当然可以把分配或 I/O 放进 timed region。但那时 benchmark 名称要诚实：

```
bench_vector_push_back_with_growth
bench_write_4KiB_sync
```

而不是假装它是纯计算 kernel。

4.2.8 Batch：短 kernel 的必要技巧

如果一次 kernel 调用只需几十纳秒，计时器开销和噪声会占很大比例。解决方法是 batch：

```
one sample:
begin timer
repeat kernel B times
end timer

sample_ns_per_batch = end - begin
ns_per_call = sample_ns_per_batch / B
```

但 batch 不是越大越好。它会改变 cache 状态、分支预测状态和热路径行为。比如同一个小数组重复求和很多次，会变成强 warm-cache 测试；如果你想测 cold-cache，batch 可能破坏实验。

一个简单策略：

- 对短 kernel 使用 batch，目标是让每个 sample 至少达到几十微秒。
- 报告 batch 大小。
- 不同输入规模可以使用不同 batch，但要在 CSV 中记录。
- 不能把 batch 后的数字和非 batch 数字混在一起比较。

```
std::uint64_t run_sum_batch(std::span<const std::uint64_t> values,
                           int batch_count)
{
    std::uint64_t combined = 0;
    for (int batch = 0; batch < batch_count; ++batch) {
        combined += sum_u64(values);
    }
    return combined;
}
```

combined 的作用是把每次结果连接起来，避免优化器删除重复调用。实际 harness 还要用 `do_not_optimize(combined)`。

常见误区

batch 会改变 workload。对 cache 敏感、状态ful、随机输入、分支预测敏感的 kernel，batch 可能让测量更稳定，也可能让测量偏离真实场景。报告里必须写明。

4.2.9 CSV：先保存原始数据

不要只打印最终汇总。原始样本是后续统计和复查的基础。

推荐至少输出两个文件：

```
raw_samples.csv
summary.csv
```

raw samples：

```
kernel,n,bytes,distribution,batch,rep,warmup,elapsed_ns,result_checksum
sum_u64,1048576,8388608,lcg,1,0,5,312000,12345
sum_u64,1048576,8388608,lcg,1,1,5,311400,12345
```

summary：

```
kernel,n,bytes,distribution,batch,repetitions,min_ns,median_ns,p95_ns,max_ns,GBps
sum_u64,1048576,8388608,lcg,1,31,309900,312000,320100,330000,26.88
```

字段设计原则：

- 每一行都能独立解释。
- 不要把单位藏在列名之外。
- 所有影响结果的参数都要进 CSV。
- 原始样本不要覆盖，只追加或写到带时间戳/配置名的文件。
- summary 可以由 raw samples 再生成。

4.2.10 环境记录：报告不是记忆

benchmark 输出目录建议包含：

```
results/2026-06-13_sum_suite/
  raw_samples.csv
  summary.csv
  environment.txt
  command.txt
  objdump.txt
  perf_stat.txt
  report.md
```

environment.txt 至少包含：

```
date -Iseconds
uname -a
lscpu
clang++ --version
g++ --version
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
cat /proc/sys/kernel/perf_event_paranoid
```

这些命令不一定每台机器都有同样输出。没关系，记录能记录的，并把失败也记录下来：

```
scaling_governor:
unavailable: /sys/devices/system/cpu/cpu0/cpufreq does not exist
```

这比假装环境不存在要可靠。

4.2.11 一个小型 benchmark suite 的数据模型

先设计数据结构。不要一开始就把所有逻辑塞进 main()。

```
#include <stdint>
#include <string>
#include <vector>

struct BenchmarkOptions {
    std::vector<std::size_t> sizes;
    int warmup_repetitions = 0;
    int measured_repetitions = 0;
    int batch_count = 0;
    std::uint64_t seed = 0;
    std::string output_prefix;
};

struct RawSample {
    std::string kernel_name;
    std::size_t elements = 0;
    std::size_t bytes = 0;
    int batch_count = 0;
    int repetition = 0;
    std::uint64_t elapsed_ns = 0;
    std::uint64_t checksum = 0;
};

struct SummaryRow {
    std::string kernel_name;
    std::size_t elements = 0;
    std::size_t bytes = 0;
    int batch_count = 0;
    int repetitions = 0;
    std::uint64_t min_ns = 0;
    std::uint64_t median_ns = 0;
    std::uint64_t p95_ns = 0;
    std::uint64_t max_ns = 0;
    double gb_per_second = 0.0;
};
```

这些结构分别对应：

- BenchmarkOptions：实验配置。

- **RawSample**: 每次测量的原始样本。
- **SummaryRow**: 由原始样本计算出的汇总。

这里先用 public data struct, 因为它们只是数据载体。如果后续需要不变量检查、解析和格式化, 可以再封装成类。

4.2.12 命令行参数: 可复现需要显式配置

硬编码常量适合第一章入门, 但 benchmark suite 应该能从命令行配置。

示例:

```
./bench_suite \
--sizes 1024,16384,262144,1048576 \
--warmup 5 \
--repetitions 31 \
--batch 1 \
--seed 1 \
--output results/sum_suite
```

解析器可以先写得简单, 但要有清晰错误:

```
#include <charconv>
#include <cstdint>
#include <cstdlib>
#include <string_view>

namespace {

bool parse_u64(std::string_view text, std::uint64_t* value_out)
{
    const char* first = text.data();
    const char* last = text.data() + text.size();
    std::uint64_t value = 0;
    const auto result = std::from_chars(first, last, value);
    if (result.ec != std::errc{} || result.ptr != last) {
        return false;
    }
    *value_out = value;
    return true;
}

} // namespace
```

不要在解析失败后悄悄用默认值。性能报告里的配置必须可信; 输入错了就应当失败并打印错误。

深入理解

成熟项目可以使用 CLI11、cxxopts、absl flags 等库。本书早期用手写解析器, 是为了让你看清楚参数如何进入实验系统。工程项目中应根据规模选择库, 避免自制复杂解析器。

4.2.13 Workload factory: 生成输入但不污染计时

对不同 kernel, 可以定义不同 workload。

```
#include <cstdint>
#include <span>
#include <vector>

struct SumWorkload {
    std::vector<std::uint64_t> values;
    std::uint64_t expected = 0;
};

struct CopyWorkload {
    std::vector<std::uint64_t> source;
    std::vector<std::uint64_t> destination;
};

struct DotWorkload {
    std::vector<float> a;
    std::vector<float> b;
    float expected = 0.0f;
};
```

factory 负责:

- 分配 buffer。
- 初始化输入。
- 计算 reference。

- 保证数据在 timed region 前准备好。

```
std::uint64_t sum_u64_ref(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}

SumWorkload make_sum_workload(std::size_t elements, std::uint64_t seed)
{
    SumWorkload workload;
    workload.values = make_u64_input(elements, seed);
    workload.expected = sum_u64_ref(workload.values);
    return workload;
}
```

对 `copy_u64`, reference 可以不是另一个 `copy` 函数, 而是校验不变量:

```
after copy:
    destination[i] = source[i] for all i
```

对 `fill_u64`:

```
after fill:
    destination[i] = fill_value for all i
```

对 `dot_f32`, reference 需要误差容忍:

```
abs(actual - expected) <= epsilon * max(abs(expected), 1.0)
```

4.2.14 Kernel API: 统一但不过度抽象

为了让 suite 能跑多个 kernel, 需要统一接口。但不要为了统一而隐藏重要差异。可以从简单函数开始:

```
std::uint64_t sum_u64_kernel(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}

void copy_u64_kernel(std::span<std::uint64_t> destination,
                    std::span<const std::uint64_t> source)
{
    for (std::size_t i = 0; i < source.size(); ++i) {
        destination[i] = source[i];
    }
}

void fill_u64_kernel(std::span<std::uint64_t> destination,
                    std::uint64_t value)
{
    for (std::uint64_t& element : destination) {
        element = value;
    }
}

float dot_f32_kernel(std::span<const float> a, std::span<const float> b)
{
    float sum = 0.0f;
    for (std::size_t i = 0; i < a.size(); ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}
```

这里没有强行把所有 kernel 包成同一个函数指针类型, 因为它们返回值和参数不同。更实际的做法是每类 kernel 有自己的 runner, 然后输出相同的 `RawSample`。

常见误区

过度抽象会污染 benchmark。虚函数调用、`std::function` type erasure、动态分派、分配和锁都可能进入热路径。抽象应放在 timed region 外, 或者确认编译器能完全内联。

4.2.15 校验策略：不同 kernel 不同方法

正确性校验不一定每次都完整扫描，但必须有明确策略。

kernel	校验方式	注意
sum_u64	比较返回值。	使用 unsigned 避免 signed overflow UB。
copy_u64	比较输出和输入。	校验会扫描内存，不放入 timed region。
fill_u64	检查元素等于常量。	测量前可能要重置 buffer。
dot_f32	误差容忍比较。	说明容忍度和 reference 顺序。

对于会修改输出 buffer 的 kernel，要注意每次测量前的初始状态。比如 fill_u64：

```
for each repetition:
    reset destination outside timed region if required
    time fill_u64(destination, value)
    validate destination outside timed region
```

如果 reset 成本很大，不能混进 timed region；如果不 reset，又要确认 kernel 的性能不依赖原始内容。

4.2.16 样本汇总：从 raw 到 summary

summary 应由 raw samples 计算。最小实现：

```
#include <algorithm>
#include <stdint>
#include <vector>

std::uint64_t percentile_sorted(const std::vector<std::uint64_t>& sorted,
                                std::size_t numerator,
                                std::size_t denominator)
{
    if (sorted.empty()) {
        return 0;
    }
    const std::size_t last_index = sorted.size() - 1;
    const std::size_t index = (last_index * numerator + denominator - 1) / denominator;
    return sorted[index];
}

SummaryRow summarize_samples(std::string kernel_name,
                              std::size_t elements,
                              std::size_t bytes,
                              int batch_count,
                              std::vector<std::uint64_t> elapsed_ns)
{
    constexpr std::size_t P95_NUMERATOR = 95;
    constexpr std::size_t PERCENT_DENOMINATOR = 100;
    constexpr double BYTES_PER_GIGABYTE = 1.0e9;
    constexpr double NANOSECONDS_PER_SECOND = 1.0e9;

    std::sort(elapsed_ns.begin(), elapsed_ns.end());

    SummaryRow row;
    row.kernel_name = std::move(kernel_name);
    row.elements = elements;
    row.bytes = bytes;
    row.batch_count = batch_count;
    row.repetitions = static_cast<int>(elapsed_ns.size());
    row.min_ns = elapsed_ns.front();
    row.median_ns = elapsed_ns[elapsed_ns.size() / 2];
    row.p95_ns = percentile_sorted(elapsed_ns, P95_NUMERATOR, PERCENT_DENOMINATOR);
    row.max_ns = elapsed_ns.back();

    const double seconds = static_cast<double>(row.median_ns) / NANOSECONDS_PER_SECOND;
    row.gb_per_second = static_cast<double>(bytes) / seconds / BYTES_PER_GIGABYTE;
    return row;
}
```

这个实现还有局限：

- 空样本只返回 0，生产代码应报告错误。
 - p95 使用简单 nearest-rank 近似。
 - GB/s 默认每个 sample 只处理一次 bytes，如果 batch 大于 1，公式要乘以 batch。
- 所以更准确：

```
effective_bytes = bytes * batch_count
GB/s = effective_bytes / seconds / 1e9
```

4.2.17 派生指标：不要只给时间

不同 kernel 应输出不同派生指标。

4.2.17.1 *sum_u64*

显式读：

```
bytes_read = n * sizeof(uint64_t)
GB/s = bytes_read * batch / seconds / 1e9
ns/element = elapsed_ns / (n * batch)
```

4.2.17.2 *copy_u64*

copy 既读又写：

```
bytes_read = n * sizeof(uint64_t)
bytes_written = n * sizeof(uint64_t)
logical_bytes = bytes_read + bytes_written
GB/s = logical_bytes * batch / seconds / 1e9
```

硬件实际流量可能更高，因为写 miss 可能触发 write allocate。后续讲内存系统时会展开。

4.2.17.3 *fill_u64*

fill 主要写：

```
bytes_written = n * sizeof(uint64_t)
GB/s = bytes_written * batch / seconds / 1e9
```

但普通 store 可能也触发 write allocate。非时序 store 是另一个话题。

4.2.17.4 *dot_f32*

每个元素通常算 1 次乘法和 1 次加法：

```
FLOPs = 2 * n
GFLOP/s = FLOPs * batch / seconds / 1e9
bytes_read = 2 * n * sizeof(float)
```

如果使用 FMA，仍通常按 2 FLOPs 计。

4.2.18 输出 CSV 的代码结构

不要在测量循环里频繁 flush。先收集样本，最后写文件。

```
#include <stdio>
#include <string>
#include <vector>

void write_raw_samples_csv(const std::string& path,
                          const std::vector<RawSample>& samples)
{
    FILE* file = std::fopen(path.c_str(), "w");
    if (file == nullptr) {
        std::perror("fopen raw samples");
        std::abort();
    }

    std::fprintf(file,
                 "kernel,n,bytes,batch,rep,elapsed_ns,checksum\n");

    for (const RawSample& sample : samples) {
        std::fprintf(file,
                     "%s,%zu,%zu,%d,%d,%llu,%llu\n",
                     sample.kernel_name.c_str(),
                     sample.elements,
                     sample.bytes,
                     sample.batch_count,
                     sample.repetition,
                     static_cast<unsigned long long>(sample.elapsed_ns),
                     static_cast<unsigned long long>(sample.checksum));
    }

    std::fclose(file);
}
```

}

这段代码使用 C stdio 是为了简单直接。生产代码可以用 RAII 封装文件句柄，或者使用 `std::ofstream`。关键不是 API 选择，而是：文件写入不在 timed region 里。

4.2.19 Runner：把一次实验组织起来

下面是一个简化的 sum runner。它展示结构，不追求覆盖所有错误处理。

```
#include <chrono>
#include <stdint>
#include <span>
#include <vector>

namespace {

using Clock = std::chrono::steady_clock;

template <class T>
inline void do_not_optimize(const T& value)
{
    asm volatile("" : : "g"(value) : "memory");
}

std::uint64_t elapsed_ns_since(Clock::time_point begin, Clock::time_point end)
{
    return static_cast<std::uint64_t>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count());
}

std::uint64_t run_sum_batch(std::span<const std::uint64_t> values,
                           int batch_count)
{
    std::uint64_t combined = 0;
    for (int batch = 0; batch < batch_count; ++batch) {
        combined += sum_u64_kernel(values);
    }
    return combined;
}

void warmup_sum(const SumWorkload& workload,
               int warmup_repetitions,
               int batch_count)
{
    for (int iter = 0; iter < warmup_repetitions; ++iter) {
        const std::uint64_t result = run_sum_batch(workload.values, batch_count);
        do_not_optimize(result);
    }
}

RawSample measure_sum_once(const SumWorkload& workload,
                          std::size_t elements,
                          int batch_count,
                          int repetition)
{
    const auto begin = Clock::now();
    const std::uint64_t result = run_sum_batch(workload.values, batch_count);
    const auto end = Clock::now();

    do_not_optimize(result);

    RawSample sample;
    sample.kernel_name = "sum_u64";
    sample.elements = elements;
    sample.bytes = elements * sizeof(std::uint64_t);
    sample.batch_count = batch_count;
    sample.repetition = repetition;
    sample.elapsed_ns = elapsed_ns_since(begin, end);
    sample.checksum = result;
    return sample;
}

} // namespace
```

注意：

- warmup 和 measurement 分开。
- timed region 只包 batch kernel。
- RawSample 记录 batch 和 checksum。
- 校验可以在外层比较 checksum 和 expected batch result。

expected batch result:

```
expected_batch = workload.expected * batch_count
```

因为使用 `std::uint64_t`，乘法和加法都按模 2^{64} 回绕，语义明确。

4.2.20 参数扫描：矩阵比单点更重要

真实 benchmark 通常是多维矩阵：

```
kernel:
    sum_u64, copy_u64, fill_u64, dot_f32

size:
    1 KiB, 16 KiB, 256 KiB, 4 MiB, 64 MiB

distribution:
    zero, lcg, increasing

cache mode:
    warm, cold-ish

batch:
    1, auto
```

不要一开始就跑满所有组合。组合爆炸会浪费时间。训练建议：

1. 先固定 distribution 和 cache mode，扫描 size。
2. 发现异常后，增加 distribution 维度。
3. 对短 kernel 引入 batch。
4. 对内存 kernel 引入 cold-ish 模式。
5. 最后再做完整矩阵。

CSV 里必须记录每个维度，否则后续无法比较。

4.2.21 自动 batch 的思路

自动 batch 的目标是让单个 sample 足够长。例如希望每个 sample 至少 100 微秒：

```
target_sample_ns = 100000
batch = 1
while measured_ns(batch) < target_sample_ns:
    batch *= 2
```

但这个校准本身也有成本和噪声。它应该在正式测量前完成，并记录最终 batch。

```
int choose_batch_count(std::span<const std::uint64_t> values,
                      std::uint64_t target_ns)
{
    constexpr int BENCHMARK_MAX_BATCH = 1 << 20;

    int batch_count = 1;
    while (batch_count < BENCHMARK_MAX_BATCH) {
        const auto begin = Clock::now();
        const std::uint64_t result = run_sum_batch(values, batch_count);
        const auto end = Clock::now();
        do_not_optimize(result);

        if (elapsed_ns_since(begin, end) >= target_ns) {
            break;
        }
        batch_count *= 2;
    }
    return batch_count;
}
```

风险：

- 校准时的数据会变热。
- batch 变大后测的是重复运行场景。
- 不同机器上 batch 不同，比较时要记录。

4.2.22 避免 harness 干扰热路径

benchmark framework 本身也会影响结果。常见干扰：

- 使用 `std::function` 包装 kernel，导致间接调用。
- 在 timed region 里捕获 lambda，编译器无法内联。
- 每次 sample 创建临时 vector。

- 每次 sample 写日志或 CSV。
- 校验代码意外进入 timed region。
- runner 过度泛型，导致编译器生成复杂控制流。

对热路径最稳的办法是：

```
outside timed region:
    choose kernel
    construct workload
    prepare function-specific runner
```

```
inside timed region:
    direct call or fully inlined call
```

如果使用函数指针：

```
using SumKernel = std::uint64_t (*)(std::span<const std::uint64_t>);
```

函数指针调用可能不能内联，但可以模拟真实动态 dispatch。如果你的生产系统就是通过函数指针做 ISA dispatch，这可能是合理的。如果目标是测 kernel 本体上限，应尽量测 direct call 或单独测 dispatch overhead。

深入理解

AI 算子库常有 dispatcher：根据 dtype、shape、ISA、线程数选择 kernel。优化时要分清两个 benchmark：一个测 end-to-end operator dispatch，一个测已经选定的 microkernel。二者都有价值，但不能混为一谈。

4.2.23 四个基础 kernel 的设计

4.2.23.1 *sum_u64*

目的：

- 训练只读 streaming kernel。
- 观察 cache 层级和内存带宽。
- 学习 DCE 防护和 checksum。

语义：

```
sum = 0
for x in values:
    sum += x
return sum
```

指标：

```
GB/s = n * sizeof(uint64_t) * batch / seconds / 1e9
ns/element = elapsed_ns / (n * batch)
```

4.2.23.2 *copy_u64*

目的：

- 训练读加写 memory kernel。
- 观察 write allocate、带宽上限和 memcpy 对比。
- 学习输出 buffer 校验。

语义：

```
for i in [0, n):
    dst[i] = src[i]
```

指标：

```
logical bytes = 2 * n * sizeof(uint64_t)
```

4.2.23.3 *fill_u64*

目的：

- 训练写密集 kernel。
- 观察 store 带宽、write allocate 和初始化成本。
- 后续可对比普通 store 与 non-temporal store。

语义：

```
for i in [0, n):
    dst[i] = value
```

4.2.23.4 dot_f32

目的：

- 训练浮点计算和内存读取混合 kernel。
- 为后续 SIMD、FMA、reduction 打基础。
- 学习 GFLOP/s 和数值误差报告。

语义：

```
sum = 0
for i in [0, n):
    sum += a[i] * b[i]
return sum
```

指标：

```
FLOPs = 2 * n
bytes_read = 2 * n * sizeof(float)
```

4.2.24 报告中的图表和文字

就算没有画图，也要让表格能表达趋势。推荐 summary 表：

```
kernel,n,bytes,median_ns,GBps,ns_per_element,notes
sum_u64,128,1024,80,12.8,0.625,L1-sized
sum_u64,32768,262144,9000,29.1,0.275,L2-sized
sum_u64,8388608,67108864,3500000,19.2,0.417,memory-sized
```

报告文字不要只写“随着 n 增大性能下降”。要写因果假设：

```
Observation:
    Throughput rises from 1 KiB to 256 KiB, then drops at 64 MiB.

Hypothesis:
    Small sizes are dominated by loop and timer overhead.
    Middle sizes benefit from cache residency.
    Large sizes are limited by memory bandwidth and TLB pressure.

Evidence:
    objdump shows scalar loop, no SIMD.
    perf stat shows cache misses increase at large n.

Limit:
    CPU frequency was not pinned; result needs repeat under performance governor.
```

4.2.25 实验 16.1：设计输入规模扫描

实验

目标：把单点 benchmark 改成输入规模扫描。

基于第 15 章的 `sum_u64` benchmark，支持下面命令：

```
./bench_sum_sizes --sizes 128,2048,32768,524288,8388608 \
    --warmup 5 \
    --repetitions 31 \
    --seed 1 \
    --output results/sum_sizes
```

任务：

1. 实现 `--sizes` 解析，逗号分隔。
2. 每个 size 单独构造 workload。
3. 每个 size 单独 warmup 和测量。
4. 输出 `raw_samples.csv` 和 `summary.csv`。
5. `summary` 中加入 GBps 和 `ns_per_element`。

交付物：

1. 源码。
2. 命令行。
3. raw CSV 前 10 行。
4. summary CSV。
5. 一段趋势解释。

4.2.26 实验 16.2: 加入 copy_u64 和 fill_u64

实验

目标: 把单 kernel benchmark 扩展成多 kernel suite。

新增:

```
void copy_u64_kernel(std::span<std::uint64_t> dst,
                    std::span<const std::uint64_t> src);

void fill_u64_kernel(std::span<std::uint64_t> dst,
                    std::uint64_t value);
```

任务:

1. 为 **copy_u64** 设计 workload。
2. 为 **fill_u64** 设计 workload。
3. 分别定义 correctness check。
4. CSV 中增加 **kernel** 列。
5. 对每个 kernel 正确计算 logical bytes。
6. 对比 **copy_u64** 和 **std::memcpy**, 但要分开报告。

交付物:

1. 扩展后的 suite 源码。
2. 三个 kernel 的 summary。
3. 每个 kernel 的指标定义。
4. **copy_u64** 与 **std::memcpy** 的对比报告。

4.2.27 实验 16.3: 数据分布对分支的影响

实验

目标: 观察输入分布如何改变分支 kernel 性能。

实现:

```
std::uint64_t count_positive(std::span<const std::int32_t> values);
```

生成 4 种输入:

- 全正数。
- 全负数。
- 正负交替。
- 固定 seed 的随机正负。

任务:

1. 每种分布使用同样 size、warmup、repetitions。
2. CSV 中记录 **distribution**。
3. 用 **perfstat** 记录 **branches** 和 **branch-misses**。
4. 反汇编核心循环, 说明是否使用分支、**cmov** 或 **setcc**。
5. 解释性能差异。

交付物:

1. 四种分布的结果表。
2. **perfstat** 输出。
3. 核心循环反汇编。
4. 800 字报告: 输入分布、预测器和性能之间的关系。

4.2.28 实验 16.4: batch 校准

实验

目标: 为短 kernel 设计自动 batch, 并分析它的副作用。

任务:

1. 实现 **--batchauto**。
2. 设定目标单样本时间, 例如 100 微秒。
3. 对每个 size 选择 batch。
4. CSV 记录最终 batch。

5. 比较固定 batch 1 和 auto batch 的结果稳定性。

交付物：

1. batch 校准代码。
2. 每个 size 的 batch 表。
3. auto batch 前后的 min、median、max。
4. 说明 batch 是否改变了 cache 状态。

4.2.29 实验 16.5：完整 benchmark 报告目录

实验

目标：生成一个可以提交给别人复现的 benchmark 结果目录。

目录：

```
results/ch16_suite/
  command.txt
  environment.txt
  raw_samples.csv
  summary.csv
  objdump.txt
  perf_stat.txt
  report.md
```

任务：

1. 运行 suite，并保存完整命令到 `command.txt`。
2. 保存 `lscpu`、`uname-a`、编译器版本。
3. 保存 `objdump-drwC-Mintel` 输出。
4. 对至少一个 kernel 运行 `perfstat`。
5. 写 `report.md`，包含目标、方法、结果、解释和限制。

交付物：

1. 完整结果目录。
2. 一份不超过 1500 字但信息完整的报告。
3. 至少 3 条你认为需要进一步实验验证的假设。

4.2.30 本章作业

习题与作业

作业 1：设计一个 *benchmark suite* 架构

画出你的 benchmark suite 模块图，至少包含：

- options parsing。
- workload factory。
- kernel runner。
- correctness checker。
- sample collector。
- summary generator。
- CSV writer。
- environment recorder。

要求说明每个模块输入、输出和不应承担的责任。

作业 2：输入规模和 *cache* 假设

选择你的机器，根据 `lscpu` 或系统信息估计 L1/L2/L3 cache 大小。为 `sum_u64` 设计 8 个输入规模：

1. 至少 2 个明显小于 L1。
2. 至少 2 个接近 L2 或 L3。
3. 至少 2 个明显超过 LLC。
4. 至少 2 个尾部边界规模，例如非 8、16、32 的倍数。

写出每个规模的元素数、字节数、预期瓶颈和验证方法。

作业 3: CSV schema 评审

设计 raw 和 summary 两个 CSV schema。要求：

- 每列都有单位或明确语义。
- 每个影响结果的参数都能记录。
- 能支持多 kernel、多 size、多 distribution、多 batch。
- 能从 raw 重新生成 summary。

提交 schema 和 5 行示例数据，并解释为什么这些列足够复现实验。

作业 4: 多 kernel suite 实现

实现一个 C++20 benchmark suite，至少包含：

- sum_u64。
- copy_u64。
- fill_u64。
- dot_f32。

要求：

1. 命令行可配置 size、warmup、repetitions、batch、seed。
2. 每个 kernel 都有 correctness check。
3. 输出 raw 和 summary CSV。
4. 不把 allocation、random generation、CSV writing 放进 timed region。
5. 提供至少一个 objdump 核心循环分析。

作业 5: Benchmark 报告

用你的 suite 做一次完整实验，提交报告：

- 环境。
- 编译命令。
- workload matrix。
- raw data 路径。
- summary 表。
- 至少 2 个 kernel 的趋势解释。
- 至少 1 个 PMU 证据。
- 至少 1 个反汇编证据。
- 实验限制。

评分标准

本章作业按下面标准评价：

1. benchmark 架构是否职责清晰。
2. 输入规模是否覆盖不同性能区域。
3. 数据分布、batch、cache 状态是否记录。
4. timed region 是否干净。
5. CSV 是否能支持复现和后续统计。
6. correctness check 是否独立可信。
7. 报告是否用数据、汇编和硬件事件支撑结论。

4.2.31 本章小结

本章把可信测量原则工程化成 benchmark suite 设计：

```
benchmark suite:
  options
  workload factory
  kernel runner
  correctness checker
  sample collector
  summary generator
  CSV writer
  environment recorder
```

```
input matrix:
  size
  distribution
  alignment
  cache mode
  batch

outputs:
  raw_samples.csv
  summary.csv
  environment.txt
  command.txt
  objdump.txt
  perf_stat.txt
  report.md
```

到这里，你应该能把一个临时代码片段升级成可复现的实验系统。下一章会继续补齐统计与噪声分析：如何理解样本分布、异常值、均值和中位数，如何比较两个版本是否真的不同，以及如何把实验复现做得更严谨。

4.3 性能统计、噪声分析和实验复现

4.3.1 本章要解决的问题

第 15 章讲了怎样避免测错，第 16 章讲了怎样把 benchmark 做成可复现工程。本章处理下一层问题：当你已经有了 `raw_samples.csv`，该如何解释这些数字？

很多性能报告会犯两个相反的错误：

- 过度相信单次结果：跑一次快了 5%，就宣布优化成功。
- 过度迷信统计术语：列出均值、方差、p95，却没有解释样本为什么长成这样。

性能统计不是为了把报告写得像数学论文，而是为了保护你不被噪声误导。现代系统里，同一段代码重复运行会受到 OS 调度、中断、CPU 频率、cache 状态、页错误、NUMA、SMT sibling、后台进程、虚拟化和热状态影响。你看到的每个样本都是：

```
observed time = true workload cost + measurement overhead + system noise
```

本章目标是让你掌握：

- 把测量结果看成样本分布，而不是一个数字。
- 理解 min、mean、median、p95、max、标准差、变异系数各自表达什么。
- 识别常见分布形状：稳定、右长尾、双峰、漂移、阶跃。
- 判断 outlier 应该解释、保留、标注，还是在明确规则下剔除。
- 比较两个版本时避免 run order、频率漂移和样本量不足。
- 写出带限制条件的性能结论。
- 保存足够信息，让未来的你或别人能复现实验。

核心思想

性能结论不是由最快一次或最慢一次决定，而是由样本分布、实验条件、二进制证据、硬件事件和可复现记录共同支持。

4.3.2 样本是什么

假设你运行了 31 次 `sum_u64`：

```
elapsed_ns:
  312000
  311800
  312300
  311900
  330000
  ...
```

每个值都是一次观测。它不是“真值”，而是某次运行在某个系统状态下的结果。我们真正想知道的是某个 workload 在给定实验条件下的性能特征：

```
typical cost:
  median or mean

best stable capability:
  min or low percentile

tail behavior:
```

```
p95, p99, max

stability:
  standard deviation, coefficient of variation, spread
```

所以 raw samples 是一等公民，summary 只是它的摘要。永远保存原始样本。

常见误区

如果你只保存 summary，后续就无法判断样本是否双峰、是否有漂移、是否有明显 outlier、是否适合使用均值。summary 不能替代 raw data。

4.3.3 先排序，再观察

最简单有效的第一步是排序。

```
raw:
  312000, 311800, 330000, 312300, 311900

sorted:
  311800, 311900, 312000, 312300, 330000
```

排序后你可以立刻看到：

- 最快值。
 - 最慢值。
 - 中间值。
 - 尾部是否突然拉长。
 - 样本是否分成多个台阶。
- 对小样本，文本表格也很有用：

```
rank,elapsed_ns
0,311800
1,311900
2,312000
3,312300
4,330000
```

当样本数变多，可以画直方图或分位数表。即使不用图，也可以分桶：

```
bucket_ns,count
311000-312000,2
312000-313000,2
330000-331000,1
```

4.3.4 常用统计量的含义

统计量	表达什么	常见误用
min	噪声最少时的最好观测。	当成平均体验。
mean	所有样本的算术平均。	被少数慢样本拉高。
median	排序后的中间样本。	忽略尾部风险。
p95	95% 样本不超过该值。	样本太少时不稳定。
max	最慢观测。	不解释来源就下结论。
stddev	样本围绕均值的离散程度。	对非正态和 outlier 敏感。
CV	标准差除以均值。	忽略分布形状。

其中 CV 是 **coefficient of variation**（变异系数）：

$$CV = \text{standard_deviation} / \text{mean}$$

它用于比较不同规模或不同单位下的相对波动。例如：

```
benchmark A:
  mean = 100 ns
  stddev = 2 ns
  CV = 2%

benchmark B:
  mean = 10 ms
  stddev = 0.2 ms
  CV = 2%
```

两者绝对波动不同，但相对稳定性相同。

深入理解

性能样本常常不是正态分布。它更常见的是右长尾：大多数样本接近稳定热路径，少数样本被调度、中断、缺页或降频拉慢。因此，median 和分位数通常比只看 mean 更直观。

4.3.5 均值和中位数的差异

看样本：

```
100, 101, 100, 102, 500
```

排序：

```
100, 100, 101, 102, 500
```

统计：

```
min    = 100
median = 101
mean   = 180.6
max    = 500
```

mean 很高，因为一个 500 ns 的慢样本把平均值拉大。median 说明典型运行仍在 101 ns 左右。这个 outlier 不能简单删除；你要问它来自哪里：

- 线程被调度走了吗？
- 发生 page fault 了吗？
- CPU 降频了吗？
- benchmark 第一次触发 lazy binding 了吗？
- 后台进程抢了资源吗？

如果它是偶发系统噪声，median 更能代表热路径典型表现。如果你的应用关心尾延迟，例如在线服务，500 ns 这个尾部就不能忽略。

4.3.6 分位数：p95 和 p99

p95 的直觉是：排序后，大约 95% 的样本不超过这个值。

nearest-rank 近似：

```
sorted samples count = N
p95 index = ceil(0.95 * N) - 1
```

如果样本数只有 10：

```
p95 index = ceil(9.5) - 1 = 9
```

也就是 max。因此样本太少时，p95 不稳定。

经验：

- 31 次样本可以初步看 median、min、max。
- 100 次样本可以更好地看 p95。
- p99 通常需要更多样本才有意义。
- 服务端尾延迟要按请求级别收集大量样本，不能只用微基准 31 次。

常见误区

不要在样本很少时严肃解读 p99。比如 31 个样本的 p99 基本就是最大值，它更像“本次最慢观测”，不是稳定尾延迟估计。

4.3.7 一个 C++20 统计汇总器

下面给一个小型 summary 计算器。它处理一组 elapsed_ns，输出常见统计量。这里仍是教学代码，不替代成熟统计库。

```
#include <algorithm>
#include <cmath>
#include <stdint>
#include <span>
#include <vector>

struct SampleSummary {
    std::uint64_t min = 0;
    std::uint64_t median = 0;
    std::uint64_t p95 = 0;
    std::uint64_t max = 0;
    double mean = 0.0;
    double stddev = 0.0;
    double coefficient_of_variation = 0.0;
```

```

};

double mean_of(std::span<const std::uint64_t> samples)
{
    double sum = 0.0;
    for (const std::uint64_t sample : samples) {
        sum += static_cast<double>(sample);
    }
    return sum / static_cast<double>(samples.size());
}

double sample_stddev_of(std::span<const std::uint64_t> samples, double mean)
{
    constexpr std::size_t STATISTICS_MIN_SAMPLES_FOR_STDDEV = 2;

    if (samples.size() < STATISTICS_MIN_SAMPLES_FOR_STDDEV) {
        return 0.0;
    }

    double squared_error_sum = 0.0;
    for (const std::uint64_t sample : samples) {
        const double error = static_cast<double>(sample) - mean;
        squared_error_sum += error * error;
    }

    const double denominator = static_cast<double>(samples.size() - 1);
    return std::sqrt(squared_error_sum / denominator);
}

std::uint64_t percentile_nearest_rank(const std::vector<std::uint64_t>& sorted,
                                     std::size_t numerator,
                                     std::size_t denominator)
{
    if (sorted.empty()) {
        return 0;
    }

    const std::size_t rank = (sorted.size() * numerator + denominator - 1) / denominator;
    const std::size_t index = std::min(rank - 1, sorted.size() - 1);
    return sorted[index];
}

SampleSummary summarize_elapsed_ns(std::span<const std::uint64_t> samples)
{
    constexpr std::size_t STATISTICS_P95_NUMERATOR = 95;
    constexpr std::size_t STATISTICS_PERCENT_DENOMINATOR = 100;

    std::vector<std::uint64_t> sorted(samples.begin(), samples.end());
    std::sort(sorted.begin(), sorted.end());

    SampleSummary summary;
    summary.min = sorted.front();
    summary.median = sorted[sorted.size() / 2];
    summary.p95 = percentile_nearest_rank(
        sorted, STATISTICS_P95_NUMERATOR, STATISTICS_PERCENT_DENOMINATOR);
    summary.max = sorted.back();
    summary.mean = mean_of(samples);
    summary.stddev = sample_stddev_of(samples, summary.mean);
    if (summary.mean != 0.0) {
        summary.coefficient_of_variation = summary.stddev / summary.mean;
    }
    return summary;
}

```

使用时要保证样本非空。生产代码应在解析 CSV 时检查空样本并报告错误，而不是让 `front()` 访问空 vector。

4.3.8 常见分布形状

4.3.8.1 稳定窄分布

```
100, 101, 100, 101, 102, 100, 101
```

特征：

- min、median、mean 接近。
- max 不远。
- CV 很低。

这种结果通常说明实验比较稳定，可以开始比较版本差异。但仍要看反汇编和环境。

4.3.8.2 右长尾

100, 101, 100, 102, 101, 130, 180, 500

特征：

- 大多数样本集中。
- 少数样本明显更慢。
- mean 大于 median。

常见原因：调度、中断、后台进程、缺页、cache 污染。

4.3.8.3 双峰分布

100, 101, 100, 102, 170, 171, 169, 172

特征：

- 样本分成两个稳定簇。
- median 可能落在其中一个簇，但不能解释全部。

常见原因：

- 线程在不同 core 间迁移。
- CPU 频率状态不同。
- cache warm 和 cold 混在一起。
- NUMA 节点不同。
- benchmark 内部有两个路径。

4.3.8.4 漂移

100, 101, 102, 104, 107, 111, 116, 122

特征：样本随时间逐渐变慢或变快。

常见原因：

- CPU 热起来后降频。
- 后台任务启动。
- 数据结构状态逐渐变化。
- 内存分配器或 cache 状态变化。

4.3.8.5 阶跃

100, 101, 100, 101, 160, 161, 160, 162

特征：某个时间点之后整体切到另一个水平。

常见原因：频率档位变化、线程迁移、系统事件、热节流。

核心思想

分布形状本身就是证据。不要只用一个 summary 行掩盖它；至少在报告里描述样本是稳定、长尾、双峰、漂移还是阶跃。

4.3.9 Outlier：删除还是解释

outlier 是明显偏离主要分布的样本。处理原则：

1. 不要默默删除。
2. 先保留 raw data。
3. 尝试解释来源。
4. 如果要删除，必须提前定义规则，并同时报告删除前后的结果。
5. 如果 outlier 代表真实用户会遇到的情况，就不能删除。

例子：

samples:

100, 101, 100, 102, 5000

如果 5000 来自另一个进程抢占 CPU，它对 microkernel 稳态吞吐的估计可能是噪声；但它对端到端延迟体验可能是真实风险。

一个保守报告方式：

```
Raw result:
  median = 101 ns
  max = 5000 ns

Investigation:
  The max sample coincided with a context switch spike.

Conclusion:
  For steady-state kernel throughput, median is representative.
  Tail latency on this shared machine is not controlled.
```

4.3.10 噪声来源地图

来源	现象和观察方法
OS 调度	偶发慢样本、长尾；用 <code>taskset</code> 、 <code>perfstat</code> 和系统负载记录辅助解释。
中断	突然慢一次；增加重复次数，必要时使用隔离核心。
频率变化	漂移或阶跃；记录 <code>governor</code> 、温度，并用 <code>TSC</code> 或 <code>cycles</code> 对照。
迁核	双峰或阶跃；用 <code>taskset</code> 固定 CPU，并记录样本所在 CPU。
SMT 共享核	波动变大；关闭 SMT，或避免 sibling 上运行其他重负载线程。
缺页	首次运行慢；增加 warmup，并观察 <code>perfstat</code> 的 <code>faults</code> 计数。
NUMA	大工作集双峰；用 <code>numactl</code> 固定内存节点和执行节点。
虚拟化	高频噪声和 counter 不准；在报告里记录 VM 或 container 环境。

这张表不是让你每次都控制所有因素，而是让你知道异常形状可能对应什么机制。

4.3.11 频率：时间和 cycles 不总是同一件事

现代 CPU 会动态调频。一个 kernel 的 wall time 可能变慢，不一定是指令变多；可能是频率低了。相反，cycles 变化也要结合频率解释。
报告中可以同时记录：

```
steady_clock elapsed_ns
perf stat cycles
perf stat instructions
IPC = instructions / cycles
```

如果版本 A 和 B：

```
A:
  time = 1.0 ms
  cycles = 3.5 M

B:
  time = 1.1 ms
  cycles = 3.2 M
```

这说明 B retired cycles 更少，但 wall time 更慢。可能原因：

- 运行 B 时频率更低。
- counter 语义或测量区域不同。
- A/B 运行顺序导致热状态不同。

不要只拿一个指标下结论。时间、cycles、instructions、汇编、环境记录要一起看。

4.3.12 A/B 比较：不要按顺序跑完 A 再跑 B

错误做法：

```
run A 100 times
run B 100 times
compare medians
```

如果机器逐渐升温降频，B 可能天然吃亏；如果 cache 或 predictor 被 A 预热，B 也可能受益或受害。更好的做法是交替：

```
for rep in repetitions:
  run A
  run B
```

或者 ABBA：

```
for block in blocks:
  run A
  run B
```



```
run B
run A
```

也可以随机化顺序，但必须固定 seed 并记录：

```
order:
  A, B, B, A, B, A, ...
seed:
  12345
```

深入理解

交替运行可以减少慢速漂移的偏差，但会引入另一种问题：A 和 B 可能互相影响 cache、branch predictor 或 shared resources。严格实验需要根据 workload 设计清理或隔离策略。

4.3.13 Speedup 的正确写法

如果指标是时间，越小越好：

```
speedup = baseline_time / optimized_time
```

例如：

```
baseline median = 120 ns
optimized median = 100 ns
speedup = 120 / 100 = 1.20x
```

如果指标是吞吐，越大越好：

```
throughput speedup = optimized_GBps / baseline_GBps
```

不要写错方向。也不要混用百分比和倍数：

```
1.20x faster:
  optimized is 20% faster

20% lower time:
  optimized_time = baseline_time * 0.80
  speedup = 1 / 0.80 = 1.25x
```

“快 20%”和“耗时降低 20%”不是同一句话。

4.3.14 版本差异是否真实

假设：

```
A median = 100.0 ns
B median = 99.5 ns
```

B 看起来快 0.5%。这是否真实？要看噪声：

```
A samples mostly between 99 and 102
B samples mostly between 98.8 and 102
```

两个分布高度重叠，0.5% 差异可能不可信。

另一个例子：

```
A median = 100 ns, p95 = 103 ns
B median = 80 ns, p95 = 82 ns
```

差异大且分布分离，更可信。

工程判断规则：

- 差异小于样本波动时，不要宣布优化成功。
- 对小差异，增加 repetitions，并在不同进程运行中重复。
- 对重要结论，换一天、换重启后、换固定频率再测。
- 结合汇编解释：代码真的少了指令、减少了 miss，还是只是噪声？

4.3.15 置信直觉

统计学里的置信区间有严格定义。本书先建立工程直觉。

如果样本近似独立、分布不太奇怪，均值的标准误差近似：

```
standard_error = stddev / sqrt(n)
```

粗略 95% 区间：

```
mean +/- 1.96 * standard_error
```

但性能样本常常有长尾、相关、非正态，所以不要机械套公式。更稳妥的工程做法：

- 报告 median、p95、min、max。
- 保存 raw samples。
- 对 A/B 使用交替或随机顺序。

- 多次独立运行整个 benchmark 进程。
- 结论使用“在本环境和本 workload 下”限定。

常见误区

不要把“95% 置信区间”写成装饰词。如果实验设计不满足基本假设，漂亮的区间数字可能比没有区间更误导。

4.3.16 独立运行：process-level repetition

同一进程内重复 100 次可以观察短期噪声，但不能覆盖所有变化。更强的复现方式是重复整个进程：

```
for run in 0 1 2 3 4; do
  ./bench_suite --output results/run_${run}
done
```

这能暴露：

- 进程启动状态差异。
- ASLR 地址布局差异。
- 动态链接和首次触页差异。
- 长时间热状态变化。

报告可以分两层：

```
within-run:
  median over repetitions inside one process

between-run:
  median of process medians
  min and max across process medians
```

如果每个进程内很稳定，但进程间差异大，说明还有未控制的环境因素。

4.3.17 复现实验日志

每次实验保存：

```
results/experiment_name/
  raw_samples.csv
  summary.csv
  runs/
    run_00/raw_samples.csv
    run_00/summary.csv
    run_01/raw_samples.csv
    run_01/summary.csv
  environment.txt
  command.txt
  git_status.txt
  binary_sha256.txt
  objdump.txt
  perf_stat.txt
  notes.md
```

notes.md 里写：

```
What changed:
  Replaced scalar sum with unrolled sum by 4 accumulators.

Hypothesis:
  Reduces loop-carried dependency and improves ILP.

Expected evidence:
  Lower cycles per element.
  Similar bytes read.
  More independent add instructions in objdump.

Observed:
  median improved by 8% at L1-sized inputs.
  no improvement at memory-sized inputs.

Limits:
  CPU frequency governor was powersave.
  perf counters unavailable due permission.
```

核心理想

复现不是“我能再跑一次”。复现是未来有人能根据你的代码、命令、输入、环境、二进制和 raw data 重建你的结论路径。

4.3.18 一个 A/B 比较表模板

不要只给一行 speedup。推荐：

```
kernel,n,metric,A_median,B_median,speedup,A_p95,B_p95,A_CV,B_CV,verdict
sum_u64,1048576,time_ns,312000,290000,1.076,330000,305000,0.018,0.012,likely faster
sum_u64,8388608,time_ns,350000,348000,1.006,370000,369000,0.021,0.022,inconclusive
```

verdict 不应只有 faster/slower，可以用：

- **likelyfaster**: 差异明显，大于噪声，证据一致。
- **likelyslower**: 明显变慢。
- **inconclusive**: 差异小于波动或证据冲突。
- **changedbehavior**: 正确性、汇编或工作量改变，不能直接比较。

4.3.19 实验 17.1: 100 次样本分布分析

实验

目标：从 raw samples 观察分布，而不是只看 summary。

选择第 16 章的 **sum_u64** 或 **copy_u64**，固定一个输入规模，运行：

```
./bench_suite --kernel sum_u64 \
  --sizes 1048576 \
  --warmup 5 \
  --repetitions 100 \
  --batch 1 \
  --seed 1 \
  --output results/ch17_dist
```

任务：

1. 保存 raw samples。
2. 排序样本。
3. 计算 min、mean、median、p95、max、stddev、CV。
4. 做一个文本分桶表。
5. 判断分布形状：稳定、右长尾、双峰、漂移或阶跃。

交付物：

1. raw samples。
2. 排序后的前 10 个和后 10 个样本。
3. 统计表。
4. 分布形状分析。
5. 对异常样本的可能解释。

4.3.20 实验 17.2: Outlier 调查

实验

目标：学习解释 outlier，而不是机械删除。

任务：

1. 找到实验 17.1 中最慢的 3 个样本。
2. 查看它们出现的 repetition 序号。
3. 检查是否集中在开头、结尾或随机分布。
4. 用 **/usr/bin/time-v** 或 **perfstat** 重新运行，观察 page faults、context switches。
5. 写出“保留 outlier”和“按规则剔除 outlier”两份 summary。

交付物：

1. outlier 样本表。
2. 你认为的原因假设。
3. 删除规则，如果你选择删除。
4. 删除前后 summary 对比。
5. 说明哪个 summary 更适合你的结论，为什么。

4.3.21 实验 17.3: A/B 交替比较

实验

目标：比较两个版本时避免运行顺序偏差。

准备两个版本：

- A：普通 `sum_u64`。
- B：4 个累加器展开版本。

运行三种顺序：

1. 先 A 100 次，再 B 100 次。
2. 交替 AB，重复 100 个 pair。
3. ABBA block，重复 50 个 block。

任务：

1. 分别计算 A 和 B 的 median、p95、CV。
2. 计算 speedup。
3. 判断三种顺序是否给出一致结论。
4. 用 `objdump` 解释 B 是否真的减少依赖链。
5. 用 `perfstat` 观察 cycles 和 instructions。

交付物：

1. 三种顺序的 raw data。
2. A/B summary 表。
3. speedup 表。
4. 汇编证据。
5. 对 run order bias 的解释。

4.3.22 实验 17.4: 频率和温度漂移

实验

目标：观察长时间运行时结果是否漂移。

任务：

1. 选择一个至少运行 10 秒的 benchmark。
2. 每个 sample 记录 repetition 序号和 `elapsed_ns`。
3. 按时间顺序观察是否变慢或变快。
4. 如果系统提供频率文件，采样当前 CPU 频率。
5. 对比 `taskset` 绑定前后结果。

交付物：

1. 时间顺序样本表。
2. 漂移判断。
3. 频率或环境记录。
4. 绑定 core 前后的对比。
5. 说明你的机器是否适合做精细 microbenchmark。

4.3.23 实验 17.5: 复现实验包

实验

目标：把一次性能实验整理成别人能复查的结果包。

目录：

```
results/ch17_repro/
  README.md
  command.txt
  environment.txt
  raw_samples.csv
  summary.csv
  ab_comparison.csv
  objdump.txt
  perf_stat.txt
  notes.md
```

要求：

1. `README.md` 写清楚如何重新运行。
2. `notes.md` 写清楚假设、结果和限制。
3. 保留所有 raw samples。
4. 至少包含一个 A/B comparison。
5. 至少包含一个你认为 inconclusive 的结果，并解释为什么。

4.3.24 本章作业

习题与作业

作业 1：统计量解释

给定样本：

```
100, 101, 100, 102, 101, 100, 500
```

要求：

1. 排序。
2. 计算 min、mean、median、max。
3. 解释 mean 和 median 为什么差异很大。
4. 判断是否存在 outlier。
5. 写出你会如何报告这组样本。

作业 2：分布形状分类

对下面四组样本分类，并说明可能原因：

```
A: 100, 101, 100, 101, 102, 100
B: 100, 101, 102, 150, 300, 900
C: 100, 101, 100, 170, 171, 169
D: 100, 102, 105, 110, 118, 127
```

要求至少提到稳定、右长尾、双峰、漂移中的三个概念。

作业 3：A/B 结论审计

给定：

```
A median = 100 ns, p95 = 103 ns, CV = 0.01
B median = 98 ns, p95 = 120 ns, CV = 0.08
```

要求：

1. 计算 median speedup。
2. 判断 B 是否可以简单称为“更快”。
3. 解释 p95 和 CV 暗示了什么风险。
4. 写一段严谨结论。

作业 4：噪声来源调查

在你的机器上运行同一个 benchmark：

1. 不绑定 core。
2. 使用 `taskset` 绑定一个 core。
3. 后台运行一个 CPU 密集程序。
4. 如果可以，在电源不同模式下运行。

提交每种条件下的 median、p95、max、CV，并解释差异。

作业 5：完整统计报告

基于第 16 章的 suite，选择一个 kernel 做完整统计报告：

- 至少 100 个 raw samples。
- 样本排序表。
- 统计 summary。
- 分布形状判断。
- outlier 调查。

- 至少一次 A/B 比较。
- 结论和限制。

评分标准

本章作业按下面标准评价：

1. 是否保存并使用 raw samples。
2. 是否能正确解释 mean、median、p95、max 和 CV。
3. 是否能识别分布形状，而不是只报一个数字。
4. outlier 处理是否透明。
5. A/B 比较是否避免运行顺序偏差。
6. 结论是否明确限制在具体环境和 workload 下。
7. 复现实验包是否足够别人复查。

4.3.25 本章小结

本章把 benchmark 输出的数字变成可解释的证据：

```
raw samples:
  preserve first, summarize second

summary:
  min, mean, median, p95, max, stddev, CV

distribution:
  stable, right-tail, bimodal, drift, step

outlier:
  explain and document; do not silently delete

A/B comparison:
  alternate or randomize order
  compare distributions, not only medians

reproducibility:
  raw data, command, environment, binary, objdump, perf, notes
```

从现在开始，你写性能结论时应当更克制：不是“优化后快了”，而是“在这台机器、这个输入规模、这个编译参数、这个样本分布下，B 的 median 比 A 低 7%，p95 也更低，汇编显示依赖链减少，PMU cycles 下降；但频率未固定，所以仍需在受控环境复测”。这才是性能工程里可信的表达方式。

4.4 Lab 00：建立可信性能实验室

4.4.1 本章定位

前面三章分别建立了性能测量的原则、benchmark suite 的工程结构、统计和噪声分析的方法。本章把这些原则落到第一个完整实验：`labs/lab00_benchmark_foundation`。

这不是一个“跑一下看看速度”的小练习，而是后续所有优化实验的地基。你以后做汇编、SIMD、cache 优化、矩阵乘法、softmax、LayerNorm、算子融合时，都必须回答同一组问题：

- 代码是否正确？
- 被测区域到底包含什么？
- 编译器有没有把工作删掉、提前算掉、合并掉？
- 输入规模和数据分布是否覆盖了你声称的场景？
- raw samples 是否保存？
- summary 是否来自 raw samples，而不是手工挑数？
- 环境、构建模式、编译器、CPU 信息是否记录？
- 结论能否被二进制、反汇编、PMU 或系统信息支持？

本章的目标不是让你马上写最快的 kernel，而是让你拥有一个能产生可信证据的实验室。没有这个地基，所谓优化很容易退化成猜测。

核心思想

可信性能实验的顺序是：先 correctness，再 measurement boundary，再 raw data，再 statistics，再 binary audit，最后才是 performance claim。任何跳过前面步骤的结论都不稳。

4.4.2 你要交付什么

完成本章后，你应该提交一个完整的 Lab00 报告，而不是一张截图。报告至少包含：

- 运行命令和构建类型。
- `results/lab00/env.txt` 的关键环境摘要。
- `ctest` 正确性结果。
- `results/lab00/foundation.csv` 的 raw sample 摘要。
- 每个 benchmark 在不同输入规模下的 min、median、max。
- 对 `results/lab00/bad_benchmarks.txt` 中错误案例的解释。
- 至少一个 kernel 的反汇编审计记录。
- 至少 5 条你现在愿意遵守的 benchmark 规则。
- 至少 3 个你仍然不能确定、需要后续章节解决的问题。

如果你不能把这些信息整理出来，说明你还没有真正掌握实验流程；如果你能稳定地产生这些信息，后面的优化工作才有可比较的基础。

4.4.3 目录地图

先从目录看这个 lab 的结构：

```
labs/lab00_benchmark_foundation/
  CMakeLists.txt
  include/lab00/
    benchmark.hpp
    kernels.hpp
  src/
    benchmark.cpp
    kernels.cpp
    main.cpp
    bad_benchmarks.cpp
  tests/
    lab00_tests.cpp
  scripts/
    run_lab00.sh
```

对应的输出目录是：

```
results/lab00/
  env.txt
  foundation.csv
  summary.md
  bad_benchmarks.txt
```

这两个目录构成了实验闭环：

```
source code
-> build
-> tests
-> environment report
-> benchmark raw samples
-> summary
-> bad examples
-> written report
```

注意这里的顺序。测试在 benchmark 之前，因为错误代码跑得再快也没有价值。环境记录在 benchmark 之前，因为后续解释数据时你需要知道机器和系统状态。raw samples 在 summary 之前，因为 summary 必须可追溯。

4.4.4 一键运行脚本不是魔法

先看最重要的运行入口：

```
bash labs/lab00_benchmark_foundation/scripts/run_lab00.sh
```

这个脚本做了几件事：

```
configure Release build
build lab00_bench, lab00_bad_benchmarks, lab00_tests
run CTest
write environment report
run foundation benchmark
summarize CSV
run intentionally bad benchmark examples
```

对应的关键命令如下：

```
cmake -S "${ROOT_DIR}" -B "${BUILD_DIR}" \
  -DCMAKE_BUILD_TYPE=Release \
  -DCPU_BUILD_LABS=ON
```



```
cmake --build "${BUILD_DIR}" \
  --target lab00_bench lab00_bad_benchmarks lab00_tests

ctest --test-dir "${BUILD_DIR}" \
  -R lab00_tests \
  --output-on-failure

"${ROOT_DIR}/tools/env_report.sh" > "${RESULT_DIR}/env.txt"

"${BUILD_DIR}/labs/lab00_benchmark_foundation/lab00_bench" \
  --sizes 1024,16384,1048576 \
  --warmup 3 \
  --iterations 10 \
  --csv "${RESULT_DIR}/foundation.csv"
```

这里有两个非常重要的设计点。

第一，构建类型是 **Release**。Debug build 的性能数据不能用于优化结论，因为它通常包含额外的调试信息、较低优化等级、不同的内联决策、不同的寄存器分配和更多框架开销。Debug build 可以用来调试 correctness，不应用来报告性能。

第二，脚本把输出保存到 **results/lab00**，而不是只打印到屏幕。性能实验必须留下产物。没有产物，就不能比较、复查、画图、重算统计量，也不能在几周后解释当时发生了什么。

常见误区

一键脚本不是为了偷懒，而是为了减少人为差异。一个实验如果只能靠你手工敲十几条命令复现，那么它迟早会因为漏一步、错一个参数或换一个目录而失真。

4.4.5 CMake 架构：为什么要拆成库、程序和测试

labs/lab00_benchmark_foundation/CMakeLists.txt 把 lab 拆成四个目标：

```
lab00_benchmark      static library
lab00_bench          benchmark executable
lab00_bad_benchmarks wrong benchmark examples
lab00_tests          correctness tests
```

为什么不把所有代码都塞进一个 **main.cpp**？

- kernel 和 runner 放进库，测试程序可以直接调用它们。
- benchmark 主程序只处理 CLI 和输出，不承担核心逻辑。
- 错误 benchmark 单独成程序，避免污染正常实验。
- 后续 lab 可以复用 runner 或对比不同实现。
- CTest 能够在运行 benchmark 前先验证基础行为。

这一点非常像大型性能库的工程结构。生产级 GEMM、FFT、DNN 算子库不会把 kernel、测试、benchmark、报告脚本混在一个文件里。它们会把 correctness、measurement、dispatch、report 分开，因为每一层都有不同的不变量。

```
add_library(lab00_benchmark STATIC
  src/benchmark.cpp
  src/kernels.cpp
)

target_include_directories(lab00_benchmark
  PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

add_executable(lab00_bench
  src/main.cpp
)

target_link_libraries(lab00_bench PRIVATE lab00_benchmark)

add_executable(lab00_tests
  tests/lab00_tests.cpp
)

target_link_libraries(lab00_tests PRIVATE lab00_benchmark)
```

这里的 **target_include_directories(...PUBLIC...)** 表示使用 **lab00_benchmark** 的目标也能看到 **include**。因此源码可以写稳定的 include：

```
#include <lab00/benchmark.hpp>
#include <lab00/kernels.hpp>
```

这比深层相对路径更稳：


```
// 不推荐
#include "../include/lab00/benchmark.hpp"
```

后者会把文件位置耦合到源码路径，目录一变就坏；前者表达的是模块接口。

4.4.6 公共 API: benchmark harness 的骨架

打开 `include/lab00/benchmark.hpp`，先看几个核心类型：

```
struct BenchmarkConfig {
    std::size_t warmup_iterations = LAB00_DEFAULT_WARMUP_ITERATIONS;
    std::size_t measured_iterations = LAB00_DEFAULT_MEASURED_ITERATIONS;
    std::vector<std::size_t> input_sizes;
    std::string only_name;
};

struct BenchmarkResult {
    std::string name;
    std::size_t input_size = 0;
    std::size_t iteration = 0;
    std::int64_t elapsed_ns = 0;
    std::uint64_t checksum = 0;
};

using BenchmarkWorkload = std::function<std::uint64_t(>>;
using BenchmarkFactory = std::function<BenchmarkWorkload(std::size_t input_size)>;

struct BenchmarkCase {
    std::string name;
    BenchmarkFactory make_workload;
};
```

这些类型看起来普通，但它们把一个 benchmark 的核心边界定义出来了。

类型	职责
<code>BenchmarkConfig</code>	描述一次实验如何运行：warmup 次数、测量次数、输入规模、是否只运行某个 case。
<code>BenchmarkResult</code>	描述一次观测：哪个 benchmark、哪个输入规模、第几次、耗时多少、结果校验值是多少。
<code>BenchmarkWorkload</code>	一个已经准备好输入、可以直接计时调用的闭包。
<code>BenchmarkFactory</code>	根据输入规模创建 workload，准备阶段不进入计时区域。
<code>BenchmarkCase</code>	把名字和 workload factory 绑定起来。

这里最关键的是 `BenchmarkFactory` 和 `BenchmarkWorkload` 的分离。

```
factory(input_size):
    allocate buffers
    initialize data
    capture buffers in closure
    return workload

workload():
    run only the hot kernel
    return checksum
```

如果没有 factory，很多初学者会把分配、初始化、随机数生成和 kernel 混在一个函数里计时。那样测到的是“整个准备流程”的成本，不是 kernel 成本。

心智模型

把 factory 看成“实验样本制备”，把 workload 看成“被测热路径”。样本制备可以复杂，但不能混入热路径；热路径应该尽量只包含你真正想比较的代码。

4.4.7 Runner 数据流

`BenchmarkRunner::run()` 是这个 lab 的心脏。它的逻辑可以压缩成下面的流程：

```
for each benchmark case:
    if --only does not match:
        skip

    for each input size:
        workload = factory(input_size)

        repeat warmup_iterations:
            checksum = workload()
```

```
do_not_optimize(checksum)

repeat measured_iterations:
    begin = steady_clock::now()
    checksum = workload()
    end = steady_clock::now()

do_not_optimize(checksum)
append BenchmarkResult
```

真实代码中的核心片段是：

```
for (std::size_t iteration = 0; iteration < this->config_.measured_iterations; ++iteration) {
    auto const begin = std::chrono::steady_clock::now();
    std::uint64_t const checksum = workload();
    auto const end = std::chrono::steady_clock::now();

    do_not_optimize(checksum);

    results.push_back(BenchmarkResult{
        .name = benchmark_case.name,
        .input_size = input_size,
        .iteration = iteration,
        .elapsed_ns = elapsed_nanoseconds(begin, end),
        .checksum = checksum,
    });
}
```

这段代码有三个边界要盯住。

第一，`begin` 和 `end` 之间只包含 `workload()`。如果这里出现日志、分配、格式化、文件写入、随机数生成，benchmark 就被污染。

第二，`do_not_optimize(checksum)` 在计时之后调用。它告诉编译器结果被观察了，不能随便删掉计算。它不应该把大量额外工作加入计时区域。

第三，`results.push_back(...)` 在计时之后发生。记录结果本身不是 kernel 的一部分。

4.4.8 steady_clock 和 elapsed_ns

这个 lab 使用 `std::chrono::steady_clock`：

```
[[nodiscard]] std::int64_t elapsed_nanoseconds(
    std::chrono::steady_clock::time_point begin,
    std::chrono::steady_clock::time_point end)
{
    return std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();
}
```

`steady_clock` 的语义是单调时间源：时间点不会因为系统时间校准而倒退。它适合做一般 benchmark 的第一层计时。它的缺点是：一次调用的开销不是零，分辨率和实现依赖平台，极短 kernel 可能被计时器开销淹没。

因此，本章实验有两个实践规则：

- 如果 kernel 太短，增大输入规模或增加 batch，避免一次测量接近计时器开销。
- 不要把 `elapsed_ns` 直接当成 CPU cycles。cycles 需要 TSC 或 PMU 辅助观察。

后续章节会引入 `rdtsc`、`rdtscp`、serializing instruction、PMU event 和 top-down analysis。本章先用 `steady_clock` 建立可移植地基。

4.4.9 do_not_optimize：和优化器达成边界协议

性能测量最常见的灾难之一是：你以为代码跑完了，编译器发现结果没人用，于是把它删了。

本 lab 的 `do_not_optimize` 是：

```
template <typename T>
inline void do_not_optimize(T const& value) noexcept
{
    #if defined(__clang__) || defined(__GNUC__)
        asm volatile("" : : "g"(value) : "memory");
    #else
        auto const volatile* volatile_value = &value;
        static_cast<void*>(volatile_value);
    #endif
}
```

对 GCC 和 Clang，这是一段空的 inline assembly。它不生成实际指令，但它告诉编译器：

- `value` 是输入操作数。
- 这段 asm 是 `volatile`，不能随意删除。
- `"memory"` clobber 表示内存可能被观察或影响，不能跨过它任意重排内存访问。

它不是性能优化指令；它是测量边界工具。它解决的是“编译器是否认为结果可删除”的问题，不解决 OS 噪声、cache 状态、频率变化、分支预测等问题。

常见误区

`do_not_optimize` 不是万能护身符。它只能降低某些编译器优化对测量的破坏，不能把一个设计错误的 benchmark 变成可信实验。

4.4.10 clobber_memory: 告诉编译器内存边界存在

对写内存的 kernel，例如 `fill_u8` 和 `copy_u8`，还需要避免编译器认为某些内存写可以被重排或删除。lab 中使用：

```
inline void clobber_memory() noexcept
{
    #if defined(__clang__) || defined(__GNUC__)
        asm volatile("" : : "memory");
    #endif
}
```

`clobber_memory()` 不会刷新 CPU cache，也不会执行内存屏障指令。它主要约束编译器，而不是约束硬件。理解这一点很重要：

```
compiler barrier:
    affects compiler reordering
    usually emits no hardware instruction

hardware memory fence:
    affects CPU memory ordering
    may emit mfence, lfence, sfence, lock-prefixed instruction, etc.
```

本 lab 暂时只需要 compiler barrier。多线程、atomic、memory ordering 会在后面的并发章节单独讲。

4.4.11 CSV: raw sample 是一等公民

`BenchmarkRunner::write_csv()` 输出：

```
name,input_size,iteration,elapsed_ns,checksum
sum_i32,1024,0,521,18446744073709551111
sum_i32,1024,1,514,18446744073709551111
sum_i32,1024,2,514,18446744073709551111
```

每一行代表一次测量，不是最终结论。这个格式有几个优点：

- `name` 支持多个 benchmark。
- `input_size` 支持规模扫描。
- `iteration` 保留重复测量顺序。
- `elapsed_ns` 保留原始耗时。
- `checksum` 帮助检查结果是否稳定。

注意 `checksum` 不是完整 correctness test。它只说明某次 workload 返回了一个被观察的值。真正的 correctness 仍然需要 `lab00_tests` 或 reference implementation。

深入理解

为什么 CSV 里不直接只写 median？因为 median 是可重新计算的派生结果，而 raw sample 不是。保存 raw sample 等于保存实验事实；保存 summary 只是保存一次解释。

4.4.12 五个基础 kernel

`src/kernels.cpp` 注册了五个 benchmark：

```
sum_i32
sum_f32
dot_f32
fill_u8
copy_u8
```

它们看起来简单，但刚好覆盖后续优化常见问题。

kernel	主要工作	后续会牵出的主题
sum_i32	整数数组规约。	load 吞吐、整数加法、循环展开、向量规约。
sum_f32	浮点数组规约。	浮点结合律、fast-math、向量化、数值误差。
dot_f32	两个数组乘加规约。	FMA、内存带宽、寄存器累加器、SIMD lane。
fill_u8	写一个字节数组。	store 带宽、写分配、memset、cache line。
copy_u8	从一个数组复制到另一个数组。	load/store 平衡、memcpy、预取、内存带宽。

这五个 kernel 不是为了炫技，而是为了建立性能分类直觉：

```
sum_i32:
  one input stream
  integer accumulation
  likely memory-sensitive for large n

sum_f32:
  one input stream
  floating accumulation
  compiler may preserve strict order unless allowed otherwise

dot_f32:
  two input streams
  multiply + add
  can use FMA if compiler and target allow

fill_u8:
  write stream
  may map to optimized memset-like code

copy_u8:
  read stream + write stream
  memory subsystem dominates for large n
```

等你后面学到 SIMD 和 microarchitecture，再回头看这些 baseline，会发现它们几乎覆盖了所有基础性模式。

4.4.13 确定性输入：实验不能依赖当天运气

以 make_i32_input 为例：

```
[[nodiscard]] std::vector<std::int32_t> make_i32_input(std::size_t input_size)
{
    std::vector<std::int32_t> values(input_size);
    for (std::size_t index = 0; index < values.size(); ++index) {
        auto const raw = static_cast<std::int32_t>(
            (index * LAB00_I32_PATTERN_MULTIPLIER + LAB00_I32_PATTERN_OFFSET)
            % LAB00_I32_PATTERN_MODULUS);
        values[index] = raw - LAB00_I32_PATTERN_CENTER;
    }
    return values;
}
```

这里没有在热路径里调用随机数生成器。输入在 factory 阶段生成，然后被 workload 捕获。

```
benchmarks.push_back(make_case("sum_i32", [](std::size_t input_size) {
    return [values = make_i32_input(input_size)]() -> std::uint64_t {
        return checksum_i64(sum_i32(values));
    };
}));
```

这个捕获动作很重要：

```
factory time:
  allocate vector
  fill vector
  move vector into closure

measured time:
  call sum_i32(values)
  return checksum
```

如果把输入生成放进 measured time，测到的就是输入生成加 kernel 的总和。对小 kernel 来说，输入生成甚至可能远远超过 kernel 本身。

4.4.14 CLI: 让实验参数显式化

src/main.cpp 支持:

```
--help
--list
--csv <path>
--sizes <a,b,c>
--warmup <count>
--iterations <count>
--only <name>
```

这不是为了界面好看, 而是为了让实验条件出现在命令行里。命令行比“改源码里的常量再编译”更适合实验记录。

列出所有 benchmark:

```
build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench --list
```

只运行 dot_f32:

```
build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench \
--only dot_f32 \
--sizes 1024,16384,1048576 \
--warmup 3 \
--iterations 10
```

这里的 `--only` 对后续调试很重要。一个完整 benchmark suite 可能有几十个 case, 如果你每次改一个 kernel 都跑全量, 会浪费大量时间, 也会让调试反馈变慢。

4.4.15 输入规模: Lab 00 的三档只是起点

默认输入规模是:

```
1024
16384
1048576
```

这三个数并不是神秘常数。它们分别代表:

- 小规模: 函数调用、循环开销、前端和计时器开销更明显。
- 中规模: 更接近 cache 内工作集。
- 大规模: 更容易触及内存带宽和缓存层级。

但是你不能永远只测这三档。本章作业会要求你加入更多规模:

```
16,64,256,1024,4096,16384,65536,262144,1048576,4194304
```

这些规模能帮助你观察曲线形状:

```
small n:
    fixed overhead dominates

medium n:
    cache behavior and vectorization become visible

large n:
    bandwidth and memory hierarchy dominate
```

真正的优化报告通常不是一个数字, 而是一条曲线。

4.4.16 测试: 正确性是性能实验的入口

tests/lab00_tests.cpp 是一个无第三方依赖的小测试程序。它覆盖:

- parse_size_list 能解析逗号分隔的输入规模。
- parse_size_list 会拒绝 0。
- sum_i32、sum_f32、dot_f32 的基础正确性。
- dot_f32 会拒绝不等长输入。
- fill_u8 和 copy_u8 会写出期望数据。
- benchmark case 数量符合预期。
- runner 只记录 measured iterations, 不记录 warmup。
- --only 过滤语义。
- CSV 输出格式。

运行命令:

```
cctest --test-dir build/lab00-release -R lab00_tests --output-on-failure
```

测试代码里有一个简单的 require:

```
void require(bool condition, std::string_view message)
{
```

```
if (!condition) {
    throw std::runtime_error(std::string(message));
}
}
```

这不是完整测试框架，但足够让本 lab 不依赖外部库。后续如果你愿意，可以迁移到 Catch2、GoogleTest 或 doctest；但无论使用什么框架，原则不变：先测 correctness，再跑 benchmark。

4.4.17 覆盖率：知道自己测到了什么

仓库里提供了：

```
bash tools/run_coverage.sh
cat results/coverage/summary.txt
```

覆盖率不是质量的全部，但它能暴露一个常见问题：你以为测试覆盖了某个分支，实际上完全没有执行过。

对 Lab 00，覆盖率重点不是追求漂亮数字，而是检查这些风险点：

- CLI 解析错误路径是否测试过。
- size list 空 token、非数字、0 是否被拒绝。
- runner 是否真的区分 warmup 和 measured iteration。
- dot_f32 不等长输入是否报错。
- CSV 输出格式是否稳定。

性能工程里，测试和 benchmark 的关系很紧：测试保证“结果可信”，benchmark 保证“速度可观察”。缺一个都不行。

4.4.18 错误 benchmark 博物馆

src/bad_benchmarks.cpp 故意展示四类错误：

```
dead result not observed
allocation inside timing
RNG inside timing
formatting inside timing
```

运行后查看：

```
cat results/lab00/bad_benchmarks.txt
```

这些错误要认真看。它们比正确示例更能训练直觉。

4.4.18.1 错误一：结果没有被观察

错误版本：

```
std::uint64_t sum = 0;
for (std::uint64_t value : input) {
    sum += value;
}
static_cast<void>(sum);
```

如果编译器能证明 sum 没有可观察效果，它可能删除整个循环。最后你测到的不是求和速度，而是空操作或残留框架开销。

受控版本：

```
std::uint64_t sum = 0;
for (std::uint64_t value : input) {
    sum += value;
}
lab00::do_not_optimize(sum);
```

这里 do_not_optimize 让结果成为编译器必须保留的输入。

4.4.18.2 错误二：把 allocation 放进计时区域

错误版本在计时区域里反复创建 vector：

```
for (std::size_t repeat = 0; repeat < LAB00_BAD_REPEAT_COUNT; ++repeat) {
    std::vector<std::uint64_t> values(LAB00_BAD_INPUT_SIZE, LAB00_BAD_VECTOR_FILL);
    checksum += std::accumulate(values.begin(), values.end(), std::uint64_t{0});
}
```

这测到的是：

```
allocation
+ initialization
+ possible page fault
+ allocator behavior
```

+ sum

如果你的目标是测 `sum`，这就是污染。如果你的目标是测分配器，那又应该换一个 problem statement。

4.4.18.3 错误三：把 RNG 放进计时区域

随机数生成器本身有成本，而且成本不小。如果你把 RNG 放在热路径里，就无法区分：

```
random generation cost
vs
kernel cost
```

正确做法通常是预生成输入，然后只计时 kernel。除非你的真实 workload 就包含 RNG，而且你明确要测整个 workload。

4.4.18.4 错误四：把格式化放进计时区域

字符串格式化、IO、日志、流对象构造都不应该混进 kernel 计时区域。性能实验里，输出应该发生在测量之后：

```
bad:
    timed region includes formatting

good:
    timed region returns numeric result
    reporting code formats after measurement
```

这个规则后续做 AI 算子也一样。不要在 kernel 计时区域里打印 tensor、保存文件、检查每个元素、分配临时字符串。

4.4.19 环境记录：没有上下文的数据没有解释力

`tools/env_report.sh` 会记录：

```
date
uname
virtualization hint
lscpu
cpu flags
compiler versions
perf availability
perf_event_paranoid
git status
```

这些信息分别回答不同问题：

字段	为什么重要
日期	<code>date</code> 给性能数据提供时间背景，系统更新、温度和配置可能变化。
内核	<code>uname</code> 记录 kernel 版本；它会影响调度、计时、PMU 权限和虚拟化表现。
CPU	<code>lscpu</code> 记录型号、核心数、cache、频率范围和 ISA flags。
指令集	<code>cpuflags</code> 用来判断 AVX2、AVX-512、FMA、VNNI 等指令是否可能使用。
编译器	编译器版本会改变优化、向量化和代码生成。
PMU 工具	<code>perfavailability</code> 判断后续能否使用 PMU 做硬件事件分析。
PMU 权限	<code>perf_event_paranoid</code> 决定普通用户能否读取某些性能计数器。
代码状态	<code>gitstatus</code> 记录工作区是否有未提交代码。

当你看到两个性能结果不同，不要马上说“代码变快了”或“代码变慢了”。先问：

- CPU 是否同一台？
- 编译器是否同一个？
- 编译参数是否一样？
- 是否在 WSL、VM 或 container 里？
- 是否有未提交代码？
- `perf_event_paranoid` 是否阻止了硬件事件读取？

4.4.20 从源码到反汇编：第一次二进制审计

性能优化不能只看 C++。即使本章还没有正式进入汇编优化，你也应该开始建立源码到反汇编的映射。先构建 Release：


```
cmake -S . -B build/lab00-release \
  -DCMAKE_BUILD_TYPE=Release \
  -DCPU_BUILD_LABS=ON

cmake --build build/lab00-release --target lab00_bench
```

找到二进制:

```
build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench
```

列出符号:

```
nm -C build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench \
  | rg "sum_i32|sum_f32|dot_f32|fill_u8|copy_u8"
```

反汇编:

```
objdump -d -Intel -C \
  build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench \
  > results/lab00/lab00_bench.objdump.txt
```

查找 `sum_i32`:

```
rg -n "lab00::sum_i32" results/lab00/lab00_bench.objdump.txt
```

你可能看到类似形状的代码:

```
0000000000035a0 <lab00::sum_i32(std::span<int const, ...>)>:
    test    rsi,rsi
    je      35d0
    lea     rdx,[rdi+rsi*4]
    xor     eax,eax
.loop:
    movsxd  rcx,DWORD PTR [rdi]
    add     rdi,0x4
    add     rax,rcx
    cmp     rdi,rdx
    jne     .loop
    ret
```

不同编译器、版本、优化参数和 CPU 目标会生成不同指令，所以不要死记这段。你要观察的是结构:

- `rdi` 是否像指向数组起点的指针?
- `rsi` 是否像元素个数?
- 是否有循环边界?
- `load` 指令是什么?
- 每次循环处理几个元素?
- 是否有 SIMD 指令?
- 返回值是否在 `rax`?

把这些问题写进报告，比单纯贴一大段反汇编更有价值。

深入理解

反汇编审计不是要求你现在完全看懂每条指令，而是训练你形成证据链：C++ 源码对应哪些符号，符号对应哪些指令，指令形状是否支持你的性能解释。

4.4.21 第一次 perfstat 审计

如果系统允许，可以运行:

```
perf stat -e cycles,instructions,branches,branch-misses,cache-misses \
  build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench \
  --only dot_f32 \
  --sizes 1048576 \
  --warmup 3 \
  --iterations 10 \
  --csv /tmp/lab00_dot.csv
```

你需要记录:

- `cycles`: 大致消耗多少 CPU 周期。
- `instructions`: 大致退休多少指令。
- `branches` 和 `branch-misses`: 分支行为是否明显。
- `cache-misses`: 大输入下是否出现缓存未命中。

如果命令失败，不要跳过。把失败原因写进报告。常见原因包括:

- 系统没有安装 `perf`。
- 当前环境是 WSL 或虚拟机，PMU 支持有限。
- `perf_event_paranoid` 权限过高。
- 容器没有暴露 `perf event`。

这也是性能工程的一部分：工具不可用本身就是环境事实。

4.4.22 怎样阅读 summary.md

`tools/summarize_bench_csv.py` 会从 raw CSV 生成摘要。你应该把它当作第一版报告，不要当作最终分析。

一个好的摘要阅读顺序是：

1. 先按 `name` 分组，看每个 kernel 是否都有结果。
2. 再按 `input_size` 分组，看规模是否覆盖预期。
3. 对每组看 `min`、`median`、`max` 的差距。
4. 如果 `max` 明显大于 `median`，回到 raw CSV 看是哪一次 iteration 慢。
5. 如果同一 kernel 随规模变化不合理，检查计时区域和输入生成。
6. 如果 `checksum` 不稳定，优先怀疑 `correctness` 或未初始化数据。

不要只复制 summary。你要写解释，例如：

```
copy_u8 at 1048576 bytes has higher elapsed_ns than sum_i32 at the same input size.
This is expected because copy_u8 performs both a read stream and a write stream.
However, the current result is only a first observation; I have not yet controlled
CPU frequency or pinned the process to one core.
```

这种写法比“copy 比 sum 慢”更专业，因为它同时给出机制解释和限制条件。

4.4.23 本章实验流程

下面是本章你应该按顺序完成的任务。不要跳步。

实验

实验 18.1：跑通 Lab 00 基线

任务：

1. 执行 `bashlabs/lab00_benchmark_foundation/scripts/run_lab00.sh`。
2. 确认 `results/lab00/env.txt` 存在。
3. 确认 `results/lab00/foundation.csv` 存在。
4. 确认 `results/lab00/summary.md` 存在。
5. 确认 `results/lab00/bad_benchmarks.txt` 存在。
6. 把脚本输出中的构建、测试和结果路径写进报告。

交付物：

- 一段运行记录。
- 一段环境摘要。
- 一段说明：为什么这个脚本先测试再 benchmark？

实验

实验 18.2：理解 raw CSV

任务：

1. 打开 `results/lab00/foundation.csv`。
2. 对每个 `name+input_size` 统计样本数。
3. 检查同组 `checksum` 是否一致。
4. 找出每组 `min`、`median`、`max`。
5. 标出 `max` 与 `median` 差距最大的三组。

交付物：

- 一张表，列出 `name`、`input_size`、`sample count`、`min`、`median`、`max`。
- 对三组波动最大的结果做解释，不能只写“系统噪声”。

实验

实验 18.3：扩大输入规模扫描

任务：

1. 手动运行 `lab00_bench`，使用更多输入规模。
2. 至少包含 16, 64, 256, 1024, 4096, 16384, 65536, 262144, 1048576, 4194304。
3. warmup 至少 3 次，measured iterations 至少 15 次。
4. 输出到 `results/lab00/foundation_wide.csv`。

5. 重新生成自己的 summary。

建议命令：

```
build/lab00-release/labs/lab00_benchmark_foundation/lab00_bench \
--sizes 16,64,256,1024,4096,16384,65536,262144,1048576,4194304 \
--warmup 3 \
--iterations 15 \
--csv results/lab00/foundation_wide.csv
```

交付物：

- 每个 kernel 的规模曲线描述。
- 指出哪个规模开始更像内存带宽问题。
- 说明你的判断依据。

实验

实验 18.4：错误 benchmark 复盘

任务：

1. 阅读 `src/bad_benchmarks.cpp`。
2. 运行 `lab00_bad_benchmarks`。
3. 对四个错误分别写出“测到了什么”和“原本想测什么”。
4. 任选一个错误，修改源码制造一个更隐蔽的版本。
5. 写出你会怎样发现它。

交付物：

- 四个错误案例解释。
- 一个自制错误案例。
- 一个检查清单。

实验

实验 18.5：反汇编审计

任务：

1. 使用 `nm-C` 找到 `sum_i32`、`sum_f32`、`dot_f32`。
2. 使用 `objdump-d-Mintel-C` 保存反汇编。
3. 选择一个 kernel，截取 20 到 80 行相关反汇编。
4. 标注入口、循环、load、add 或 multiply、branch、return。
5. 判断是否出现 SIMD 指令。

交付物：

- 反汇编文件路径。
- 一段带标注的反汇编。
- 一段源码到指令的解释。

4.4.24 高质量作业：新增 triad_f32

本章的大作业是给 Lab 00 增加一个新 benchmark： `triad_f32`。

目标函数语义：

```
// out[i] = alpha * x[i] + y[i]
std::uint64_t triad_f32(
    std::span<const float> x,
    std::span<const float> y,
    std::span<float> out,
    float alpha);
```

为什么选 `triad_f32`？

- 它有两个输入流和一个输出流。
- 它有乘法和加法，后续可以观察 FMA。
- 它接近 STREAM benchmark 的 triad 模式。
- 它是很多 AI 算子中线性组合的简化模型。
- 它会迫使你处理 correctness、checksum、输入生成、输出缓冲、benchmark 注册、测试和报告。

4.4.24.1 实现要求

你需要修改：

```
labs/lab00_benchmark_foundation/include/lab00/kernels.hpp
labs/lab00_benchmark_foundation/src/kernels.cpp
labs/lab00_benchmark_foundation/tests/lab00_tests.cpp
```

函数要求：

- 如果 `x`、`y`、`out` 长度不一致，抛出 `std::invalid_argument`。
- 不使用递归。
- 不在 `kernel` 内部分配内存。
- 使用普通标量 C++ 实现，不要手写 SIMD。
- 返回一个低成本 checksum，避免输出完全不可观察。
- `factory` 阶段生成输入和输出缓冲。
- `measured region` 只调用 `triad_f32`。

参考结构：

```
std::uint64_t triad_f32(
    std::span<const float> x,
    std::span<const float> y,
    std::span<float> out,
    float alpha)
{
    if (x.size() != y.size() || x.size() != out.size()) {
        throw std::invalid_argument("triad_f32 requires equal input sizes");
    }

    for (std::size_t index = 0; index < x.size(); ++index) {
        out[index] = alpha * x[index] + y[index];
    }

    clobber_memory();
    return checksum_f32(out.empty() ? 0.0F : out[out.size() / 2]);
}
```

这段只是结构提示。你提交时要根据项目现有命名、常量和测试风格整理实现。

4.4.24.2 测试要求

至少增加这些测试：

- 小数组计算结果正确。
- 空数组行为明确。
- 长度不一致会抛出异常。
- `make_foundation_benchmarks` 注册数量增加。
- `runner` 能通过 `--onlytriad_f32` 运行这个 case。

如果你只加 `benchmark` 不加测试，这个作业不合格。性能实验不是绕过 `correctness` 的捷径。

4.4.24.3 报告要求

报告必须回答：

1. `triad_f32` 每个元素读取多少字节、写入多少字节？
2. 如果不考虑 `cache` 复用，它的 `bytes per element` 是多少？
3. 每个元素有多少浮点操作？
4. 它更可能是 `compute-bound` 还是 `memory-bound`？为什么？
5. 你的反汇编里有没有 `FMA` 指令？
6. 如果没有 `FMA`，你认为原因可能是什么？
7. 与 `dot_f32` 相比，它的内存流数量有什么不同？
8. 与 `copy_u8` 相比，它多了哪些算术工作？

4.4.25 评分标准

项目	分值	要求
跑通基线	10	脚本成功，四个结果文件齐全。
环境记录	10	能解释 CPU、编译器、构建模式、PMU 权限。
raw CSV 分析	15	按组统计样本，解释波动和 checksum。
错误 benchmark 复盘	15	能指出每个错误测到的真实成本。
反汇编审计	15	能把源码、符号、循环和关键指令对应起来。
triad_f32 实现	15	API、实现、注册和热路径边界正确。
测试与覆盖	10	新增测试覆盖正常路径和错误路径。
性能解释	10	能用 bytes、FLOPs、输入规模和限制条件支撑结论。

4.4.26 提交前检查清单

提交前逐项检查：

- 是否使用 Release build?
- 是否运行了 ctest?
- 是否保存了 raw CSV?
- 是否保存了环境记录?
- 是否说明了输入规模?
- 是否说明了 warmup 和 measured iterations?
- 是否检查 checksum 是否稳定?
- 是否解释了 min、median、max 的差距?
- 是否避免把 allocation、RNG、formatting 放进热路径?
- 是否保存了至少一个 kernel 的反汇编?
- 是否说明结论的限制条件?

4.4.27 本章总结

Lab 00 建立的是后续所有章节共用的实验纪律：

```
correctness first
measurement boundary explicit
inputs deterministic
raw samples preserved
summary reproducible
environment recorded
binary audited
claims limited by evidence
```

从下一部分开始，我们会进入编译器和 IR：C++ 源码如何被前端解析、如何变成中间表示、优化器如何改写循环、别名和未定义行为如何影响优化、为什么同一段 C++ 在不同编译参数下会生成完全不同的汇编。Lab 00 的价值在于：当你改变代码或编译参数时，你已经有了判断“是否真的变快”的工具。

第二卷：编译器、CPU、内存和 SIMD 性能模型

第 5 章 编译器、LLVM IR 与优化

5.1 编译器流水线：从源码到 IR

5.1.1 本章要解决的问题

从本章开始，我们进入编译器部分。对性能优化来说，编译器不是一个黑盒按钮，也不是一个只负责“把 C++ 变成汇编”的工具。它是一条逐层建立事实、逐层丢失信息、逐层引入假设、逐层重写程序的流水线。

你写下的 C++ 源码不会直接变成机器指令。它通常会经历：

```
source file
-> preprocessing
-> tokens
-> AST
-> semantic analysis
-> IR generation
-> optimization passes
-> instruction selection
-> register allocation
-> assembly
-> object file
-> link
-> executable or shared library
```

如果你只看源码，会误以为代码结构就是执行结构。如果你只看汇编，又会很难判断编译器为什么生成这种结构。IR 的价值在这里：它比源码更接近执行，比汇编更保留高级语义，是理解优化器的桥。

本章目标是让你掌握：

- 编译器流水线每一层的职责和不变量。
- token、AST、语义分析、IR、优化、后端之间的区别。
- AST 和 IR 为什么不是同一种东西。
- 编译器为什么需要中间表示，而不是直接从 AST 生成汇编。
- -O0、-O2、-O3 大致改变哪些事情。
- 如何用 Clang 生成 AST dump、LLVM IR、汇编和优化报告。
- 如何把 C++ 源码、IR、汇编和 benchmark 证据连起来。

核心思想

性能优化不是“源码优化”或“汇编优化”单点工作，而是跨越源码语义、编译器假设、IR 变换、目标代码和硬件执行的证据链。你必须知道每一层保留了什么信息、丢失了什么信息、引入了什么假设。

5.1.2 一个贯穿例子

本章使用一个很小的函数：

```
#include <cstdint>
#include <cstdint>
#include <span>

std::int64_t sum_i32(std::span<const std::int32_t> values)
{
    std::int64_t sum = 0;
    for (std::int32_t value : values) {
        sum += value;
    }
    return sum;
}
```

这个函数足够简单，刚学完 C++ 的读者能看懂；也足够真实，因为它包含性能优化里最常见的结构：

- 一个输入数组。
- 一个循环。
- 一个累加变量。
- 一个返回值。
- 一次从 32 位整数到 64 位整数的扩展。

我们会反复问：

```
At source level:
    What does the code mean?
```

```
At AST level:
```

```
What syntax and semantic structure did the frontend build?

At IR level:
  What values, memory operations, branches and loops exist?

At optimized IR level:
  What did the optimizer prove and rewrite?

At assembly level:
  What instructions actually run?

At benchmark level:
  What evidence supports the performance claim?
```

5.1.3 流水线总览

先看各阶段的职责。

阶段	主要职责
预处理	展开 <code>#include</code> 、宏和条件编译，形成翻译单元。
词法分析	把字符流切成 token，例如关键字、标识符、字面量、运算符。
语法分析	根据 C++ 语法把 token 组织成 AST。
语义分析	做名字查找、类型检查、重载决议、模板实例化、访问控制等。
IR 生成	把已检查的语义结构 lowering 成便于优化的中间表示。
优化	通过一系列 pass 改写 IR，例如 DCE、inline、LICM、向量化。
后端	选择指令、分配寄存器、安排栈帧、生成目标汇编或目标文件。
链接	解析符号、合并目标文件、处理重定位、生成最终二进制。

每一层都有不变量。所谓不变量，就是下一层可以依赖的事实。

```
lexer output invariant:
  token sequence has source ranges and token kinds

parser output invariant:
  AST shape follows grammar

semantic output invariant:
  expressions have types, names are resolved, overloads are selected

IR invariant:
  control flow is explicit, values have definitions, operations are well-typed

optimized IR invariant:
  program behavior is preserved under language rules and optimization assumptions

backend invariant:
  generated machine code implements the target-level calling convention
```

如果某一层的不变量不清楚，后面的优化就会变成危险猜测。工业编译器通常会在 IR 层提供 verifier，用来检查基本块、类型、控制流、SSA use-def 等结构是否满足规则。

5.1.4 预处理：真正进入编译器的源码不等于你看到的文件

C++ 编译的第一个大变化通常发生在预处理阶段。看这段源码：

```
#include <cstdint>

#define SCALE 4

std::int32_t f(std::int32_t x)
{
    return x * SCALE;
}
```

预处理后，`#include<cstdint>` 可能展开成大量声明，`SCALE` 会被替换成 `4`。可以用：

```
clang++ -std=c++20 -E example.cpp -o example.ii
```

`example.ii` 往往非常大，因为标准库头文件会被展开。性能学习中你不需要每天阅读完整预处理输出，但你必须知道：编译器前端真正看到的是翻译单元，不是你屏幕上那个小文件。

预处理会影响性能相关工作：

- 宏可能隐藏分支、循环、函数调用和常量。
- 条件编译可能让不同平台编译出完全不同的代码。
- header-only 库会把大量实现暴露给编译器，增加 inline 和优化机会。

- 包含顺序和宏定义可能影响 ABI 或代码路径。

常见误区

不要只看一个 .cpp 文件就断言编译器输入是什么。宏和头文件可能让真实输入比你看到的源码复杂得多。

5.1.5 词法分析：字符流变成 token

词法分析器不理解“求和函数是否高效”。它的任务更基础：把字符切成 token。

例如：

```
sum += value;
```

大致会形成：

```
identifier    "sum"
operator      "+="
identifier    "value"
punctuator    ";;"
```

token 需要携带位置信息：

```
file
line
column
byte offset
token kind
spelling
```

没有准确位置，编译器就不能给出高质量诊断，也不能让 IDE、格式化工具、静态分析器、代码导航工具稳定工作。

词法阶段的性能意义在于：后面所有诊断和工具能力都依赖它的 source range。你在优化代码时看到的 warning、vectorization report、optimization remark，最终都要定位到源码位置。

5.1.6 语法分析：token 变成 AST

Parser 根据语言语法把 token 组织成 AST。AST 是 Abstract Syntax Tree，抽象语法树。

例如：

```
return x * 4 + y;
```

可以想成下面的树：

```
ReturnStmt
  BinaryOperator '+'
    BinaryOperator '*'
      DeclRefExpr x
      IntegerLiteral 4
    DeclRefExpr y
```

AST 保留源码结构。它适合表达：

- 函数、类、命名空间、模板。
- 声明和语句层次。
- 表达式优先级和嵌套。
- 源码位置。
- 类型、名字和语义信息的附着点。

生成 AST dump：

```
clang++ -std=c++20 \
-Xclang -ast-dump \
-fsyntax-only sum.cpp
```

对 sum_i32，你会看到很多标准库类型信息。为了学习，可以先用裸指针版本：

```
#include <cstddef>
#include <cstdint>

std::int64_t sum_i32_ptr(const std::int32_t* values, std::size_t size)
{
    std::int64_t sum = 0;
    for (std::size_t index = 0; index < size; ++index) {
        sum += values[index];
    }
    return sum;
}
```

它的 AST 更容易读，因为少了 std::span 的模板展开噪声。

5.1.7 语义分析：源码结构变成确定含义

AST 只说明“长什么样”，语义分析说明“是什么意思”。C++ 语义分析非常复杂，至少包括：

- 名字查找：sum 指的是哪个变量？
- 类型检查：values[index] 是什么类型？
- 隐式转换：std::int32_t 如何加到 std::int64_t？
- 重载决议：调用的是哪个函数或运算符？
- 模板实例化：std::span<const std::int32_t> 展开成什么？
- 生命周期：引用、临时对象、析构何时发生？
- 可见性和访问控制：名字是否可访问？

对 sum_i32_ptr 这句：

```
sum += values[index];
```

语义分析需要知道：

```
sum:
  local variable
  type: std::int64_t

values:
  parameter
  type: pointer to const std::int32_t

index:
  local variable
  type: std::size_t

values[index]:
  lvalue expression
  type: const std::int32_t

sum += values[index]:
  read sum
  load values[index]
  convert int32 to int64
  add
  store back to sum
```

注意最后两行已经接近 IR 思维：读、加载、转换、加、写回。优化器需要的不是“源码看起来像加法”，而是明确的值和副作用。

5.1.8 为什么 AST 不适合直接优化

初学者常问：既然 AST 已经表示程序结构，为什么不直接在 AST 上优化并生成汇编？可以，但不好。原因是 AST 太贴近源码语法，很多执行细节不够显式。

问题	AST 和 IR 的差异
控制流表达式	AST 中 for、if、while 是不同语法节点；IR 通常统一成基本块和分支。AST 保留嵌套表达式的源码形状；IR 把表达式拆成有定义、有使用的中间值。
变量	AST 要承载局部变量、成员和引用语义；IR 更倾向于表达 SSA 值以及显式的 load 和 store。
优化目标无关性	AST 优化要处理大量语法特例；IR pass 可以在统一结构上工作。AST 仍然带有强烈语言特征；IR 更适合跨语言、跨目标共享优化。

举例，源码里有很多循环写法：

```
for (std::size_t i = 0; i < n; ++i) { ... }

while (i < n) { ...; ++i; }

do { ...; ++i; } while (i < n);
```

AST 中它们是不同节点；IR 中它们都会降低成 basic block、条件分支和回边。优化器更愿意在这种统一结构上工作。

心智模型

AST 是给语言语义服务的，IR 是给程序变换服务的。AST 关心“源码如何表达”，IR 关心“执行效果如何表示”。

5.1.9 IR：优化器的工作语言

IR 是 Intermediate Representation，中间表示。不同编译器有不同 IR，例如 LLVM IR、GCC GIMPLE 和 RTL、MLIR dialect、JVM bytecode、WebAssembly、Cranelift IR。

本书主要使用 LLVM IR，因为 Clang 能方便地产生它，工具链成熟，结构适合学习。

生成 IR：

```
clang++ -std=c++20 -O0 -emit-llvm -S sum.cpp -o sum_00.ll
clang++ -std=c++20 -O3 -emit-llvm -S sum.cpp -o sum_03.ll
```

-O0 的 IR 更接近源码和调试需求，常有大量 `alloca`、`load`、`store`。

-O3 的 IR 已经过优化，局部变量可能消失，循环可能展开或向量化，函数可能 `inline`，死代码可能被删除。

一个简化后的 LLVM IR 片段可能像这样：

```
define i64 @sum_i32_ptr(ptr %values, i64 %size) {
entry:
    br label %loop

loop:
    %index = phi i64 [ 0, %entry ], [ %next, %loop ]
    %sum = phi i64 [ 0, %entry ], [ %sum.next, %loop ]
    %ptr = getelementptr inbounds i32, ptr %values, i64 %index
    %v32 = load i32, ptr %ptr, align 4
    %v64 = sext i32 %v32 to i64
    %sum.next = add nsw i64 %sum, %v64
    %next = add nuw i64 %index, 1
    %cond = icmp eq i64 %next, %size
    br i1 %cond, label %exit, label %loop

exit:
    ret i64 %sum.next
}
```

这段 IR 已经揭示了很多源码里不直观的事实：

- 循环被 `basic block` 和 `branch` 表达。
- `index` 和 `sum` 在循环里通过 `phi` 表达。
- 数组访问通过 `getelementptr` 计算地址。
- 读取内存是 `load`。
- 32 位到 64 位有符号扩展是 `sext`。
- 循环条件是 `icmp` 加条件分支。

本章先建立直觉，下一章会系统讲 SSA、`phi`、`basic block`、CFG 和 `dominance`。

5.1.10 Basic block：控制流的基本单位

`Basic block` 是一段顺序执行的指令序列。控制流只能从 `block` 开头进入，从 `block` 末尾离开。LLVM IR 中每个 `basic block` 通常以 `terminator` 结束，例如：

```
br label %loop
br i1 %cond, label %exit, label %loop
ret i64 %value
```

对循环来说，常见结构是：

```
entry:
    initialize
    jump to loop

loop:
    phi values
    body
    compute next index
    branch to loop or exit

exit:
    return result
```

这比源码的 `for` 更底层。硬件执行的也是分支和跳转，不是 `for` 这个语法概念。

5.1.11 SSA：值为什么只定义一次

SSA 是 Static Single Assignment。简化管理：每个 SSA value 只定义一次。

源码：

```
sum = 0;
sum = sum + value;
sum = sum + value2;
```

SSA 思维：

```
sum0 = 0
sum1 = sum0 + value
sum2 = sum1 + value2
```

这对优化非常有用，因为 use-def chain 清楚。优化器能问：

- 这个值在哪里定义？
- 它被谁使用？
- 如果替换这个值，会影响哪些地方？
- 如果它没有使用者，是否可以删除？

循环里的变量会遇到“值从哪里来”的问题，所以需要 phi：

```
%sum = phi i64 [ 0, %entry ], [ %sum.next, %loop ]
```

含义是：

- 如果控制流从 entry 来，sum 是初始值 0。
- 如果控制流从 loop 回来，sum 是上一轮的 sum.next。

你不需要在本章完全掌握 phi，但要先知道：它不是普通函数调用，也不是运行时选择表；它是 SSA 在控制流汇合处表达值来源的方式。

5.1.12 优化 pass：编译器不是一次性“变聪明”

优化器通常不是一个巨大函数，而是一串 pass。每个 pass 做某类分析或变换。

常见 pass 包括：

- Dead Code Elimination：删除没有可观察用途的代码。
- Constant Propagation：传播常量，把能算出的值提前算出。
- InstCombine：合并和简化局部指令模式。
- Inlining：把函数调用替换成函数体。
- Loop Invariant Code Motion：把循环不变量移出循环。
- Induction Variable Simplification：简化循环归纳变量。
- Loop Unrolling：展开循环，减少分支开销，增加 ILP。
- SLP Vectorization：在基本块内寻找可打包的标量操作。
- Loop Vectorization：把循环变成 SIMD 形式。

一个 pass 通常有前提：

```
input:
  valid IR
  required analyses available

transform:
  rewrite some instructions, blocks, metadata, attributes

output:
  valid IR
  preserve or invalidate analyses
```

这就是为什么 IR verifier 很重要。每个 pass 改完后，IR 仍然必须满足规则。

5.1.13 优化级别：-O0、-O2、-O3 的区别

不同编译器细节不同，但大方向可以这样理解：

级别	常见特征
-O0	面向可调试性，保留更多源码结构，优化少，性能结论通常不能使用。
-O1	做基础优化，编译时间和调试体验相对保守。
-O2	常用 Release 优化级别，启用大量标量优化和部分循环优化。
-O3	更激进，增加更多循环优化、向量化、展开和内联，可能增加代码尺寸。
-Ofast	允许更激进语义放松，尤其是浮点规则；不等价于“总是更好”。

不要把 -O3 理解成“最大优化，所以一定最好”。它可能：

- 增加代码尺寸，导致 instruction cache 或 frontend 压力。
- 过度展开，增加 register pressure。
- 改变浮点优化策略。
- 编译更慢。

- 对小函数更快，对大工作集无明显收益。
性能实验必须把优化级别写进报告。

5.1.14 Dead Code Elimination 和 Lab 00 的关系

第 18 章反复强调 `do_not_optimize`。现在从编译器角度看原因。

源码：

```
void bad(const std::uint64_t* values, std::size_t size)
{
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < size; ++i) {
        sum += values[i];
    }
}
```

如果 `sum` 没有可观察用途，优化器可以认为循环没有必要。优化后函数可能接近：

```
define void @bad(ptr %values, i64 %size) {
entry:
    ret void
}
```

这不是编译器作弊，而是语言规则允许它删除没有可观察效果的计算。`benchmark` 如果没有观察结果，就可能测到一个被删除的 `loop`。

`do_not_optimize(sum)` 的作用是人为制造一个优化器必须尊重的观察点，阻止这类删除。

5.1.15 Constant propagation: 编译器会提前算出能证明的值

源码：

```
int f()
{
    int x = 2;
    int y = 3;
    return x * y + 4;
}
```

优化后可能直接变成：

```
define i32 @f() {
entry:
    ret i32 10
}
```

汇编可能是：

```
mov eax, 10
ret
```

这对普通程序是好事；对 `benchmark` 可能是灾难。如果你的被测输入是编译期常量，编译器可能提前算完。性能实验应使用运行时输入、外部数据、参数化规模或适当的 `observation barrier`。

5.1.16 Inlining: 性能优化的入口放大器

Inlining 把函数调用替换成函数体。

收益：

- 消除 `call` 和 `ret` 开销。
- 暴露常量参数。
- 暴露别名和内存访问模式。
- 让后续 `pass` 能跨函数优化。

代价：

- 增加代码尺寸。
- 可能增加 `instruction cache` 压力。
- 可能增加 `register pressure`。
- 可能让前端取指和解码成为瓶颈。

对性能工程来说，`inline` 不是“越多越好”。一个小函数 `inline` 后可能变快；一个大模板函数到处 `inline` 可能让热路径变胖，反而变慢。后面的 `frontend` 和 `code size` 章节会重新回到这个问题。

5.1.17 Lowering: 高级语义向低级操作下降

Lowering 是把高级结构转换成更低级结构的过程。它不只发生一次，而是贯穿编译器很多层。

例子：

```
C++ range-for
-> begin/end iterator loop
-> pointer/index loop
-> IR basic blocks
-> machine branch
```

再看 `std::span`:

```
std::span<const std::int32_t> values
```

在 ABI 层，它常常可以理解成类似：

```
struct span_like {
    const std::int32_t* data;
    std::size_t size;
};
```

但这是帮助理解的模型，不是标准规定的 ABI 承诺。实际布局和调用细节要看标准库实现、编译器和 ABI。

当你在汇编里看到 `rdi` 和 `rsi` 像指针和长度时，本质上是在观察 lowering 之后的结果。

5.1.18 后端：从 IR 到机器

LLVM 后端大致还会经历：

```
LLVM IR
-> target-independent optimizations
-> instruction selection
-> machine IR
-> register allocation
-> instruction scheduling
-> assembly emission
-> object emission
```

后端要处理源码和 IR 不关心的目标细节：

- x86-64 有哪些指令可以表达这个操作？
- 哪些值放进寄存器，哪些 spill 到栈？
- 函数参数和返回值遵守哪套 calling convention？
- 栈帧如何布局？
- 是否使用 SIMD 指令？
- 分支、跳转、重定位如何编码？

生成汇编：

```
clang++ -std=c++20 -O3 -S -masm=intel sum.cpp -o sum_03.s
```

生成目标文件并反汇编：

```
clang++ -std=c++20 -O3 -c sum.cpp -o sum_03.o
objdump -d -Mintel -C sum_03.o
```

你应该同时保存 IR 和汇编，因为它们回答不同问题：

- IR 解释优化器看到了什么、证明了什么。
- 汇编解释后端实际选择了什么指令。

5.1.19 源码、IR、汇编和性能报告如何连接

一个合格的性能分析应该能形成这样的链：

```
C++ source:
    loop over int32 array and accumulate into int64

IR evidence:
    load i32
    sext i32 to i64
    add i64
    phi for index and sum

assembly evidence:
    load from memory
    sign-extend
    add to accumulator
    branch back to loop

benchmark evidence:
    raw samples for several input sizes
    median and tail behavior
    environment and compiler flags recorded

claim:
```

under this compiler, flags, input sizes and machine,
the scalar baseline behaves as observed

这条链越完整，你的结论越可信。只说“我改了一行，快了 20%”不够。你需要说明：

- 编译器是否真的生成了你以为的代码？
- 优化前后 IR 有什么变化？
- 汇编是否支持你的解释？
- 测量是否可复现？
- 输入规模是否覆盖声称的场景？

5.1.20 优化器不是数学真理机

编译器优化基于语言规则和它能证明的事实。它不能读懂你的意图。

三个常见限制：

- 别名：两个指针是否可能指向同一块内存？
- 副作用：函数调用是否可能读写未知内存、抛异常、访问 volatile？
- 浮点语义：重排浮点加法是否改变结果？

例如：

```
void saxpy(float* y, const float* x, float a, std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        y[i] += a * x[i];
    }
}
```

如果 x 和 y 可能重叠，编译器就要谨慎。它可能生成 runtime alias check，也可能拒绝某些向量化。你知道两个数组不重叠，不等于编译器能证明它们不重叠。

再看浮点求和：

```
float sum_f32(const float* values, std::size_t n)
{
    float sum = 0.0F;
    for (std::size_t i = 0; i < n; ++i) {
        sum += values[i];
    }
    return sum;
}
```

严格浮点语义下，编译器不能随便把加法重排成树形规约，因为浮点加法不满足严格结合律。使用 `-ffast-math` 或相关 `fast-math` flags 会放松语义，但也会改变数值行为。性能优化必须知道自己付出了什么语义代价。

5.1.21 优化报告：让编译器解释自己

Clang 可以输出优化 remark。对向量化，常用：

```
clang++ -std=c++20 -O3 -march=native \
-Rpass=loop-vectorize \
-Rpass-missed=loop-vectorize \
-Rpass-analysis=loop-vectorize \
sum.cpp -c -o /tmp/sum.o
```

GCC 常用：

```
g++ -std=c++20 -O3 -march=native \
-fopt-info-vec-optimized \
-fopt-info-vec-missed \
sum.cpp -c -o /tmp/sum.o
```

优化报告不是最终证据，但它很有用。它能告诉你：

- 哪个 loop 被向量化。
- 哪个 loop 没有被向量化。
- 原因是 legality 问题还是 profitability 问题。
- 源码位置在哪里。

你要把优化报告和 IR、汇编对照起来。报告说 vectorized，不代表最终汇编一定符合你的预期；报告说 missed，也要看 missed 的原因是否可以通过源码重写、alias 信息或编译参数改善。

5.1.22 实验 19.1：建立源码到产物目录

实验

目标：建立本章实验目录，保存每一层编译产物。

任务：

1. 新建 `results/ch19`。
2. 写一个 `results/ch19/sum.cpp`，包含 `sum_i32_ptr` 和 `sum_f32`。
3. 使用 Clang 生成预处理输出、AST dump、-00 IR、-03 IR、-03 汇编和目标文件。
4. 所有产物都放在 `results/ch19`。

参考命令：

```
mkdir -p results/ch19

clang++ -std=c++20 -E \
  results/ch19/sum.cpp \
  -o results/ch19/sum.ii

clang++ -std=c++20 -Xclang -ast-dump -fsyntax-only \
  results/ch19/sum.cpp \
  > results/ch19/sum.ast.txt

clang++ -std=c++20 -00 -emit-llvm -S \
  results/ch19/sum.cpp \
  -o results/ch19/sum_00.ll

clang++ -std=c++20 -03 -emit-llvm -S \
  results/ch19/sum.cpp \
  -o results/ch19/sum_03.ll

clang++ -std=c++20 -03 -S -masm=intel \
  results/ch19/sum.cpp \
  -o results/ch19/sum_03.s
```

交付物：

- 文件列表。
- 每个文件对应的编译阶段说明。
- 哪个文件最接近源码？哪个文件最接近机器执行？

5.1.23 实验 19.2：比较 -00 和 -03 IR

实验

目标：观察优化等级如何改变 IR。

任务：

1. 打开 `results/ch19/sum_00.ll`。
2. 打开 `results/ch19/sum_03.ll`。
3. 统计 `alloca`、`load`、`store`、`br`、`phi` 出现次数。
4. 找出 -03 中是否还有局部变量形态的 `sum`。
5. 写出 -00 更适合调试、-03 更适合性能的原因。

建议命令：

```
rg -n "alloca|load|store|br | phi " results/ch19/sum_00.ll
rg -n "alloca|load|store|br | phi " results/ch19/sum_03.ll
```

交付物：

- 一张对比表。
- 5 条你实际观察到的差异。
- 一段解释：为什么不能用 Debug 或 -00 benchmark 做优化结论？

5.1.24 实验 19.3：从 IR 找循环

实验

目标：在 LLVM IR 中识别循环结构。

任务：

1. 在 -03 IR 中找到 `sum_i32_ptr`。

2. 标出 entry block、loop block、exit block。
3. 找到 `phi`。
4. 找到 `getelementptr`。
5. 找到 `load`、`sext`、`add`。
6. 用自己的话解释一次循环迭代如何发生。

交付物：

- 一段带标注的 IR。
- 一张 block 关系图，可以用文本画。
- 一段源码语句到 IR 指令的映射。

5.1.25 实验 19.4：对照汇编

实验

目标：把优化后 IR 和 x86-64 汇编联系起来。

任务：

1. 打开 `results/ch19/sum_03.s`。
2. 找到 `sum_i32_ptr` 函数。
3. 标出循环入口、循环体和退出。
4. 找到 `load`、`sign-extension`、`add`、`branch`。
5. 判断返回值使用哪个寄存器。
6. 对照第 13 章 ABI 解释参数寄存器。

交付物：

- 一段带注释的汇编。
- 一段 IR 到汇编的映射。
- 一段说明：哪些 IR 信息在汇编里已经看不见？

5.1.26 实验 19.5：优化报告和实际产物是否一致

实验

目标：学习使用优化报告，但不盲信报告。

任务：

1. 对 `sum_f32` 运行 Clang 向量化报告。
2. 如果报告 missed，记录原因。
3. 尝试加入 `-ffast-math` 再运行。
4. 比较 IR 和汇编是否发生变化。
5. 写出 fast-math 的语义代价。

参考命令：

```
clang++ -std=c++20 -O3 -march=native \
  -Rpass=loop-vectorize \
  -Rpass-missed=loop-vectorize \
  -Rpass-analysis=loop-vectorize \
  results/ch19/sum.cpp \
  -c -o /tmp/sum_strict.o

clang++ -std=c++20 -O3 -ffast-math -march=native \
  -Rpass=loop-vectorize \
  -Rpass-missed=loop-vectorize \
  -Rpass-analysis=loop-vectorize \
  results/ch19/sum.cpp \
  -c -o /tmp/sum_fast.o
```

交付物：

- 两次优化报告。
- 两份汇编或 `objdump` 摘要。
- 一段结论：性能机会和数值语义风险分别是什么？

5.1.27 本章作业

习题与作业

作业 19.A：流水线解释

用你自己的话解释下面每一层输出给下一层的“不变量”：

1. lexer 到 parser。
2. parser 到 semantic analysis。
3. semantic analysis 到 IR generation。
4. IR generation 到 optimizer。
5. optimizer 到 backend。
6. backend 到 assembler 或 object emitter。

要求：每层至少写 3 条具体事实，不能只写“更底层”。

习题与作业

作业 19.B：AST 和 IR 对比

选择一个包含 `if`、`for`、函数调用和数组访问的小函数，生成 AST dump 和 LLVM IR。

回答：

- AST 中哪些节点保留了源码结构？
- IR 中哪些 block 表达了控制流？
- 函数调用在 AST 和 IR 中分别如何出现？
- 数组访问在 IR 中变成了哪些操作？
- 哪些信息在 IR 中更清楚？哪些信息在 IR 中更难看？

习题与作业

作业 19.C：benchmark 防 DCE 证明

写两个函数：

```
void bad_sum(const std::uint64_t* values, std::size_t size);
std::uint64_t good_sum(const std::uint64_t* values, std::size_t size);
```

`bad_sum` 计算结果但不返回、不写出、不调用 observation barrier。`good_sum` 返回结果。

任务：

1. 生成 -O3 IR。
2. 生成 -O3 汇编。
3. 证明 `bad_sum` 的循环是否被删除。
4. 解释为什么这会破坏 benchmark。

习题与作业

作业 19.D：编译参数报告规范

为以后的每个性能实验设计一个“编译参数记录模板”，至少包含：

- 编译器名称和版本。
- C++ 标准版本。
- 优化等级。
- `-march` 或目标 CPU。
- 是否启用 LTO。
- 是否启用 fast-math。
- 是否定义 `NDEBUG`。
- 生成 IR 和汇编的命令。

要求：模板要能直接放进实验报告。

5.1.28 本章总结

本章建立了从源码到 IR 的主线：

```
source:
    human-written C++ with syntax and intent

AST:
    parsed and semantically attached language structure

IR:
```

```
explicit values, memory operations and control flow

optimized IR:
    transformed program under language rules and compiler assumptions

assembly:
    target-specific instructions and registers

benchmark:
    measured behavior under a concrete environment
```

接下来第 20 章会深入 LLVM IR、SSA、basic block、CFG 和 phi。本章你只需要牢牢记住：编译器优化不是魔法，而是一系列在明确中间表示上进行的程序变换。看懂这些变换，你才能解释为什么 C++ 写法会影响汇编，为什么 benchmark 会被优化器误导，为什么某些循环能向量化而另一些不能。

5.2 LLVM IR、SSA 和控制流图

5.2.1 本节要解决的问题

上一节建立了编译器流水线。现在我们要进入真正能解释优化行为的层次：LLVM IR。对高性能计算和 AI CPU 算子优化来说，LLVM IR 不是“编译器内部细节”，而是一种观察窗口。它能告诉你：

- 编译器认为哪些值是普通整数、哪些值是指针、哪些值来自内存。
- 循环是否已经变成可分析的控制流结构。
- 一个变量是否真的存在于内存里，还是只存在为 SSA 值。
- 优化器能不能证明一段代码没有副作用。
- 载入、存储、分支、函数调用、别名和属性如何影响优化。

初学者看 IR 最容易犯两个错误。第一个错误是把 IR 当成“另一种汇编”。它不是。IR 没有目标寄存器、没有真实栈帧、没有端口和 uop。第二个错误是把 IR 当成“压扁后的 C++”。它也不是。IR 已经丢掉大量源码语法结构，并显式化了控制流和值依赖。

核心思想

LLVM IR 的价值在于它把程序变成优化器容易证明的形式：值有类型，控制流由 basic block 和 branch 显式描述，普通临时值大多遵循 SSA，内存访问由 load/store 显式描述。读懂 IR，就是读懂优化器“能证明什么”和“不能证明什么”。

5.2.2 IR 的三种形态

平时你最常看到的是文本形式的 .ll 文件。但 LLVM IR 实际上有三种常见形态：

形态	作用
文本 IR	人能阅读的 .ll，适合学习、调试、写报告。
bitcode	二进制形式的 .bc，适合保存、传输、LTO。
内存 IR	编译器进程内部的数据结构，pass 真正操作的是它。

生成文本 IR 的常用命令：

```
clang++ -std=c++20 -O0 -S -emit-llvm sum.cpp -o sum_00.ll
clang++ -std=c++20 -O3 -S -emit-llvm sum.cpp -o sum_03.ll

生成 bitcode:
clang++ -std=c++20 -O3 -c -emit-llvm sum.cpp -o sum.bc
llvm-dis sum.bc -o sum_from_bc.ll
```

学习阶段优先看 .ll。但要记住：文本 IR 只是打印结果，不是优化器内部对象本身。优化 pass 操作的是内存中的 module、function、basic block、instruction 和 use-def 关系。

5.2.3 一个最小函数的 IR

先从一个不涉及循环的函数开始：

```
#include <stdint>

extern "C" std::int32_t add3(std::int32_t a,
                             std::int32_t b,
                             std::int32_t c)
{
    return a + b + c;
}
```

优化后 IR 通常接近下面这种形态：

```
define dso_local i32 @add3(i32 noundef %0,
                          i32 noundef %1,
                          i32 noundef %2) {
entry:
  %4 = add nsw i32 %1, %0
  %5 = add nsw i32 %4, %2
  ret i32 %5
}
```

这里每一行都值得慢慢拆：

- **define** 表示函数定义，不是函数声明。
- **i32** 是返回类型。
- **@add3** 是全局符号名。
- 参数也是带类型的值。
- **entry:** 是 basic block 标签。
- **add** 产生一个新值。
- **ret** 是 terminator，结束当前 basic block。

nsw 的意思是 no signed wrap。它不是 CPU 指令的一部分，而是 IR 层的语义标记：如果有符号加法溢出，C++ 程序已经进入未定义行为，因此优化器可以把“不会有有符号溢出”当成假设。这一点会在 5.6 中展开。

常见误区

不要把 **%4**、**%5** 理解成真实机器寄存器。它们是 IR 值名。后端可能把它们放进寄存器，也可能重算、合并、删除，甚至整个函数都被内联后消失。

5.2.4 Module、Function、Basic Block、Instruction

LLVM IR 的层级可以这样理解：

```
module
-> global variables
-> function declarations
-> function definitions
    -> basic blocks
        -> instructions
            -> operands
```

一个 **module** 大致对应一个翻译单元或链接后模块。它包含目标三元组、数据布局、全局变量、函数声明、函数定义、metadata 等信息。

一个 **function** 是控制流图的边界。函数内部由 basic block 组成。basic block 的关键规则是：

- 只能从 block 的第一条指令进入。
- 除了最后的 terminator，中间不能跳出。
- 最后一条必须是 terminator，例如 **ret**、**br**、**switch**、**invoke**。

basic block 是许多优化的基本单位。比如 DCE 会删除无用指令，CFG simplify 会合并空 block，loop pass 会识别循环头、循环体、退出边和回边。

5.2.5 控制流图：程序不再是文本顺序

看一个分支函数：

```
extern "C" int abs_i32(int x)
{
  if (x < 0) {
    return -x;
  }
  return x;
}
```

低优化等级下可能出现类似结构：

```
define dso_local i32 @abs_i32(i32 noundef %0) {
entry:
  %cmp = icmp slt i32 %0, 0
  br i1 %cmp, label %then, label %exit

then:
  %neg = sub nsw i32 0, %0
  br label %exit

exit:
  %result = phi i32 [ %neg, %then ], [ %0, %entry ]
}
```

```
ret i32 %result
}
```

这段 IR 的 CFG 可以画成：

```
entry
| true
v
then
|
v
exit
^
| false
+----- from entry
```

源码中的 `if` 语句已经变成两个显式分支目标。优化器不再需要猜 “if 的范围在哪里”，它只需要遍历 basic block 和边。

5.2.6 SSA：值只定义一次

SSA 是 Static Single Assignment 的缩写。直观说，一个 SSA 值只被定义一次。上面 `add3` 中的每个 IR 临时值都只出现一次定义：

```
%4 = add nsw i32 %1, %0
%5 = add nsw i32 %4, %2
```

这样做的好处是 use-def chain 非常清楚：

```
%1 ----\
      add -> %4 ----\
%0 ----/          add -> %5 -> ret
%2 -----/
```

如果一个值只定义一次，优化器回答问题会容易得多：

- 这个值从哪里来？
- 哪些指令使用它？
- 如果删除它，会影响哪些地方？
- 两个表达式是否计算同一个值？
- 循环里某个值是否每轮都不变？

没有 SSA，优化器需要反复分析 “变量在这个程序点到底是哪一次赋值后的值”。SSA 把这个问题变成图结构上的 def-use 关系。

5.2.7 phi：控制流汇合处的值选择

SSA 遇到控制流汇合时会出现一个问题：如果一个变量在不同路径上有不同定义，汇合后该用哪个值？C++ 写法：

```
extern "C" int choose(int x, int y, bool flag)
{
    int v = x;
    if (flag) {
        v = y;
    }
    return v + 1;
}
```

SSA 不能让同一个 `v` 被赋值两次。IR 会用 `phi` 表达 “根据前驱 block 选择值”：

```
entry:
br i1 %flag, label %then, label %exit

then:
br label %exit

exit:
%v = phi i32 [ %y, %then ], [ %x, %entry ]
%ret = add nsw i32 %v, 1
ret i32 %ret
```

`phi` 的含义不是运行时调用一个函数，也不是从内存读一个变量。它是 SSA 图上的控制流选择：

```
if predecessor is then:
    %v = %y

if predecessor is entry:
    %v = %x
```

循环中的 induction variable 也经常由 `phi` 表达：

```
loop:
```

```
%i = phi i64 [ 0, %entry ], [ %i.next, %loop ]
...
%i.next = add nuw i64 %i, 1
%done = icmp eq i64 %i.next, %n
br i1 %done, label %exit, label %loop
```

这段结构的含义是：第一次进入循环时 `%i` 来自 `entry` 的 0；之后每次从 `loop` 回来时，`%i` 来自上一轮的 `%i.next`。

心智模型

`phi` 是读 LLVM IR 时最重要的门槛之一。看到 `phi` 时，不要问“它对应哪条机器指令”，而要问“这个 block 有哪些前驱，每条前驱带来哪个值”。后端最终可能用寄存器分配、`move`、条件 `move` 或直接合并寄存器来实现它。

5.2.8 内存不是自动 SSA

SSA 主要描述普通 IR 值。内存是另一回事。看这个例子：

```
extern "C" int add_local(int a, int b)
{
    int x = a;
    x += b;
    return x;
}
```

在 `-O0` 下，Clang 往往保留局部变量的内存槽，IR 里会看到 `alloca`、`store`、`load`：

```
%3 = alloca i32, align 4
%4 = alloca i32, align 4
store i32 %0, ptr %3, align 4
store i32 %1, ptr %4, align 4
%5 = load i32, ptr %3, align 4
%6 = load i32, ptr %4, align 4
%7 = add nsw i32 %5, %6
store i32 %7, ptr %3, align 4
%8 = load i32, ptr %3, align 4
ret i32 %8
```

在优化后，内存槽通常会被提升为 SSA 值：

```
%3 = add nsw i32 %1, %0
ret i32 %3
```

这个过程常被称为 `mem2reg`。它把满足条件的栈上局部变量提升成 SSA 值。能不能提升，取决于地址是否逃逸、是否被不透明调用使用、是否存在复杂别名等。

常见误区

C++ 变量和 LLVM IR 值不是一一对应关系。一个源码变量可能在 `-O0` 下是内存槽，在 `-O3` 下完全消失；一个 IR 值也可能没有源码变量名，只是优化器为了表达中间计算生成的临时值。

5.2.9 load/store：优化器眼中的内存边界

IR 中的 `load` 和 `store` 是理解性能的关键。很多优化本质上都在问：

- 这次 `load` 能不能提前？
- 两次 `load` 是否一定读同一个值？
- 这次 `store` 会不会影响后面的 `load`？
- 两个指针会不会指向同一片内存？
- 这个循环里的 `load` 能不能向量化？

例如：

```
extern "C" void saxpy(float* y, const float* x, float a, int n)
{
    for (int i = 0; i < n; ++i) {
        y[i] = a * x[i] + y[i];
    }
}
```

这个循环至少有两个 `load` 和一个 `store`：

```
load x[i]
load y[i]
compute a * x[i] + y[i]
store y[i]
```

如果 `x` 和 `y` 可能 `alias`，优化器必须更谨慎。它可能需要插入 `runtime alias check`，也可能拒绝某些重排。第 5.5 和 5.6 会继续把这个例子展开。

5.2.10 attribute 和 metadata

LLVM IR 不只包含指令，还包含属性和 metadata。它们经常决定优化器能不能大胆改写程序。

常见函数或参数属性包括：

- **readonly**：函数只读内存，不写内存。
- **readnone**：函数不读也不写可观察内存。
- **noalias**：某个返回值或参数在特定语义下不与其他对象别名。
- **nonnull**：指针不为空。
- **noundef**：值不是未定义值。
- **align**：内存对齐信息。

常见 metadata 包括：

- **TBAA**：type-based alias analysis 信息。
- **loop metadata**：循环展开、向量化、unroll hints。
- **debug metadata**：源码位置、变量名、调试信息。
- **profile metadata**：分支概率、执行频率。

属性和 metadata 的重要性在于：同样的指令，如果携带不同事实，优化空间完全不同。例如一个 **load** 如果有更强的对齐信息，向量化器可能选择更高效的载入；一个函数如果能证明 **readnone**，DCE 可能直接删除它的调用。

5.2.11 新版 LLVM 的 opaque pointer

你可能在不同资料里看到两种指针写法：

```
older style:
i32* %p
float* %q

opaque pointer style:
ptr %p
ptr %q
```

较新的 LLVM IR 默认使用 opaque pointer，也就是统一写成 **ptr**，指针本身不携带 pointee type。被访问的数据类型由 **load**、**store**、**getelementptr** 等指令表达：

```
%v = load i32, ptr %p, align 4
store float %x, ptr %q, align 4
```

读资料时要注意版本差异。看到旧资料里的 **i32*** 不必困惑，它表达的是旧 IR 打印风格。核心问题仍然是：指针值、被访问类型、地址计算和 alias 信息如何共同决定优化。

5.2.12 读 IR 的顺序

初学者读 IR 不应该从第一行逐字符读到最后一行。建议按下面顺序：

1. 看函数签名：参数类型、返回类型、属性。
2. 找 basic block 标签：**entry**、**loop**、**exit**。
3. 找 terminator：**br**、**ret**、**switch**。
4. 画 CFG：block 之间怎么跳。
5. 找 **phi**：循环变量和路径汇合值。
6. 找 **load/store**：真正的内存访问在哪里。
7. 找 **call**：不透明调用通常是优化边界。
8. 找 metadata 和 attributes：优化器有哪些额外事实。

5.2.13 实验 20.1：标注一个 sum loop 的 IR

实验

目标：建立读 LLVM IR 的基本动作。

准备文件 **results/ch20/sum_loop.cpp**：

```
#include <cstdint>
#include <cstdint>

extern "C" std::int64_t sum_i32_ptr(const std::int32_t* values,
                                   std::size_t size)
{
    std::int64_t sum = 0;
    for (std::size_t i = 0; i < size; ++i) {
```

```

        sum += values[i];
    }
    return sum;
}

```

生成 IR:

```

clang++ -std=c++20 -O0 -S -emit-llvm \
    results/ch20/sum_loop.cpp -o results/ch20/sum_00.ll

clang++ -std=c++20 -O2 -S -emit-llvm \
    results/ch20/sum_loop.cpp -o results/ch20/sum_02.ll

```

任务:

1. 在 `sum_00.ll` 中找出所有 `alloca`、`load`、`store`。
2. 在 `sum_02.ll` 中找出循环 `header`、`latch`、`exit`。
3. 标注所有 `phi`，说明每个输入来自哪个前驱 `block`。
4. 找到数组访问对应的地址计算和 `load`。
5. 写出 `-O0` 到 `-O2` 后哪些源码变量消失了。

交付物:

- 一张 CFG 手绘图或文本图。
- 一份带注释的 IR 摘录。
- 一段 500 字报告: SSA 如何让循环变量和累加器变得可分析。

5.2.14 实验 20.2: 观察 mem2reg

实验

目标: 理解局部变量从内存槽提升为 SSA 值。

任务:

1. 写一个只有局部整数变量的小函数。
2. 生成 `-O0` IR，记录 `alloca/load/store`。
3. 生成 `-O1` IR，记录这些指令是否消失。
4. 修改函数: 把局部变量地址传给外部函数。
5. 再次比较 IR，解释为什么地址逃逸会阻碍提升。

建议使用外部函数:

```
extern "C" void escape_int(int* p);
```

报告要求:

- 必须贴出关键 IR。
- 必须说明“变量在源码中存在”和“变量在优化后 IR 中存在”的区别。
- 必须解释地址逃逸为什么会让优化器更保守。

5.2.15 本节作业

习题与作业

作业 20.A: IR 结构解释

选择一个包含 `if`、`for`、数组访问和函数调用的 C++ 函数，生成 `-O1` IR。回答:

1. 这个函数有几个 `basic block`?
2. 每个 `basic block` 的 `terminator` 是什么?
3. CFG 中有几条边?
4. 哪些值是 `phi` 产生的?
5. 哪些指令是真正的内存访问?
6. 哪个调用最可能阻止优化? 为什么?

习题与作业

作业 20.B: SSA 手工转换

把下面伪代码手工改写成 SSA 风格:

```

x = a + b
if cond:
    x = x * 2

```



```

else:
    x = x - 1
y = x + c
return y

```

要求：

- 给每次定义起不同名字。
- 在控制流汇合处写出 **phi**。
- 画出 def-use 关系。

习题与作业

作业 20.C：版本差异记录

记录你机器上的 LLVM/Clang 版本，并回答：

- IR 中指针显示为 **ptr** 还是类似 **i32***？
- **sum_i32_ptr** 的参数有哪些 attributes？
- **load** 和 **store** 是否带有 **align**？
- 是否出现 TBAA metadata？

要求：以后每次引用 IR 片段时，都要在报告中写明编译器版本和命令。

5.2.16 本节总结

本节建立了后续优化章节的共同语言：

```

LLVM IR:
    typed operations + explicit control flow + explicit memory operations

SSA:
    each ordinary value has one definition

basic block:
    straight-line instruction sequence ended by a terminator

CFG:
    graph formed by basic blocks and branch edges

phi:
    value selection at control-flow join points

load/store:
    where optimization meets memory and alias analysis

```

接下来讨论标量优化时，你要一直带着这套问题：这个 pass 删除、移动、合并或改写了哪些 IR 指令？它依赖哪些 SSA 事实、控制流事实和内存事实？如果这些事实不成立，优化为什么不能做？

5.3 标量优化：DCE、常量传播、内联和 LICM

5.3.1 本节要解决的问题

标量优化处理的是“单个值、表达式、分支、调用和循环不变量”这类结构。它不像 SIMD 那样直接改变一次处理的数据宽度，也不像线程并行那样改变执行资源数量，但它是所有高性能代码的前置条件。很多看似高级的优化，都依赖一批标量 pass 先把程序整理成可分析形态。

本节要回答：

- 编译器如何证明一段计算没有用，从而删除它？
- 常量为什么能穿过函数、分支和循环传播？
- 两次相同计算什么时候能合并？
- 内联为什么既可能加速，也可能让代码变慢？
- 循环里不变的表达式为什么能移到循环外？
- benchmark 为什么特别容易被标量优化“优化没了”？

核心思想

标量优化的核心不是“编译器猜到程序员想快一点”，而是“编译器在 IR 上证明某些变换保持可观察行为”。你读优化后的 IR 时，要追问每个变化背后的证明条件：这个值是否被使用？这个表达式是否无副作用？这个 load 是否可能被 store 改写？这个调用是否可能读写内存？

5.3.2 可观察行为: 优化的边界

优化器不是随意改写程序。它必须保留语言规则定义下的可观察行为。粗略说, 下面这些通常会成为边界:

- 函数返回值。
- 对 volatile 对象的访问。
- 可能逃逸到外部的内存写入。
- I/O、系统调用、原子操作、同步操作。
- 可能有副作用的不透明函数调用。
- 未定义行为边界带来的优化假设。

对 benchmark 来说, 最危险的问题是: 你以为循环在测量某个计算, 但结果没有被观察, 优化器可以证明计算没有影响可观察行为, 于是整个循环消失。

```
extern "C" void bad_bench(const int* values, int n)
{
    long long sum = 0;
    for (int i = 0; i < n; ++i) {
        sum += values[i];
    }
}
```

如果 sum 没有返回、没有写到外部、没有被 observation barrier 使用, 这段循环很可能在 -O3 下被删除。改成:

```
extern "C" long long good_bench(const int* values, int n)
{
    long long sum = 0;
    for (int i = 0; i < n; ++i) {
        sum += values[i];
    }
    return sum;
}
```

返回值成为可观察结果, 循环就不能被简单删除。

5.3.3 Dead Code Elimination

DCE 删除不会影响可观察行为的代码。最简单的例子:

```
extern "C" int dce_example(int x)
{
    int a = x + 1;
    int b = x + 2;
    return a;
}
```

b 的计算结果没有被使用。优化后 IR 可能只剩:

```
define dso_local i32 @dce_example(i32 @noundef %0) {
entry:
    %a = add nsw i32 %0, 1
    ret i32 %a
}
```

DCE 在 SSA 上非常自然。如果一个 instruction 产生的值没有 users, 并且指令本身没有副作用, 就可以删除。删除之后可能让它的 operands 也变成无用值, 于是 DCE 会继续传播。

```
%a = add i32 %x, 1
%b = mul i32 %a, 7
; %b has no users

delete %b
; now %a may also have no users
delete %a
```

但不是所有“结果没用”的指令都能删。例如:

```
call void @puts(ptr %s)
store i32 1, ptr %p
volatile load i32, ptr %q
```

这些操作可能有可观察副作用。DCE 必须尊重副作用边界。

5.3.4 Constant Folding 和 Constant Propagation

constant folding 是在编译期计算常量表达式:

```
extern "C" int folded()
{
    return (3 + 4) * 8;
}
```

```
}
```

优化后：

```
ret i32 56
```

constant propagation 是把已知常量沿 use-def 链传播：

```
extern "C" int branch_fold(int x)
{
    const int scale = 4;
    int y = x * scale;
    if (scale == 4) {
        return y + 1;
    }
    return y - 1;
}
```

因为 `scale==4` 总为真，分支可以被折叠。优化器可能得到：

```
%y = shl nsw i32 %x, 2
%ret = add nsw i32 %y, 1
ret i32 %ret
```

这里还出现了 strength reduction：乘以 4 被替换为左移。实际后端是否真的发出 `shl`，还取决于目标机器、后端选择和后续优化。

5.3.5 Sparse Conditional Constant Propagation

真实编译器通常不会只做“文本替换式”的常量传播。控制流也会参与。SCCP 会同时追踪值是否为常量，以及 basic block 是否可达。

例子：

```
extern "C" int sccp_example(int x)
{
    int mode = 1;
    if (mode == 0) {
        return x * 100;
    }
    return x + 7;
}
```

优化器知道 `mode` 为 1，于是 `mode==0` 的分支不可达，乘法分支可以删除。

这类优化对模板和 `constexpr` 很重要。C++ 常常通过编译期常量生成多种路径，优化器再把不可达路径消掉。很多高性能库利用这个机制把通用模板代码变成针对特定类型、大小、布局的专门代码。

5.3.6 CSE 和 GVN

CSE 是 Common Subexpression Elimination，公共子表达式消除。GVN 是 Global Value Numbering，全局值编号。它们都在寻找“已经算过的等价值”。

简单例子：

```
extern "C" int cse_example(int a, int b)
{
    int x = (a + b) * 3;
    int y = (a + b) * 5;
    return x + y;
}
```

`a+b` 出现两次。优化器可以只算一次：

```
%sum = add nsw i32 %b, %a
%x = mul nsw i32 %sum, 3
%y = mul nsw i32 %sum, 5
%ret = add nsw i32 %y, %x
ret i32 %ret
```

GVN 处理的问题更广：不同写法是否产生同一个值。例如：

```
(x + 1) + 1
x + 2
```

在某些语义条件下可以等价，但有符号溢出、浮点舍入、NaN、异常、内存副作用都可能阻止这种等价。整数优化和浮点优化的差异，后面会反复出现。

5.3.7 内存表达式为什么更难

纯 SSA 值容易做 CSE，内存 load 更难。

```
extern "C" int load_twice(const int* p)
{
    int a = *p;
```

```
int b = *p;
return a + b;
}
```

如果中间没有可能写 `*p` 的操作，两次 `load` 可能合并。但如果加一个不透明调用：

```
extern "C" void unknown();

extern "C" int load_call_load(const int* p)
{
    int a = *p;
    unknown();
    int b = *p;
    return a + b;
}
```

优化器不能随便把第二次 `load` 合并到第一次。因为 `unknown()` 可能修改 `p` 指向的内存，除非函数属性、别名分析或链接时信息能证明不会。

心智模型

对优化器来说，普通 SSA 值像一条清楚的电线；内存像一个共享的大盒子。只要指针、调用或 `store` 可能碰到同一个盒子，很多看似明显的优化就不能做。

5.3.8 Inlining：优化的入口放大器

内联把被调用函数的函数体复制到调用点。它最直接的收益是消除调用开销，但更重要的收益是暴露优化机会。

```
static int scale4(int x)
{
    return x * 4;
}

extern "C" int use_scale(int x)
{
    return scale4(x) + 1;
}
```

内联后，调用边界消失：

```
%mul = shl nsw i32 %x, 2
%ret = add nsw i32 %mul, 1
ret i32 %ret
```

如果不内联，优化器可能看不到被调用函数内部，不知道它只是乘以 4。内联之后，constant propagation、CSE、DCE、LICM、向量化都可能获得更多事实。

但内联不是越多越好。过度内联会造成：

- 代码体积膨胀。
- instruction cache 压力增加。
- 前端 decode/uop cache 压力增加。
- 寄存器压力增加。
- 编译时间增加。

现代编译器有内联 cost model。它会根据函数大小、调用频率、参数常量、是否热路径、是否会暴露优化机会等因素决策。PGO 会进一步影响这些决策。

5.3.9 LICM：Loop Invariant Code Motion

LICM 把循环里每轮结果都相同的计算移到循环外。

```
extern "C" int sum_scaled(const int* values, int n, int scale)
{
    int sum = 0;
    int bias = scale * 7 + 3;
    for (int i = 0; i < n; ++i) {
        sum += values[i] + bias;
    }
    return sum;
}
```

如果 `bias` 写在循环外，当然没有问题。即使写在循环内：

```
for (int i = 0; i < n; ++i) {
    int bias = scale * 7 + 3;
    sum += values[i] + bias;
}
```

优化器也可能把 `scale*7+3` 移到循环外，因为它不依赖 `i`，也不被循环内可能的副作用改变。

LICM 的证明条件包括：

- 表达式 operands 在循环中不变。
- 指令没有不可移动副作用。
- 移动后不会改变异常、未定义行为或内存可观察行为。
- 如果涉及 load，必须证明循环内没有 store 或 call 会改变它读到的内存。

内存 load 的 LICM 很常见也很危险：

```
extern "C" int sum_with_factor(const int* values,
                              const int* factor,
                              int n)
{
    int sum = 0;
    for (int i = 0; i < n; ++i) {
        sum += values[i] * (*factor);
    }
    return sum;
}
```

能不能把 `*factor` 移出循环，取决于优化器能否证明 `values[i]` 的访问不会写到 `factor`，且中间没有其他可能修改 `*factor` 的操作。因为 `values` 是 `const int*` 只限制通过该指针写入，不自动证明它和 `factor` 不 alias。第 5.6 会深入讨论。

5.3.10 Strength Reduction

strength reduction 把昂贵操作换成等价的便宜操作。常见例子：

```
i * 4      -> i << 2
base + i*4 -> addressing mode or induction increment
x / 8      -> shift with signedness caveats
```

在 x86-64 上，数组地址计算经常不需要显式乘法，因为寻址模式可以表达：

```
[base + index*1 + displacement]
[base + index*2 + displacement]
[base + index*4 + displacement]
[base + index*8 + displacement]
```

因此 C++ 中的 `values[i]` 可能变成汇编里的 scale addressing，而不是单独的乘法指令。读汇编时要把地址计算和数据访问放在一起看。

5.3.11 InstCombine：小变换的大集合

LLVM 中有一个非常重要的 pass 叫 InstCombine。它做大量局部代数化简和规范化，例如：

```
x + 0      -> x
x * 1      -> x
x * 0      -> 0
(x << 2) << 1 -> x << 3
icmp eq x, x -> true, if x is not poison/undef sensitive
```

InstCombine 的目标不只是“少几条指令”，还包括把 IR 变成后续 pass 更容易识别的标准形态。编译器优化经常是多轮协作：

```
inline
-> exposes constants
constant propagation
-> makes branches unreachable
simplify CFG
-> deletes blocks
instcombine
-> canonicalizes expressions
loop passes
-> recognize loops better
```

5.3.12 benchmark 中的优化陷阱

性能实验最常见的错误，不是计时 API 用错，而是被优化器改变了实验对象。

错误模式 1：结果没人用。

```
for repeat:
    compute sum
; sum discarded
```

错误模式 2：输入是编译期常量。

```
for repeat:
    compute f(known_constant)
; compiler precomputes or folds
```

错误模式 3：benchmark 函数被内联到调用点后，整个计算被证明可删除。

错误模式 4：测量的是空循环、分支框架或计时框架，而不是目标 kernel。

最低限度的防护：

- 返回或校验计算结果。
- 使用运行时输入。
- 查看优化后 IR 和汇编。
- 对照不同输入规模的运行时间。
- 用 PMU 计数确认指令和访存确实发生。

5.3.13 实验 21.1：让 DCE 删除你的 benchmark

实验

目标：亲眼观察 benchmark 计算被删除。

写两个函数：

```
#include <cstdint>
#include <cstdint>

extern "C" void bad_sum(const std::uint64_t* values, std::size_t size)
{
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < size; ++i) {
        sum += values[i];
    }
}

extern "C" std::uint64_t good_sum(const std::uint64_t* values,
                                  std::size_t size)
{
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < size; ++i) {
        sum += values[i];
    }
    return sum;
}
```

任务：

1. 生成 -O0 和 -O3 IR。
2. 生成 -O3 汇编。
3. 解释 bad_sum 的循环是否存在。
4. 解释 good_sum 为什么不能被同样删除。
5. 用 perfstat 比较两者运行时的指令数。

交付物：

- IR 摘录。
- 汇编摘录。
- perfstat 输出。
- 一段报告：这个实验如何改变你对 benchmark 的理解。

5.3.14 实验 21.2：观察内联暴露常量

实验

目标：理解内联如何放大其他优化。

写：

```
static int affine(int x, int a, int b)
{
    return x * a + b;
}

extern "C" int call_affine(int x)
{
    return affine(x, 8, 3);
}
```

任务：

1. 生成 -O0、-O1、-O3 IR。
2. 找到 affine 是否仍然作为独立函数存在。

3. 记录乘以 8 是否变成 shift 或地址形式。
4. 尝试给 `affine` 加 `__attribute__((noinline))`。
5. 对比 `noinline` 前后的 IR 和汇编。

报告必须回答：内联本身省掉了什么？内联之后又暴露了哪些新的优化机会？

5.3.15 实验 21.3: LICM 和 alias 的边界

实验

目标：观察循环不变量移动什么时候失败。

准备两个函数：

```
extern "C" int sum_factor_plain(const int* values,
                               const int* factor,
                               int n);

extern "C" int sum_factor_restrict(const int* __restrict values,
                                   const int* __restrict factor,
                                   int n);
```

函数体都计算：

```
int sum = 0;
for (int i = 0; i < n; ++i) {
    sum += values[i] * (*factor);
}
return sum;
```

任务：

1. 对两个函数生成 -O3 IR。
2. 找 `*factor` 的 load 在循环内还是循环外。
3. 查看函数参数是否带有 `noalias` 相关属性。
4. 生成汇编并标注 load 位置。
5. 解释 `restrict` 改变了什么证明条件。

5.3.16 本节作业

习题与作业

作业 21.A: DCE 判定

对下面每条 IR 风格指令，判断如果结果没有 users，能不能删除，并说明理由：

```
%x = add i32 %a, %b
%y = load i32, ptr %p
store i32 %v, ptr %p
call void @unknown()
%z = call i32 @pure_func(i32 %a)
%q = volatile load i32, ptr %p
```

要求：必须区分“值无用”和“操作无副作用”。

习题与作业

作业 21.B: 优化链条报告

选择一个包含小 helper 函数、常量参数、分支和循环的例子。生成 -O0、-O1、-O2、-O3 IR，写一份报告：

- 哪些函数被内联？
- 哪些分支被删除？
- 哪些表达式被常量折叠？
- 哪些循环不变量被移出？
- 哪些变化在汇编里可见？

习题与作业

作业 21.C: benchmark 防优化清单

为自己的性能实验写一份 checklist。至少包含：

1. 如何保证输入不是编译期常量。
2. 如何保证结果被观察。
3. 如何检查循环没有被 DCE 删除。

4. 如何记录编译器版本和参数。
5. 如何用 PMU 证明计算确实发生。
6. 如何解释优化器造成的意外差异。

5.3.17 本节总结

标量优化的主线可以压缩成一句话：优化器在 SSA、CFG、属性和内存事实上证明哪些计算可以删除、合并、提前、常量化或跨函数传播。DCE 保护你不浪费指令，也会毁掉写错的 benchmark；内联省掉调用，更重要的是暴露上下文；LICM 减少循环体工作量，但遇到内存和 alias 就必须谨慎。

下一节进入循环优化。循环优化建立在本节之上：先有清楚的 induction variable、循环不变量、内存访问和控制流，才谈得上展开、重排、归约、向量化和并行。

5.4 循环优化：展开、重排、归约和依赖

5.4.1 本节要解决的问题

高性能计算和 AI 算子的大部分时间都花在循环里。GEMM 是循环，卷积是循环，softmax 是循环，layernorm 是循环，attention 的核心也是循环和矩阵块操作。理解循环优化，就是理解现代性能优化的主战场。

本节讨论编译器如何分析和改写循环：

- 循环在 IR 里如何被识别为 loop。
- induction variable、trip count、latch、exit 是什么。
- loop-carried dependency 为什么会限制优化。
- unroll 如何增加 ILP、减少分支开销、暴露向量化机会。
- peeling、unswitching、interchange 分别解决什么问题。
- reduction 为什么是特殊但常见的循环模式。
- 编译器为什么有时拒绝优化一个看似简单的循环。

核心思想

循环优化不是“把循环变快”这么笼统。每个变换都有明确目标：减少循环控制开销、暴露独立操作、改善访存局部性、移动不变量、消除分支、匹配 SIMD 形态或降低依赖链长度。分析循环时必须先判断瓶颈来自控制流、依赖、访存、前端、后端端口还是内存层级。

5.4.2 循环在 IR 中的基本形态

优化器希望循环有规范结构。一个常见循环：

```
extern "C" int sum_i32(const int* values, int n)
{
    int sum = 0;
    for (int i = 0; i < n; ++i) {
        sum += values[i];
    }
    return sum;
}
```

在 IR 的概念结构上接近：

```
entry:
    br label %loop

loop:
    %i = phi i32 [ 0, %entry ], [ %i.next, %loop ]
    %sum = phi i32 [ 0, %entry ], [ %sum.next, %loop ]
    %ptr = getelementptr i32, ptr %values, i32 %i
    %v = load i32, ptr %ptr
    %sum.next = add i32 %sum, %v
    %i.next = add i32 %i, 1
    %done = icmp eq i32 %i.next, %n
    br i1 %done, label %exit, label %loop

exit:
    ret i32 %sum.next
```

几个术语：

术语	含义
header	循环入口 block，通常包含关键 <code>phi</code> 。
latch	跳回 header 的 block。简单循环中 header 和 latch 可能是同一个 block。
backedge	从 latch 回到 header 的边。
preheader	进入循环前的唯一前驱，LICM 常把不变量放这里。
exit	离开循环后到达的 block。
induction variable	按规律变化的循环变量，例如 <code>i=i+1</code> 。
trip count	循环执行次数，可能是常量、运行时值或不可证明。

LoopSimplify、LCSSA 等 pass 会把循环整理成更适合后续分析的形态。你不必马上记住所有 pass 名字，但要理解：很多高级优化依赖循环先被规范化。

5.4.3 loop-carried dependency

循环优化最核心的问题是依赖。依赖分为 loop-independent 和 loop-carried。loop-carried dependency 指某一轮迭代依赖前一轮或更早迭代的结果。

最典型的是累加：

```
sum += values[i];
```

第 `i+1` 轮的 `sum` 依赖第 `i` 轮的 `sum`。这形成一条串行依赖链：

```
sum0 -> add v0 -> sum1 -> add v1 -> sum2 -> add v2 -> sum3
```

如果每次整数加法延迟是 1 cycle，这条链可能还不严重；如果是浮点加法、FMA 或复杂操作，依赖链就可能限制吞吐。编译器和手写优化常用多个累加器打断依赖链：

```
std::int64_t s0 = 0;
std::int64_t s1 = 0;
std::int64_t s2 = 0;
std::int64_t s3 = 0;
for (std::size_t i = 0; i + 3 < n; i += 4) {
    s0 += values[i + 0];
    s1 += values[i + 1];
    s2 += values[i + 2];
    s3 += values[i + 3];
}
std::int64_t sum = s0 + s1 + s2 + s3;
```

现在依赖链变成四条较短链：

```
s0 chain: s0 -> add v0 -> add v4 -> ...
s1 chain: s1 -> add v1 -> add v5 -> ...
s2 chain: s2 -> add v2 -> add v6 -> ...
s3 chain: s3 -> add v3 -> add v7 -> ...
```

这样 CPU 乱序执行可以并行推进多条链，提升 ILP。

5.4.4 循环展开

loop unrolling 把多轮迭代合并到一个更大的循环体里。原始循环：

```
for i = 0 .. n-1:
    body(i)
```

展开 4 倍：

```
for i = 0 .. n-4 step 4:
    body(i + 0)
    body(i + 1)
    body(i + 2)
    body(i + 3)
tail loop for remaining elements
```

收益：

- 减少分支和循环计数开销。
- 暴露更多独立指令给调度器。
- 暴露多个 load/store，帮助隐藏延迟。
- 给向量化、SLP、指令选择创造更明显模式。

代价：

- 代码体积增加。
- instruction cache 和 uop cache 压力增加。
- 寄存器压力增加，可能导致 spill。

- tail 处理更复杂。

所以展开不是越大越好。真正的优化需要结合指令延迟、吞吐、寄存器数量、前端带宽、代码体积和数据规模。

5.4.5 完全展开和部分展开

如果 trip count 是小常量，编译器可能完全展开：

```
extern "C" int sum4(const int* p)
{
    int s = 0;
    for (int i = 0; i < 4; ++i) {
        s += p[i];
    }
    return s;
}
```

优化后可能没有循环：

```
load p[0]
load p[1]
load p[2]
load p[3]
add tree
ret
```

如果 trip count 是运行时值，只能部分展开并保留 tail：

```
main loop:
    process 4 elements per iteration

tail:
    process n % 4 elements
```

这也是 SIMD 向量化中 remainder loop 的来源之一。

5.4.6 Loop Peeling

peeling 把循环前几轮剥离出来，剩下的循环获得更好的性质。

常见目的：

- 处理对齐，让主体循环从对齐地址开始。
- 处理第一次迭代的特殊情况。
- 消除循环体内某个分支。
- 让剩余循环满足向量化条件。

例子：

```
while pointer is not aligned:
    scalar process one element

vector loop:
    process aligned chunks

tail:
    scalar process remaining elements
```

现代 x86 对非对齐 load 的支持已经很好，但跨 cache line、跨 page、老架构或特定指令仍可能受影响。peeling 是否值得，需要结合目标 CPU 和数据分布。

5.4.7 Loop Unswitching

如果循环内有一个条件在整个循环期间不变，unswitching 可以把分支移到循环外：

```
extern "C" void scale_or_copy(float* y,
                             const float* x,
                             float a,
                             int n,
                             bool scale)
{
    for (int i = 0; i < n; ++i) {
        if (scale) {
            y[i] = a * x[i];
        } else {
            y[i] = x[i];
        }
    }
}
```

如果 scale 在循环中不变，可以变成：

```

if scale:
    for i:
        y[i] = a * x[i]
else:
    for i:
        y[i] = x[i]

```

收益是主体循环不再每轮执行分支，向量化器也更容易处理两个独立循环。代价是代码体积增加。对于 AI 算子中的模式分支、是否加 bias、是否 ReLU、是否 mask，这类变换非常常见。生产库通常会手动 dispatch 到不同 kernel，本质上也是把循环内不变分支外提。

5.4.8 Loop Interchange

loop interchange 改变嵌套循环顺序。它直接影响内存局部性。

行主序二维数组：

```

for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        sum += a[i * cols + j];
    }
}

```

内层 j 连续访问内存，cache line 利用好。如果交换：

```

for (int j = 0; j < cols; ++j) {
    for (int i = 0; i < rows; ++i) {
        sum += a[i * cols + j];
    }
}

```

内层 stride 是 cols，可能每次访问不同 cache line，性能差很多。

编译器做 interchange 需要证明依赖允许交换。如果循环写入和读取之间存在跨迭代依赖，交换可能改变结果。

5.4.9 Reduction：特殊的依赖模式

reduction 是把一组元素归约成一个值：

```

sum += x[i]
prod *= x[i]
max = max(max, x[i])
count += predicate(x[i])

```

reduction 有 loop-carried dependency，但它有特殊结构。对于整数加法，如果没有溢出语义障碍，编译器可以用多累加器、向量规约、树形归约等方式改变执行顺序。对于浮点加法，顺序改变会改变舍入结果，因此严格 IEEE 语义下编译器更保守。

这就是为什么同一个 sum_f32：

```

clang++ -O3 sum.cpp
clang++ -O3 -ffast-math sum.cpp

```

可能产生完全不同的向量化结果。fast-math 允许编译器放弃某些浮点语义约束，例如重新结合和忽略 NaN/Inf 的一部分严格行为。第 5.6 会具体拆。

5.4.10 依赖距离和内存依赖

循环中读写数组时，优化器还要判断跨迭代内存依赖：

```

for (int i = 1; i < n; ++i) {
    a[i] = a[i - 1] + b[i];
}

```

第 i 轮读取 a[i-1]，这个值由第 i-1 轮写入。存在明确 loop-carried dependency。不能简单并行化或向量化。

另一个例子：

```

for (int i = 0; i < n; ++i) {
    a[i] = b[i] + c[i];
}

```

如果 a、b、c 不 alias，各迭代基本独立，非常适合向量化。但如果它们可能指向重叠内存，编译器可能需要 runtime alias check：

```

if arrays do not overlap:
    vector loop
else:
    scalar loop

```

这就是很多高性能 C/C++ 接口使用 restrict、明确 shape、对齐、stride 和 non-overlap contract 的原因。

5.4.11 循环优化和机器执行

循环优化最终要落到 CPU 执行。分析时不要只问“编译器有没有展开”，还要问：

- 展开后是否增加了独立 load?
- 多累加器是否降低了依赖链长度?
- 寄存器是否够用，是否出现 spill?
- 代码体积是否让前端压力变大?
- tail loop 是否成为小规模输入的主要成本?
- 分支预测是否稳定?
- 内存访问是否连续，cache line 是否充分利用?

这也是为什么第 6 章微架构、第 7 章内存和第 8 章 SIMD 要建立在循环优化之后。循环结构决定了硬件看到的指令流和数据流。

5.4.12 实验 22.1：手写多累加器

实验

目标：观察多累加器如何改变依赖链和汇编。

实现三个版本：

```
std::int64_t sum1(const std::int32_t* values, std::size_t size);
std::int64_t sum2(const std::int32_t* values, std::size_t size);
std::int64_t sum4(const std::int32_t* values, std::size_t size);
```

要求：

- sum1 使用一个累加器。
- sum2 使用两个累加器，每轮处理 2 个元素。
- sum4 使用四个累加器，每轮处理 4 个元素。
- 每个版本都处理 tail。
- 所有版本返回相同结果。

任务：

1. 生成 `-O3 -fno-vectorize -fno-slp-vectorize` 汇编。
2. 标注每个版本的累加依赖链。
3. benchmark 不同 size。
4. 用 `perfstat` 记录 cycles、instructions、IPC。
5. 判断性能差异来自依赖链、分支、load 还是其他因素。

5.4.13 实验 22.2：观察 unroll 和 tail

实验

目标：理解展开后的主体循环和尾部循环。

使用同一个 `sum_i32`，分别生成：

```
clang++ -std=c++20 -O3 -S -emit-llvm sum.cpp -o sum_O3.ll
clang++ -std=c++20 -O3 -fno-unroll-loops -S -emit-llvm sum.cpp -o sum_no_unroll.ll
clang++ -std=c++20 -O3 -funroll-loops -S -emit-llvm sum.cpp -o sum_unroll.ll
```

任务：

1. 比较三个 IR 文件里的循环数量。
2. 找主体循环每轮处理多少元素。
3. 找 remainder 或 tail 路径。
4. 对小 n 和大 n 分别 benchmark。
5. 写出展开对小规模和大规模输入的不同影响。

5.4.14 实验 22.3：循环顺序和 cache

实验

目标：亲自测量 loop interchange 对局部性的影响。

准备一个行主序矩阵 `std::vector<float>a(rows*cols)`，实现：

```
float sum_row_major(const float* a, int rows, int cols);
float sum_col_major(const float* a, int rows, int cols);
```

任务：

1. 固定总元素数，改变 `rows/cols` 组合。
2. 记录两种循环顺序的运行时间。
3. 用 `perfstat` 记录 cache miss 相关事件。
4. 解释 stride 如何影响 cache line 利用。
5. 画出地址访问序列。

报告要求：不能只写“行优先更快”，必须结合地址、cache line 和测量证据解释。

5.4.15 本节作业

习题与作业

作业 22.A：循环结构标注

给一个优化后 IR 中的循环，标出：

- header。
- latch。
- preheader。
- exit。
- induction variable。
- trip count 判断。
- reduction `phi`。

要求：每个标注都要说明对应哪一行 IR。

习题与作业

作业 22.B：依赖分类

判断下面循环是否存在 loop-carried dependency，并说明是否容易向量化：

```
for (int i = 0; i < n; ++i) a[i] = b[i] + c[i];
for (int i = 1; i < n; ++i) a[i] = a[i - 1] + b[i];
for (int i = 0; i < n; ++i) sum += a[i];
for (int i = 0; i < n; ++i) a[index[i]] += b[i];
```

要求：区分标量依赖、内存依赖和归约模式。

习题与作业

作业 22.C：循环优化报告

选择一个真实小 kernel，例如 `dot`、`saxpy`、`relu` 或二维矩阵求和。写一份循环优化报告：

1. baseline 源码。
2. 优化前 IR 和汇编摘要。
3. 你认为的瓶颈。
4. 你做的循环变换。
5. 正确性验证。
6. benchmark 数据。
7. PMU 证据。
8. 为什么这个优化有效或无效。

5.4.16 本节总结

循环优化把前面学到的 SSA、CFG、DCE、LICM、alias 和机器执行全部连起来。一个循环是否好优化，取决于控制流是否简单、trip count 是否可知、依赖是否可证明、内存访问是否规律、浮点语义是否允许重排、目标 CPU 是否能利用暴露出的并行性。

下一节进入自动向量化。向量化不是凭空发生的，它通常需要一个已经被标量优化整理好的循环：清晰的 induction variable、可证明的连续访问、可处理的归约、可接受的 alias 关系和明确的尾部策略。

5.5 自动向量化

5.5.1 本节要解决的问题

SIMD 是 CPU 高性能计算的关键能力之一。手写 AVX2/AVX-512 intrinsic 很重要，但在进入手写 SIMD 之前，必须先理解编译器自动向量化。原因很简单：很多循环本来就能被编译器向量化；很多不能向量化的循环，失败原因会告诉你手写 SIMD 也会遇到什么语义障碍。

本节要讲清楚：

- 编译器把 scalar loop 改写成 SIMD loop 的基本过程。
- loop vectorizer 和 SLP vectorizer 的区别。
- legal 和 profitable 分别是什么意思。
- alias、alignment、trip count、reduction、fast-math 如何影响向量化。
- 如何读 Clang/GCC 的向量化报告。
- 如何写代码帮助编译器向量化，而不是盲目依赖它。

核心思想

自动向量化有两道门：legal 和 profitable。legal 问的是“这样改写是否保持程序语义”；profitable 问的是“这样改写是否值得”。一个循环没有向量化，不一定是编译器弱，也可能是语言语义、别名、浮点、尾部成本、目标 CPU 或成本模型给出的合理结论。

5.5.2 从 scalar loop 到 vector loop

标量循环：

```
extern "C" void add_f32(float* out,
                      const float* a,
                      const float* b,
                      int n)
{
    for (int i = 0; i < n; ++i) {
        out[i] = a[i] + b[i];
    }
}
```

向量化后的概念结构：

```
main vector loop:
    load vector a[i : i+VF-1]
    load vector b[i : i+VF-1]
    add vector
    store vector out[i : i+VF-1]
    i += VF

remainder loop:
    handle n % VF elements
```

其中 VF 是 vectorization factor，也就是每个向量处理多少个元素。对于 AVX2：

```
float:
    256-bit vector / 32-bit element = 8 lanes

int32:
    256-bit vector / 32-bit element = 8 lanes

double:
    256-bit vector / 64-bit element = 4 lanes
```

如果使用 AVX-512，VF 通常更大，但尾部处理、频率、寄存器压力和平台差异也更复杂。后面 SIMD 卷会专门讲。

5.5.3 Loop Vectorizer 和 SLP Vectorizer

LLVM 中常见两类向量化：

类型	直觉
Loop Vectorizer	把循环的多次迭代合并成一次向量迭代。
SLP Vectorizer	在同一 basic block 中把相似的独立标量操作打包成向量。

loop vectorizer 处理：

```
for (int i = 0; i < n; ++i) {
    out[i] = a[i] + b[i];
}
```

SLP 处理：

```
float x0 = a0 + b0;
float x1 = a1 + b1;
float x2 = a2 + b2;
float x3 = a3 + b3;
```

在 HPC 和 AI 算子中，两者都会出现。循环向量化负责长数组、矩阵行、连续元素；SLP 常出现在小固定大小向量、展开后的循环体、结构体字段并行计算中。

5.5.4 legal: 语义上能不能改

编译器首先要证明向量化合法。常见问题：

- 不同迭代之间是否独立？
- load/store 是否可能互相 alias？
- 是否存在 loop-carried dependency？
- 浮点运算是否允许改变结合顺序？
- 循环内调用是否可向量化或可安全移动？
- 异常、越界、未定义行为和内存访问是否可处理？

容易向量化：

```
for (int i = 0; i < n; ++i) {
    out[i] = a[i] + b[i];
}
```

明显不容易：

```
for (int i = 1; i < n; ++i) {
    a[i] = a[i - 1] + b[i];
}
```

第二个循环有跨迭代依赖。第 i 轮写的 $a[i]$ 会被第 $i+1$ 轮读取。不能简单把多轮同时执行。

5.5.5 profitable: 值不值得改

合法不等于值得。成本模型还要判断：

- 向量指令是否比标量指令吞吐更好。
- 需要多少 shuffle、permute、gather、scatter。
- 是否需要 runtime alias check。
- remainder loop 成本是否太高。
- 循环 trip count 是否太小。
- 向量化是否增加寄存器压力。
- 目标 CPU 上宽向量是否影响频率。

例如带 gather 的间接访问：

```
for (int i = 0; i < n; ++i) {
    out[i] = table[index[i]];
}
```

理论上可以用 gather，但如果目标 CPU 上 gather 很慢，或者访问模式随机导致 cache miss，向量化不一定划算。

5.5.6 alias 和 runtime check

看 SAXPY：

```
extern "C" void saxpy(float* y,
                    const float* x,
                    float a,
                    int n)
{
    for (int i = 0; i < n; ++i) {
        y[i] = a * x[i] + y[i];
    }
}
```

如果 x 和 y 重叠，向量化可能改变语义吗？某些重叠关系下，先读一批 x 再写一批 y ，可能和标量逐个处理不同。编译器有几种选择：

- 拒绝向量化。
- 生成 runtime alias check，不重叠时走向量路径，可能重叠时走标量路径。
- 依赖 restrict、函数属性或更强接口约定证明不重叠。

概念结构：

```
if no_overlap(x, y, n):
    vectorized saxpy
else:
    scalar saxpy
```

这就是优化报告里经常看到 runtime checks 的原因。它不是“多余代码”，而是编译器为了在保持语义的同时获得快速路径。

5.5.7 alignment

对齐影响 load/store 成本和可选指令。现代 x86 支持非对齐向量 load/store，但对齐仍可能影响：

- 是否跨 cache line。
- 是否跨 page。
- 某些老指令或平台上的异常/性能。
- 编译器选择 aligned 还是 unaligned 指令。
- 循环 peeling 是否有收益。

C++ 层可用的表达方式包括：

```
auto* aligned = static_cast<float*>(
    __builtin_assume_aligned(ptr, 32));
```

但这类假设必须真实。如果指针没有按 32 字节对齐，你告诉编译器它对齐了，程序可能产生错误或崩溃。优化假设不是注释，是契约。

5.5.8 trip count、VF 和 remainder

如果 n 不是 VF 的整数倍，向量循环处理主体，剩余元素交给 tail：

```
vector loop:
    i = 0
    while i + VF <= n:
        process VF elements
        i += VF

scalar tail:
    while i < n:
        process 1 element
        i += 1
```

AVX-512 有 mask，可以用 masked operations 处理尾部；AVX2 通常使用标量 tail 或较小向量 tail。不同策略影响小 n 性能。

如果 trip count 很小，向量化前的准备、检查和 tail 成本可能超过收益。生产算子常常为不同 size 做多版本 dispatch：

```
if n < small_threshold:
    scalar or small-kernel path
else:
    vectorized path
```

5.5.9 浮点 reduction 和 fast-math

浮点规约是自动向量化的经典难题：

```
extern "C" float sum_f32(const float* values, int n)
{
    float sum = 0.0f;
    for (int i = 0; i < n; ++i) {
        sum += values[i];
    }
    return sum;
}
```

标量顺序：

```
((v0 + v1) + v2) + v3) + ...
```

向量归约可能变成：

```
(v0 + v4 + v8 + ...)
(v1 + v5 + v9 + ...)
(v2 + v6 + v10 + ...)
(v3 + v7 + v11 + ...)
horizontal add at end
```

加法顺序变了，浮点舍入结果可能不同。严格语义下编译器可能拒绝向量化，或者只做非常保守的转换。启用 `-ffast-math`、`-fassociative-math` 等选项后，编译器获得重新结合浮点表达式的自由，可能向量化。

常见误区

fast-math 不是“让浮点更快”的普通开关，而是改变语义契约。它可能影响 NaN、Inf、有符号零、舍入、异常、结合律和 FMA contraction。AI 推理中很多场景可以接受近似差异，但数值代码必须明确误差预算。

5.5.10 向量化报告

Clang 常用：

```
clang++ -std=c++20 -O3 -march=native \
  -Rpass=loop-vectorize \
  -Rpass-missed=loop-vectorize \
  -Rpass-analysis=loop-vectorize \
  kernel.cpp -c -o kernel.o
```

GCC 常用：

```
g++ -std=c++20 -O3 -march=native \
  -fopt-info-vec \
  -fopt-info-vec-missed \
  kernel.cpp -c -o kernel.o
```

报告常见信息：

- loop vectorized, VF 是多少, interleave count 是多少。
- missed: cannot identify array bounds。
- missed: unsafe dependent memory operations。
- missed: value that could not be identified as reduction。
- analysis: runtime pointer checks needed。
- analysis: cost model indicates vectorization is not beneficial。

读报告时要和 IR/汇编对照。报告说 vectorized，不代表最终汇编一定是你期待的 AVX2/FMA；报告说 missed，也不代表没有任何其他优化。

5.5.11 帮助编译器向量化的代码风格

通常更容易向量化的代码：

- 简单 counted loop。
- 连续数组访问。
- 清楚的边界和正向步长。
- 尽量少的循环内分支。
- 函数调用可内联或可向量化。
- 明确 non-overlap 契约。
- 明确对齐和 size。
- reduction 形态清晰。

容易失败的代码：

- 复杂 while 条件。
- 间接索引和随机访问。
- 指针之间可能 alias。
- 循环内不透明调用。
- 复杂控制流和 early exit。
- 浮点严格语义下的规约。
- 数据结构是 AoS 且字段访问跨 stride。

这并不意味着你要为了编译器牺牲所有抽象。正确策略是：热路径 kernel 边界明确，内部循环写得让编译器和人都容易分析；外层保持良好工程结构。

5.5.12 实验 23.1：SAXPY 向量化报告

实验

目标：读懂一个真实的向量化报告。
实现两个版本：

```
extern "C" void saxpy_plain(float* y,
                           const float* x,
                           float a,
                           int n);
```



```
extern "C" void saxpy_restrict(float* __restrict y,
                             const float* __restrict x,
                             float a,
                             int n);
```

任务：

1. 使用 Clang 生成向量化报告。
2. 比较 plain 和 restrict 报告差异。
3. 生成 `-O3-march=native` 汇编。
4. 找是否有 runtime alias check。
5. benchmark 两个版本。
6. 解释 restrict 是否改变性能，以及为什么。

交付物：

- 优化报告原文摘录。
- 汇编中向量 load、FMA 或 multiply/add 的位置。
- 一张性能表。
- 一段说明：alias 信息如何影响向量化。

5.5.13 实验 23.2: dot product 和 fast-math

实验

目标：观察浮点规约在严格语义和 fast-math 下的差异。

实现：

```
extern "C" float dot_f32(const float* a,
                        const float* b,
                        int n)
{
    float sum = 0.0f;
    for (int i = 0; i < n; ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}
```

分别编译：

```
clang++ -std=c++20 -O3 -march=native \
-Rpass=loop-vectorize \
-Rpass-missed=loop-vectorize \
dot.cpp -c -o dot_strict.o

clang++ -std=c++20 -O3 -march=native -ffast-math \
-Rpass=loop-vectorize \
-Rpass-missed=loop-vectorize \
dot.cpp -c -o dot_fast.o
```

任务：

1. 记录两次向量化报告。
2. 比较汇编是否出现向量 FMA。
3. 用随机输入比较结果误差。
4. 用特殊输入测试 NaN、Inf、非常大/非常小数。
5. 写出 fast-math 的收益和语义代价。

5.5.14 实验 23.3: ReLU 的分支和向量化

实验

目标：观察简单分支如何被向量化成 compare/select/max。

实现两个版本：

```
extern "C" void relu_branch(float* out,
                            const float* x,
                            int n)
{
    for (int i = 0; i < n; ++i) {
        out[i] = x[i] > 0.0f ? x[i] : 0.0f;
    }
}
```

```

}

extern "C" void relu_max(float* out,
                        const float* x,
                        int n)
{
    for (int i = 0; i < n; ++i) {
        out[i] = __builtin_fmaxf(x[i], 0.0f);
    }
}

```

任务：

1. 生成向量化报告。
2. 比较两种写法的 IR。
3. 比较汇编中是 branch、compare/select 还是 max。
4. 测量不同正负数分布下的性能。
5. 解释分支预测和 SIMD select 的关系。

5.5.15 本节作业

习题与作业

作业 23.A: legal/profitable 分类

对下面循环判断向量化障碍，并区分 legal 和 profitable：

```

for (int i = 0; i < n; ++i) out[i] = a[i] + b[i];
for (int i = 1; i < n; ++i) a[i] = a[i - 1] + b[i];
for (int i = 0; i < n; ++i) out[i] = table[index[i]];
for (int i = 0; i < n; ++i) sum += x[i];
for (int i = 0; i < n; ++i) out[i] = f(x[i]);

```

要求：不能只写“能/不能”，必须解释依赖、访存、调用、成本模型或浮点语义。

习题与作业

作业 23.B: 向量化报告归档

选择三个 kernel: saxpy、dot、relu。为每个 kernel 建立报告目录：

```

results/ch23/<kernel>/
  source.cpp
  compile.txt
  report_clang.txt
  report_gcc.txt
  ir_03.ll
  asm_03.s
  perf.csv
  notes.md

```

要求：报告中必须同时包含编译器报告、IR 证据、汇编证据和运行时间证据。

习题与作业

作业 23.C: 帮助编译器优化

选择一个最初没有向量化的循环，逐步修改源码：

1. 简化控制流。
2. 移出循环不变量。
3. 加入 non-overlap 契约。
4. 改善数据布局或访问顺序。
5. 在可接受时调整浮点语义。

每一步都必须记录：源码变化、向量化报告变化、汇编变化、性能变化。最后说明哪一步真正起作用。

5.5.16 本节总结

自动向量化连接了编译器语义和硬件 SIMD。它需要清晰循环、可证明依赖、可接受成本、明确内存行为和合适的数值语义。对性能工程师来说，自动向量化报告不是结论，而是线索：它告诉你编译器看到了什么、担心什么、为什么愿意或不愿意生成 SIMD。

下一节讨论 alias、未定义行为和浮点语义。它们是自动向量化背后的根本约束：优化器越能相信你的代码满足某些契约，就越能大胆重排、合并、向量化；契约越模糊，优化器越保守。


```

                                int n)
{
    for (int i = 0; i < n; ++i) {
        out[i] = a[i] + b[i];
    }
}

```

这告诉编译器：在该指针的有效访问范围内，不会有其他 `restrict` 指针访问同一对象。它是强契约，不是优化建议。如果调用者违反契约，行为不再可靠。

对高性能 kernel，`restrict` 可以显著减少 runtime alias check，帮助向量化和 load/store 重排。但它也把责任从编译器转移给接口设计和调用者。

5.6.5 TBAA：基于类型的别名分析

strict aliasing 允许编译器根据类型判断某些指针不应该指向同一对象。例如 `int*` 和 `float*` 通常不能随便指向同一个对象并通过不同类型访问它。

LLVM IR 中常见 TBAA metadata：

```

%v = load float, ptr %p, align 4, !tbaa !5
store i32 %x, ptr %q, align 4, !tbaa !9

```

TBAA 告诉优化器某些访问属于不同类型树，从而辅助 alias analysis。

但 C++ 有例外，例如 `char`、`std::byte` 可以用于查看对象表示。还有 union、placement new、生命周期、类型擦除等复杂问题。性能代码中不要用违反 strict aliasing 的技巧做“类型重解释”。应使用明确、安全的机制，例如 `std::bit_cast` 做值表示转换，或用 `std::memcpy` 表达字节复制。

常见误区

违反 strict aliasing 的代码经常在 `-O0` 下看起来正常，在 `-O3` 下突然出错。原因不是编译器“坏”，而是程序给了优化器错误契约。

5.6.6 未定义行为不是运行时异常

Undefined Behavior 的意思不是“程序会抛异常”或“结果随机”。它的意思是：一旦程序执行到这种行为，语言标准不再对程序行为作保证；编译器可以假设这种情况不会发生，并基于这个假设优化。

常见 UB：

- 有符号整数溢出。
- 数组越界访问。
- 解引用空指针。
- 使用未初始化的自动变量。
- 违反对象生命周期规则。
- 违反 strict aliasing。
- 数据竞争。

性能优化中最常见的是 signed overflow 和 out-of-bounds。

5.6.7 signed overflow 如何改变优化

例子：

```

extern "C" bool greater_after_add(int x)
{
    return x + 1 > x;
}

```

数学整数上，这总为真。但机器上的 32 位补码如果 `x==INT_MAX`，`x+1` 会 wrap 到 `INT_MIN`。C++ 规定有符号整数溢出是 UB，因此编译器可以假设 `x+1` 不溢出，于是返回 true。

IR 中你可能看到：

```

%add = add nsw i32 %x, 1
ret i1 true

```

nsw 就是 no signed wrap。它告诉优化器：产生 signed overflow 的路径不需要保持定义良好的行为。

如果你需要明确 wrap 语义，应使用无符号整数，或使用编译器提供的 checked arithmetic：

```

std::uint32_t y = x + 1;           // modulo arithmetic

int out = 0;
bool overflow = __builtin_add_overflow(a, b, &out);

```

5.6.8 越界访问和优化

数组越界不仅可能读错数据，还会给优化器错误前提。

```
extern "C" int first_or_zero(const int* p, int n)
{
    if (n <= 0) {
        return 0;
    }
    return p[n - 1];
}
```

这里有边界保护，较安全。反过来：

```
extern "C" int unsafe(const int* p, int n)
{
    int x = p[n - 1];
    if (n <= 0) {
        return 0;
    }
    return x;
}
```

当 $n \leq 0$ 时，前面的 $p[n-1]$ 已经越界，程序进入 UB。优化器可能根据“程序不会越界”推导 $n > 0$ ，从而重排或删除后续判断。不要把 UB 当成“反正这条路径不会用结果”。

5.6.9 未初始化读取

未初始化局部变量读取也是 UB：

```
extern "C" int maybe_uninit(bool flag)
{
    int x;
    if (flag) {
        x = 42;
    }
    return x;
}
```

如果 $flag == false$ ，读取 x 未定义。优化器可能生成你意想不到的代码。性能代码中为了“省初始化”留下未初始化路径非常危险。正确做法是：

- 明确所有路径都赋值。
- 使用工具检查，例如 UBSan、MSan。
- 在热路径中用结构保证初始化，而不是依赖注释。

5.6.10 浮点语义：不是普通实数代数

浮点数不是数学实数。IEEE 浮点有：

- 舍入。
- NaN。
- 正负 Inf。
- 正零和负零。
- subnormal。
- 异常标志和环境。

因此：

```
(a + b) + c
a + (b + c)
```

在浮点里不一定相等。优化器在严格浮点语义下不能随便重新结合。dot product、sum、mean、variance、softmax 都会受影响。

5.6.11 fast-math 到底放弃了什么

`-ffast-math` 通常不是一个单一假设，而是一组假设的集合。不同编译器细节不同，但常见含义包括：

- 允许重新结合浮点表达式。
- 假设没有 NaN 或 Inf，或不严格保留相关行为。
- 忽略有符号零差异。
- 允许 reciprocal 近似。
- 允许 FMA contraction。
- 不严格维护浮点异常和环境。

例子：

```
sum += a[i] * b[i];
```

严格语义下，编译器可能必须保留逐次累加顺序。fast-math 下，编译器可以用向量 FMA 形成多条累加链，最后水平归约。

```
strict:
    scalar order, predictable language-level accumulation order

fast-math:
    vector lanes accumulate independently
    horizontal reduction changes order
    FMA may change rounding
```

AI 推理通常更关心吞吐和可接受误差，fast-math 或等价局部假设经常可用。但训练、科学计算、金融、几何鲁棒性和测试基准可能要求更严格语义。工程上必须写清楚误差标准。

5.6.12 FMA contraction

FMA 把乘法和加法融合成一次舍入：

```
a * b + c
-> fma(a, b, c)
```

性能上，FMA 通常吞吐好；数值上，它和先乘后加不一定完全一样，因为舍入次数不同。编译器是否自动收缩，取决于选项和语义：

```
clang++ -O3 -ffp-contract=off
clang++ -O3 -ffp-contract=fast
clang++ -O3 -ffast-math
```

在 GEMM、dot、convolution 中，FMA 是核心性能来源。你需要同时理解它的性能价值和数值差异。

5.6.13 高性能接口的契约设计

性能 kernel 不应该只写函数签名，还要写清楚契约。例如：

```
// Contract:
// - out, a, b point to at least n valid float elements.
// - out, a, b do not overlap.
// - pointers are at least 32-byte aligned.
// - n >= 0.
// - NaN/Inf behavior follows the selected math mode.
extern "C" void add_kernel(float* __restrict out,
                          const float* __restrict a,
                          const float* __restrict b,
                          int n);
```

对应实现和测试必须验证或约束这些条件：

- debug 版本加断言。
- benchmark 生成满足契约的数据。
- 文档写明 alias/alignment/size。
- 对外 API 不轻易暴露危险假设。
- 内部 hot kernel 可以有更强 precondition。

生产库常见做法是外层安全 API 做检查和 dispatch，内层 microkernel 假设输入满足严格契约。这是性能和安全之间的工程分层。

5.6.14 Sanitizer：找 UB 的工程工具

学习阶段要养成用 sanitizer 验证性能代码的习惯：

```
clang++ -std=c++20 -O1 -g -fsanitize=undefined,address \
    test.cpp -o test_asan_ubsan

./test_asan_ubsan
```

常用工具：

- ASan：越界、use-after-free 等内存错误。
- UBSan：signed overflow、非法 shift、空指针等 UB。
- MSan：未初始化读取。
- TSan：数据竞争。

注意 sanitizer 会改变性能，不能用于最终 benchmark。但它非常适合 correctness 和契约验证。

5.6.15 实验 24.1: signed overflow 改变汇编

实验

目标：观察 signed overflow 假设如何影响优化。
写：

```
#include <climits>

extern "C" bool greater_after_add(int x)
{
    return x + 1 > x;
}

extern "C" bool greater_after_add_unsigned(unsigned x)
{
    return x + 1 > x;
}
```

任务：

1. 生成 -O0、-O3 IR。
2. 比较 signed 和 unsigned 版本。
3. 找 nsw 或相关优化痕迹。
4. 生成汇编，观察 signed 版本是否直接返回 true。
5. 用 UBSan 运行 greater_after_add(INT_MAX)。

报告要求：解释为什么 signed 版本和 unsigned 版本语义不同。

5.6.16 实验 24.2: strict aliasing

实验

目标：理解错误类型重解释为什么危险。
准备一个违反 alias 规则的例子，并和安全写法对比：

```
#include <bit>
#include <cstdint>
#include <cstring>

float unsafe_bits_to_float(std::uint32_t bits)
{
    return *reinterpret_cast<float*>(&bits);
}

float safe_bits_to_float(std::uint32_t bits)
{
    return std::bit_cast<float>(bits);
}
```

任务：

1. 使用 -O3-Wall-Wstrict-aliasing 编译。
2. 生成 IR，查看 load 的 TBAA metadata。
3. 用不同优化等级运行测试。
4. 用 std::memcpy 再写一个安全版本。
5. 总结什么时候应使用 bit_cast、什么时候用 memcpy。

5.6.17 实验 24.3: fast-math 误差和性能

实验

目标：把 fast-math 的性能收益和数值风险同时量化。
实现 dot_f32，生成三种版本：

```
clang++ -std=c++20 -O3 -march=native dot.cpp -o dot_strict
clang++ -std=c++20 -O3 -march=native -ffp-contract=fast dot.cpp -o dot_fma
clang++ -std=c++20 -O3 -march=native -ffast-math dot.cpp -o dot_fast
```

任务：

1. 比较三种版本的汇编。
2. 记录是否使用 FMA、是否向量化。
3. 使用随机数据、极大极小混合数据、包含 NaN/Inf 的数据测试结果差异。
4. benchmark 大小为 1K、1M、64M 的输入。

5. 写出你能接受 fast-math 的条件。
交付物必须包含误差表和性能表，不能只给结论。

5.6.18 本节作业

习题与作业

作业 24.A：契约审计

选择一个你前面写过的 kernel，例如 `saxpy`、`dot` 或 `relu`。写一份契约审计：

- 输入指针是否允许为空？
- size 是否允许为 0？
- 指针是否允许重叠？
- 是否要求对齐？
- 是否可能越界？
- 是否有 signed overflow 风险？
- 浮点 NaN/Inf/负零如何处理？
- fast-math 是否可接受？

要求：把每条契约落实到代码、断言、测试或文档。

习题与作业

作业 24.B：UB 示例报告

构造三个 UB 示例：

1. signed overflow。
2. out-of-bounds。
3. uninitialized read。

对每个示例：

- 运行 `-O0` 和 `-O3`。
- 生成 IR 和汇编。
- 用 sanitizer 检测。
- 写明为什么不能依赖 `-O0` 的表现判断程序正确。

习题与作业

作业 24.C：浮点策略说明书

为 AI CPU 算子项目写一页浮点策略说明：

- 哪些 kernel 允许 fast-math。
- 哪些 kernel 只允许 FMA contraction。
- 哪些 kernel 必须严格保持顺序。
- 误差用绝对误差、相对误差还是 ULP 衡量。
- 如何构造极端输入测试。
- benchmark 报告中如何记录浮点编译参数。

5.6.19 本节总结

Alias、UB 和浮点语义决定了优化器的边界。你给出的契约越清楚，编译器越能做强优化；契约越模糊，编译器越保守；契约错误，程序就可能在高优化等级下崩塌。性能工程不是绕开语言规则，而是在规则内给优化器足够明确、真实、可验证的信息。

至此，第 5 章建立了从编译器流水线到 IR、SSA、标量优化、循环优化、自动向量化和语义契约的完整主线。后面进入现代 CPU 微架构时，我们会继续把这些 IR 和汇编变化映射到前端、后端、端口、延迟、吞吐、cache 和 TLB。

第 6 章 现代 CPU 微架构性能模型

6.1 Pipeline、前端和后端总览

6.1.1 本节定位

本章从极简 CPU 过渡到现代乱序 CPU，把 front-end、back-end、retire 和 memory subsystem 放到同一张图里。

6.1.2 核心问题

- 指令从字节到执行结果经过哪些阶段？
- 前端和后端分别可能如何成为瓶颈？
- 为什么 ISA 指令不是 CPU 内部执行的最小单位？

6.1.3 必须讲清楚的底层机制

- fetch、decode、uop cache、rename、dispatch、scheduler、execute、retire。
- pipeline hazard、dependency、flush。
- retire in order 与 execute out of order。

6.1.4 实验和交付物

实验

用简单汇编和 `llvm-mca` 观察指令吞吐，画出一个 loop 的前后端数据流。

6.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

6.2 Latency、Throughput、依赖链和 ILP

6.2.1 本节定位

本章建立性能分析最核心词汇：延迟、吞吐、critical path、instruction-level parallelism。

6.2.2 核心问题

- 为什么单条指令 latency 不能直接预测函数耗时？
- 多累加器为什么能加速 reduction？
- critical path 如何决定下界？

6.2.3 必须讲清楚的底层机制

- latency vs reciprocal throughput。
- data dependency、false dependency、dependency chain。
- ILP、unroll、multiple accumulators 和 register pressure。

6.2.4 实验和交付物

实验

实现 `sum1/sum2/sum4/sum8`，比较汇编、`llvm-mca` 和 benchmark。

6.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

6.3 uops、执行端口和 llvm-mca

6.3.1 本节定位

本章讲 x86 指令如何被解码为 micro-ops，以及如何用 `llvm-mca` 建立静态后端模型。

6.3.2 核心问题

- 一条汇编指令为什么可能是多个 uops?
- port pressure 如何限制吞吐?
- `llvm-mca` 能解释什么，不能解释什么?

6.3.3 必须讲清楚的底层机制

- uop decomposition、execution resources、load/store ports。
- block throughput、resource pressure、timeline。
- 静态模型与实测差异。

6.3.4 实验和交付物

实验

抽取 SAXPY/FMA loop，用 `llvm-mca` 分析 uops、resource pressure 和 block throughput。

6.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

6.4 分支预测和推测执行

6.4.1 本节定位

本章讲控制流为什么会影晌性能，以及 CPU 如何猜测未来控制流来保持流水线工作。

6.4.2 核心问题

- 可预测分支和随机分支为什么差异巨大?
- branchless 什么时候更好，什么时候更差?
- speculative execution 为什么既提升性能又带来复杂性?

6.4.3 必须讲清楚的底层机制

- branch predictor、BTB、return stack buffer。
- mispredict flush、speculative work、bad speculation。
- `cmov`、`setcc`、mask 和 SIMD blend。

6.4.4 实验和交付物

实验

构造全真、全假、随机分支输入；比较 branch、branchless、`cmov` 和 SIMD mask。

6.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

6.5 前端带宽、l-cache、uop cache 和代码尺寸

6.5.1 本节定位

本章讲为什么过度 inline、过度 unroll 和模板膨胀会让代码在前端变慢。

6.5.2 核心问题

- 代码越展开为什么不一定越快？
- l-cache 和 uop cache 如何影响 hot loop？
- operator fusion 如何在减少内存流量和增加代码尺寸之间取舍？

6.5.3 必须讲清楚的底层机制

- instruction fetch、decode bandwidth、uop cache。
- code layout、hot/cold path、function alignment。
- loop body size、unroll factor 和 front-end bound。

6.5.4 实验和交付物

实验

写不同 unroll 因子的 kernel，记录 text size、汇编大小和 benchmark。

6.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

6.6 乱序执行、寄存器重命名和提交

6.6.1 本节定位

本章深入现代 CPU 如何在保持单线程语义的前提下乱序执行。

6.6.2 核心问题

- 为什么 cache miss 后 CPU 不一定完全停住？
- 寄存器重命名如何消除假依赖？
- reorder buffer 为什么需要按序提交？

6.6.3 必须讲清楚的底层机制

- physical register file、ROB、reservation station。
- load/store queue、memory disambiguation。
- precise exception 和 speculation rollback。

6.6.4 实验和交付物

实验

构造独立链和依赖链，比较乱序执行能隐藏哪些延迟，哪些隐藏不了。

6.6.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 7 章 内存层级、数据布局、TLB 与 Roofline

7.1 内存层级：寄存器、cache、DRAM

7.1.1 本节定位

本章从 CPU 与 DRAM 速度差距讲起，建立 memory hierarchy 的第一性原理。

7.1.2 核心问题

- 为什么需要多级 cache?
- SRAM 和 DRAM 的角色差别是什么?
- 工作集大小如何决定瓶颈?

7.1.3 必须讲清楚的底层机制

- register file、L1/L2/L3、DRAM。
- latency、bandwidth、capacity 的三角关系。
- inclusive/exclusive/non-inclusive cache 的概念边界。

7.1.4 实验和交付物

实验

做工作集扫描，观察 ns/element 和 GB/s 随数组大小变化的台阶。

7.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

7.2 Cache line、局部性和 miss 类型

7.2.1 本节定位

本章讲 cache line、temporal/spatial locality、compulsory/capacity/conflict miss。

7.2.2 核心问题

- 为什么 stride=1 比 stride=16 快很多?
- cache line 利用率如何计算?
- conflict miss 为什么容量够也会慢?

7.2.3 必须讲清楚的底层机制

- cache line、set associativity、tag/index/offset。
- replacement policy 的工程直觉。
- hardware prefetcher 与规律访问。

7.2.4 实验和交付物

实验

做 stride 扫描；设计 conflict miss 实验；解释 cache line 浪费。

7.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

7.3 TLB、页和地址翻译性能

7.3.1 本节定位

本章把操作系统虚拟内存和 CPU TLB 连接到性能模型。

7.3.2 核心问题

- TLB miss 为什么会让随机大跨度访问变慢？
- 4 KiB page 和 huge page 对性能有什么影响？
- page walk 和 cache 有什么关系？

7.3.3 必须讲清楚的底层机制

- virtual address decomposition。
- TLB hierarchy、page table walk、page fault。
- huge page、NUMA locality 和 memory allocator。

7.3.4 实验和交付物

实验

按 page stride 访问大数组，观察访问延迟；比较连续访问和每页一次访问。

7.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

7.4 Load/Store、store buffer、prefetch 和写入策略

7.4.1 本节定位

本章讲内存访问在 CPU 内部如何排队、发射、等待和写回。

7.4.2 核心问题

- load miss 如何影响依赖链？
- store 为什么可能引发 read-for-ownership？
- prefetch 什么时候有用？

7.4.3 必须讲清楚的底层机制

- address generation unit、load queue、store queue。
- store buffer、store forwarding、write allocate。
- hardware/software prefetch 和 non-temporal store。

7.4.4 实验和交付物

实验

实现 copy、triad、stream store、pointer chasing，分析 load/store 瓶颈。

7.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发, 逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念; 每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

7.5 数据布局: AoS、SoA、AoSoA 和 tensor layout

7.5.1 本节定位

本章讲数据布局如何决定 cache、SIMD 和算子性能。

7.5.2 核心问题

- AoS 和 SoA 什么时候各自合理?
- tensor 的 NHWC/NCHW/blocked layout 为什么影响性能?
- layout 改变如何换取更简单的 SIMD 数据流?

7.5.3 必须讲清楚的底层机制

- struct padding、cache line utilization。
- contiguous load vs gather。
- blocked/tiled layout 和 AI tensor memory format。

7.5.4 实验和交付物

实验

实现粒子更新 AoS/SoA 对比; 设计一个小 tensor layout 转换 benchmark。

7.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发, 逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念; 每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

7.6 Roofline: FLOP、Bytes 和 Arithmetic Intensity

7.6.1 本节定位

本章建立判断 compute-bound 与 memory-bound 的基础模型。

7.6.2 核心问题

- 为什么只数 FLOP 会误导?
- arithmetic intensity 如何计算?
- roofline 能解释什么, 不能解释什么?

7.6.3 必须讲清楚的底层机制

- FLOP counting、byte traffic、write allocate。
- peak compute、memory bandwidth、attainable performance。
- cache-aware roofline 和多层 roofline 初步。

7.6.4 实验和交付物

实验

对 vector add、SAXPY、dot、naive GEMM 计算 AI 并测量 GFLOP/s、GB/s。

7.6.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 8 章 SIMD、AVX2/FMA 与低精度计算

8.1 SIMD 思维模型

8.1.1 本节定位

本章从数据并行的角度讲 SIMD：一个指令处理多个 lane，为什么它适合数组和 tensor。

8.1.2 核心问题

- SIMD lane 是什么？
- scalar loop 如何变成 vector loop？
- tail、alignment 和 mask 为什么不可避免？

8.1.3 必须讲清楚的底层机制

- XMM/YMM/ZMM register。
- packed integer/floating operations。
- vector length、lane、horizontal vs vertical operation。

8.1.4 实验和交付物

实验

写 scalar vector add，观察自动向量化生成的 AVX2 汇编。

8.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

8.2 AVX2 Intrinsics 从入门到可审计代码

8.2.1 本节定位

本章讲如何用 intrinsics 写可读、可测试、可审计的 AVX2 代码。

8.2.2 核心问题

- intrinsic 和汇编指令是什么关系？
- 如何查 Intel Intrinsics Guide？
- 如何写 scalar reference 和 tail path？

8.2.3 必须讲清楚的底层机制

- `_mm256`、load/store、add/mul/max。
- aligned/unaligned load。
- compiler flags、target attributes、ISA dispatch 前置。

8.2.4 实验和交付物

实验

实现 AVX2 vector add、mul、ReLU；用测试覆盖非 8 倍数长度。

8.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

8.3 FMA、归约和浮点误差

8.3.1 本节定位

本章讲 FMA 指令、dot product、horizontal reduction 和浮点累加顺序。

8.3.2 核心问题

- FMA 为什么既影响性能也影响数值？
- SIMD reduction 为什么比 elementwise 更复杂？
- 多累加器如何隐藏 FMA latency？

8.3.3 必须讲清楚的底层机制

- vfmadd 指令族。
- horizontal add、shuffle reduction。
- error model、reassociation、fast-math。

8.3.4 实验和交付物

实验

实现 scalar dot、auto-vectorized dot、AVX2 FMA dot，并写误差报告。

8.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

8.4 Shuffle、Permute、Gather 和数据移动

8.4.1 本节定位

本章讲 SIMD 中经常比计算更贵的数据重排。

8.4.2 核心问题

- 为什么 shuffle 可能比 multiply 更贵？
- gather 为什么不能当连续 load 用？
- layout 如何减少 shuffle/gather？

8.4.3 必须讲清楚的底层机制

- lane 内/跨 lane shuffle。
- unpack、blend、broadcast、permute。
- gather/scatter 的微架构成本。

8.4.4 实验和交付物

实验

实现 4x4 transpose、AoS to SoA、gather vs contiguous load benchmark。

8.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

8.5 Tail、Mask、Alignment 和边界策略

8.5.1 本节定位

本章讲 SIMD kernel 的工程边界：非整齐长度、小 shape、对齐、padding 和 remainder kernels。

8.5.2 核心问题

- tail 处理为什么影响小 shape 延迟？
- padding、scalar tail、masked load/store 如何取舍？
- 对齐优化什么时候值得？

8.5.3 必须讲清楚的底层机制

- prologue/main loop/epilogue。
- AVX2 mask load/store 的限制。
- small-size dispatch 与 code size。

8.5.4 实验和交付物

实验

为 ReLU、vector add、dot 设计三种 tail policy 并 benchmark。

8.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

8.6 int8 量化、AVX-VNNI 和低精度推理

8.6.1 本节定位

本章讲 AI 推理中的 int8 数据路径，从量化误差到 dot product 指令。

8.6.2 核心问题

- scale 和 zero-point 如何把 float 映射到 int8？
- int8 dot 为什么累加到 int32？
- AVX-VNNI 相比 AVX2 fallback 优势在哪里？

8.6.3 必须讲清楚的底层机制

- quantize/dequantize、rounding、saturation。
- signed/unsigned int8 combinations。
- VNNI dot-product 指令和 fallback 实现。

8.6.4 实验和交付物

实验

实现 int8 dot reference、AVX2 fallback、AVX-VNNI 版本，写误差与性能报告。

8.6.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第三卷：HPC、AI 算子、生产工具和综合项目

第 9 章 HPC Kernel：从循环优化到 MiniBLAS

9.1 Loop Nest 性能模型

9.1.1 本节定位

本章讲多层循环如何决定数据复用、向量化、并行划分和 cache 行为，是 GEMM、stencil、tensor 算子的共同基础。

9.1.2 核心问题

- loop order 如何改变内存访问模式？
- interchange、blocking、fusion、fission 分别解决什么？
- 如何从循环结构推导 bytes 和 FLOPs？

9.1.3 必须讲清楚的底层机制

- iteration space、dependence、reuse distance。
- loop transformation correctness。
- compiler optimization 与手工重排的边界。

9.1.4 实验和交付物

实验

对 matrix-vector、matrix transpose、2D stencil 改变 loop order 并 benchmark。

9.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

9.2 Transpose、Stencil 和 Reduction

9.2.1 本节定位

本章用三个典型 memory-bound kernel 训练 cache blocking、边界处理和归约设计。

9.2.2 核心问题

- transpose 为什么 naive 很慢？
- stencil 如何复用邻域数据？
- parallel/SIMD reduction 如何处理依赖和误差？

9.2.3 必须讲清楚的底层机制

- cache blocking for transpose。
- stencil halo、boundary、temporal blocking 初步。
- reduction tree、multiple accumulators、determinism。

9.2.4 实验和交付物

实验

实现 naive/blocked transpose、1D/2D stencil、scalar/SIMD reduction，并用 roofline 解释。

9.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

9.3 GEMM: 从 naive 到 cache blocking

9.3.1 本节定位

本章从矩阵乘第一性原理出发，解释为什么 GEMM 可以接近峰值，以及 naive 为什么做不到。

9.3.2 核心问题

- GEMM 的 FLOP 和 bytes 如何计算？
- loop order 如何影响 B 矩阵访问？
- block size 如何与 cache 容量关联？

9.3.3 必须讲清楚的底层机制

- arithmetic intensity of GEMM。
- ijk/ikj/jik loop order。
- L1/L2/L3 blocking 和 C tile reuse。

9.3.4 实验和交付物

实验

实现 naive GEMM、loop-order variants、blocked GEMM，扫描 tile size 并写 roofline 报告。

9.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

9.4 GEMM Packing 和 Panel Layout

9.4.1 本节定位

本章讲生产 GEMM 为什么需要 packing，以及 packing 如何把复杂访问变成连续流。

9.4.2 核心问题

- packing 的成本为什么值得付？
- packed A/B panel 应该如何布局？
- edge panel 如何处理？

9.4.3 必须讲清楚的底层机制

- panel-major layout。
- cache reuse 与 TLB locality。
- packing overhead、amortization、prefetch friendliness。

9.4.4 实验和交付物

实验

为 blocked GEMM 增加 A/B packing，对比无 packing 版本，分析不同矩阵大小下 packing overhead。

9.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

9.5 AVX2 GEMM Microkernel

9.5.1 本节定位

本章讲 register blocking 和 AVX2 FMA microkernel 的设计。

9.5.2 核心问题

- MR/NR 为什么由寄存器数量、FMA latency 和 vector width 共同决定？
- accumulator 如何常驻寄存器？
- microkernel 如何与 packing 约定接口？

9.5.3 必须讲清楚的底层机制

- register tile。
- broadcast A、vector load B、FMA accumulate。
- unroll K、load/FMA scheduling、store C。

9.5.4 实验和交付物

实验

实现 4x8 或 6x8 fp32 AVX2 microkernel，用 `llvm-mca` 和 benchmark 分析。

9.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

9.6 MiniBLAS: 把 GEMM 做成可维护库

9.6.1 本节定位

本章把前面 kernel 组织成一个教学版 BLAS，实现测试、dispatch、benchmark 和报告。

9.6.2 核心问题

- kernel 如何从实验代码变成库接口？
- 如何处理 shape、leading dimension、edge case？
- 如何与 OpenBLAS/BLIS 做公平对比？

9.6.3 必须讲清楚的底层机制

- API design、row/column major、lda/ldb/ldc。
- kernel selection、packing buffers、threading 前置。
- correctness matrix 和 benchmark suite。

9.6.4 实验和交付物

实验

交付 MiniBLAS `sgemm`，含 reference、tests、benchmarks、assembly audit 和与 OpenBLAS 对比。

9.6.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 10 章 AI CPU 算子优化

10.1 AI CPU 算子性能模型

10.1.1 本节定位

本章建立 AI 算子的通用分析框架：shape、dtype、layout、FLOPs、bytes、reuse、vectorization 和数值误差。

10.1.2 核心问题

- 一个 AI 算子如何从数学公式变成 CPU kernel?
- tensor layout 如何影响 SIMD 和 cache?
- latency-oriented 和 throughput-oriented 优化有什么不同?

10.1.3 必须讲清楚的底层机制

- tensor shape/stride/layout。
- dtype、quantization、broadcast、reduction。
- operator benchmark、framework overhead 与 kernel overhead。

10.1.4 实验和交付物

实验

选择 5 个小算子，写 shape、FLOPs、bytes、expected bottleneck 表。

10.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

10.2 Softmax：数值稳定性、归约和向量化

10.2.1 本节定位

本章讲 softmax 的稳定实现、两遍/三遍扫描、SIMD exp 近似和内存流量。

10.2.2 核心问题

- 为什么 naive softmax 会溢出?
- max/sum reduction 如何 vectorize?
- streaming softmax 解决什么问题?

10.2.3 必须讲清楚的底层机制

- stable softmax formula。
- vector reduction、exp approximation。
- row-wise shape、tail、cache behavior。

10.2.4 实验和交付物

实验

实现 stable softmax reference、optimized row softmax，比较误差和性能。

10.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

10.3 LayerNorm 和 RMSNorm

10.3.1 本节定位

本章讲 normalization 算子的 reduction、数值稳定性、SIMD、内存访问和 fusion 机会。

10.3.2 核心问题

- mean/variance 如何稳定计算？
- RMSNorm 为什么少一个 mean？
- gamma/beta、residual、epsilon 如何影响数据流？

10.3.3 必须讲清楚的底层机制

- two-pass vs one-pass variance。
- Welford algorithm。
- vectorized reduction、broadcast scale、tail。

10.3.4 实验和交付物

实验

实现 layernorm/RMSNorm reference 和 AVX2 优化版，写误差报告。

10.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

10.4 GELU、激活函数和近似计算

10.4.1 本节定位

本章讲非线性激活在 CPU 上的近似、向量化和误差控制。

10.4.2 核心问题

- GELU exact 和 tanh/polynomial approximation 如何取舍？
- 激活函数通常是 compute-bound 还是 memory-bound？
- 近似误差如何报告？

10.4.3 必须讲清楚的底层机制

- erf/tanh approximation。
- polynomial evaluation、Horner scheme、FMA。
- vector math library 与手写近似。

10.4.4 实验和交付物

实验

实现 ReLU、Sigmoid、GELU 近似，测误差和性能。

10.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

10.5 Attention、KV Cache 和 Streaming Softmax

10.5.1 本节定位

本章讲 transformer attention 在 CPU 上的数据流、瓶颈和优化方向。

10.5.2 核心问题

- prefill 和 decode 阶段瓶颈为何不同？
- KV cache layout 如何决定带宽和 TLB 行为？
- streaming softmax 如何避免 materialize 大矩阵？

10.5.3 必须讲清楚的底层机制

- QK、softmax、PV 三段模型。
- head_dim、seq_len、batch 对 FLOP/byte 的影响。
- KV cache paging、stride、layout。

10.5.4 实验和交付物

实验

实现单头 attention reference、streaming softmax、小型 KV cache layout benchmark。

10.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

10.6 Operator Fusion：收益、代价和边界

10.6.1 本节定位

本章讲 fusion 如何减少内存流量，也如何增加寄存器压力、代码尺寸和测试复杂度。

10.6.2 核心问题

- fusion 为什么常常能加速 AI 算子？
- 哪些 fusion 会让性能变差？
- epilogue fusion 如何和 GEMM microkernel 结合？

10.6.3 必须讲清楚的底层机制

- producer-consumer locality。
- register pressure、front-end pressure、code generation。
- bias、activation、scale、residual fusion。

10.6.4 实验和交付物

实验

实现 matmul+bias、matmul+bias+ReLU、layernorm+residual，对比 separate kernels。

10.6.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

10.7 MiniDNN: 教学版 AI CPU 算子库

10.7.1 本节定位

本章把 normalization、activation、matmul、attention 组织成一个小算子库。

10.7.2 核心问题

- operator API 如何设计？
- primitive cache 和 shape specialization 如何工作？
- 如何保证不同 ISA 路径输出一致？

10.7.3 必须讲清楚的底层机制

- tensor descriptor、memory layout、primitive descriptor。
- ISA dispatch、workspace、scratchpad。
- correctness tests、benchmark suite、regression tracking。

10.7.4 实验和交付物

实验

交付 MiniDNN，小规模复现 layernorm、GELU、matmul epilogue 和 attention kernel。

10.7.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 11 章 性能工具、并行、NUMA 与生产库阅读

11.1 perf、PMU 和 Top-Down 分析

11.1.1 本节定位

本章讲硬件性能计数器如何把“猜测瓶颈”变成证据。

11.1.2 核心问题

- cycles、instructions、IPC 能说明什么，不能说明什么？
- Top-Down 的 Retiring、Bad Speculation、Frontend Bound、Backend Bound 如何解读？
- WSL2 为什么可能无法使用完整 PMU？

11.1.3 必须讲清楚的底层机制

- PMU event、sampling、multiplexing。
- perfstat、perfrecord/report。
- Top-Down methodology。

11.1.4 实验和交付物

实验

在裸机或可用环境下对前面 kernel 做 perf stat/report；在 WSL2 记录限制并用替代证据分析。

11.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

11.2 VTune、AMD uProf 和图形化性能分析

11.2.1 本节定位

本章讲工业 profiling 工具如何定位热点、内存瓶颈、线程扩展性和微架构问题。

11.2.2 核心问题

- 图形化工具相比 perf 提供了什么？
- 如何避免被漂亮图表误导？
- 如何把 profiler 结论回到源码和汇编？

11.2.3 必须讲清楚的底层机制

- hotspot、microarchitecture exploration、memory access analysis。
- call stack、source/assembly correlation。
- profiling overhead 和采样误差。

11.2.4 实验和交付物

实验

对 MiniBLAS 或 MiniDNN kernel 做一次完整工具分析，生成性能事故报告。

11.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

11.3 线程、Atomics 和内存模型入门

11.3.1 本节定位

本章讲多线程性能基础，解释为什么加线程不等于加速。

11.3.2 核心问题

- work partition 如何影响 scaling?
- atomic 为什么贵?
- memory order 是什么层面的约束?

11.3.3 必须讲清楚的底层机制

- thread creation、thread pool、chunking。
- atomic RMW、cache coherence。
- acquire/release/relaxed 的工程直觉。

11.3.4 实验和交付物

实验

实现 parallel reduction，比较 mutex、atomic、local accumulation 三种方式。

11.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

11.4 False Sharing、Cache Coherence 和 NUMA

11.4.1 本节定位

本章讲多核共享缓存一致性、false sharing 和 NUMA locality 对性能的影响。

11.4.2 核心问题

- 为什么不同线程写不同变量也会互相拖慢?
- cache coherence 如何维护一致性?
- NUMA 下内存分配位置为什么重要?

11.4.3 必须讲清楚的底层机制

- cache line ownership、MESI 直觉。
- padding/alignment 防 false sharing。
- NUMA node、first touch、thread pinning。

11.4.4 实验和交付物

实验

写 false sharing benchmark；通过 padding 修复；在支持环境下做 NUMA locality 实验。

11.4.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

11.5 CPUID、ISA Dispatch 和多版本 Kernel

11.5.1 本节定位

本章讲生产库如何在不同 CPU feature 上选择最优实现。

11.5.2 核心问题

- 如何检测 AVX2、FMA、AVX-VNNI?
- runtime dispatch 的开销应该放在哪里?
- fallback 路径如何测试?

11.5.3 必须讲清楚的底层机制

- CPUID、XCR0、OS support check。
- function pointer dispatch、ifunc、target attribute。
- primitive cache 和 first-call overhead。

11.5.4 实验和交付物

实验

实现 scalar/AVX2/AVX-VNNI 三路径 dispatch，确保输出一致并测 first-call 与 steady-state。

11.5.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

11.6 生产库阅读：oneDNN、BLIS、OpenBLAS

11.6.1 本节定位

本章训练阅读真实高性能库，把教学实现和工业实现连接起来。

11.6.2 核心问题

- 生产库为什么比教学 kernel 复杂很多?
- BLIS 如何组织 GEMM microkernel?
- oneDNN primitive descriptor 和 primitive cache 如何工作?

11.6.3 必须讲清楚的底层机制

- packing、blocking、threading、dispatch。
- primitive descriptor、memory descriptor、post-op fusion。
- portability、testing、maintenance。

11.6.4 实验和交付物

实验

追踪一次 BLIS/OpenBLAS GEMM 或 oneDNN matmul 调用路径，写生产库阅读报告。

11.6.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 12 章 Capstone：大师级性能工程项目

12.1 Capstone 项目设计

12.1.1 本节定位

本章定义最终大项目，要求把全书知识整合成一个可测试、可测量、可解释的系统。

12.1.2 核心问题

- 选择 MiniBLAS、MiniDNN、int8 GEMV/GEMM 还是 memory-bound suite?
- 如何定义正确性、性能和工程边界?
- 如何安排里程碑?

12.1.3 必须讲清楚的底层机制

- 项目范围控制。
- benchmark suite、test suite、report pipeline。
- baseline、目标、风险和验收标准。

12.1.4 实验和交付物

实验

提交 capstone proposal，包含目标、接口、实验、风险、参考实现和性能目标。

12.1.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

12.2 性能事故复盘方法

12.2.1 本节定位

本章讲如何像工程师一样复盘一次性能问题，而不是只报告“优化后更快”。

12.2.2 核心问题

- 如何定义事故现象?
- 如何建立假设并逐步排除?
- 如何把源码、汇编、模型、PMU、benchmark 统一起来?

12.2.3 必须讲清楚的底层机制

- symptom、hypothesis、experiment、evidence、conclusion。
- regression tracking。
- failed experiments as knowledge。

12.2.4 实验和交付物

实验

选择一个真实失败优化，写完整性能事故复盘报告。

12.2.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

12.3 最终评审：从学习者到性能工程师

12.3.1 本节定位

本章定义最终答辩和自检标准，帮助读者评估是否真正具备独立优化能力。

12.3.2 核心问题

- 你能否独立从源码追到机器执行？
- 你能否设计可信 benchmark？
- 你能否解释性能瓶颈并提出可验证优化？

12.3.3 必须讲清楚的底层机制

- 知识图谱回顾。
- 能力矩阵。
- 后续长期路线：生产库、论文、真实 workload。

12.3.4 实验和交付物

实验

提交最终项目、报告、代码、benchmark 数据、汇编审计和答辩提纲。

12.3.5 扩写标准

本节后续扩写时必须从具体 C/C++ 例子出发，逐层连接到编译器表示、汇编、机器执行、系统机制和性能证据。不能只列概念；每个核心概念都必须解释它为什么存在、没有它会发生什么、底层如何实现、如何用工具观察、对性能有什么影响。

第 A 章 x86-64 速查表

A.1 通用寄存器

寄存器	常见用途
rax/eax	返回值、通用计算
rdi/edi	System V 第 1 个整数/指针参数
rsi/esi	System V 第 2 个整数/指针参数
rdx/edx	System V 第 3 个整数/指针参数
rcx/ecx	System V 第 4 个整数/指针参数
r8/r8d	System V 第 5 个整数/指针参数
r9/r9d	System V 第 6 个整数/指针参数
rsp	栈指针
rbp	栈帧指针或通用 callee-saved 寄存器

A.2 常见指令

指令	简化语义	备注
movdst,src	dst=src	复制位模式
leaddst,[addr]	dst=addr	不访问内存
adddst,src	dst+=src	更新 flags
subdst,src	dst-=src	更新 flags
cmpa,b	计算 a-b 更新 flags	不保存结果
testa,b	计算 a&b 更新 flags	常用于判零
jmplabel	无条件跳转	改变 rip
callf	调用函数	压入返回地址
ret	返回	弹出返回地址

第 B 章 Linux 与二进制工具命令速查

B.1 编译与反汇编

```
g++ -E main.cpp -o main.i
g++ -S -O3 -masm=intel main.cpp -o main.s
g++ -c -O3 main.cpp -o main.o
g++ main.o -o app
objdump -drwC -Mintel main.o
```

B.2 ELF 和符号

```
file app
readelf -h app
readelf -S app
readelf -l app
readelf -r app
nm -C app
```

B.3 调试

```
g++ -O0 -g main.cpp -o app
gdb -q ./app
```

```
break main
run
stepi
info registers
x/10i $rip
disassemble /m main
```

第 C 章 LaTeX 写作规范

C.1 章节规则

每章必须从具体问题开始，不能只列概念。所有术语第一次出现时使用术语宏：

```
\term{中文术语}{English explanation}
\engterm{English term}
```

C.2 代码规则

短代码使用：

```
\code{...}
```

多行代码使用 `lstlisting`。

C.3 实验和习题

实验使用 `labbox`，习题使用 `exercisebox`，常见误区使用 `warningbox`。

C.4 扩写节奏

先扩写 Part 0 和 Part 1，确保刚学完 C/C++ 的读者能进入。之后再扩写汇编、测量、编译器和微架构。

第 D 章 参考资料和延伸阅读

D.1 公开课程

- CMU 15-213 / 18-213: Intro to Computer Systems。
- MIT 6.172: Performance Engineering of Software Systems。
- UC Berkeley CS61C: Great Ideas in Computer Architecture。
- Stanford CS149: Parallel Computing。
- Georgia Tech CS 6290: High Performance Computer Architecture。

D.2 官方和一手资料

- Intel 64 and IA-32 Architectures Software Developer's Manual。
- Intel 64 and IA-32 Architectures Optimization Reference Manual。
- AMD64 Architecture Programmer's Manual。
- LLVM documentation and `llvm-mca` command guide。
- Intel Intrinsics Guide。

D.3 性能数据和生产库

- uops.info。
- Agner Fog optimization resources。
- BLIS。
- OpenBLAS。
- oneDNN。

第 E 章 实验和作业评分标准

E.1 正确性

所有优化实现必须有 reference。没有 reference 的性能数字没有意义。

E.2 复现性

报告必须记录 CPU、OS、编译器版本、编译参数、输入规模、运行次数和统计口径。

E.3 证据链

高质量报告必须包含源码、汇编或 IR、性能数据和解释模型。涉及微架构时，应尽量包含 `llvm-mca`、PMU 或替代证据。

E.4 失败实验

每个大作业至少记录一个失败实验。失败实验必须解释假设、结果和下一步推断。

术语表

中文	英文	简要解释
翻译单元	translation unit	一个源文件经过预处理后的编译输入。
中间表示	intermediate representation	编译器用于优化和 lowering 的内部表示。
架构状态	architectural state	ISA 规定的软件可见状态，如寄存器、内存、flags。
微架构	microarchitecture	CPU 内部实现 ISA 的方式，如 pipeline、cache、uops。
延迟	latency	操作输入 ready 到输出 ready 的时间。
吞吐	throughput	稳态下单位时间能完成的操作数量。
算术强度	arithmetic intensity	FLOPs 与数据搬运字节数的比值。

索引

- ABI, 5, 104
- alignment, 42
- AoS, 49

- basic block, 84
- big-endian, 34
- bit, 32
- byte, 32

- coefficient of variation, 172

- ELF, 13
- endianness, 34

- harness, 140

- immediate, 63
- induction variable, 92
- instruction
 - add, 3, 17, 19, 20, 29, 67, 73, 75, 83, 119, 133
 - add dword ptr [mem], edx, 70
 - add eax, 1, 95
 - add eax, dword ptr [...], 26
 - add eax, esi, 18, 19, 63
 - add rax, rbx, 3
 - add reg, 1, 75
 - and, 77, 78, 82, 83
 - call, 22, 28, 30, 84, 94, 95, 97, 102–104, 106, 110, 111, 113, 119, 120, 125–127, 129, 131, 134, 135, 137
 - call 0x401180, 95
 - call rax, 96
 - call rbx, 96
 - call target, 95
 - cmov, 27, 168
 - cmovcc, 84, 89, 90, 99
 - cmovle, 98
 - cmp, 26, 27, 67, 73, 78–80, 83, 84, 86
 - cmp a, b, 78
 - cmp edi, 0, 86
 - cmp edi, esi, 20, 67, 87
 - cmp rdi, rdx, 91
 - cpuid, 142
 - cvtsi2sd, 106
 - cvtss2sd, 106
 - dec, 75
 - imul, 75, 76, 81, 83
 - inc, 75
 - ja, 86, 87, 94, 103
 - jae, 86
 - jb, 86, 87
 - jbe, 86
 - jcc, 27, 84, 86, 97, 99, 103
 - je, 86
 - jg, 20, 85–87, 94
 - jge, 86
 - jge done, 23
 - jl, 26, 86, 87
 - jle, 85, 86
 - jle .Lelse, 88
 - jmp, 94, 97
 - jmp rax, 96
 - jmp target, 95
 - jna, 86
 - jnae, 86
 - jnb, 86
 - jnbe, 86
 - jne, 86
 - jng, 86
 - jnge, 86
 - jnl, 86
 - jnle, 86
 - jnz, 86
 - jz, 86
 - lea, 20, 29, 38, 41, 49–51, 54, 66, 67, 69, 70, 73, 75, 76, 80, 81, 83, 128, 132, 133, 135
 - lea eax, [rdi+rsi], 20
 - lfence, 130, 142, 151
 - mfence, 130
 - mov, 2, 17, 20, 50, 55, 70, 73, 74, 83
 - mov eax, dword ptr [0x100c], 39
 - mov eax, dword ptr [rdi], 65
 - mov eax, edi, 19, 20, 85
 - mov rax, qword ptr [rdi], 65
 - mov reg, 0, 74
 - movaps, 110, 115, 116
 - movsd, 55
 - movsd xmm0, qword ptr [rdi], 65
 - movss, 41, 55
 - movss xmm0, dword ptr [rdi], 65
 - movsx, 73, 76, 77, 79, 81, 83
 - movsx eax, byte ptr [p], 82
 - movsxd, 54, 76, 77, 81, 83
 - movsxd rsi, esi, 30, 77
 - movups, 116
 - movzx, 40, 55, 70, 73, 76, 77, 79, 81, 83
 - movzx eax, byte ptr [p], 82
 - movzx eax, byte ptr [rdi], 65
 - neg edi, 80
 - not, 77, 78
 - or, 77, 78, 82, 83
 - pop, 109, 126, 131
 - push, 109, 110, 122, 126, 131
 - push rbp, 106
 - rdtsc, 137, 142, 151
 - rdtscp, 142

- ret, 17, 19–22, 25, 28, 30, 84, 95–97, 102, 104, 131
- sal, 77
- sar, 73, 77, 83
- sar 1, 77
- sbb, 80
- sbb eax, eax, 80
- seta, 89, 102
- setb, 87, 89, 102
- setcc, 27, 84, 89, 99, 168
- sete, 89
- setg, 89, 102
- setg al, 67, 89
- setl, 87, 89, 102
- setne, 78, 89
- shl, 41, 51, 54, 77
- shl rax, 4, 50
- shl rsi, 4, 41
- shr, 73, 77, 83
- shr 1, 77
- sub, 67, 75, 83, 133
- sub reg, 1, 75
- sub reg, reg, 75
- sub rsp, ..., 106, 110, 111
- sub rsp, 16, 116
- sub rsp, 24, 116
- test, 27, 67, 78, 79, 82–85
- test a, b, 78, 79
- test edi, edi, 23, 86
- vfmadd, 149
- xor, 74, 77, 78
- xor eax, eax, 74, 89
- xor reg, reg, 74, 75, 83
- ISA, 3, 18
- kernel, 140
- little-endian, 34
- name mangling, 112, 124
- object representation, 33
- padding, 43
- red zone, 111
- reference, 140
- register
 - al, 19, 64, 67, 89, 111, 112, 116, 117, 130
 - ax, 19, 64
 - bl, 64
 - bp, 64
 - bpl, 64
 - bx, 64
 - cl, 64, 77
 - cx, 64
 - di, 64
 - dil, 64
 - dl, 64
 - dx, 64
 - eax, 17, 19, 21, 22, 24, 26, 29, 30, 39, 40, 63–65, 70, 74, 89, 90, 105, 107, 117, 130, 137, 142
 - ebp, 19, 64
 - ebx, 19, 64
 - ecx, 19, 64, 90, 105
 - edi, 17, 19–21, 23, 26, 29, 63, 64, 74, 87, 96, 105
 - edx, 19, 29, 64, 105, 106, 137, 142
 - esi, 17, 19–22, 26, 29, 30, 63–65, 69, 90, 96, 105, 106
 - esp, 19, 64
 - r10, 19, 64
 - r10b, 64
 - r10d, 19, 64
 - r10w, 64
 - r11, 19, 64, 109
 - r11b, 64
 - r11d, 19, 64
 - r11w, 64
 - r12, 19, 64, 109, 115, 120, 137
 - r12b, 64
 - r12d, 19, 64
 - r12w, 64
 - r13, 19, 64, 137
 - r13b, 64
 - r13d, 19, 64
 - r13w, 64
 - r14, 19, 64
 - r14b, 64
 - r14d, 19, 64
 - r14w, 64
 - r15, 3, 19, 64, 109, 120
 - r15b, 64
 - r15d, 19, 64
 - r15w, 64
 - r8, 3, 19, 64, 105, 109, 122
 - r8b, 64
 - r8d, 19, 64, 98, 105
 - r8w, 64
 - r9, 19, 64, 105, 122
 - r9b, 64
 - r9d, 19, 64, 105
 - r9w, 64
 - rax, 3, 4, 17, 19, 21, 22, 29, 39, 50, 63, 64, 105, 107, 109, 118, 120, 122, 128, 130, 131, 192
 - rbp, 3, 19, 64, 106, 109, 110, 117, 120, 122
 - rbx, 3, 17, 19, 64, 109, 115, 117, 120, 126, 135
 - rcx, 3, 17, 19, 26, 51, 64, 90, 91, 105, 109, 120, 122
 - rdi, 3, 17, 19, 22, 23, 26, 30, 37, 39–41, 47, 49, 50, 64–66, 69, 90, 96, 105, 108, 109, 122, 128, 132, 192, 205
 - rdx, 3, 17, 19, 64, 105, 107, 109, 122, 128
 - rip, 3, 17–23, 26, 27, 29, 30, 63, 83–85, 89, 90, 94, 95, 102, 103, 131, 137
 - rsi, 3, 17, 19, 23, 26, 37, 39–41, 49, 64–66, 77, 105, 108, 109, 122, 192, 205

- rsp, 3, 5, 17, 19, 21, 22, 26, 27, 29, 30, 63, 64, 95, 102, 103, 106, 109–112, 117, 122, 123, 126, 127, 131, 135
- si, 64
- sil, 64
- sp, 64
- spl, 64
- xmm, 108, 117
- xmm0, 37, 41, 106, 107, 109, 124, 131
- xmm15, 109
- xmm7, 106
- SoA, 50
- tail call, 95
- timed region, 140
- two's complement, 35
- word, 32
- workload, 140
- 中间表示, 11
- 吞吐, 24
- 延迟, 24
- 微架构, 18
- 抽象语法树, 11
- 时序逻辑, 17
- 架构状态, 18
- 汇编器, 12
- 组合逻辑, 17
- 翻译单元, 10
- 进程, 14
- 链接器, 13
- 预处理器, 10